



中华人民共和国国家标准

GB/T 16262.1—2025/ISO/IEC 8824-1:2021

代替 GB/T 16262.1—2006

信息技术 抽象语法记法一(ASN.1) 第1部分:基本记法规范

Information technology—Abstract Syntax Notation One(ASN.1)—
Part 1:Specification of basic notation

(ISO/IEC 8824-1:2021, IDT)

2025-03-28 发布

2025-10-01 实施

国家市场监督管理总局 发布
国家标准化管理委员会

目 次

前言	III
引言	V
1 范围	1
2 规范性引用文件	1
3 术语和定义	2
4 缩略语	13
5 记法	13
6 类型扩展的 ASN.1 模块	16
7 编码规则的可扩展性要求	16
8 标记	17
9 编码指令	18
10 ASN.1 记法的使用	18
11 ASN.1 字符集	19
12 ASN.1 词项	20
13 模块定义	30
14 引用类型和值定义	35
15 支持引用 ASN.1 组件的记法	37
16 类型和值的赋值	38
17 类型和值的定义	39
18 布尔类型的记法	43
19 整数类型的记法	44
20 枚举类型的记法	45
21 实数类型的记法	46
22 位串类型的记法	48
23 八位位组串类型的记法	50
24 空类型记法	50
25 序列类型的记法	51
26 单一序列类型的记法	54
27 集合类型的记法	57
28 单一集合类型的记法	58
29 选择类型的记法	59
30 精选类型的记法	61
31 前缀类型的记法	62

32	对象标识符类型的记法	64
33	相对对象标识符类型记法	66
34	OID 国际化资源标识符类型的记法	67
35	相对 OID 国际化资源标识符类型的记法	68
36	嵌入式 pdv 类型的记法	69
37	外部类型的记法	70
38	时间类型	72
39	字符串类型	81
40	字符串类型的记法	82
41	受限制字符串类型的定义	82
42	命名字符、集合和属性类别集	87
43	字符的常规顺序	91
44	无限制字符串类型的定义	91
45	第 46 章~第 48 章中定义的类型的记法	93
46	通用时间	93
47	世界时间	94
48	对象描述符类型	95
49	受约束类型	95
50	元素集规范	97
51	子类型元素	99
52	扩展标志	106
53	例外标识符	108
54	编码控制部分	109
附录 A (规范性)	ASN.1 正则表达式	110
附录 B (规范性)	定义的时间类型	114
附录 C (规范性)	类型和值兼容的规则	120
附录 D (规范性)	指派的对象标识符和 OID 国际化资源标识符值	129
附录 E (规范性)	编码引用	132
附录 F (资料性)	国际对象标识符树中弧的分配和使用	133
附录 G (资料性)	举例和提示	134
附录 H (资料性)	ASN.1 字符串的辅导附录	161
附录 I (资料性)	类型扩展 ASN.1 模块的辅助附录	164
附录 J (资料性)	TIME 类型的辅导附录	170
附录 K (资料性)	TIME 类型的值记法分析	175
附录 L (资料性)	ASN.1 记法总结	179
参考文献		199

前 言

本文件按照 GB/T 1.1—2020《标准化工作导则 第 1 部分：标准化文件的结构和起草规则》的规定起草。

本文件是 GB/T 16262《信息技术 抽象语法记法一(ASN.1)》的第 1 部分,GB/T 16262 已经发布了以下 4 个部分:

- 第 1 部分:基本记法规范;
- 第 2 部分:信息客体规范;
- 第 3 部分:约束规范;
- 第 4 部分:ASN.1 规范的参数化。

本文件代替 GB/T 16262.1—2006《信息技术 抽象语法记法一(ASN.1) 第 1 部分:基本记法规范》,与 GB/T 16262.1—2006 相比,除结构调整和编辑性改动外,主要技术变化如下:

- a) 增加了“编码指令”“OID 国际化资源标识符类型的记法”“相对 OID 国际化资源标识符类型的记法”“时间类型”“编码控制部分”“定义的时间类型”“编码引用”(见第 9 章、第 34 章、第 35 章、第 38 章、第 54 章、附录 B、附录 E);
- b) 增加了“简单字符串词项”“时间值字符串”“XML 时间值字符串词项”“属性和名称设置词项”“编码引用”“整数值 Unicode 标号”“非整数值 Unicode 标号”“XML 布尔扩展真项”“XML 布尔扩展假项”“XML 实非数字项”和“XML 实无穷项”等 ASN.1 词项(见第 12 章);
- c) 增加了通用类别标记的分配(见 8.4 的表 1)、ASN.1 字符“不换行字符”(见 11.1 的表 2)、“空白的字符表示”(见 12.1.6)、“xmlasnltypename 的字符”(见 12.36.2 的表 4)、“保留字”(见 12.38)、“前缀类型的记法”(见 31.1 和 31.3);
- d) 增加了“命名字符、集合和属性类别集”(见 42.1.5 和 42.1.6)、“子类型元素”中的“属性设置”“时间长度范围”“时间点范围”和“重复范围”(见 51.10、51.11、51.12 和 51.13);
- e) 更改了“ModuleDefinition”的产生式(见 13.1,2006 年版的 12.1)、“BuiltinType”记法和“固有类型”记法(见 17.2,2006 年版的 16.2)、“BuiltinValue”记法(见 17.9,2006 年版的 16.9)、“XMLBuiltinValue”记法(见 17.10,2006 年版的 16.10)、“XMLBooleanValue”记法(见 18.3,2006 年版的 17.3)、“XMLIntegerValue”记法(见 19.9,2006 年版的 18.9)、“XMLEnumeratedValue”记法(见 20.8,2006 年版的 19.8)、“RealValue”和“XMLRealValue”记法(见 21.6,2006 年版的 20.6)、“XMLBitStringValue”记法(见 22.9,2006 年版的 21.9)、“XMLSequenceOfValue”记法(见 26.3,2006 年版的 25.3)、“XMLSetOfValue”记法(见 28.3,2006 年版的 27.3)、“Named-Type”记法(见 31.2.1,2006 年版的 30.1)、“SubtypeElements”记法(见 51.1,2006 年版的 47.1);
- f) 更改了“GB/T 13000.1 中定义的命名字符和集”章标题为“命名字符、集合和属性类别集”(见第 42 章,2006 年版的第 38 章)、“通用时间”的有关规定(见第 46 章,2006 年版的第 42 章);更改了附录 C“指派的对象标识符值”为附录 D“指派的对象标识符和 OID 国际化资源标识符值”,并对内容作了适当增加(见附录 D,2006 年版的附录 C);
- g) 删除了“字符的常规顺序”的有关规定(见 2006 年版的 39.5)。

本文件等同采用 ISO/IEC 8824-1:2021《信息技术 抽象语法记法一(ASN.1) 第 1 部分:基本记法规范》。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别专利的责任。

本文件由全国信息技术标准化技术委员会(SAC/TC 28)提出并归口。

本文件起草单位:中国电子技术标准化研究院、中国科学院计算技术研究所、深圳赛西信息技术有限公司、联想(北京)有限公司、深圳市中兴微电子技术有限公司、山东省计算中心(国家超级计算济南中心)、西门子(中国)有限公司、广州鲁邦通物联网科技股份有限公司、浪潮电子信息产业股份有限公司、富泰华工业(深圳)有限公司、北京东土科技股份有限公司、神木市信息产业发展集团有限公司。

本文件主要起草人:张弛、杨宏、王婷、刘敏、郭雄、王云浩、蔡廷晓、孙波、李刚、苏静茹、孙胜、程远、张少联、汤松柏、卓兰、鲁璐、张学琴、刘生强、杨四雄、孙金洋、李靖、朱彬、赵伟、白欣璐、史喆。

本文件及其所代替文件的历次版本发布情况为:

——2006年首次发布为GB/T 16262.1—2006;

——本次为第一次修订。

引 言

GB/T 16262 旨在规范抽象语法记法,GB/T 16262 的编制基于 ISO/IEC 8824。根据 ISO/IEC 8824, GB/T 16262 拟由 4 个部分构成。

- 第 1 部分:基本记法规范。目的在于定义数据类型、值及数据类型的约束。
- 第 2 部分:信息客体规范。目的在于提供规定信息客体类别、信息客体和信息客体集合的记法。
- 第 3 部分:约束规范。目的在于提供规定用户定义的约束、表约束和内容约束的记法。
- 第 4 部分:ASN.1 规范的参数化。目的在于定义 ASN.1 规范的参数化记法。

信息技术 抽象语法记法一(ASN.1)

第 1 部分:基本记法规范

1 范围

本文件提供一个称为抽象语法记法一(ASN.1)的标准记法,该记法用来定义数据类型、值及数据类型的约束。

本文件:

- 定义了一些简单的类型及其标记,也规定了引用这些类型和规定这些类型值的记法;
- 定义了从多个基本类型构造新类型的机制,也规定了定义这些类型及为它们指派标记和规定这些类型值的记法;
- 定义了 ASN.1 内使用的字符集(通过引用其他标准)。

当需要定义信息的抽象语法时即可采用 ASN.1。

ASN.1 记法供其他定义 ASN.1 类型编码规则的标准引用。

2 规范性引用文件

下列文件中的内容通过文中的规范性引用而构成本文件必不可少的条款。其中,注日期的引用文件,仅该日期对应的版本适用于本文件;不注日期的引用文件,其最新版本(包括所有的修改单)适用于本文件。

注:本文件基于 GB/T 13000—2010 和 Unicode 标准版本 3.2.0:2002。本文件不能应用于这两个标准的最新版本。

GB/T 2311—2000 信息技术 字符代码结构与扩充技术(ISO/IEC 2022:1994,IDT)

GB/T 7408.1—2023 日期和时间 信息交换表示法 第 1 部分:基本原则(ISO 8601-1:2019,IDT)

GB/T 13000—2010 信息技术 通用多八位编码字符集(UCS)(ISO/IEC 10646:2003,IDT)

GB/T 16262.2—2025 信息技术 抽象语法记法一(ASN.1) 第 2 部分:信息客体规范(ISO/IEC 8824-2:2021,IDT)

GB/T 16262.3—2025 信息技术 抽象语法记法一(ASN.1) 第 3 部分:约束规范(ISO/IEC 8824-3:2021,IDT)

GB/T 16262.4—2025 信息技术 抽象语法记法一(ASN.1) 第 4 部分:ASN.1 规范参数化(ISO/IEC 8824-4:2021,IDT)

ISO/IEC 8825-1:2021 Information technology—ASN.1 encoding Rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)

注:GB/T 16263.1—2006 信息技术 ASN.1 编码规则 第 1 部分:基本编码规则(BER)、正则编码规则(CER)和非典型编码规则(DER)规范(ISO/IEC 8825-1:2002,IDT)

ISO/IEC 8825-2:2021 Information technology—ASN.1 encoding rules: Specification of Packed Encoding Rules (PER)

注:GB/T 16263.2—2006 信息技术 ASN.1 编码规则 第 2 部分:紧缩编码规则(PER)规范(ISO/IEC 8825-2:2002,IDT)

ISO/IEC 8825-4:2021 Information technology—ASN.1 encoding rules: XML Encoding Rules (XER)

GB/T 16262.1—2025/ISO/IEC 8824-1:2021

注：GB/T 16263.4—2015 信息技术 ASN.1 编码规则 第4部分：XML 编码规则(XER)(ISO/IEC 8825-4:2008,IDT)

ISO/IEC 646 信息技术 信息交换用 ISO 七位编码字符集(Information technology—ISO 7-bit coded character set for information interchange)

注：GB/T 1988—1998 信息技术 信息交换用七位编码字符集(ISO/IEC 646:1991,EQV)

ISO/IEC 6523:1998 数据互换 用于识别组织和组织部分的结构(Data interchange—Structures for the identification of organizations and organization parts)

Rec.ITU-T X.660(2011)| ISO/IEC 9834-1:2012 信息技术 开放系统互连 OSI 登记机构的操作规程 一般规程和国际对象标识符树的顶级弧(Information technology—Open Systems Interconnection—Procedures for the operation of OSI Registration Authorities: General procedures and top arcs of the ASN.1 International Object Identifier tree)

注：GB/T 17969.1—2015,信息技术 开放系统互连 OSI 登记机构的操作规程 第1部分：一般规程和国际对象标识符树的顶级弧(ISO/IEC 9834-1:2008,NEQ)

Unicode 标准 版本 3.2.0;2002.Unicode 联盟(读物,MA,Addison-Wesley)

注：因为提供控制字符的名称并规范字符分类,所以包括这项文件。

W3C XML 1.0:2008 可扩展置标语言(XML)1.0(第5版),W3C 建议,版权 © 2008 W3C,(MIT,ERCIM,Keio),<http://www.w3.org/TR/2008/REC-xml-20081126/>

3 术语和定义

下列术语和定义适用于本文件。

3.1 国际对象标识符树规范

本文件使用 Rec.ITU-T X.660(2011)|ISO/IEC 9834-1:2012 中定义的下列术语：

- a) 整数值 Unicode 标号 integer-valued Unicode label;
- b) 国际对象标识符树 international object identifier tree;
- c) **OID 国际化资源标识符** OID internationalized resource identifier;
- d) 长弧 long arc;
- e) 对象标识符 object identifier;
- f) 主整数值 primary integer value;
- g) 辅标识符 secondary identifier;
- h) Unicode 标号 Unicode label.

3.2 信息客体规范

本文件使用 GB/T 16262.2—2025 中定义的下列术语：

- a) 信息客体 information object;
- b) 信息客体类别 information object class;
- c) 信息客体集合 information object set;
- d) 单一实例类型 instance-of type;
- e) 对象类别字段类型 object class field type.

3.3 约束规范

本文件使用 GB/T 16262.3—2025 中定义的下列术语：

- a) 成分关系约束 component relation constraint;
- b) 表约束 table constraint.

3.4 ASN.1 规范的参数化

本文件使用 GB/T 16262.4—2025 中定义的下列术语：

- a) 参数化类型 parameterized type;
- b) 参数化值 parameterized value。

3.5 组织标识的结构

本文件使用 ISO/IEC 6523:1998 中定义的下列术语：

- a) 发布组织 issuing organization;
- b) 组织代码 organization code;
- c) 国际代码指示符 International Code Designator。

3.6 通用多八位编码字符集(UCS)

本文件使用 GB/T 13000—2010 中定义的下列术语：

- a) 基本多文种平面(BMP) Basic Multilingual Plane(BMP);
- b) 字符元 cell;
- c) 组合用字符 combining character;
- d) 图形符号 graphic symbol;
- e) 组 group;
- f) 有限子集 limited subset;
- g) 平面 plane;
- h) 行 row;
- i) 选择子集 selected subset。

3.7 日期和时间的表示

本文件使用 GB/T 7408.1—2023 中定义的下列术语：

- a) 基本格式 basic format;
- b) 日历日期 calendar date;
- c) 平年 common year;
- d) 时间长度 duration;
- e) 扩展格式 extended format;
- f) 公历 Gregorian calendar;
- g) 瞬时 instant;
- h) 闰秒 leap second;
- i) 闰年 leap year;
- j) 当地时间 local time of day;
- k) 顺序日期 ordinal date;
- l) 循环时间间隔 recurring time interval;
- m) 时间轴 time axis;
- n) 时间间隔 time interval;
- o) 时间 time;
- p) 时间刻度 time-scale;

- q) 协调世界时 UTC;
- r) 星期日期 week date。

3.8 附加定义

3.8.1

抽象字符 abstract character

用于组织、控制或表示文本数据的抽象值。

注：附录 H 提供术语抽象字符更完整的描述。

3.8.2

抽象值 abstract value

其定义仅基于用来携带某些语义的类型，而与其在任何编码中的表达方式无关的值。

注：抽象值的示例是整数类型、布尔类型、字符串类型或者整数和布尔的序列（或选择）类型等的值。

3.8.3

附加时间类型 additional time type

定义为时间类型的子类型（见 3.8.83），通过将属性设置子类型记法应用于时间类型或有用的或已定义的时间类型。

3.8.4

ASN.1 字符集 ASN.1 character set

第 11 章中规定的用于 ASN.1 记法的字符集。

3.8.5

ASN.1 规范 ASN.1 specification

一个或多个 ASN.1 模块的集合。

3.8.6

关联类型 associated type

仅用于定义类型的值及子类型记法的类型。

注：在本文件中，当有必要明确某一类型在 ASN.1 中的定义方式与其编码方式之间可能存在显著差异时，会定义关联类型。关联类型不会出现在用户规范中。

3.8.7

位串类型 bitstring type

其非典型值是 0 个、1 个或多个二进制位的有序序列的简单类型。

注：当需要携带抽象值的嵌入编码时，不赞成使用没有内容约束（见 GB/T 16262.3—2025 的第 11 章）的位串（或八位位组串）类型。而使用嵌入 pdv 类型（见第 36 章）提供更灵活的机制，允许声明抽象语法及嵌入的抽象值的编码。

3.8.8

布尔类型 boolean type

具有两个可区分值的简单类型。

3.8.9

字符性质 character property

与定义字符汇的表中的字符元相关的信息集。

注：信息通常将包含部分或全部下面的项：

- a) 图形符号；
- b) 字符名称；
- c) 在特定环境下使用时，与字符相关的功能的定义；

- d) 它是否表示数字；
- e) 只有在(大/小)写中不同的关联字符。

3.8.10

字符抽象语法 character abstract syntax

其值规定为 0 个、1 个或多个字符的字符串集的任何抽象语法,这些字符取自字符的某些已规定的集合。

3.8.11

字符汇 character repertoire

对字符怎样编码没有任何隐含说明的字符集中的那些字符。

3.8.12

字符串类型 character string types

其值取自某些已定义字符集的字符串的简单类型。

3.8.13

字符传送语法 character transfer syntax

字符抽象语法的任何传送语法。

3.8.14

选择类型 choice types

通过引用不同类型列表来定义的类型;选择类型的每个值从其中一个成分类型的值衍生而来。

3.8.15

成分类型 component type

定义 CHOICE、SET、SEQUENCE、SET OF 或 SEQUENCE OF 时引用的某种类型。

3.8.16

约束 constraint

可与类型一起使用来定义该类型的子类型的记法。

3.8.17

内容约束 contents constraint

规定内容为规定的 ASN.1 类型的编码,或规定的程序用来产生和处理内容的位串或八位位组串类型的约束。

3.8.18

控制字符 control character

出现在某些给出名称(和也许是有关某些环境的已定义功能),但没有分配图形符号,也不是间隔字符的字符汇中的字符。

注: HORIZONTAL TABULATION(9)和 LINE FEED(10)是打印环境里指派了格式化功能的控制字符示例。

DATA LINK ESCAPE(16)是通信环境中指派了功能的控制字符示例。

3.8.19

协调世界时 Coordinated Universal Time;UTC

国际时间局所保持的时标,构成标准频率和时间信号协调传播的基础

注 1: 此定义的来源是 ITU-R Rec.TF.460-5。ITU-R 也用 UTC 作为协调世界时的缩写。

注 2: UTC 和格林尼治标准时间(GMT)是两个可替换的时间标准,在大部分实用情况下确定同一时间的的时间标准。

3.8.20

(模块的)默认编码引用 default encoding reference(for a module)

在模块头文件中规定的编码引用,并在不包含编码引用的所有类型前缀中假定。

注: 如果在模块头文件中没有规定默认编码引用,那么所有不包含编码引用的类型前缀都被置标记。

3.8.21

已定义时间类型 defined time type

在附录 B 中定义的时间类型(见 3.8.83)的子类型,应用设计者在其应用需要时导入。

3.8.22

元素 element

支配类型的值或支配信息客体类别的信息客体,能分别从相同类型或相同类别信息客体的所有其他值中区别开来。

3.8.23

元素集 element set

所有支配类型的值或支配类别的信息客体的元素集合。

注: ISO/IEC 8824-2 的 3.4.7 中定义了支配类别。

3.8.24

嵌入 pdv 类型 embedded-pdv type

其值集是所有可能的抽象语法中值集形式上合并的类型。这一类型可用于希望在其协议中携带抽象值的 ASN.1 规范中,其类型可能在那个 ASN.1 规范的外部定义。它也携带被携带抽象值的抽象语法(类型)的标识以及用来编码那个抽象值的编码规则的标识。

3.8.25

编码 encoding

编码规则集应用于抽象值上产生的位图。

3.8.26

编码控制部分 encoding control section

ASN.1 模块的一部分,它能指派编码指令给 ASN.1 模块中定义或使用的类型。

3.8.27

编码指令 encoding instruction

与使用类型前缀或编码控制部分与类型关联的信息,并且通过一个或多个 ASN.1 编码规则影响该类型的编码。

注: 编码指令不会影响类型的抽象值,并且不期望出现在应用中。

3.8.28

编码引用 encoding reference

用于标识受类型前缀或编码控制部分中的编码指令影响的编码规则的一种名称(见附录 E)。

注: 编码引用 TAG 可用于规定类型前缀是指派标记,而不是编码指令(见 31.2)。

3.8.29

(ASN.1)编码规则 (ASN.1)encoding rules

在传送 ASN.1 类型值期间规定其表示的规则。编码规则也能使值从给出类型知识的表示法中恢复。

注: 为了规定编码规则,能为固有类型(和值)提供替换记法的所有被引用类型(和值)记法没有关系。

3.8.30

枚举类型 enumerated types

一个简单类型,其值作为类型记法一部分被给定不同标识符。

3.8.31

扩展附加 extension addition

扩展序列中增加的记法之一。对于集合、序列和选择类型,每个扩展附加是单个扩展附加组或单个成分类型的附加。对于枚举类型,它是单个进一步枚举的附加。对于约束,它是(只有)一个子类型元素

的附加。

注：扩展附加既按文本排列（紧接扩展标志），也按逻辑排列（有递增枚举值和在 CHOICE 替换时的递增标记）。

3.8.32

扩展附加组 **extension addition group**

在版本括号内分组的集合、序列或选择类型的一或多个成分。扩展附加组用来清楚地标识加在 ASN.1 模块特定版本中集合、序列或选择类型的成分，也能用简单整数标识那个版本。

3.8.33

扩展附加类型 **extension addition type**

包含于扩展附加组中的类型或本身是扩展附加（这时它不包含在扩展附加组中）的单个成分类型。

3.8.34

可扩展约束 **extensible constraint**

在外层带扩展标志，或使用带值的可扩展集的集合运算来扩展的子类型约束。

3.8.35

扩展插入点（或插入点） **extension insertion point(or insertion point)**

类型定义中插入扩展附加的位置。如果类型定义中有单个省略号，该位置在扩展序列中紧贴前面类型的类型记法末尾，或者如果类型定义中有扩展标志对，该位置紧贴在第二个省略号之前。

注：在任何选择、序列或集合类型组件内最多能有一个插入点。

3.8.36

扩展标志 **extension marker**

包含在构成扩展序列部分的所有类型中的语法标志（省略号）。

3.8.37

扩展标志对 **extension marker pair**

插入扩展附加之间的一对扩展标志。

3.8.38

相关扩展 **extension-related**

有相同扩展根，通过在一个上加 0 个或多个扩展附加而产生另一个的两个类型。

3.8.39

扩展根 **extension root**

扩展序列中第一个类型的可扩展类型。它要么携带不含附加记法、只有注解和扩展标志与相配的“}”或“)”之间空白的扩展标志；要么携带不含附加记法，只有单个逗号、注解和扩展标志之间空白的扩展标志对。

注：只有扩展根能是扩展序列中的第一个类型。

3.8.40

扩展序列 **extension series**

能按通过在扩展插入点附加文本形成序列中每个连续类型的方式排列的 ASN.1 类型序列。

3.8.41

可扩展类型 **extensible type**

有扩展标志或者应用了扩展约束的类型。

注：扩展标记能够以文本形式出现，也能够通过 EXTENSIBILITY-IMPLIED 插入（见 13.4）。

3.8.42

外部引用 **external reference**

类型引用、值引用、信息客体类别引用、信息客体引用或（可能参数化的）信息客体集合引用，他们在某些其他而不是正在被它引用的模块中定义，并且通过在被引用项上加模块名作为前缀来引用。

例如:ModuleName.TypeReference

3.8.43

外部类型 external type

携带对该 ASN.1 规范而言其类型可能是外部定义值的 ASN.1 规范一部分的类型。它也携带被携带值的类型标识。

3.8.44

假 false

布尔类型中的非典型值的一个(也见“真”)。

3.8.45

支配(类型);支配者 governing(type);governor

影响部分 ASN.1 语法解释、要求那部分 ASN.1 语法引用支配类型中的值的类型定义或引用。

3.8.46

等同类型定义 identical type definition

完成附录 B 中规定的转换后,如果 ASN.1“Type”产生式中的两个实例(见第 17 章)是相同词项的相同顺序列表(见第 12 章),则它们定义为等同类型定义。

3.8.47

OID 国际化资源标识符类型 OID internationalized resource identifier type

所有 OID 国际化资源标识符的集合。

注 1: 这是一种简单的类型,其值是 Unicode 标号序列,标识一系列从根到国际对象标识符树的节点的弧,由 Rec. ITU-T X.660|ISO/IEC 9834 系列标准所规定。

注 2: Rec.ITU-T X.660|ISO/IEC 9834-1 的规则允许各种权威机构独立地将 Unicode 标号与树的弧关联起来。

3.8.48

整数类型 integer type

具有非典型值的简单类型,值是正整数或负整数,包括 0(作为单一值)。

注: 当特定的编码规则限制整数的范围时,选择这种限制不至影响 ASN.1 任何用户。

3.8.49

词项 lexical item

取自 ASN.1 字符集(在第 12 章中规定),用来形成 ASN.1 记法的字符的已命名序列。

3.8.50

模块 module

类型、值、值集、信息客体类别、信息客体和信息客体集合(以及它们的参数化变体)采用 ASN.1 记法的一个或多个实例,用 ASN.1 模块记法来定界(见第 13 章)。

注: 术语信息客体类别(等)在 GB/T 16262.2—2025 中规定,而参数化在 GB/T 16262.4—2025 中规定。

3.8.51

空类型 null type

由单一值组成的简单类型,也称为空。

3.8.52

对象 object

精确定义的一段信息、定义或规范,它要有名称以便标识其在通信实例中的用途。

注: 这种对象可是 GB/T 16262.2—2025 中定义的信息客体。

3.8.53

对象描述符类型 object descriptor type

其非典型值是提供对信息客体简要描述的人可读的文本的类型。

注: 对象描述符值常常与单个对象相关,只有对象标识符值无歧义地标识一个对象。

3.8.54

对象标识符类型 object identifier type

其值是 Rec.ITU-T X.660|ISO/IEC 9834 系列规定的一个主整数序列的一个简单类型,标识从国际对象标识树的根到节点的一系列弧。

注 1: Rec.ITU-T X.660(2011)|ISO/IEC 9834-1:2012 的规则允许各种权威机构独立地将一个主整数与树的一个弧关联起来。

注 2: 在对象标识符类型的值记法(以及该类型的 XML 编码)中,可能包含弧的辅标识符。

3.8.55

八位位组串类型 octetstring type

其非典型值是 0 个、1 个或多个八位位组的有序序列的简单类型。每个八位位组是 8 个二进制的有序序列。

3.8.56

开放系统互连 open systems interconnection

提供许多以缩写“OSI”开始、用于本文件的术语的计算机通信结构。

注: 如果需要,这些术语的意义能从 ITU-T Rec.X.200 系列和相当的 ISO/IEC 标准中获得。如果 ASN.1 用于 OSI 环境,这些术语是唯一适用的。

3.8.57

开放类型记法 open type notation

用来表示取自不只一个 ASN.1 类型的值的集合的 ASN.1 记法。

注 1: 在本文件的正文中,术语“开放类型”和“开放类型记法”同义使用。

注 2: 所有 ASN.1 编码规则为单个 ASN.1 类型值提供无歧义编码,不必为“开放类型记法”提供无歧义的编码,因为“开放类型记法”携带取自在规范时刻通常没有确定的 ASN.1 类型的值。能无歧义确定那一字段的抽象值前需要“开放类型记法”中被编码值的类型的知识。

注 3: 本文件中是“开放类型记法”的唯一记法是 GB/T 16262.2—2025 第 14 章中规定的“ObjectClassField-Type”。这里的“FieldName”指明类型字段或者可变类型值字段。

3.8.58

(子类型的)双亲类型 parent type(of a subtype)

定义子类型时受约束的、并支配子类型记法的类型。

注: 双亲类型本身可能是某些其他类型的子类型。

3.8.59

产生式 production

用于规定 ASN.1 的形式记法的一部分(也叫作语法规则或 Backus-Naur Form,BNF)。

3.8.60

实数类型 real type

一个简单类型,其非典型值(第 21 章中规定)是实数集合的一个成员。

3.8.61

(类型的)递归定义 recursive definition(of a type)

ASN.1 的定义的一个集合,不能对这些定义重新排序,因此,结构中使用的所有类型在定义构造之前定义。

注: ASN.1 中允许递归定义:记法的用户有责任保证所用(产生类型的)这些值有限定的表示法并且与类型相关的值集至少包含一个值。

3.8.62

相对 OID 国际化资源标识符类型

relative OID internationalized resource identifier type

通过其相对于某些已知 OID 国际化资源标识符的位置来标识对象的值。

3.8.63

相对对象标识符 **relative object identifier**

通过其相对于某些已知对象标识符的位置来标识对象的值。

3.8.64

相对对象标识符类型 **relative object identifier type**

其值是所有可能相对对象标识符集的简单类型。

3.8.65

受限制字符串类型 **restricted character string type**

其字符取自类型规范中标识的固定字符汇的字符串类型。

3.8.66

精选类型 **selection types**

通过引用选择类型的成分类型定义的类型,而且其值精确地为那个成分类型的值。

3.8.67

序列类型 **sequence types**

通过引用固定的、有序的类型列表(有些可能声明是可选的)定义的类型;序列类型的每个值是取自每个成分类型的值的有序列表。

注: 当一个成分类型声明为可选时,序列类型的值不必包含那个成分类型的值。

3.8.68

单一序列类型 **sequence-of types**

通过引用单个成分类型定义的类型;单一序列类型中的每个值是成分类型的 0 个、1 个或多个值的有序列表。

3.8.69

(约束的)连续应用 **serial application(of constraints)**

约束应用在已经受约束的双亲类型。

3.8.70

集合运算 **set arithmetic**

使用如 50.2 中规定的并集、交集和差集(使用 EXCEPT)等算法的值或信息客体的新集的形式。

注: 术语“集合运算”没有覆盖约束连续应用的结果。

3.8.71

(时间属性的)设置 **setting(of a time property)**

能与给定时间属性关联的一系列值之一(见 3.8.82 和 J.4.2 中的注释)。

注: 任何适用于特定时间抽象值的时间属性只有一个设置(见表 6)。

3.8.72

集合类型 **set types**

通过引用固定的、无序的、类型(有些声明是可选的)列表定义的类型。集合类型中的每个值是一个无序的值列表,列表中的各个值取自相应的成分类型。

注: 当成分类型声明为可选时,集合类型的值不必包含那个成分类型的值。

3.8.73

单一集合类型 **set-of types**

通过引用单个成分类型定义的类型,单一集合类型中的每个值是成分类型的 0 个、1 个或多个值的无序列表。

3.8.74

简单类型 simple types

通过直接规定其值集来定义的类型。

3.8.75

间隔字符 spacing character

字符集中的字符,它用来包括字符串打印中的图形字符,但用空间隔在物理解释中表示。通常,不认为它是控制字符(见 3.8.18)。

注:字符集中可能有单个间隔字符,或宽度可变的多个间隔字符。

3.8.76

(双亲类型的)子类型 subtype(of a parent type)

其值是某些其他类型(双亲类型)值的子集(或完整集)的类型。

3.8.77

标记 tag

与类型的抽象值分离的附加信息,这些信息与每个 ASN.1 类型相关联,能通过类型前缀进行更改或扩充。

注:在某些编码规则中使用标记信息,以确保编码不会有歧义。标记信息不同于编码指令,因为标记信息与所有 ASN.1 类型相关联,即使它们没有类型前缀。

3.8.78

已标记类型 tagged type

通过引用单个现存类型和标记来定义的类型,新类型与该现存类型同构,但与它不等同。

3.8.79

置标记 tagging

将新标记指派给类型,替换或添加到已有(可能是默认的)标记。

3.8.80

时间抽象值 time abstract value

时间类型的抽象值。

3.8.81

时间组件 time component

用于规定时间抽象值定义的一部分。

注:时间组件的例子有日期组件(可能有年份组件)、当日时间组件或时差组件。

3.8.82

(时间抽象值的)时间属性 time property(of a time abstract value)

用来描述时间抽象值的众多术语之一(见 3.8.80)。

注:用来描述时间抽象值的时间属性通常取决于该抽象值的其他时间属性的设置。表 6 的第 1 列列出了时间属性。

3.8.83

时间类型 time type

TIME 类型,支持 GB/T 7408.1—2023 隐式定义的所有抽象值。

3.8.84

传送语法 transfer syntax

用来交换抽象语法中抽象值的位串的集合,通常是将编码规则应用在抽象语法上获得。

注:术语“传送语法”与“编码”同义。

3.8.85

真 true

布尔类型中的非典型值的一个(也见“假”)。

3.8.86

类型 type

已命名的值集合。

3.8.87

类型前缀 type prefix

用于将编码指令或标记指派给类型的 ASN.1 记法的一部分。

3.8.88

类型引用名 type reference name

在某些上下文中唯一与类型相联系的名稱。

注：指派引用名给本文件中定义的类型，在 ASN.1 中是普遍存在的。其他引用名在其他标准中定义，并只适用于那个标准的上下文中。

3.8.89

未限制字符串类型 unrestricted character string type

其抽象值取自字符抽象语法，并且带字符抽象语法及用于其编码的字符传送语法的标识的值的类型。

3.8.90

有用时间类型 useful time type

一种内置的供应用设计者直接使用的时间类型的子类型(见 3.8.83)。

3.8.91

(ASN.1)用户 user(of ASN.1)

用 ASN.1 定义一段特定信息的抽象语法的个人或组织。

3.8.92

值映射 value mapping

能使引用那些值的一个用来引用其他值的两个类型中的值之间的 1-1 对应关系。例如，这能用在规定子类型和默认值中(见附录 C)。

3.8.93

值引用名 value reference name

在某些上下文中唯一与值相联系的名稱。

3.8.94

值集(合) value set

类型的值的集合，语义上相当于子类型。

3.8.95

版本括号 version brackets

一对用来描画扩展附加组开始和结束的相邻左括号和右括号(“[[”或“]]”)。紧接在左括号对后面能有选择地给出扩展附加组版本号的数字。

3.8.96

版本号 version number

能与版本括号相联系的数字(见 I.1.8)。

注：版本号不能加在不是扩展附加组一部分的扩展附加上，也不能加在非选择、序列或集合的任意类型的扩展附

加上。

3.8.97

空白 white-space

任何在打印页上产生间隔的格式化动作,例如:空格或制表。

4 缩略语

下列缩略语适用于本文件。

ASN.1 抽象语法记法一(Abstract Syntax Notation One)

BER ASN.1 基本编码规则(Basic Encoding Rules of ASN.1)

BMP 基本多文种平面(Basic Multilingual Plane)

DCC 数据国家代码(Data Country Code)

DNIC 数据网络标识代码(Data Network Identification Code)

ECN ASN.1 编码控制记法(Encoding Control Notation of ASN.1)

ICD 国际代码指示符(International Code Designator)

IRI 国际化资源标识符(Internationalized Resource Identifier)

OID 对象标识符(Object Identifier)

OSI 开放系统互连(Open Systems Interconnection)

PER ASN.1 紧缩编码规则(Packed Encoding Rules of ASN.1)

ROA 公认运营机构(Recognized Operating Agency)

UCS 通用多八位编码字符集(Universal Multiple-Octet Coded Character Set)

URI 通用资源标识符(Universal Resource Identifier)

UTC 协调世界时(Coordinated Universal Time)

XML 可扩展置标语言(Extensible Markup Language)

5 记法

5.1 概述

5.1.1 ASN.1 记法由取自第 11 章规定的 ASN.1 字符集的字符序列组成。

5.1.2 每次使用 ASN.1 记法包括从 ASN.1 字符集中抽取字符并组合为词项。第 12 章规定了组成词项的所有字符序列,并命名了每个项。

5.1.3 在第 13 章(以及以下几章中),ASN.1 记法的规定是通过对组成 ASN.1 记法有效实例的那些词项序列的规定和命名,及对每个序列的 ASN.1 语义的规定来实现的。

5.1.4 为了规定允许的项序列,本文件使用下面各条中定义的形式记法。

5.2 产生式

5.2.1 所有的词项都已命名(见第 12 章),并且允许的项序列也都已命名。

5.2.2 一个新的(更复杂的)允许的项序列是通过产生式来定义的。该定义使用词项的名称和允许的项序列的名称,并形成一个新命名的允许的项序列。

5.2.3 每个产生式由下面几个部分组成,占一行或几行,次序是:

a) 新的允许的项序列的名称;

b) 字符

::=;

c) 一个或多个 5.3 中所定义的词项的替换序列,使用下面字符分隔
|。

5.2.4 一个词项序列若在一个或多个替换序列中出现,则它在对应的新的允许的词项序列中出现。在本文件中,新的允许的词项序列通过上述 5.2.3a) 中的名称来被引用。

注:若相同的词项序列出现在多个替换序列中,由此所产生的记法中的任何语义歧义性由相关文本解决。

5.3 替换项集

5.3.1 产生式[见 5.2.3c)]中的每个替换由一个名称列表来规定。每一个名称或者是一个词项名,或者是一个由某些其他产生式定义和命名的允许的词项序列的名称。

5.3.2 由每个替换项定义的允许的词项序列包括所有由以下操作获得的序列,取任何一个与第一个名称相关的序列(或词项),后连与任何一个与第二个名称相关的序列(或词项),再后连任何一个与第三个名称相关的序列(或词项)组合,以此类推,直到替换中最后一个名称(或词项)被包含在内。

5.4 非间隔指示符

如果产生式序列的某些项之间插入非间隔指示符“&”(AMPERSAND),那么它前面的词项和它后面的词项不应用空白隔开。

注:这个指示符仅用于描述 XML 值记法的产生式中。例如,它用来规定词项“<”之后紧接 XML 标记名的。

5.5 产生式的示例

5.5.1 产生式:

```
ExampleProduction::=
    bstring
    | hstring
    | "{"IdentifierList"}"
```

将名称“ExampleProduction”与下列词项序列相关联:

- a) 任何“bstring”(一个词项);或
- b) 任何“hstring”(一个词项);或
- c) 任何与“IdentifierList”相关联的词项序列,以“{”开始并以“}”结束。

注:“{”和“}”分别是含有单个字符‘{’和‘}’的词项名称(见 12.37)。

5.5.2 本例中,“IdentifierList”会由定义“ExampleProduction”的产生式之前或之后的另一个产生式定义。

5.6 样式

本文件中使用的每个产生式前面或后面都有一个空行。产生式中没有空行。产生式可在一行上或者分布在几行上。样式并没有区别性。

5.7 递归

本文件中的产生式经常是递归的。在这种情况下,只要有新的序列产生,产生式就要继续重复。

注:在很多情况下,这种重复应用导致一个允许的词项序列的无穷集合,集合中的某些或者所有序列本身可能包含无穷的词项数目。这种情况并不是一种错误。

5.8 词项允许序列的引用

本文件通过引用产生式中出现在“: =”之前的名称来引用允许的词项序列 (ASN.1 记法的一部分); 除非这个名称作为产生式的一部分出现, 否则, 该名称用 QUOTATION MARK (34) 字符 (") 括起来, 以便把该名称和自然语言文本区分开来。

5.9 词项的引用

本文件通过使用词项的名称来引用词项, 当名称在自然语言文本中出现, 并且可能与该文本混淆时, 那么, 把该名称用 QUOTATION MARK (34) 字符 (") 括起来。

5.10 缩写记法

为了使产生式更加准确和更可读, 下面的缩写记法用于本文件的允许的词项序列定义中, 且也用在 GB/T 16262.2—2025、GB/T 16262.3—2025、GB/T 16262.4—2025 中。

- a) “A”和“B”两个名称之后的星号 (*) 表示“empty”词项 (见 12.7), 或与“A”相关的允许的词项序列之一, 或与“A”相关的词项序列之一和与“B”相关的词项序列之一的替换序列, 两者都以与“A”相关一个开始和结束。因此:

$$C ::= A B *$$

相当于:

$$C ::= D | \text{empty}$$

$$D ::= A | A B D$$

“D”是不会有在产生式中其他地方出现的辅助名。

例如: “ $C ::= A B *$ ”是 C 的下列替换项的缩写记法:

empty

A

A B A

A B A B A

A B A B A B A

...

- b) 加号 (+) 与 a) 中的星号类似, 只是不包括“empty”词项。因此:

$$E ::= A B +$$

相当于:

$$E ::= A | A B E$$

例如: “ $E ::= A B +$ ”是 E 的下列替换项的缩写记法:

A

A B A

A B A B A

A B A B A B A

...

- c) 名称后的问号 (?) 表示“empty”词项 (见 12.7) 或与“A”相关的允许的词项序列, 因此:

$$F ::= A ?$$

相当于:

$F ::= \text{empty} \mid A$

注：这些缩写记法优先于产生式序列中并列的词项（见 5.2.2）。

5.11 值引用和值的类型

5.11.1 ASN.1 值赋值记法能把名称给与规定类型的值。只要需要引用那个值时就能采用这个名称。附录 C 描述和规定了允许用一个类型值的值引用名称来标识第二个（相似）类型的值的值映射机制。因此，只要要求引用第二个类型中的值就能使用第一个值的引用。

5.11.2 在 ASN.1 标准正文中，用标准英语文本来规定包含不止一个类型构造的合法性（或其他）。这些合法性规范通常要求两个或更多个类型“兼容”。例如，使用值引用时，要求用于定义值引用的类型与支配类型“兼容”。附录 C 采用值映射概念来为任意给出的 ASN.1 构造是否合法给出精确的描述。

6 类型扩展的 ASN.1 模块

当解码可扩展类型时，解码器可检测：

- a) 序列或集合类型中缺少预期的扩展附加；或
- b) 序列或集合类型（若有）中定义的那些扩展附加之外出现任意意外的扩展附加，或者选择类型中出现未知的替换项，或者枚举类型中出现未知的枚举，或可扩展其约束的类型出现意外的长度或值。

在形式术语中，可扩展类型“X”定义的抽象语法包含的不仅是类型“X”的值，还包括与“X”扩展关联的所有类型的值。因此，无论发现上面 a) 或 b) 的哪种情形，解码过程都不会报错。每种情形中采取的行为由 ASN.1 规定者确定。

注：所采取的行为经常会忽略出现的意外扩展附加，并且会为期望但是未出现扩展附加使用默认值或“丢失”指示器。

可扩展类型中的解码器检测到的意外扩展附加能被包含在该类型的后续编码中（以转发回发送者，或某些第三方），只要在后续转发中使用相同的传送语法。类型扩展 ASN.1 模块的辅助附录见附录 I。

7 编码规则的可扩展性要求

注：这些要求适用于标准化的编码规则。它们不适用于用 ECN 定义的编码规则[见 Rec.ITU-T X.692(2021) | ISO/IEC 8825-3:2021]。

7.1 所有 ASN.1 编码规则应允许对可扩展类型“X”的值的编码能被与“X”扩展关联的可扩展类型“Y”解码。而且，编码规则应允许用“Y”解码的值被重新编码（用“Y”）并可被与“Y”（进而与“X”）扩展关联的第三个可扩展类型“Z”解码。

注：类型“X”“Y”“Z”可能在扩展系列中以任何顺序出现。

如果可扩展类型“X”的一个值被编码，并且随后被传递（直接或通过用扩展有关的类型“Z”的传递应用）到另一个用与“X”扩展关联的可扩展类型“Y”解码的应用，然后使用类型“Y”的解码器获得由如下构成的抽象值；

- a) 扩展根类型的抽象值；
- b) 在“X”和“Y”中都出现的每个扩展附加的抽象值；
- c) 只有“X”中有而“Y”中没有的每个扩展附加（如果有的话）的无限制编码。

如果应用层要求，c) 中的编码应能包含在“Y”值后面的编码中，其中后面的编码应是“X”值的有效

编码。

指导示例:如果系统 A 使用带有可选整数类型扩展附加的序列类型或集合类型的可扩展根类型(类型“X”),而系统 B 使用与扩展有关的类型(类型“Y”),它有两个扩展附加,其中每个扩展都是可选整数类型,那么,B 传输的省略第一个扩展附加的整数值而包括第二个的“Y”类型的一个值,绝不能与其所知的 A 的只有首个扩展附加的“X”类型相混淆。而且,如果应用协议要求的话,A 应能重新编码带有作为第一个整数类型出现的值、随后跟有从 B 收到的第二个整数值的“X”类型的值。

7.2 所有的 ASN.1 编码规则应规定以如下方式编码和解码枚举类型和选择类型的值:如果传送的值是在由编码器和解码器共同保持的扩展附加集中,那么,该值被成功解码,否则,解码器将可能为该值的编码划界,并将其标识为(未知)扩展附加值。

7.3 所有的 ASN.1 编码规则应规定以如下方式编码和解码有可扩展约束的类型:如果传送的值是在由编码器和解码器共同保持的扩展附加集中,那么,它被成功地解码,否则,解码器将可能限定它的编码并把它标识为(未知)扩展附加值。

在所有的情况下,出现扩展附加不应影响当带扩展标志的类型套入某些其他类型内时识别后面材料的能力。

注 1: ASN.1 基本编码规则的所有变量和 ASN.1 的紧缩编码规则满足所有这些要求。用 ECN 定义的编码规则不必满足所有这些要求,但是也可能满足。

注 2: PER 和 BER 不标识扩展附加编码中的版本号。用 ECN 规定的编码可能提供也可能没提供这样的标识。

8 标记

8.1 标记是通过给出它的类别和类别中的号码来说明的(在本文件的文本中,或通过使用类型前缀),类别是下列之一:

- 通用,
- 应用,
- 专用,
- 上下文规定。

8.2 号码是一个非负整数,用十进制记法规定。

8.3 31.2 中规定了 ASN.1 用户指派的标记的限制。

注: 31.2 中包括不允许本记法的用户显式地规定他们的 ASN.1 规范中通用类别标记的限制。标记的用法之间与另外三个类别没有形式上的差别。当采用应用类别标记时,通常应用专用或上下文规定类别标记替代,作为用户选择和风格。出现三个类别主要是出于历史原因,但是 G.2.12 中以通常采用类别的方式给出了指南。

8.4 表 1 总结了在本文件中规定的通用类别里标记的分配。

8.5 有些编码规则要求标记有常规顺序。为了一致,8.6 中定义了标记的常规顺序。

8.6 标记的常规顺序以每个类型的最外层标记为基础并定义如下:

- a) 那些带通用类别标记的元素或替换项应首先出现,接着是那些带应用类别标记的,再是那些带上下文规定标记的,然后是那些带专用类别标记的;
- b) 在标记的每个类别内,元素或替换项应以其标记数字的递增顺序出现。

表 1 通用类别标记分配

通用类别 0	保留为编码规则使用
通用类别 1	布尔类型
通用类别 2	整数类型
通用类别 3	位串类型
通用类别 4	八位位组串类型
通用类别 5	空类型
通用类别 6	对象标识符类型
通用类别 7	对象描述符类型
通用类别 8	外部类型和单一实例类型
通用类别 9	实数类型
通用类别 10	枚举类型
通用类别 11	嵌入 pdv 类型
通用类别 12	UTF8 串类型
通用类别 13	相对对象标识符类型
通用类别 14	时间类别
通用类别 15	为本文件的将来版本保留
通用类别 16	序列和单一序列类型
通用类别 17	集合和单一集合类型
通用类别 18-22,25-30	字符串类型
通用类别 23-24	UTCTime 和 GeneralizedTime
通用类别 31-34	分别为 DATE, TIME-OF-DAY, DATE-TIME 和 DURATION
通用类别 35	OID 国际化资源标识符类型
通用类别 36	相对 OID 国际化资源标识符类型
通用类别 37-...	为本文件的附录保留

9 编码指令

9.1 使用类型前缀(见 31.3)或编码控制部分(见第 54 章)指派编码指令给类型。

9.2 类型前缀可包含编码引用。如果没有,则编码引用由模块的默认编码引用确定(见 13.5)。

9.3 编码控制部分总是包含编码引用。可能有多个编码控制部分,但每个编码控制部分应具有不同的编码引用。

9.4 编码指令由本文件规定的词项序列组成,并由编码引用确定(见附录 E)。

9.5 可将具有相同或不同编码引用的多个编码指令指派给一个类型(使用类型前缀和编码控制部分中的一个或两个)。使用给定的编码引用指派的编码指令与使用不同的编码引用指派的和使用任何类型前缀来执行置标记的编码指令是独立的。

9.6 为几个编码指令指派相同的编码引用(使用类型前缀和编码控制部分中的一个或两个)所造成的影响由编码引用(见附录 E)所确定的本文件规定,并在本章中未规定。

9.7 如果一条编码指令指派给“TypeAssignment”中的“Type”,它就会与该类型相关,并应用在任何使用“TypeAssignment”的“typereference”的地方。这包括通过导出和导入语句在其他模块中使用。

10 ASN.1 记法的使用

10.1 类型定义的 ASN.1 记法为“Type”(见 17.1)。

10.2 类型的值的 ASN.1 记法为“Value”(见 17.7)。

注：在不知道类型的有关知识时，通常不能解释值的记法。

10.3 将类型指派给类型引用名的 ASN.1 记法应为“TypeAssignment”(见 16.1)、“ValueSetTypeAssignment”(见 16.6)、“ParameterizedTypeAssignment”(见 GB/T 16262.4—2025 的 8.2)，或者“ParameterizedValueSetTypeAssignment”(见 GB/T 16262.4—2025 的 8.2)。

10.4 将值指派给值引用名的 ASN.1 记法应为“ValueAssignment”(见 16.2)或者“ParameterizedValueAssignment”(见 GB/T 16262.4—2025 的 8.2)。

10.5 记法“Assignment”的产生式替换项应只用在记法“ModuleDefinition”(13.1 注 2 中规定的除外)内。

11 ASN.1 字符集

11.1 除 11.2、11.3 和 11.4 的规定外，词项应由表 2 中列出的字符的序列组成。在表 2 中，字符由 GB/T 13000—2010 给出的名称标识。

表 2 ASN.1 字符

A~Z	拉丁大写字母 A 至拉丁大写字母 Z(LATIN CAPITAL LETTER A 到 LATIN CAPITAL LETTER Z)
a~z	拉丁小写字母 A 至拉丁小写字母 Z(LATIN SMALL LETTER A 到 LATIN SMALL LETTER Z)
0~9	数字 0 到数字 9(DIGIT ZERO 到 DIGIT 9)
!	感叹号(EXCLAMATION MARK)
"	双引号(QUOTVHQN MARK)
&	和(AMPERSAND)
'	撇号(APOSTROPHE)
(	左圆括号(LEFT PARENTHESE)
)	右圆括号(RIGHT PARENTHESE)
*	星号(ASTERISK)
,	逗号(COMMA)
—	负号(HYPHEN-MINUS)
.	句号(FULL STOP)
/	斜线(SOLIDUS)
:	冒号(COLON)
;	分号(SEMICOLON)
<	小于符号(LESS-THAN SIGN)
=	等于符号(EQUALS SIGN)
>	大于符号(GREATER-THAN SIGN)
@	商用 AT 符号(COMMERCIAL AT)
[	左方括号(LEFT SQUARE BRACKET)
]	右方括号(RIGHT SQUARE BRACKET)
^	向上箭号(CIRCUMFLEX BRACKET)
_	下横线(LOW LINE)
{	左花括号(LEFT CURLY BRACKET)
	竖线(VERTICAL LINE)
}	右花括号(RIGHT CURLY BRACKET)
-	不换行字符(NON-BREAKING HYPHEN)

注：在由国家标准机构制定的等效衍生标准中，可能会在以下词汇项中出现附加字符：

- typerreference(见 12.2)；
- identifier(见 12.3)；
- valuerreference(见 12.4)；
- modulereference(见 12.5)。

当附加字符用在一种大小写无区别的文种时，由以上某些词项的第一个字符的不同情况导致的语义区别将用别的办法来处理。这就允许有效的 ASN.1 规范以各种文种书写。

11.2 当用该记法来规定字符串类型的值时，在 ASN.1 记法中能够出现已定义字符集中的所有字符，括以双引号 QUOTATION MARK(34)字符(")(见 12.14)。

11.3 附加(任意)的图形符号可出现在“comment”词项中(见 12.6)。

11.4 当用该记法来规定 Unicode 标号的值时，Unicode 标号中允许的所有字符都能够出现在 ASN.1 记法中。

11.5 印刷的字体、大小、色彩、亮度或其他显示特性无语义区别。

11.6 大写字母和小写字母应视为不同。

11.7 ASN.1 定义在词项之间也可包含空白字符(见 12.1.6)。

11.8 在所有名称(包括保留字)中，NON-BREAKING HYPHEN 和 HYPHEN-MINUS 宜被视为相同的。

注：像 My-Type 这样的名称无论是包含 HYPHEN-MINUS 还是 NON-BREAKING HYPHEN，都是指相同的名称。

12 ASN.1 词项

12.1 一般原则

12.1.1 下列各条规定词项中的字符。在每种情况下，都给出词项的名称，以及形成词项的字符序列的定义。ASN.1 记法总结见附录 L。

12.1.2 第 12 章的各条中规定的每个词项(除多行“comment”“bstring”“xmlbstring”“hstring”“xmlhstring”“cstring”“xmlestring”和“simplestring”外)不应包含空白(见 12.6、12.10、12.11、12.12、12.13、12.14、12.15 和 12.16)。一个单行“comment”不应包含“new-line”字符。“non-integerUnicodeLabel”词项可包含 NON-BREAKING-SPACE 字符。

12.1.3 行的长度不受限制。

12.1.4 词项可用一或多次空白(见 12.1.6)或注解(见 12.6)隔开，除非使用非间隔指示符“&.”(见 5.4)。在“XMLTypeValue”产生式(见 16.2)中，词项之间可出现空白，但不应出现“comment”词项。

注：这将避免“xmlestring”词项内出现相邻连字符或星号和斜线产生的歧义。当“XMLTypeValue”产生式内出现这样的字符时，它们从不指明“comment”词项的开始点。

12.1.5 若后续词项的起始字符(或多个字符)是前项词项中字符末尾包括的允许字符(或多个字符)时，词项与后续词项间要用一个或多个空白或注解实例隔开。

12.1.6 本文件使用术语“新行”和“空白”。在机器可读的规范中的表示(行尾)空白和新行中，任意一个或多个下面的字符可用在任意组合中(对每个字符，给出 Unicode 编码标准规定的字符名称和字符代码)：

对于空白：

HORIZONTAL TABULATION(9)

LINE FEED(10)

VERTICAL TABULATION(11)
 FORM FEED(12)
 CARRIAGE RETURN(13)
 SPACE(32)
 NO-BREAK SPACE({0,0,0,160})

对于新行:

LINE FEED(10)
 VERTICAL TABULATION(11)
 FORM FEED(12)
 CARRIAGE RETURN(13)

注:任何是有效新行的字符或字符序列也是有效的空白。

12.2 类型引用

词项名——typereference

12.2.1 一个“typereference”应由任意个(一个或多个)字母、数字和连字符组成,以大写字母开头。不能用连字符结尾。一个连字符不能紧接另一个连字符。

注:有关连字符的规则是为了避免与(可能后随的)注解歧义。

12.2.2 “typereference”不应是 12.38 中列出的保留字符序列之一。

12.3 标识符

词项名——identifier

一个“identifier”应由任意个(一个或多个)字母、数字和连字符组成,以小写字母开头。不能用连字符结尾。一个连字符不能紧接另一个连字符。

注:有关连字符的规则是为了避免与(可能后随的)注解歧义。

12.4 值引用

词项名——valuereference

一个“valuereference”应由 12.3 中为一个“identifier”规定的字符序列组成。分析使用这一记法的实例时,一个“valuereference”由其出现的上下文来与一个“identifier”区别。

12.5 模块引用

词项名——modulereference

“modulereference”应由 12.2 中规定为“typereference”的字符序列组成。分析使用这一记法的实例时,“modulereference”由其出现的上下文来与“typereference”区别。

12.6 注解

词项名——comment

12.6.1 在 ASN.1 记法的定义中不引用“comment”。但它可在任意时候出现在别的词项之间,而且没有语法意义。

注:虽然如此,在使用 ASN.1 的本文件上下文中,ASN.1 注解可含有与应用语义有关的标准文本或对语法的约束。

12.6.2 词项“comment”能有两种形式:

a) 如 12.6.3 定义的以“--”开始的一行注解;

b) 如 12.6.4 定义的以“/*”开始的多行注解。

12.6.3 只要“comment”以一对相连的连字符开始,它就应以下一对相连的连字符或行尾为结束,无论哪一个在前面。注解除了开始的一对连字符及结尾的一对连字符(若有的话)外,不能有一对相连的连字符。如果以“--”开始的注解包括相连的字符“/*”或“*/”,那么它们没有特别的意义并且认为是注解的一部分。注解可包括没有在 11.1 中规定的字符集中的图形符号(见 11.3)。

12.6.4 只要“comment”以“/*”开始,它就应以对应的“*/”结束,不管这个“*/”是否在同一行。如果在“*/”之前发现另一个“/*”,那么当为每个“/*”发现配对的“*/”时注解应终止。如果以“/*”开始的注解包含两个相连的连字符“--”,这些连字符没有特别的意义并且认为是注解的一部分。注解可包括在 11.1 中规定的字符集中没有的图形符号(见 11.3)。

注:允许用户注解已经含有注解的 ASN.1 模块部分(不管它们是以“--”或“/*”开始),通过简单地在被注解的部分开头插入“/*”和其结尾插入“*/”,只要被注解出的部分内没有含有“/*”或“*/”的字符串值。

12.7 空词项

词项名——empty

“empty”项不包括字符。当规定了产生式序列的替换项集时,它用于第 5 章的记法中以指明所有替换项可能缺失。

12.8 数

词项名——number

一个“number”应由一个或多个数字组成。除非“number”是单个数字,否则其第一位数字不应是 0。

注:通过将它解释成十进制记法,“number”词项总是映射成整数值。

12.9 实数

词项名——realnumber

一个“realnumber”应由一串一位或多位数字、而且选择十进制小数点(.)的整数部分组成。十进制小数点能够选择地后接一位或多位数字的分数部分。整数部分、十进制小数点或分数部分(不管哪个先出现)能够有选择地后接 e 或 E 和是一位或多位数字的选择签名指数。“realnumber”的起始数字不应是 0,除非它是唯一的数字,或者后面紧跟一个小数点,小数点后面跟着至少有一位非 0 的小数部分。

一个“number”也是“realnumber”的一个有效实例。在分析使用这种记法的实例时,“realnumber”由其出现的上下文来与“number”区别。

12.10 二进制数串

词项名——bstring

一个“bstring”应由任意个(可能 0 个)字符 0、1 组成,可能混杂着空白、前置 APOSTROPHE(39) 字符('),后随一对字符'B。

例如:'011011100'B

二进制数串词项中出现空白并无语义区别。

12.11 XML 二进制数串项

项名——xmlbstring

一个“xmlbstring”应由任意个(可能 0 个)0、1 或者空白组成。二进制数串项中出现空白并无语义区别。

例如:01101100

这一字符序列也是有效的“xmlhstring”和“xmlestring”实例。分析使用这一记法的实例时,“xmlbstring”由其出现的上下文来与“xmlhstring”或“xmlestring”区别。

12.12 十六进制数串

词项名——hstring

12.12.1 一个“hstring”应由任意个(可能 0 个)字符:ABCDEF0123456789 组成,可能混杂着空白,前置 APOSTROPHE(39)字符“'”,后随一对字符'H。

例如:'AB0196'H

十六进制数串词项中出现空白并无语义区别。

12.12.2 每个字符用十六进制表示法来表示 4 位的值。

12.13 XML 十六进制数串项

词项名——xmlhstring

12.13.1 一个“xmlhstring”应由任意个(可能 0 个)0123456789ABCDEFabcdef 或者空白组成。十六进制数串项中出现空白并无语义区别。

例如:Ab0196

12.13.2 每个字符用十六进制表示法来表示 4 位的值。

12.13.3 所有“xmlhstring”实例也是有效的“xmlestring”实例,有些“xmlhstring”实例也是有效的“xmlbstring”实例。分析使用这一记法的实例时,“xmlhstring”由其出现的上下文来与“xmlbstring”或“xmlestring”区别。

12.14 字符串

词项名——cstring

12.14.1 一个“cstring”应由任意个(可能 0 个)图形符号和来自字符串类型引用的字符集中的间隔字符组成,其前、后置一个 QUOTATION MARK(34)字符(“)。如果字符集中含有 QUOTATION MARK(34)字符,该字符(如果出现在用“cstring”表示的字符串中)应用在不插入间隔字符的同一行中的一对 QUOTATION MARK(34)字符在“cstring”中表示。“cstring”可能跨过几行文字,在这种情况下,被描述的字符串不应在“cstring”中紧接行尾的前后位置上包含有间隔字符。紧接“cstring”中行尾的前或后出现空白并无语义区别。

注 1:“cstring”只能用来无歧义地(在打印页上)表示字符串,该被描述的字符串中的每个字符要么指派一个图形符号,要么是一个间隔字符。当要在打印表示中指明包含控制字符的字符串时,可用替换项 ASN.1 语法(见第 39 章)。

注 2:“cstring”表示的字符串应由与图形符号相关的字符和间隔字符组成。紧接“cstring”中任意行尾前或后的间隔字符不是所表示的字符串的一部分(它们被忽略掉),当“cstring”中含有间隔字符串时,或字符集中的图形符号在打印表示中不是无歧义时,“cstring”指示的字符串可能在那个打印表示中有歧义。

例 1:“屎 屍 市 弑”

例 2:“cstring”

"ABCDE FGH

IJK" "XYZ"

可用来表示类型 IA5String 的字符串值。所表示的值由下列字符组成：

ABCDE FGHIJK"XYZ

如果打印规范中使用成比例的间隔字体(如上所用的),或者如果字符汇包含多个不同宽度的间隔字符,则 E 和 F 之间计划的空格的准确数目在打印表示中可能有歧义。

12.14.2 当字符是组合字符时(见附录 H),它应在“cstring”的打印表示中指明为一个单独的字符。它不应用组成它的字符套印(这样保证了打印版本中无歧义地定义了串值中组成字符的顺序)。

例如:小写的“e”和重音组成字符在 GB/T 13000—2010 中是两个字符,因此,在对应的“cstring”中打印为两个字符而不是单个字符 é。

12.15 XML 字符串项

词项名——xmlestring

12.15.1 一个“xmlestring”应由任意个(可能 0 个)下面的 GB/T 13000—2010 字符组成:

- a) HORIZONTAL TABULATION(9);
- b) LINE FEED(10);
- c) CARRIAGE RETURN(13);
- d) 其 GB/T 13000—2010 字符代码在 32(十六进制 20)~55 295(十六进制 D7FF)(含)范围内的任意字符;
- e) 其 GB/T 13000—2010 字符代码在 57 344(十六进制 E000)~65 533(十六进制 FFFD)(含)范围内的任意字符;
- f) 其 GB/T 13000—2010 字符代码在 65 536(十六进制 10000)至 1 114 111(十六进制 10FFFF)(含)范围内的任意字符。

要求强加的附加限制是采用的实例中“xmlestring”应只包含支配字符串类型允许的字符。

12.15.2 字符“&.”(AMPERSAND)、“<”(LESS-THAN SIGN)或“>”(GREATER-THAN SIGN)应只作为 12.15.4 或 12.15.5 规定的字符序列之一的一部分出现。

12.15.3 一个“xmlestring”用来表示受限制字符串(见 41.9)的值,并且能直接或通过用下面规定的转义序列表示 GB/T 13000—2010 字符的所有组合。

注 1: 一个“xmlestring”不能用来表示不在 GB/T 13000—2010 中出现的字符,例如能在 GeneralString 中出现的某些控制字符,也不能表示用代码超过 10FFFF 十六进制的 GB/T 13000—2010 字符定义的字符。

注 2: 当用符合 XML 处理器处理时,不能区分字符 LINE FEED(10)和 CARRIAGE RETURN(13)及一对 CARRIAGE RETURN+LINE FEED。

12.15.4 如果字符“&.”(AMPERSAND)、“<”(LESS-THAN SIGN)或“>”(GREATER-THAN SIGN)在用“xmlestring”(见 41.9)表示的抽象字符串值中出现,在“xmlestring”中他们应以下列方式之一表示:

- a) 12.15.8 中规定的转义序列;或
- b) 各自的转义序列“amp;”“<”“>”这些转义序列不应包含空白(见 12.1.6)。

12.15.5 如果一个具有表 3 中第一列的 GB/T 13000—2010 的字符编码的字符在用“xmlestring”(见 41.9)表示的抽象字符串值中出现,它应用表 3 中第二列的字符序列表示。这些字符序列不应包含空白(见 12.1.6)。

注: 这里不包括带十进制字符代码 9、10 和 13 及这些字符序列中所有小写的字母。

表 3 xmlcstring 中控制字符的转义序列

GB/T 13000—2010 字符编码	xmlcstring 表示法	GB/T 13000—2010 字符编码	xmlcstring 表示法
0(十六进制 0)	<nul/>	17(十六进制 11)	<dc1/>
1(十六进制 1)	<soh/>	18(十六进制 12)	<dc2/>
2(十六进制 2)	<stx/>	19(十六进制 13)	<dc3/>
3(十六进制 3)	<etx/>	20(十六进制 14)	<dc4/>
4(十六进制 4)	<eot/>	21(十六进制 15)	<nak/>
5(十六进制 5)	<enq/>	22(十六进制 16)	<syn/>
5(十六进制 5)	<ack/>	23(十六进制 17)	<etb/>
7(十六进制 7)	<bel/>	24(十六进制 18)	<can/>
8(十六进制 8)	<bs/>	25(十六进制 19)	
11(十六进制 B)	<vt/>	26(十六进制 1A)	<sub/>
12(十六进制 C)	<ff/>	27(十六进制 1B)	<esc/>
14(十六进制 E)	<so/>	28(十六进制 1C)	<is4/>
15(十六进制 F)	<si/>	29(十六进制 1D)	<is3/>
16(十六进制 10)	<dle/>	30(十六进制 1E)	<is2/>
		31(十六进制 1F)	<is1/>

12.15.6 当“xmlcstring”用在形成部分 XER 编码(见 ISO/IEC 8825-4:2021 的“XMLTypedValue”中(见 16.2)时,它可能包含相连的 HYPHEN-MINUS(45)字符。当用在 ASN.1 模块中的 XML 值记法实例内时,它不应包含两个相连的 HYPHEN-MINUS 字符。如果这一字符序列在用 ASN.1 模块中的“xmlcstring”表示的抽象字符串值中出现,那么,12.15.8 中规定的转义序列应该表示至少一个相连的 HYPHEN-MINUS 字符。

12.15.7 当“xmlcstring”用在形成部分 XER 编码(见 ISO/IEC 8825-4:2021 的“XMLTypedValue”中时,它可能包含以任意顺序相连的 ASTERISK(42)和 SOLIDUS(47)字符。当用在 ASN.1 模式中的 XML 值记法实例内时,它不应包含(以任意顺序)相连的 ASTERISK 和 SOLIDUS 字符。如果这一字符序列在用“xmlcstring”表示的抽象字符串值中出现,那么,12.15.8 中规定的转义序列应表示至少一个相连的 ASTERISK 和 SOLIDUS 字符。

12.15.8 能直接在“xmlcstring”中出现的任何字符也能在“xmlcstring”中用形式“&#n;”(其中 n 是十进制记法中的 GB/T 13000—2010 字符代码)或形式“&#xn;”(其中 n 是十六进制记法中的 GB/T 13000—2010 字符代码)的转义序列表示。这些转义序列不应包含空白(见 12.1.6)。

注 1: 在“n”的十进制和十六进制值中允许以 0 开始,而且“A”-“F”的大小写都能用在十六进制值中。

注 2: 如果转义序列“&#n;”和“&#xn;”用于不在基本多文种平面(BMP)中的 GB/T 13000—2010 字符,“n”值将比 65 535(十六进制 FFFF)大。

示例——“xmlcstring”:

AECDéFGHîJK&XYZ

能用来表示类型 UTF8String 的字符串值。表示的值由下面字符组成:

ABCDé FGHjK&XYZ

如果在规范中采用成比例的间隔字体(例如上面),其中 é 和 F 之间的精确间隔字符在印刷媒介上可能有歧义。

12.16 简单字符串词项

词项名——simplestring

一个“simplestring”应由一个或多个对应字符码在 32~126 之间的 GB/T 13000—2010 字符组成,其前、后各置一个 QUOTATION MARK(34)字符(“)。它不应包含 QUOTATION MARK(34)字符(“)。“simplestring”可跨越多个文本行,在这种情况下,任何表示行尾的字符都应被视为空格字符。在分析使用这种记法的实例时,“simplestring”由其出现的上下文来与“cstring”区别。

注:“simplestring”词项仅用于时间类型的子类型记法。

12.17 时间值字符串

词项名——tstring

一个“tstring”应由一个或多个字符组成:

0 1 2 3 4 5 6 7 8 9 + - : . / C D H M R P S T W Y Z

其前、后各置一个 QUOTATION MARK(34)字符(“)。

注:“tstring”词项仅在时间类型的值记法中使用。

12.18 XML 时间值字符串词项

词项名——xmltstring

一个“xmltstring”应由一个或多个字符组成:

0 1 2 3 4 5 6 7 8 9 + - : . / C D H M R P S T W Y Z

注:“xmltstring”词项仅在时间类型的 XML 值记法中使用。

12.19 属性和名称设置词项

词项名——psname

一个“psname”应由任意数目(一个或多个)的字母、数字和连字符组成。首字符应是大写字母。最后一个字符不应是连字符。连字符后不应立即跟着另一个连字符。

注:“psname”词项仅用于时间类型的子类型记法中的“simplestring”内容。

12.20 赋值词项

词项名——": :="

该词项应由字符序列": :="组成。

注:该序列不包含任何空白字符(见 12.1.2)。

12.21 范围分隔符

词项名——".."

该词项应由字符序列..组成。

注:该序列不包含任何空白字符(见 12.1.2)。

12.22 省略号

词项名——"..."

该词项应由字符序列...组成。

注:该序列不包含任何空白字符(见 12.1.2)。

12.23 左版本括号

词项名——"`[`"

该词项应由字符序列`[`组成。

注：该序列不包含任何空白字符(见 12.1.2)。

12.24 右版本括号

词项名——"`]`"

该词项应由字符序列`]`组成。

注：该序列不包含任何空白字符(见 12.1.2)。

12.25 编码引用

词项名——`encodingreference`

一个“`encodingreference`”应由 12.2 中的“`typereference`”规定的字符序列组成,但不应包括小写字母。

当前定义的编码引用符合附录 E 的要求,它规定了相应编码指令的语法和语义。“`encodingreference`”应仅包括本文件或本文件未来版本中附录 E 中列出的序列。

12.26 整数值 Unicode 标号

词项名——`integerUnicodeLabel`

该词项应由一个任意长的范围从 0(数字 0)到 9(数字 9)的 GB/T 13000—2010 字符序列组成,用来标识国际对象标识符树的一个弧。除非它只有一个字符,并且国际对象标识符树的相关弧的主整数值为 0,否则它不应以 0(数字 0)字符开始。

12.27 非整数值 Unicode 标号

词项名——`non-integerUnicodeLabel`

该词项应由满足 Rec.ITU-T X.660(2011)| ISO/IEC 9834-1:2012 的 7.5 中规定约束的任意长的 GB/T 13000—2010 字符序列组成,并标识国际对象标识符树的一个弧。为了词法分析目的,它不应仅由能够使其被标识为“`integer-UnicodeLabel`”的字符组成。

12.28 XML 结尾标记开始项

词项名——"`</`"

该词项应由字符序列`</`组成。

注：该序列不包含任何空白字符(见 12.1.2)。

12.29 XML 单个标记结尾项

词项名——"`/>`"

该词项应由字符序列`/>`组成。

注：该序列不包含任何空白字符(见 12.1.2)。

12.30 XML 布尔真项

词项名——"`true`"

12.30.1 该词项应由字符序列 `true` 组成。

12.30.2 分析使用这一记法的实例时,“true”由其出现的上下文来与“valuereference”或“identifier”或一个 XML 布尔值“extended-true”的实例区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.31 XML 布尔扩展真项

词项名——extended-true

12.31.1 该词项应由字符序列 true 组成,或者是单个字符 1(数字 1)组成。

12.31.2 分析使用这一记法的实例时,“extended-true”由其出现的上下文来与“valuereference”或“identifier”或一个 XML 布尔值“true”的实例区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.32 XML 布尔假项

词项名——“false”

12.32.1 该词项应由字符序列 false 组成。

12.32.2 分析使用这一记法的实例时,“false”由其出现的上下文来与“valuereference”或“identifier”或者一个 XML 布尔值“extended-false”的实例区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.33 XML 布尔扩展假项

词项名——extended-false

12.33.1 该词项应由字符序列 false 组成,或者是单个字符 0(数字 0)组成。

12.33.2 分析使用这一记法的实例时,“extended-false”由其出现的上下文与“valuereference”或“identifier”或一个 XML 布尔值“false”的实例区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.34 XML 实非数字项

词项名——“NaN”

12.34.1 该词项应由字符序列“NaN”组成。

12.34.2 分析使用这一记法的实例时,“NaN”由其出现的上下文来与任何其他以大写字母开头的词项区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.35 XML 实无穷项

词项名——“INF”

12.35.1 该词项应由字符序列“INF”组成。

12.35.2 分析使用这一记法的实例时,“INF”由其出现的上下文来与任何其他以大写字母开头的词项区别。

注:该序列不包含任何空白字符(见 12.1.2)。

12.36 ASN.1 类型的 XML 标记名称

词项名——xmlasn1typename

12.36.1 当 ASN.1 固有类型将被用作 XML 标记名称时,本文件采用项“xmlasn1typename”。

12.36.2 表 4 为 17.2 中列出的每个 ASN.1 固有类型列出形成“xmlasn1typename”的字符序列。ASN.1 固有类型通过它的产生式名称在表 4 的第一列中标识。应用于“xmlasn1-typename”的字符序列在表 4 的第二列中标识,这些字符序列的前或后没有空白。

12.36.3 用于各个“UsefulType”的“xmlasn1typename”(见 45.1)应是其定义中所用到的“typereference”。

12.36.4 “ObjectClassFieldType”和“InstanceOfType”的“xmlasn1typename”项中的字符序列在 GB/T 16262.2—2025 的 14.1 和附录 C 中规定。

12.36.5 如果 ASN.1 固有类型是“PrefixedType”,那么,确定“xmlasn1typename”的类型应该是“PrefixedType”中的“Type”(见 31.1.5)。如果这本身是“PrefixedType”,那么,应该递归地用本条。

注: 26.10 中规定了用于“SelectionType”和“ConstrainedType”的“Type”

表 4 xmlasn1typename 的字符

ASN.1 类型产生式名称	xmlasn1typename 中的字符
BitStringType	BIT_STRING
BooleanType	BOOLEAN
ChoiceType	CHOICE
DateType	DATE
DateTimeType	DATE_TIME
DurationType	DURATION
EmbeddedPDVType	SEQUENCE
EnumeratedType	ENUMERATED
ExternalType	SEQUENCE
InstanceOfType	SEQUENCE
IntegerType	INTEGER
IRIType	OID_IRI
NullType	NULL
ObjectClassFieldType	见 GB/T 16262.2—2025 的 14.10 和 14.11
ObjectIdentifierType	OBJECT_IDENTIFIER
OctetStringType	OCTET_STRING
PrefixedType	见 12.36.5
RealType	REAL
RelativeIRIType	RELATIVE_OID_IRI
RelativeOIDType	RELATIVE_STRING
RestrictedCharacterStringType	类型名称(如:IA5String)
SequenceType	SEQUENCE
SequenceOfType	SEQUENCE_OF
SetType	SET
SetOfType	SET_OF
TimeType	TIME
TimeOfDayType	TIME_OF_DAY
UnrestrictedCharacterStringType	SEQUENCE

12.37 单个字符词项

词项名——

“{”“}”“<”“>”“,”“.”“/”“(”“)”“[”“]”“-”(HYHEN-MINUS)“:”“=”“”(QUOTATION MARK)“'”(APOSTROPHE)“;”“@”“|”“!”“~”

带任意上面所列名称的词项应由无引号的单个字符组成。

12.38 保留字

保留字名——

ABSENT	ENCODED	INTERSECTION	SEQUENCE
ABSTRACT-SYNTAX	ENCODING-CONTROL	ISO646String	SET
ALL	END	MAX	SETTINGS
APPLICATION	ENUMERATED	MIN	SIZE
AUTOMATIC	EXCEPT	MINUS-INFINITY	STRING
BEGIN	EXPLICIT	NOT-A-NUMBER	SYNTAX
BIT	EXPORTS	NULL	T61String
BMPString	EXTENSIBILITY	NumericString	TAGS
BOOLEAN	EXTERNAL	OBJECT	TeletexString
BY	FALSE	ObjectDescriptor	TIME
CHARACTER	FROM	OCTET	TIME-OF-DAY
CHOICE	GeneralizedTime	OF	TRUE
CLASS	GeneralString	OID-IRI	TYPE-IDENTIFIER
COMPONENT	GraphicString	OPTIONAL	UNION
COMPONENTS	IA5String	PATTERN	UNIQUE
CONSTRAINED	IDENTIFIER	PDV	UNIVERSAL
CONTAINING	IMPLICIT	PLUS-INFINITY	UniversalString
DATE	IMPLIED	PRESENT	UTCTime
DATE-TIME	IMPORTS	PrintableString	UTF8String
DEFAULT	INCLUDES	PRIVATE	VideotexString
DEFINITIONS	INSTANCE	REAL	VisibleString
DURATION	INSTRUCTIONS	RELATIVE-OID	WITH
EMBEDDED	INTEGER	RELATIVE-OID-IRI	

带上面名称的词项应由名称中的字符序列组成,且是保留的字符序列。

注 1: 这些序列中没有空白。

注 2: 本文件中不用关键字: CLASS、CONSTRAINED、CONTAINING、ENCODED、INSTANCE、SYNTAX 和 UNIQUE,它们在 GB/T 16262.2—2025、GB/T 16262.3—2025、GB/T 16262.4—2025 中使用。

13 模块定义

13.1 “ModuleDefinition”由下面的产生式规定:

ModuleDefinition::=

ModuleIdentifier

DEFINITIONS

EncodingReferenceDefault

TagDefault

ExtensionDefault

"::="

BEGIN

ModuleBody

EncodingControlSections

END

ModuleIdentifier::=

modulereference

DefinitiveIdentification

DefinitiveIdentification::=

DefinitiveOID

| DefinitiveOIDandIRI

| empty

DefinitiveOID::=

"{ "DefinitiveObjIdComponentList" }

DefinitiveOIDandIRI::=

DefinitiveOID

IRIValue

DefinitiveObjIdComponentList::=

DefinitiveObjIdComponent

| DefinitiveObjIdComponent DefinitiveObjIdComponentList

DefinitiveObjIdComponent::=

NameForm

| DefinitiveNumberForm

| DefinitiveNameAndNumberForm

DefinitiveNumberForm::=number

DefinitiveNameAndNumberForm::=identifier("DefinitiveNumberForm")

EncodingReferenceDefault::=

encodingreference INSTRUCTIONS

| empty

TagDefault::=

EXPLICIT TAGS

| IMPLICIT TAGS

| AUTOMATIC TAGS

| empty

ExtensionDefault::=

EXTENSIBILITY IMPLIED

| empty

ModuleBody::=

Exports Imports AssignmentList

```

|         empty
Exports::=
    EXPORTS SymbolsExported";"
|         EXPORTS ALL";"
|         empty
SymbolsExported::=
    SymbolList
|         empty
Imports::=
    IMPORTS SymbolsImported";"
|         empty
SymbolsImported::=
    SymbolsFromModuleList
|         empty
SymbolsFromModuleList::=
    SymbolsFromModule
|         SymbolsFromModuleList SymbolsFromModule
SymbolsFromModule::=
    SymbolList FROM GlobalModuleReference SelectionOption
SelectionOption::=
    empty
|         WITH"SUCCESSORS"
|         WITH"DESCENDANTS"
GlobalModuleReference::=
    modulereference AssignedIdentifier
AssignedIdentifier::=
    ObjectIdentifierValue
|         DefinedValue
|         empty
SymbolList::=
    Symbol
|         SymbolList", "Symbol
Symbol::=
    Reference
|         ParameterizedReference
Reference::=
    typereference
|         valuereference
|         objectclassreference
|         objectreference
|         objectsetreference
AssignmentList::=
    Assignment

```



```

|      AssignmentList Assignment
Assignment ::=
|      TypeAssignment
|      ValueAssignment
|      XMLValueAssignment
|      ValueSetTypeAssignment
|      ObjectClassAssignment
|      ObjectAssignment
|      ObjectSetAssignment
|      ParameterizedAssignment

```

注 1：在 GB/T 16262.4—2025 中规定了“Exports”和“Imports”列表里的“ParameterizedReference”的用法。

注 2：例如（及对于带通用类别标记类型的本文件中的定义），“ModuleBody”可用在“ModuleDefinition”的外部。

注 3：第 16 章中规定了“TypeAssignment”“ValueAssignment”“XMLValueAssignment”和“ValueSetTypeAssignment”产生式。

注 4：为模块定义的“TagDefault”值只影响那些在模块中显式已定义类型，它不影响引入类型的解释。

注 5：字符分号不在赋值列表规范或它的任何下一级产生式中出现，并保留为 ASN.1 工具开发者使用。

13.2 如果“TagDefault”是“empty”，则将其取为“EXPLICIT TAGS”。

注：31.2 中给出了 EXPLICIT TAGS、IMPLICIT TAGS 和 AUTOMATIC TAGS 的意义。

13.3 当选择“TagDefault”的替换项 AUTOMATIC TAGS 时，就是说选择了为模块自动加标记，否则就说没有选择。自动加标记是（带附加条件）应用于模块定义内的“ComponentTypeLists”和“AlternativeTypeLists”产生式上的语法转换。25.8～25.10、27.3 和 29.2～29.5 形式上分别规定了有关序列类型、集合类型和选择类型记法的这一转换。

13.4 EXTENSIBILITY IMPLIED 选项相当于允许的模块中每个类型定义中的文本插入扩展标志（“...”）。隐式扩展标志的位置是允许有显式规定扩展标志的类型中的最后位置。无 EXTENSIBILITY IMPLIED 意味着只为扩展标志显式存在的模块内的那些类型提供可扩展性。

注：EXTENSIBILITY IMPLIED 只影响类型。它不影响对象集和子类型约束。

13.5 “EncodingReferenceDefault”规定“encodingreference”是模块的默认编码引用。如果“EncodingReferenceDefault”为“empty”，则该模块的默认编码引用为 TAG。

注：附录 E 包含了允许的编码引用列表，以及本文件规定了相应编码指令的形式和含义。

13.6 在“ModuleIdentifier”产生式中出现的“modulereference”称为模块名称。

注：用赋予相同“modulereference”的“ModuleBody”的几个事件定义单个 ASN.1 模块的可能性在早期的规范中是（可论证）允许的。本文件中不允许这样。

13.7 模块名在模块定义的应用范围内应只能使用一次（13.10 规定的除外）。

13.8 如果“DefinitiveIdentifier”不是空的，指示对象标识符和任何可选的“IRIValue”值无歧义和唯一地标识所定义模块的标识 OID 树的同一节点。未定义值可用于定义对象标识符值。“IRIValue”产生式在 34.3 中规范。

注 1：强烈推荐至少为该模块指派一个对象标识符值（最好是一个对象标识符值加上一个 OID 国际化资源标识符值），以便能够无歧义地引用该模块。

注 2：本文件不阐述模块变化需要新“DefinitiveIdentifier”的问题。

13.9 如果“AssignedIdentifier”不是空，则“ObjectIdentifierValue”和“DefinedValue”替换项无歧义和唯一地标识引入引用名的模块。当使用“AssignedIdentifier”的“DefinedValue”替换项时，它应是类型对象标识符值。“AssignedIdentifier”内文本出现的每个“valuereference”应满足下列规则之一：

- a) 它在所定义模块的“AssignmentList”中定义，且在赋值描述右边文本出现的所有“valuereference”也满足本规则（“a”规则）或下一个规则（“b”规则）；

- b) 在“AssignmentIdentifier”不能文本包含任何“valuereference”的“SymbolsFromModule”中,它表现为“Symbol”。

注 1: 推荐赋予一个对象标识符以便其他地方能无歧义地引用模块。

注 2: 该语法不提供将 OID 国际化资源引用(如果赋值)包含到被引用的模块中,但推荐将其包含在 ASN.1 注解中。

13.10 “SymbolsFromModule”中的“GlobalModuleReference”应在另一个模块的“ModuleDefinition”中出现,除非它包含非空的“DefinitiveIdentifier”,“modulereference”可能在两种情况下有所不同。

注: 当符号是从两个名字相同(不考虑 13.7 已命名模块)的模块中引入时,应只能用不同于用于其他模块中的“modulereference”。采用替换项的不同名字使这些名字可用于模块体中(见 13.16)。

13.11 当“modulereference”和非空“AssignedIdentifier”都用来引用一个模块时,后者应认为是确定性的。

13.12 当引用的模块有非空的“DefinitiveIdentifier”时,引用那个模块的“GlobalModuleReference”不应有空的“AssignedIdentifier”。

13.13 当选择“SymbolsExported”替换“Exports”时:

- a) “SymbolsExported”中的每个“Symbol”应满足一个且只能是一个下面的条件:
 - i) 只在所构建的模块中定义;或
 - ii) 在“Imports”的“SymbolsImported”替换项中,只准确地出现一次;
- b) 每个适合从模块外面引用的“Symbol”应包含在“SymbolsExported”中且只有这些“Symbol”能从模块外部引用(根据 13.14 的规定放宽限制);和
- c) 如果没有这类“Symbol”,那么应选择空替换“SymbolsExported”(不是替换“Exports”)。

13.14 当选择“empty”或者 EXPORTS ALL 替换“Exports”时,模块中定义或模块引入的每个“Symbol”可从其他受到 13.13a)中规定限制的模块中引用。

注: 含有“empty”替换“Exports”是为了向后兼容。

13.15 如果定义它们的 typereference 是输出的或作为输出类型内的组件(或子组件)出现,则“NamedNumberList”“Enumeration”或“NamedBitList”中出现的标识符是隐式输出。

13.16 当选择“SymbolsImported”替换“Imports”时:

- a) 在“SymbolFromModule”中的每个“Symbol”应在“SymbolFromModule”中的“GlobalModuleReference”指明模块的模块体中定义,或者在“Imports”章出现。如果在那一章中只有一次“Symbol”事件,则只允许引入被引用模块的“Imports”章中出现的“Symbol”,而且“Symbol”不能在引用模块中定义。

注 1: 不禁止在两个来自被引入到另一个模块的不同模块中定义相同的符号名。然而,如果相同的“Symbol”名字在模块 A 的“Imports”章中出现不止一次,那个“Symbol”名就不能从 A 输出以引入到另一个模块 B。

- b) 如果在“SymbolsFromModule”中的“GlobalModuleReference”指明的模块的定义中选择了“SymbolsExported”替换“Exports”,则“Symbol”应出现在其“SymbolsExported”中。
- c) 只有那些出现在“SymbolsFromModule”的“SymbolList”之中的“Symbol”可在任何有由那个“SymbolsFromModule”的“GlobalModuleReference”指明的“modulereference”的“External<X>Reference”(这里<X>是“Value”“Type”“Object”、“Objectclass”或“Objectset”)中作为符号出现。
- d) 如果没有这样的“Symbol”,那么应选择“empty”替换“SymbolsImported”。

注 2: c)和 d)的作用是语句 IMPORTS;意味着模块不能包含“External<X>Reference”。

- e) 在“SymbolsFromModuleList”中的所有“SymbolsFromModule”应包含“GlobalModuleReference”出现,因此:
 - i) 其中的“modulereference”彼此完全不同,而且不同于与引用模块相关的“modulereference”;和
 - ii) 非空时,“AssignedIdentifier”指明彼此完全不同的对象标识符值,而且不同于(若有的话)与引用模块相关的对象标识符值。

- f) 如果“SymbolsFromModule”有一个非空的“SelectionOption”，则“GlobalModuleReference”中的“AssignedIdentifier”不应为空，被引用模块应按如下方式确定：
- i) 如果“SelectionOption”是 WITH SUCCESSORS，则由“GlobalModuleReference”指明的模块具有一个最后节点可递增 0 次或更多次的对象标识符的 DefinitiveIdentification。如果多个模块满足此条件，则指明的模块是其对象标识符具有最多数量增量的最后节点的模块。
- ii) 如果“SelectionOption”是 WITH DESCENDANTS，则由“GlobalModuleReference”指明的模块具有 DefinitiveIdentification，该 DefinitiveIdentification 标识由“GlobalModuleReference”或其下属之一所标识的节点。如果多个模块满足此条件，则指明的模块具有最大的对象标识符。对于这个比较，依次比较弧，直到一个弧不同（选择最大的弧）或到达一个对象标识符的末端（选择较长的对象标识符）。

13.17 当选择“empty”替换“Imports”时，模块仍可通过“External<X>Reference”的方法引用其他模块中定义的“Symbol”。

注：含有“empty”替换“Import”是为了向后兼容。

13.18 如果定义它们的 typereference 是引入的或作为引入类型的组件（或子组件）出现的，则在“NamedNumberList”“Enumeration”或“NamedBitList”中出现的标识符是隐式引入。

13.19 “SymbolsFromModule”中的“Symbol”可在“ModuleBody”中作为“Reference”出现，与“Symbol”相关的意义是它在对应的“GlobalModuleReference”指明的模块中出现。

13.20 当“Symbol”也出现在“AssignmentList”（不赞成），或者在“SymbolsFromModule”的一个或多个其他实例中出现时，它应仅用在“External<X>Reference”中。它没有这样出现时，应直接用作“Reference”。

13.21 本文件中随后的几章里定义“Assignment”的各种替代式，另外标注的除外：

赋值替代式	定义章节
“TypeAssignment”	16.1
“ValueAssignment”	16.2
“XMLValueAssignment”	16.2
“ValueSetTypeAssignment”	16.6
“ObjectClassAssignment”	GB/T 16262.2—2025, 9.1
“ObjectAssignment”	GB/T 16262.2—2025, 11.1
“ObjectSetAssignment”	GB/T 16262.2—2025, 12.1
“ParameterizedAssignment”	GB/T 16262.4—2025, 8.1

每个“Assignment”的第一个符号是“Reference”的替换项之一，指明被定义的引用名。“AssignmentList”内的两个赋值中不应有相同的引用名。

13.22 “EncodingControlSections”在第 54 章中规定。

14 引用类型和值定义

14.1 被定义类型和值产生式：

```

DefinedType ::=
    ExternalTypeReference
    |
    typereference
    |
    ParameterizedType
    |
    ParameterizedValueSetType

```

```

DefinedValue ::=
    ExternalValueReference
|
    Valuereference
|
    ParameterizedValue

```

规定应用于引用类型和值定义的序列。GB/T 16262.4—2025 中规定了“ParameterizedType”和“ParameterizedValueSetType”标识的类型和“ParameterizedValue”标识的值。

14.2 “NonParameterizedTypeName”产生式：

```

NonParameterizedTypeName ::=
    ExternalTypeReference
|
    typereference
|
    xmlasn1typename

```

被采用，当需要 XML 标记名表示 ASN.1 类型时。如果生成的 XML 标记名以字母“XML”开头，则应预先添加一个 LOW LINE(95)以形成“Non-ParameterizedTypeName”。

14.3 如果“xmlasn1typename”是“CHOICE”“ENUMERATED”“SEQUENCE”“SEQUENCE_OF”“SET”或“SET_OF”，当 XML 值记法用于 ASN.1 模块中时，第三个替换项不应用作“XMLValueAssignment”(见 16.2)或“XMLOpenTypeFeildVal”(见 GB/T 16262.2—2025 的 14.6)的“XMLTypeValue”中的“NonParameterizedTypeName”。

注：因为“xmlasn1typename”不定义 ASN.1 类型，因此，在用于 ASN.1 模块中的 XML 值记法中这一限制是强制的。

因为不用从“xmlasn1typename”形成的 XML 标记来确定所编码的类型，因此，对于编码规则（例如 XER，见 ISO/IEC 8825-4:2021 中使用这一记法不出现该限制。

14.4 除 13.19 规定的外，除非引用是在类型或值指派给（见 16.1 和 16.2）“typereference”或“valuereference”的“ModuleBody”内，否则不应使用“typereference”“valuereference”“ParameterizedType”“ParameterizedValueSetType”或“ParameterizedValue”替代项。

14.5 除非对应的“typereference”或“valuereference”在用于定义对应的“modulereference”的“ModuleBody”内

- a) 分别被赋予类型或值（见 16.1 和 16.2）；或
- b) 出现在“Imports”章中，

否则不应使用“ExternalTypeReference”和“ExternalValueReference”。如果在那一章中，“Symbol”事件不超过一次，则仅允许引用另一个模块的“Imports”章中的名称。

注：这禁止在两个来自被引入到另一模块的不同模块中定义的不同“Symbol”，然而，如果相同的“Symbol”在模块 A 的 IMPORTS 章中出现不止一次，那么不能在外部引用中用模块 A 引用那个“Symbol”。

14.6 在只引用在不同模块中定义的模块名的模块中应采用外部引用，而且用下面的产生式规定：

```

ExternalTypeReference ::=
    Modulereference
    "."
    typereference
ExternalValueReference ::=
    modulereference
    "."
    valuereference

```

注：GB/T 16262.2—2025 中规定了附加外部引用产生式（“ExternalClassReference”“ExternalObjectReference”和“ExternalObjectSetReference”）。

14.7 当用“Imports”的“SymbolsImported”替换项定义引用模块时,“SymbolsImported”里只有一个“SymbolsFromModule”的“GlobalModuleReferetice”中应出现外部引用中的“modulereference”。当用“Imports”的“empty”替换项定义引用模块时,定义“Reference”的模块(不同于引用模块)的“ModuleDefinition”中应出现外部引用的“modulereference”。

14.8 在“DefinedType”用作“Type”支配的部分记法时(例如,在“SubtypeConstraint”中),那么,“DefinedType”应与附录 C.6.2 中规定的支配“Type”相兼容。

14.9 “DefinedValue”的 ASN.1 规范内的每次出现由“Type”支配,而且“DefinedValue”应引用与 C.6.2 中规定的支配“Type”相兼容的类型的值。

15 支持引用 ASN.1 组件的记法

15.1 在很多情况下,需要形式引用 ASN.1 类型、值等的组件。一个这样的实例是写出用于标识某些 ASN.1 模块内的规定类型的文本需求。本章定义能用来提供这些引用的记法。

15.2 记法使集合或序列类型(强制或可选地出现在类型中)的任何组件能被标识。

15.3 通过使用“AbsoluteReference”语法结构:

```
AbsoluteReference ::=
    "@ " ModuleIdentifier
    "."
    ItemSpec
ItemSpec ::=
    typereference
    | ItemId "." ComponentId
ItemId ::= ItemSpec
ComponentId ::=
    identifier
    | number
    | "*"

```

能引用任何 ASN.1 类型定义的任何部分。

注: AbsoluteReference 产生式不可用在本文档的其他地方。提供它是为 15.1 的陈述。

15.4 “ModuleIdentifier”标识 ASN.1 模块(见 13.1)。

15.5 当“DefinitiveIdentifier”的第一个或第二个替换项用作部分“ModuleIdentifier”时,“DefinitiveIdentifier”无歧义和唯一地标识名称被引用的模块。

15.6 “typereference”引用由“ModuleIdentifier”标识的模块中定义的任何 ASN.1 类型。

15.7 每个“ItemSpec”中的“ComponentId”标识由“ItemId”标识的类型的组件。如果它标识的组件不是集、序列、单一集合、单一序列或选择类型,它应是最后的“ComponentId”。

15.8 如果父类“ItemId”是集或序列类型,并且要求是该集或序列“ComponentTypeLists”中“NamedType”的“identifier”之一,能够使用“ComponentId”的“identifier”形式。如果“ItemId”标识选择类型,且然后要求是那个选择类型“AlternativeTypeLists”中“NamedType”的“identifier”之一,也可使用“ComponentId”的“identifier”形式。它不能在任何其他情况下使用。

15.9 只有“ItemId”是单一序列或单一集合类型的条件下能使用“ComponentId”的数字形式。数字值标识单一序列或单一集合中的类型实例,用值“1”标识类型的第一个实例。值 0 标识概念整数类型组件(不在传送中显式出现),该组件包含封闭类型的值中出现的单一序列或单一集合中的类型实例数字的计数。

15.10 只有“ItemId”是单一序列或单一集合时能够使用“ComponentId”的“*”形式。任何与使用“ComponentId”的“*”形式相关的语义适用于单一序列和单一集合的所有组件。

注：下面的示例中：

```
M DEFINITIONS ::= BEGIN
    T ::= SEQUENCE {
        a BOOLEAN
        b SET OF INTEGER
    }
END
```

组件“T”能通过 ASN.1 模块外(或注解中)的文本引用,如:

——if(@M.T.b.0 is odd)then;

—— (@M.T.b.* shall be an odd integer)

用来陈述如果“b”里的组件数字是奇数,“b”的所有组件必定是奇数。

16 类型和值的赋值

16.1 通过“TypeAssignment”产生式规定的记法将“typereference”赋予类型:

```
TypeAssignment ::=
    typereference
    "."
    Type
```

“typereference”不应是 ASN.1 的保留字(见 12.38)。

16.2 通过“ValueAssignment”或“XML ValueAssignment”产生式规定的记法将“valuereference”赋予值:

```
ValueAssignment ::=
    valuereference
    Type
    " : : ="
    Value

XML ValueAssignment ::=
    valuereference
    " : : ="
    XMLTypedValue
```

```
XMLTypedValue ::=
    "<"&.NonParameterizedTypeName">"
    XMLValue
    "</"&.NonParameterizedTypeName">"
    |
    "<"&.NonParameterizedTypeName"/>"
```

赋给“ValueAssignment”中“valuereference”的值应是“Value”,并且由“Type”支配和应是由“Type”定义的类型的地值的记法(如 16.3 中规定的)。赋给“XMLValueAssignment”中“valuereference”的值应是“XMLValue”(见 17.7),并且应是“NonParameterizedTypeName”定义的地类型的值的记法(如 16.4 中规定的)。如果这是“xmlasn1typename”项,那么它标识表 4 中对应行的 ASN.1 固有类型(也见 14.3)。允许“XMLTypedValue”中的“XMLValue”周围的空格,除非明确禁止(见 41.9 和 ISO/IEC 8825-4:2021 的 31.3.4.1)。

16.3 “Value”是 17.7 中规定类型的值的记法。

16.4 如果“XMLValue”是类型的“XMLBuiltinValue”记法(见 17.10),则“XMLValue”是类型的值的记法。

16.5 只有“XMLValue”产生式实例为空时,能够使用“XMLTypedValue”的第二个替换项(采用 XML 空元素标记)。

注:如果“XMLValue”产生式是只包含空白的“xmlcstring”,这将不能为空,而且,不能使用第二个替换项。

16.6 通过“ValueSetTypeAssignment”产生式规定的记法可将“typereference”赋给值集:

```
ValueSetTypeAssignment ::=
    typereference
    Type
    " ::= "
    ValueSet
```

这一记法将“typereference”赋给定义为“Type”指示的类型的子类型,并且确定包含“ValueSet”中规定或允许的值的类型。“typereference”不应是 ASN.1 的保留字(见 12.38),且可引用作为类型。在 16.7 中定义了“ValueSet”。

16.7 某些类型支配的值集应由记法“ValueSet”规定:

```
ValueSet ::= "{ "ElementSetSpecs" }"
```

值集包含“ElementSetSpecs”规定的所有值(见第 50 章),至少应包含有一个。

16.8 “ValueSetTypeAssignment”产生式扩大成:

```
typereference
Type
" ::= "
"("ElementSetSpecs")"
```

为了所有目的,包括应用编码规则,定义它是准确地相当于使用具有相同的“Type”和“ElementSetSpecs”规范的产生式:

```
typereference
" ::= "
Type
"("ElementSetSpecs")"
```

17 类型和值的定义

17.1 类型由下列记法“Type”规定:

```
Type ::= BuiltinType | ReferencedType | ConstrainedType
```

17.2 记法“BuiltinType”规定 ASN.1 的固有类型,定义如下:

```
BuiltinType ::=
    BitStringType
| BooleanType
| CharacterStringType
| ChoiceType
| DateType
| DateTimeType
| DurationType
```

	EmbeddedPDVType
	EnumeratedType
	ExternalType
	InstanceOfType
	IntegerType
	IRIType
	NullType
	ObjectClassFieldType
	ObjectIdentifierType
	OctetStringType
	RealType
	RelativeIRIType
	RelativeOIDType
	SequenceType
	SequenceOfType
	SetType
	SetOfType
	PrefixedType
	TimeType
	TimeOfDayType

以下几章(除非另有说明,否则是指本文件中的)定义各种“固有类型”记法:

BitStringType	22
BooleanType	18
CharacterStringType	40
ChoiceType	29
DateType	38.4.1
DateTimeType	38.4.3
DurationType	38.4.4
EmbeddedPDVType	36
EnumeratedType	20
ExternalType	37
InstanceOfType	GB/T 16262.2—2025 的附录 C
IntegerType	19
IRIType	34
NullType	24
ObjectClassFieldType	GB/T 16262.2—2025 的 14.1
ObjectIdentifierType	32
OctetStringType	23
RealType	21
RelativeIRIType	35
RelativeOIDType	33
SequenceType	25
SequenceOfType	26

SetType	27
SetOfType	28
PrefixedType	31
TimeType	38.1.1
TimeOfDayType	38.4.2

17.3 记法“ReferencedType”规定 ASN.1 引用类型：

ReferencedType ::=

DefinedType
| UsefulType
| SelectionType
| TypeFromObject
| ValueSetFromObject

“ReferencedType”记法提供了引用某些其他类型（最终到固有类型）的替换方法。各种“ReferencedType”记法以及确定它们引用的类型的方法在本文件中的下列位置规定，除非另有说明：

DefinedType	14.1
UsefulType	45.1
SelectionType	30
TypeFromObject	GB/T 16262.2—2025 的第 15 章
ValueSetFromObject	GB/T 16262.2—2025 的第 15 章

17.4 “ConstrainedType”在第 49 章中定义。

17.5 本文件要求在 规定集类型、序列类型和选择类型的组件时采用记法“NamedType”。“NamedType”的记法是：

NamedType ::= identifier Type

17.6 “identifier”用来无歧义地引用值记法、内部子类型约束及成分关系约束中的集类型、序列类型或选择类型的组件（见 GB/T 16262.3—2025）它不是类型的一部分，因此，不影响类型。

17.7 有些类型的值应由下列记法“Value”或记法“XMLValue”来规定：

Value ::=

BuiltinValue
| ReferencedValue
| ObjectClassFieldValue

XMLValue ::=

XMLBuiltinValue
| XMLObjectClassFieldValue

注 1：“ObjectClassFieldValue”和“XMLObjectClassFieldValue”在 GB/T 16262.2—2025 的 14.6 中定义。

注 2：“XMLValue”只用于“XMLTypedValue”中。

17.8 如果“XMLValue”产生式的任意部分导致 XML 开始标记后紧接着 XML 结束标记，可能由以 12.1.4 允许插入的空白隔开（例如，<字段 1></字段 1>），这两个 XML 标记以及任意插入的空白能用单个 XML 空-元素标记（<字段 1/>）代替。

注：如果任意空白字符，12.1.4 允许插入的空白除外，出现在开始标记的最后“>”字符和结束标记的初始“<”字符之间，就不满足上面的条件。

17.9 ASN.1 的固有类型值能用记法“XMLBuiltinValue”（见 17.10）或“BuiltinValue”规定，定义如下：

BuiltinValue ::=

BitStringValue

	BooleanValue
	CharacterStringValue
	ChoiceValue
	EmbeddedPDVValue
	EnumeratedValue
	ExternalValue
	InstanceOfValue
	IntegerValue
	IRIValue
	NullValue
	ObjectIdentifierValue
	OctetStringValue
	RealValue
	RelativeIRIValue
	RelativeOIDValue
	SequenceValue
	SequenceOfValue
	SetValue
	SetOfValue
	PrefixedValue
	TimeValue

各种“BuiltinValue”记法的每一个在上面 17.2 中列出的,对应的“BuiltinType”记法的同一条中定义。

17.10 “XMLBuiltinValue”定义如下:

XMLBuiltinValue ::=

	XMLBitStringValue
	XMLBooleanValue
	XMLCharacterStringValue
	XMLChoiceValue
	XMLEmbeddedPDVValue
	XMLEnumeratedValue
	XMLExternalValue
	XMLInstanceOfValue
	XMLIntegerValue
	XMLIRIValue
	XMLNullValue
	XMLObjectIdentifierValue
	XMLOctetStringValue
	XMLRealValue
	XMLRelativeIRIValue
	XMLRelativeOIDValue
	XMLSequenceValue
	XMLSequenceOfValue

```

| XMLSetValue
| XMLSetOfValue
| XMLPrefixedValue
| XMLTimeValue

```

各种“XMLBuiltinValue”记法的每一个在上面 17.2 中列出的,对应的“BuiltinType”记法的同一章中定义。

17.11 用记法“ReferencedValue”规定 ASN.1 引用的值;

ReferencedValue::=

DefinedValue

```

| ValueFromObject

```

“ReferencedValue”记法提供了引用某些其他值(最终到固有值)的替换方法。各种“Referenced-Value”记法以及确定它们引用的值的方法在下列位置规定(除非另有说明,否则在本文件中):

DefinedValue 14.1

ValueFromObject GB/T 16262.2—2025 的第 15 章

17.12 不管类型是“BuiltinType”“ReferencedType”或“ConstrainedType”,其值能够用那个类型的“BuiltinValue”或“ReferencedValue”规定。

17.13 用记法“NamedType”引用的类型的值应用记法“NamedValue”定义,或者当用作部分“XMLValue”时,用记法“XMLNamedValue”。这些产生式是:

NamedValue::=identifier Value

XMLNamedValue::="<"&identifier">"XMLValue"</"&identifier">"

这里,“identifier”与“NamedType”记法中使用的一样。

注:“identifier”是记法的一部分,它不形成值本身的部分。它用来无歧义地引用集类型、序列类型或选择类型的组件。

17.14 类型定义中隐式(见 13.4)或显式出现的扩展标志(见第 6 章)不影响值记法。也就是,有扩展标志的类型的值记法准确地好像缺少扩展标志一样。

注:50.11 禁止用于取自引用不在双亲类型扩展根中的值的子类型约束的值记法。

18 布尔类型的记法

18.1 应通过记法“BooleanType”引用布尔类型(见 3.8.8):

BooleanType::=BOOLEAN

18.2 用本记法定义类型的标记是通用类别,编号为 1。

18.3 布尔类型(见 3.8.85 和 3.8.44)的值应由记法“BooleanValue”定义,或者当用作“XMLValue”时,用记法“XMLBooleanValue”。这些产生式是:

BooleanValue::=TRUE|FALSE

XMLBooleanValue::=

EmptyElementBoolean

```

| TextBoolean

```

EmptyElementBoolean::=

"<"&"true"/>"

```

| "<"&"false"/>"

```

TextBoolean::=

extended-true

| extended-false

18.4 如果“EmptyElementBoolean”出现在“XMLValueAssignment”中,那么在“XMLValueAssignment”中不应出现“TextBoolean”。

19 整数类型的记法

19.1 整数类型(见 3.8.48)应由记法“IntegerType”来引用:

```
IntegerType ::=
    INTEGER
    |
    INTEGER "{NamedNumberList}"
NamedNumberList ::=
    NamedNumber
    |
    NamedNumberList ", " NamedNumber
NamedNumber ::=
    identifier ("SignedNumber")
    |
    identifier ("DefinedValue")
SignedNumber ::=
    number
    |
    "-" number
```

19.2 如果“number”是 0,则不应使用“SignedNumber”的第二个替换项。

19.3 “NamedNumberList”在类型定义中没有语义区别,它仅用于 19.9 中规定的值记法。

19.4 “DefinedValue”中的“valuereference”应是整数类型。

注:由于“identifier”不能用来规定与“NamedNumber”相关的值,“DefinedValue”永远不会被错认为是“IntegerValue”。在下面的情况下:

```
a INTEGER ::= 1
T1 ::= INTEGER {a(2)}
T2 ::= INTEGER {a(3), b(a)}
c T2 ::= b
d T2 ::= a
```

其中,c 指示值 1,由于它既不能引用 a 的第二、也不能第三个出现,而 d 指示值 3。

19.5 “NamedNumberList”中出现的每个“SignedNumber”或“DefinedValue”的值应不同,而且代表整数类型的不同值。

19.6 在“NamedNumberList”中出现的每个“identifier”应不同。

19.7 在“NamedNumberList”中的多个“NamedNumber”的顺序无语义区别。

19.8 由本记法定义的类型标记是通用类别,编号为 2。

19.9 整数类型的值应由记法“IntegerValue”来定义,或者当用作“XMLValue”时,则用记法“XMLIntegerValue”。这些产生式是:

```
IntegerValue ::=
    SignedNumber
    |
    identifier
XMLIntegerValue ::=
    XMLSignedNumber
    |
    EmptyElementInteger
    |
    TextInteger
```

```

XMLSignedNumber ::=
    number
|
    "-" & number
EmptyElementInteger ::=
    "<" & identifier ">"
TextInteger ::=
    identifier

```

19.10 如果“EmptyElementInteger”出现在“XMLValueAssignment”中,那么在“XMLValueAssignment”中不应出现“TextInteger”。

19.11 在“IntegerValue”及“XMLIntegerValue”后两个替换项中的“identifier”应是与值相关,且应表示对应的数字的“IntegerType”中的“identifier”之一。

注:当引用已定义“identifier”的整数值时,用“IntegerValue”的“identifier”形式和“XMLIntegerValue”的“identifier”形式之一更好。

19.12 有“NamedNumberList”的整数类型的值记法实例内,取自“NamedNumberList”的“identifier”和引用名都是名称的任何事件应解释为“identifier”。

19.13 如果“number”为0,则不应使用“XMLSignedNumber”的第二种替换项。

20 枚举类型的记法

20.1 枚举类型(见 3.8.30)应用记法“EnumeratedType”来引用:

```

EnumeratedType ::=
    ENUMERATED "{ "Enumerations" }"
Enumerations ::=
    RootEnumeration
|
    RootEnumeration, "... "ExceptionSpec
|
    RootEnumeration, "... "ExceptionSpe", "AdditionalEnumeration
RootEnumeration ::= Enumeration
AdditionalEnumeration ::= Enumeration
Enumeration ::=
    EnumerationItem | EnumerationItem, "Enumeration
EnumerationItem ::= identifier | NamedNumber

```

注 1:“EnumeratedType”的每个值有一个与不同整数相关的标识符。然而,不希望值本身有任何整数语义。规定“Enumerationitem”的“NamedNumber”替换项提供控制值的表达式以便有利于可兼容的扩展。

注 2:在“RootEnumeration”中的“NamedNumber”内的数值不必有序或者相邻,而“AdditionalEnumeration”中的“NamedNumber”内的数值是有序的但不必相邻。

20.2 对于每个“NamedNumber”,“identifier”和“SignedNumber”应与“Enumeration”中的所有其他“identifier”和“SignedNumber”不同。19.2 和 19.4 也适用于每个“NamedNumber”。

20.3 (在“EnumeratedType”中)每个是“identifier”的“EnumerationItem”连续地被赋给不同的非负整数。对于“RootEnumeration”,赋值从 0 开始、但不包括是“NamedNumber”的“EnumerationItem”中采用的任何整数的连续整数。

注:整数值与“EnumerationItem”相关,以帮助定义编码规则。否则,它没有在 ASN.1 规则中采用。

20.4 每个新“EnumerationItem”的值应比类型中所有以前定义的“AdditionalEnumeration”要大。

20.5 当“NamedNumber”用于定义“AdditionalEnumeration”中的“EnumerationItem”时,与之相关的

值应与(本类型中)所有前面定义“EnumerationItem”的值不同,无论前面定义的“EnumerationItem”在枚举根中存在与否。例如:

A::=ENUMERATED{a,b,...,c(0)} --无效,因为 a 和 c 都为 0

B::=ENUMERATED{a,b,...,c,d(2)} --无效,因为 c 和 d 都为 2

C::=ENUMERATED{a,b(3),...,c(1)} --有效,因为 c=1

D::=ENUMERATED{a,b,...,c(2)} --有效,因为 c=2

20.6 对于“EnumerationItem”不是在“RootEnumeration”中定义时,与作为“identifier”(而不是“NamedNumber”)的“AdditionalEnumeration”替换项中的第一个“EnumerationItem”相关的值应是最小值,而且“AdditionalEnumeration”(若有的话)中所有前面的“EnumerationItem”较小。例如,下面都是有效的:

A::=ENUMERATED{a,b,...,c} --c=2

B::=ENUMERATED{a,b,c(0),...,d} --d=3

C::=ENUMERATED{a,b,...,c(3),d} --d=4

D::=ENUMERATED{a,z(25),...,d} --d=1

20.7 枚举类型的标记是通用类别,编号为 10。

20.8 枚举类型的值应由记法“EnumeratedValue”定义,或者当用作“XMLValue”时,用记法“XMLEnumeratedValue”。这些产生式是:

EnumeratedValue::=identifier

XMLEnumeratedValue::=

EmptyElementEnumerated

| TextEnumerated

EmptyElementEnumerated::="<"&identifier"/>"

TextEnumerated::=identifier

20.9 如果“EmptyElementEnumerated”出现在“XMLValueAssignment”中,那么“XMLValueAssignment”中不应出现“TextEnumerated”。

20.10 在“EnumeratedValue”和“XMLEnumeratedValue”的第二种替换项中的“identifier”应等于与值相关“EnumeratedType”序列中的“identifier”。

20.11 枚举类型值记法的实例内,取自“Enumeration”的“identifier”和引用名都是名称的任何事件应解释为“identifier”。

21 实数类型的记法

21.1 实数类型(见 3.8.60)应用记法“RealType”来引用:

RealType::=REAL

21.2 实数类型的标记是通用类别,编号为 9。

21.3 实数类型的抽象值是特殊值 PLUS-INFINITY、MINUS-INFINITY 和 NOT-A-NUMBER,以及由正 0 或负 0 组成的数字实数,或者能用包含三个整数 M、B 和 E 的下列公式规定:

$$M \times B^E$$

这里,M(非 0)称作尾数,B(2 或 10)称作底数,E 称作指数。B=2 的值(“base”是 2 的抽象值)和 B=10 的值(“base”是 10 的抽象值)被定义为不同的抽象值。否则,相同的数值通过 $M \times B^E$ 计算得到值是单个抽象值。

注:负 0 和正 0 是数学 0 的两个不同的抽象值,“base”2 和“base”10 的抽象值与所有其他数字实数的抽象值不同。

21.4 实数类型有一个相关的类型,该类型用来支持实数类型的值和子类型记法(除了实数类型的特殊

值、正 0 和负 0 的记法之外)。

注：编码规则可定义用来规定编码的不同类型，或者不引用相关类型可规定编码。特别是，如果“base”是 10，PER 和 BER 中的编码提供代码的十进制(BCD)编码，和如果“base”是 2，提供允许来自于硬件浮点表达式的有效转换的编码。

21.5 数值的值定义(以及子类型的定义)的相关类型是(有标准注解)：

```
SEQUENCER{
    mantissa INTEGER,
    base INTEGER(2|10),
    exponent INTEGER
    --相关的数学实数是“mantissa”乘以“base”为底的“exponent”次幂
}
```

注 1：尽管“base”2 和“base”10 表示的非 0 值确定同样的实数值，它们还是被认为是不同的抽象值，而且可携带不同的应用层语义。

注 2：记法“REAL(WITH COMPONENT{...,base(10)})”能用来限制值集为“base”是 10 的数字值(对“base”是 2 的数字值也类似)。这种记法不包括不能用相关类型表示的值(特殊的实数值、和正 0 和负 0)。如果需要，使用集合运算来添加这些。

注 3：本类型能携带一个能以典型浮点硬件储存的任何数字的准确有限表达式及任何有限的十进制字符表达式数字。

21.6 实数类型的值应用记法“RealValue”来定义，或者当用于“XMLValue”时，用记法“XMLRealValue”定义：

```
RealValue ::=
    NumericRealValue
    | SpecialRealValue
NumericRealValue ::=
    realnumber
    | "-"realnumber
    | SequenceValue
SpecialRealValue ::=
    PLUS-INFINITY
    | MINUS-INFINITY
    | NOT-A-NUMBER
```

注：“NumericRealValue”的第三个替换项不应用于正负 0 值。这些抽象值分别由第一和第二个替换项规定，“realnumber”使用单个“0”字符。

```
XMLRealValue ::=
    XMLNumericRealValue | XMLSpecialRealValue
XMLNumericRealValue ::=
    realnumber
    | "&"realnumber
XMLSpecialRealValue ::=
    EmptyElementReal
    | TextReal
EmptyElementReal ::=
    "<&PLUS-INFINITY/>"
    | "<&MINUS-INFINITY/>"
```

```

| "<&NOT-A-NUMBER"/>"
TextReal ::=
    "INF"
| "-"&"INF"
| "NaN"

```

21.7 如果“EmptyElementReal”出现在“XMLValueAssignment”中,则“XMLValueAssignment”中不应出现“TextReal”。

21.8 当使用“realnumber”记法时,它标识出对应以 10 为“base”的抽象值或者正 0。当“realnumber”值前面有“-”时,它标识出对应以 10 为“base”的负数抽象值或者负 0。如果“RealType”被约束为以 2 为“base”,“realnumber”或“-”“realnumber”标识以 2 为“base”的抽象值对应“realnumber”规定的十进制值,或如果不能准确表示时对应本机系统定义的精确度。

22 位串类型的记法

22.1 位串类型(见 3.8.7)应用记法“BitStringType”来引用:

```

BitStringType ::=
    BIT STRING
    BIT STRING{"NamedBitList"}
NamedBitList ::=
    NamedBit
| NamedBitList,"NamedBit"
NamedBit ::=
    identifier("number")
| identifier("DefinedValue")

```

22.2 位串中的第一位叫做起始位,位串中的最后一位叫作结尾位。

注:这一术语用于规定值记法及定义编码规则。

22.3 “DefinedValue”应引用类型整数的非负值。

22.4 出现在“NamedBitList”中的每个“number”或“DefinedValue”的值应不同,而且是位串值中的不同位的数字。位串的起始位用“number”零标识,连续的位上有连续的值。

22.5 出现在“NamedBitList”中的每个“identifier”应不同。

注 1:在“NamedBitList”中的“NamedBit”产生式序列的顺序没有意义。

注 2:由于出现在“NamedBitList”内的“identifier”不能用来规定与“NamedBit”相关的值,“Defined-Value”从不可能被误以为是“IntegerValue”。因此在下列情况下:

```

a INTEGER ::= 1
T1 ::= INTEGER{a(2)}
T2 ::= BIT STRING{a(3),b(a)}

```

其中 a 的最后事件指明值 1,因为它既不能引用 a 的第二个也不能引用第三个事件。

22.6 “NamedBitList”的出现不会影响本类型的抽象值的集。允许是包含 1 位而非已命名位的值。

22.7 当“NamedBitList”用来定义位串类型时,ASN.1 编码规则随意地在被编码或解码的值上加上(或移去)任意多个结尾 0 位。因此,应用设计者应保证不同的语义与这些仅在结尾 0 位不同的值不相关。

22.8 该类型的标记是通用类别,编号为 3。

22.9 位串类型的值应用记法“BtStringValue”来定义,或者当用作“XMLValue”时,用记法“XMLBitStringValue”。这些产生式是:


```

BitStringValue ::=
    bstring
  |
    hstring
  |
    "{"IdentifierList"}"
  |
    "{}"
  |
    CONTAINING Value
IdentifierList ::=
    identifier
  |
    IdentifierList "," identifier
XMLBitStringValue ::=
    XMLTypedValue
  |
    xmlbstring
  |
    XMLIdentifierList
  |
    empty
XMLIdentifierList ::=
    EmptyElementList
  |
    TextList
EmptyElementList ::=
    "<"&identifier"/>"
  |
    EmptyElementList "<"&identifier"/>"
TextList ::=
    identifier
  |
    TextList identifier

```

22.10 如果“EmptyElementList”出现在“XMLValueAssignment”中，那么在“XMLValueAssignment”中不应出现“TextList”。

22.11 除非位串有包含 ASN.1 类型但不包括 ENCODED BY 的内容约束，否则，不应使用“XMLTypedValue”替换项。如果使用这一替换项，则“XMLTypedValue”应是内容约束中的 ASN.1 类型的值。

22.12 除非位串有“NamedBitList”，否则不应使用“XMLIdentifierList”替换项。

22.13 在“BitStringValue”或“XMLBitStringValue”中的每个“identifier”应和与该值相关的“BitStringType”产生式序列中的“identifier”相同。

22.14 “empty”替换项指明带空位的位串。

22.15 如果位串有已命名位，则“BitStringValue”或“XMLBitStringValue”记法用对应“identifier”的数字规定的位的位置中的值指明位串值，而且，所有其他位是 0。

注：对有“NamedBitList”的“BitStringType”，“BitStringValue”中的“{ }”产生式序列和“XMLBitStringValue”中的“empty”用来指明没有包含任何位的位串。

22.16 当使用“bstring”或“xmlbstring”记法时，位串值的起始位在左边，而位串值的结束位在右边。

22.17 当使用“hstring”记法时，每个十六进制数字的最高有效位对应位串中最左边的位。

注：这一记法在任何情况下都不约束解码规则将位串置成用来传送 8 位位组的方式。

22.18 除非位串值由 4 位的倍数组成，否则不应使用“hstring”记法。

例如：

'A98A'H

和

'1010100110001010'B

是同一位串值的替换记法。如果用“NamedBitList”定义类型,则(单个)结尾 0 不能形成长度是 15 位的值的一部分。如果没有用“NamedBitList”定义类型,则结尾 0 形成长度是 16 位的值的一部分。

22.19 如果具有包括 CONTAINING 的位串类型内容约束,则只能使用 CONTAINING 替换项。然后,“Value”应是“ContentsConstraint”中“Type”的值的值记法(见 GB/T 16262.3—2025 第 11 章)。

注:由于 GB/T 16262.3—2025 的 11.3 禁止“ContentsConstraint”之后的进一步约束,因此,该值记法从不可能在子类型约束中出现,而且,除非支配者有“ContentsConstraint”,否则,上面的文本禁止其使用。

22.20 如果有不包含 ENCODED BY 的位串类型的内容约束,应使用 CONTAINING 替换项。

23 八位位组串类型的记法

23.1 八位位组串类型(见 3.8.55)应由记法“OctetStringType”引用:

OctetStringType ::= OCTET STRING

23.2 该类型的标记是通用类别,编号为 4。

23.3 八位位组串类型的值应用记法“OctetstringValue”来定义,或者当用作“XMLValue”时,用记法“XMLOctetStringValue”,这些产生式是:

```
OctetStringValue ::=
    bstring
    | hstring
    | CONTAINING Value
XMLOctetStringValue ::=
    XMLTypedValue
    | xmlhstring
```

23.4 除非八位位组串有包含 ASN.1 类型但不包括 ENCODED BY 的内容约束,否则不应使用“XMLTypedValue”替换项。如果使用这一替换项,则“XMLTypedValue”应是内容约束中的 ASN.1 类型的值。

23.5 在规定八位位组串的编码规则时,用术语首位八位位组和结尾位八位位组引用八位位组,而对八位位组中的位用术语最高有效位和最低有效位引用。

23.6 当使用“bstring”记法时,“bstring”记法最左位应是八位位组串值的首位八位位组的最高有效位。若“bstring”不是八位的整数倍时,应理解为好像它含有附加 0 结束位,以形成下一个八的整数倍。

23.7 当使用“hstring”或“xmlhstring”记法时,最左的十六进制数字应是第一个八位位组的最高有效半八位位组。

23.8 如果“hstring”是十六进制数字奇数,应理解为好像它含有单个附加结束位 0 十六进制数字。“xmlhstring”不应是十六进制数字奇数。

23.9 如果有包括 CONTAINING 的八位位组串类型内容约束,就只能使用 CONTAINING 替换项。然后,“Value”应是“ContentsConstraint”中“Type”的值的值记法(见 GB/T 16262.3—2025 的第 11 章)。

注:由于 GB/T 16262.3—2025 的 11.3 禁止“ContentsConstraint”之后的进一步约束,因此,这一值记法从不可能在子类型约束中出现,而且,除非支配者有“ContentsConstraint”,否则,上面的文本禁止其使用。

23.10 如果有不包含 ENCODED BY 的八位位组串类型的内容约束,应使用 CONTAINING 替换项。

24 空类型记法

24.1 空类型(见 3.8.51)应用记法“NullType”来引用:

NullType ::= NULL

24.2 该类型的标记是通用类别,编号为 5。

24.3 空类型的值由记法“NullValue”引用,或者当用作“XMLValue”时,用记法“XMLNullValue”。这些产生式是:

NullValue ::= NULL

XMLNullValue ::= empty

25 序列类型的记法

25.1 定义序列类型(见 3.8.67)的记法应是“SequenceType”:

SequenceType ::=

SEQUENCE{"{"}

| SEQUENCE{"ExtensionAndException OptionalExtensionMarker}"

| SEQUENCE{"ComponentTypeLists"}

ExtensionAndException ::= "... | ..."ExceptionSpec

OptionalExtensionMarker ::= ", "... | empty

ComponentTypeLists ::=

RootComponentTypeList

| RootComponentTypeList, "ExtensionAndException ExtensionAdditions
OptionalExtensionMarker

| RootComponentTypeList, "ExtensionAndException ExtensionAdditions
ExtensionEndMarker", "RootComponentTypeList

| ExtensionAndException ExtensionAdditions ExtensionEndMarker", "
RootComponentTypeList

| ExtensionAndException ExtensionAdditions OptionalExtensionMarker

RootComponentTypeList ::= ComponentTypeList

ExtensionEndMarker ::= ", "...

ExtensionAdditions ::=

", "ExtensionAdditionList

| empty

ExtensionAdditionList ::=

ExtensionAddition

| ExtensionAdditionList, "ExtensionAddition

ExtensionAddition ::=

ComponentType

| ExtensionAdditionGroup

ExtensionAdditionGroup ::= "["VersionNumber ComponentTypeList"]]"

VersionNumber ::= empty | number : "

ComponentTypeList ::=

ComponentType

| ComponentTypeList, "ComponentType

ComponentType ::=

NamedType

	NamedType OPTIONAL
	NamedType DEFAULT Value
	COMPONENTS OF Type

25.2 为了方便后续章节,一个“PrefixedType”被定义为文本标记的类型,如果满足以下两者之一:

- a) “PrefixedType”是“TaggedType”;或
- b) “PrefixedType”中的“Type”是文本标记的类型。

25.3 当选择自动加标记的模块的定义内存在“ComponentTypeList”产生式(见 13.3),而且,“ComponentType”的前三个替换项中的任何一个的“NamedType”都不是文本已标记类型(见 25.2),那么为整个“ComponentTypeList”选择自动标记转换,否则不能这样。

注 1: 为序列类型采用组件列表定义内的“TaggedType”记法把标记控制交给了规定者,正好与通过自动标记机制的自动赋值相反。因此,在下面的情况下:

T ::= SEQUENCE { a INTEGER, b [1] BOOLEAN, c OCTET STRING }

即使选择了自动标记的模块内有序列类型 T 的定义,自动标记也不适用于组件 a、b、c 列表。

注 2: 只有那些选择了自动标记的模块内出现的“ComponentTypeList”产生式事件是通过自动标记转换的候选者。

25.4 单独对每个“ComponentTypeLists”事件采用自动标记转换并且是在 25.5 规定的 COMPONENTS OF 转换之前。然而,正如 25.8~25.10 的规定,自动标记转换(如果适用的话)应用于 COMPONENTS OF 转换之后。

注: 它的作用是通过显式出现在“ComponentTypeList”中的标记抑制自动标记的应用,但是不能通过“COMPONENTS OF”之后“Type”中出现的标记来抑制。

25.5 在“COMPONENT OF Type”记法中的“Type”应是序列类型。“COMPONENT OF Type”记法应用来定义(在组件列表的这点上)引用类型的所有成分类型(至少应有一个)的内容,可在“Type”中出现的任何扩展标记和扩展附加除外。[只包含“COMPONENT OF Type”中“Type”的“RootComponentTypeList”;“COMPONENT OF Type”记法忽略了扩展标志和扩展附加(如果有的话)]。该转换忽略了作用于被引用类型的任何子类型约束。

注: 这一转换逻辑上在满足下面各条的要求之前完成。

25.6 下面各条每个都标识根或扩展附加或两者中的“ComponentType”序列事件。25.6.1 的规则应适用于所有这些序列。

25.6.1 当有一个或多个都标有 OPTIONAL 或 DEFAULT 的连续的“ComponentType”事件时,这些“ComponentType”的标记及系列中任何紧接的成分类型的标记应不同(见 31.2)。如果选择自动标记,标记不同的要求仅适用于自动标记完成之后,而且,将总能满足。

25.6.2 25.6.1 应适用于根中的“ComponentType”序列。

25.6.3 25.6.1 以类型定义中它们事件的文本顺序(忽略所有的版本括号和省略记法),应适用于根或扩展附加中“ComponentType”的整个序列(也见 52.7)。

25.7 当使用“ComponentTypeLists”的第三个或第四个替换项时,扩展附加中的所有“ComponentType”应有标记,该标记与文本上紧接“ComponentType”的标记直到并包括结尾“RootComponentTypeList”中没有标志 OPTIONAL 或 DEFAULT(如果有的话)的第一个这种“ComponentType”不同(也见 52.7)。

25.8 “ComponentTypeLists”的自动置标记转换逻辑上在 25.5 规定的转换之后完成,但是只有 25.3 确定它应适用于哪个“ComponentTypeLists”事件。自动标记转换通过用 25.10 中规定的替代项“TaggedType”事件代替“NamedType”产生式中原有的“Type”来影响“ComponentTypeLists”的每个“ComponentType”。

25.9 如果自动置标记有效而且扩展根中的“ComponentType”没有标记,那么,“ExtensionAdditionList”内的“ComponentType”不应是文本已标记类型。

25.10 如果自动置标记有效,“TaggedType”的替代项规定如下:

- a) 替代“TaggedType”记法使用“Tag Type”替换项;
- b) 替代“TaggedType”的“Class”是空(即:置标记是上下文规定);
- c) 替代“TaggedType”中的“ClassNumber”对于“RootComponentTypeList”中的第一个“ComponentType”是标记值 0,第二个为 1,以此类推,按递增标记数字继续下去;
- d) 如果“RootComponentTypeList”丢失,“ExtensionAdditionList”中的第一个“ComponentType”的替代“TaggedType”中的“ClassNumber”是 0,否则,就是比“RootComponentTypeList”中的最大“ClassNumber”大 1,而“ExtensionAdditionList”中的下一个“ComponentType”的“ClassNumber”比第一个大 1,以此类推,按递增标记数字继续进行;
- e) 替代“TaggedType”中的“Type”是被替代的原有“Type”。

注 1: 由 31.2.7 提供了支配替代“TaggedType”的隐式置标记或显式置标记的规范的规则。自动置标记总是隐式置标记,除非“Type”是选择类型或者开放类型记法或者是“DummyReference”(见 GB/T 16262.4—2025 的 8.3),这时,它是显式置标记。

注 2: 一旦 25.8 满足,组件的标记就完全确定了,而且不能修改。即使序列类型在应用自动置标记转换的另一个“ComponentTypeList”内组件的定义中被引用。因此,在下面的情况下:

$$T ::= \text{SEQUENCE}\{a \text{ Ta}, b \text{ Tb}, c \text{ Tc}\}$$

$$E ::= \text{SEQUENCE}\{f1 \text{ E1}, f2 \text{ T}, f3 \text{ E3}\}$$

作用于 E 的组件的自动置标记决不会影响附加在 T 的组件 a、b 和 c 上的标记,而不管 T 的置标记环境如何。如果 T 是自动置标记环境中定义的,而 E 不是在自动置标记环境中,则自动置标记仍然适用于 T 的组件 a、b 和 c。

注 3: 当序列类型作为“Type”出现在“COMPONENT OF Type”中时,其中的每个“ComponentType”事件由采用 25.5 在可能将自动置标记应用于引用序列类型之前复制。因此,在下面的情况下:

$$T ::= \text{SEQUENCE}\{a \text{ Ta}, b \text{ SEQUENCE}\{b1 \text{ T1}, b2 \text{ T2}, b3 \text{ T3}\}, c \text{ Tc}\}$$

$$W ::= \text{SEQUENCE}\{x \text{ Wx}, \text{COMPONENTS OF T}, y \text{ Wy}\}$$

如果 W 是在自动置标记环境中定义的, T 中的 a、b 和 c 的标记不必与 W 中的 a、b 和 c 的标记相同,但是 b1、b2 和 b3 的标记在 T 和 W 中都一样。换句话说,自动置标记转换只能在给出的“ComponentTypeLists”上应用一次。

注 4: 分成子类型不影响自动置标记。

注 5: 当自动置标记在某一位置时,在扩展插入点(见 3.8.35)以外的任意位置插入新组件,由于改变标记的副作用而引起与规范旧版本互工作的问题,可能导致其他组件变化。

25.11 如果出现“OPTIONAL”或者“DEFAULT”,对应的值可从新类型的值中忽略。

25.12 如果“DEFAULT”存在,省略那个类型的值准确地相当于插入由“Value”定义的值,该“Value”应是“NamedType”产生式序列中的由“Type”定义的类型值的值记法。

25.13 对应于“ExtensionAdditionGroup”(全部组件一起)的值是可选的。然而,如果出现这样的值,那么,应出现对应于“ComponentTypeList”内没有标有 OPTIONAL 或 DEFAULT 的组件的值。

25.14 在“ComponentTypeList”(与那些通过 COMPONENT OF 扩展获得的一起)的所有“NamedType”产生式序列中的“identifier”应都是不同的。

25.15 除非具有为逻辑上位于扩展附加类型与扩展根之间的没有标志 OPTIONAL 或 DEFAULT 的所有扩展附加类型所规定的值,否则,不应规定给定扩展附加类型的值。

注 1: 当类型通过附加扩展附加从扩展根(第一版)经第二版发展到第三版时,第三版任何附加的编码中的出现要求没有标志 OPTIONAL 或 DEFAULT 的第二版中所有附加的编码的出现。

注 2: 如果没有标志 OPTIONAL 或 DEFAULT,本身是扩展类型但不包含在“ExtensionAdditionGroup”内的“ComponentType”总是要被编码,抽象值正从使用没有定义“ComponentType”的抽象语法早期版本的发送者转送除外。

注 3: 由于下面的原因,推荐使用“ExtensionAdditionGroup”产生式:

- a) 它能根据编码规则产生更简洁的编码(如,PER);
- b) 语法更精确,因为它清楚地指明如果本身被定义的扩展附加组被编码(和注1比较),“ExtensionAdditionGroup”中定义的和没有标志 OPTIONAL 或 DEFAULT 的类型的值总出现在编码中;
- c) 语法明确了“ExtensionAdditionGroup”中的哪些类型一定作为一个组被应用层支持。

25.16 只有模块内的所有“ExtensionAddition”和“ExtensionAdditionAlternatives”是带“VersionNumber”的“ExtensionAdditionGroup”或“ExtensionAdditionAlternativesGroup”,才应使用“VersionNumber”。在“ExtensionAdditionGroup”的每个“VersionNumber”中的“number”应大于或等于2,而且应比插入点内任何前面的“ExtensionAdditionGroup”中的“number”大。

注1:这里使用的惯例是没有扩展附加组的规范是版本1,因此,第一个附加的扩展附加组将有大于或等于2的数字。为“ExtensionAdditions”需要单个“ExtensionAddition”时,“ExtensionAdditionGroup”能够和“ExtensionAddition”一起使用。

注2:使用“VersionNumber”的约束仅适用于单个模块内而且不强加约束给引入的类型。

25.17 所有序列类型的标记都是通用类别,编号为16。

注:单一序列类型有和序列类型(见26.2)一样的标记。

25.18 定义一个序列类型的值的记法应是“SequenceValue”,或者当用作“XMLValue”时是“XMLSequenceValue”。这些产生式是:

```
SequenceValue ::=
    "{" ComponentValueList "}"
    |
    "{" "}"
ComponentValueList ::=
    NamedValue
    |
    ComponentValue, "NamedValue"
XMLSequenceValue ::=
    XMLComponentValueList
    |
    empty
XMLComponentValueList ::=
    XMLNamedValue
    |
    XMLComponentValueList XMLNamedValue
```

25.19 “{”}”或“empty”记法只用于下列条件下:

- a) 在“SequenceType”中的所有“ComponentType”序列被标记为 DEFAULT 或 OPTIONAL,而且忽略所有的值;或者
- b) 类型记法是 SEQUENCE{ }。

25.20 没有标志为 OPTIONAL 或 DEFAULT 的“SequenceType”中的每个“NamedType”应有一个“NamedValue”或“XMLNamedValue”,而且,值应与对应“NamedType”序列的顺序一样。

26 单一序列类型的记法

26.1 由其他类型定义单一序列类型(见3.8.68)的记法应是“SequenceOfType”。

```
SequenceOfType ::= SEQUENCE OF Type | SEQUENCE OF NamedType
```

如果用于“SequenceOfType”XML值记法的XML标记名称需要大写的起始字母,那么宜采用第一个替换项(XML标记名称则从“Type”的名称形成)。

26.2 所有单一序列类型的标记都是通用类别,编号为16。

注:序列类型有与单一序列类型(见25.17)一样的标记。

26.3 用于定义单一序列类型值的记法是“SequenceOfValue”,或者当用作“XMLValue”时,是

“XMLSequenceOfValue”。这些产生式是：

```
SequenceOfValue ::=
    "{"ValueList"}"
|    "{"NamedValueList"}"
|    "{""}"
ValueList ::=
    Value
|    ValueList", "Value
NamedValueList ::=
    NamedValue
|    NamedValueList", "NamedValue
XMLSequenceOfValue ::=
    XMLValueList
|    XMLDelimitedItemList
|    empty
XMLValueList ::=
    XMLValueOrEmpty
|    XMLValueOrEmpty XMLValueList
XMLValueOrEmpty ::=
    XMLValue
|    "<"&NonParameterizedTypeName"/>"
XMLDelimitedItemList ::=
    XMLDelimitedItem
|    XMLDelimitedItem XMLDelimitedItemList
XMLDelimitedItem ::=
    "<"&NonParameterizedTypeName">"XMLValue
    "</"&NonParameterizedTypeName">"
|    "<"&identifier">"XMLValue"</"&identifier">"
```

"{""}"或“Empty”记法用于“SequenceOfValue”或“XMLSequenceOfValue”是空列表时。

语法意义可按这些值的顺序放置。

26.4 如果组件的“XMLValue”是“empty”，则应选择“XMLValueOrEmpty”的第二个替换项代表组件的那个值。

注：仅发生在 SEQUENCE OF NULL 时。

26.5 应根据表 5 的第 2 列使用“XMLValueList”或“XMLDelimitedItemList”产生式，其中，组件的“Type”在第 1 列中列出。

表 5 ASN.1 类型的“XMLSequenceOfValue”和“XMLSetOfValue”记法

ASN.1 类型	XML 值记法
BitStringType	XMLDelimitedItemList
BooleanType	见 26.6
CharaterStringType	XMLDelimitedItemList
ChoiceType	XMLValueList
EmbeddedPDVType	XMLDelimitedItemList
EnumeratedType	见 26.7
ExternalType	XMLDelimitedItemList
InstanceOfType	见 GB/T 16262.2—2025 的 C.9
IntegerType	XMLDelimitedItemList
IRIType	XMLDelimitedItemList
NullType	XMLValueList
ObjectClassFieldType	见 GB/T 16262.2—2025 的 14.10 和 14.11
ObjectIdentifierType	XMLDelimitedItemList
OctetStringType	XMLDelimitedItemList
RealType	XMLDelimitedItemList
RelativeIRIType	XMLDelimitedItemList
RelativeOIDType	XMLDelimitedItemList
SelectionType	XMLDelimitedItemList
SequenceType	XMLDelimitedItemList
SequenceOfType	XMLDelimitedItemList
SetType	XMLDelimitedItemList
SetOfType	XMLDelimitedItemList
PrefixedType	见 26.10.1
UsefulType(GeneralizedTime)	XMLDelimitedItemList
UsefulType(UTCTime)	XMLDelimitedItemList
UsefuType(ObjectDescriptor)	XMLDelimitedItemList
TypeFromObject	见 GB/T 16262.2—2025 的 15.6
ValueSetFromObjects	见 GB/T 16262.2—2025 的 15.6

26.6 如果“EmptyElementBoolean”用于布尔类型的值,则应使用“XMLValueList”;否则,应使用“XMLDelimitedItemList”。

26.7 如果“EmptyElementEnumerated”用于枚举类型的值,则应使用“XMLValueList”;否则,应使用“XMLDelimitedItemList”。

26.8 如果组件的“Type”是“DefinedType”,那么确定“XMLSequenceOfValue”记法的类型应是“DefinedType”引用的类型(见 14.1)。

26.9 如果有且只有“SequenceOfType”包含“identifier”,应使用“XMLDelimitedItem”的第二个替换项,而且,“XMLDelimitedItem”中的“identifier”应是那个“identifier”。

26.10 如果使用“XMLDelimitedItem”的第一个替换项,那么,如果单一序列类型的组件(忽略任何出现的“TypePrefix”之后)是“typereference”或“ExternalTypeReference”,那么,“NonParameterized-TypeName”应分别是“typereference”或“ExternalTypeReference”中的“typereference”,否则,它应是对应组件固有类型的表 4 中规定的“xmlasnltypename”。

26.10.1 如果组件的“Type”是“PrefixedType”,那么确定“XMLSequenceOfValue”替换项和“xmlasnltypename”(如果需要)的类型应是“PrefixedType”中的“Type”(见 31.1.5)。如果它本身是一个“PrefixedType”“ConstrainedType”或者“SelectionType”,那么 26.10 的小节应递归地应用。

26.10.2 如果组件的“Type”是“ConstrainedType”,那么确定“XMLSequenceOfValue”替换项和“xmlasnltypename”(如果需要)的类型应是“ConstrainedType”中的“Type”(见 49.1)。如果它本身是一个“PrefixedType”“ConstrainedType”或者“SelectionType”,那么 26.10 的小节应递归地应用。

26.10.3 如果组件的“Type”是“SelectionType”,那么确定“XMLSequenceOfValue”替换项和“xmlasnltypename”(如果需要)记法的类型应是“SelectionType”引用的类型(见第 30 章)。如果它本身是一个“PrefixedType”“ConstrainedType”或者“SelectionType”,那么 26.10 的小节应递归地应用。

26.11 如果使用“SequenceOfType”的第一个替换项,那么,应使用“SequenceOfValue”的第一个替换项。“SequenceOfValue”的“ValueList”中的每个“Value”,以及“XMLSequenceOfValue”替换项中的每个“XMLValue”应是“SequenceOfType”中规定的类型。

26.12 如果使用“SequenceOfType”的第二个替换项,那么,应使用“SequenceOfValue”的第二个替换项,而且“NamedValueList”中的每个“NamedValue”应包含“SequenceOfType”的“NamedType”规定类型的“Value”。“NamedValue”中的“identifier”应是“SequenceOf-Type”的“NamedType”中的“identifier”。

27 集合类型的记法

27.1 由其他类型定义集合类型(见 3.8.72)的记法应是“SetType”。

```
SetType ::=
    SET"{" "}"
    | SET"{" "ExtensionAndException OptioanalExtensionMarker" }"
    | SET"{" "ComponentTypeLists" }"
```

“ComponentTypeLists”“ExtensionAndException”及“OptioanalExtensionMarke”在 25.1 中规定。

27.2 在“COMPONENT OF Type”记法中的“Type”应是集合类型。“COMPONENTS OF Type”记法应用于定义在组件列表中包含的引用类型的所有组件类型,“类型”中可能存在的任何扩展标记和扩展添加除外。[只包含“COMPONENTS OF Type”中“Type”的“RootComponentTypeList”,“COMPONENTS OF Type”记法忽略了扩展标记和扩展添加(若适用)],该转换忽略了应用于引用类型的任何子类型约束。

注:该转换逻辑上在满足下面各条的要求之前完成。

27.3 在集合类型中的“ComponentType”类型应都有不同的标记(见 31.2)。加在“Extension-Additions”的每个新“ComponentType”的标记应正则地比“ExtensionAdditions”中的其他组件的要大(见 8.6)。

注：如果出现该记法的模块的“TagDefault”是 AUTOMATIC TAGS, 不管实际“ComponentType”如何, 作为应用 25.8 的结果, 实现了这一要求(见 52.7)。

27.4 25.3 和 25.8~25.14 也适用于集合类型。

27.5 所有集合类型的标记都是通用类别, 编号为 17。

注：单一集合类型与集合类型(见 28.2)具有相同的标记。

27.6 语义与集合类型中值的顺序无关。

27.7 定义集合类型值的记法应是“SetValue”, 或者当用作“XMLValue”时是“XMLSetValue”。这些产生式是：

```
SetValue ::=
    "{"ComponentValueList"}"
|
    "{"""}"
XMLSetValue ::=
    XMLComponentValueList
|
    empty
```

在 25.18 中规定了“ComponentValueList”和“XMLComponentValueList”。

27.8 “SetValue”和“XMLSetValue”只应分别是“{”“}”和“empty”, 如果：

- a) 在“SetType”中的所有“ComponentType”序列标为“DEFAULT”或“OPTIONAL”, 且所有的值都被省略; 或者
- b) 类型记法是 SET{ }。

27.9 未标有 OPTIONAL 或 DEFAULT 的“SetType”中的每个“NamedType”应有一个“NamedValue”或“XMLNamedValue”。

这些“NamedValue”或“XMLNamedValue”可任意顺序出现。

28 单一集合类型的记法

28.1 由其他类型定义单一集合类型(见 3.8.73)的记法应是“SetOfType”。

```
SetOfType ::=
    SET OF Type
|
    SET OF NamedType
```

如果用于“SetOfType”的 XML 值记法的 XML 标记名称需要大写的起始字母, 那么宜采用第一个替换项(XML 标记名称则从“Type”的名称形成)。

28.2 所有单一集合类型的标记都是通用类别, 编号为 17。

注：集合类型有与单一集合类型具有相同的标记(见 27.5)。

28.3 用于定义单一集合类型值的记法应是“SetOfValue”, 或者用作“XMLValue”时是“XMLSetOfValue”。这些产生式是：

```
SetOfValue ::=
    "{"ValueList"}"
|
    "{"NamedValueList"}"
|
    "{"""}"
```

```
XMLSetOfValue ::=
    XMLValueList
  | XMLDelimitedItemList
  | empty
```

在 26.3 中规定了“ValueList”“NamedValueList”和“XMLSetOfValue”的替换项,替换项的选择与使用“XMLSequenceOfValue”时相同。“{ }”或“empty”记法用在“SetOfValue”或“XMLSetOfValue”是空列表时。

语义重要性不宜按照这些值的次序给出。

注 1: 不要求编码规则来保持这些值的顺序。

注 2: 单一集合类型不是值的数学集合,因此,作为示例,对于 SET OF INTEGER,值{1}和{1 1}是不同的。

28.4 如果使用“SetOfType”的第一个替换项,那么,应使用“SetOfValue”的第一个替换项。“SetOfValue”的“ValueList”中的每个“Value”,以及“XMLSetOfValue”替换项中的每个“XMLValue”应是“SetOfType”中规定的类型。

28.5 如果使用“SetOfType”的第二个替换项,那么,应使用“SetOfValue”的第二个替换项,而且,“NamedValueList”中的每个“NamedValue”序列应包含“SetOfType”的“NamedType”中规定的类型的“Value”。“NamedValue”中的“identifier”应是“SetOfType”的“NamedType”中的“identifier”。

29 选择类型的记法

29.1 由其他类型定义选择类型(见 3.8.14)的记法应是“ChoiceType”

```
ChoiceType ::= CHOICE " { " AlternativeTypeList " } "
AlternativeTypeList ::=
    RootAlternativeTypeList
  | RootAlternativeTypeList " , "
    ExtensionAndException ExtensionAdditionAlternatives
    OptionalExtensionMarker
RootAlternativeTypeList ::= AlternativeTypeList
ExtensionAdditionAlternatives ::=
    " , " ExtensionAdditionAlternativeList
  | empty
ExtensionAdditionAlternativesList ::=
    ExtensionAdditionAlternative
  | ExtensionAdditionAlternativesList " , " ExtensionAdditionAlternative
ExtensionAdditionAlternative ::=
    ExtensionAdditionAlternativesGroup
  | NamedType
ExtensionAdditionAlternativeGroup ::= "[ [ " VersionNumber AlternativeTypeList " ] ] "
AlternativeTypeList ::=
    NamedType
  | AlternativeTypeList " , " NamedType
```

注: “T ::= CHOICE {a A}”和 A 不是同一个类型,而且根据编码规则可能被不同地编码。

29.2 当选择自动置标记的模块的定义内存在“AlternativeTypeLists”产生式时(见 13.3),且任何“AlternativeTypeList”中的“NamedType”都不是文本已标记类型(见 25.2),则为整个“AlternativeTypeLists”选择自动置标记转换,否则不是。

29.3 在“AlternativeTypeLists”内“AlternativeTypeList”产生式中定义的类型应有不同的标记(见 31.2 和 52.7)。如果选择自动置标记,标记不同的要求仅适用于自动置标记完成之后,而且应总是满足的。

29.4 如果自动置标记有效而且在扩展根中的“NamedType”没有标记,那么,“ExtensionAdditionAlternativesList”内的“NamedType”都不应是文本标记类型。

29.5 自动置标记转换用替代“TaggedType”代替“NamedType”产生式中的原有“Type”影响“AlternativeTypeLists”的每个“NamedType”。替代“TaggedType”规定如下:

- a) 替代“TaggedType”记法使用“Tag Type”替换项;
- b) 替代“TaggedType”的“Class”是空(例如,置标记是上下文规定的);
- c) 替代“TaggedType”中的“ClassNumber”,对“RootAlternativeTypeList”中的第一个“NamedType”是标记值零,第二个是 1,以此类推,以递增标记数字继续;
- d) 在“ExtensionAdditionAlternativesList”中第一个“NamedType”的替代“TaggedType”中的“ClassNumber”比“RootAlternativeTypeList”中最大的“ClassNumber”大 1,“ExtensionAdditionAlternativesList”中下一个“NamedType”的“ClassNumber”比第一个大 1,以此类推,以递增标记数字继续;
- e) 替代“TaggedType”中的“Type”是被代替的原有“Type”。

注 1: 由 31.2.7 提供了支配为替代“TaggedType”的隐式置标记或显式置标记规范的规则。自动置标记总是隐式置标记,除非“Type”是无标记的选择类型或无标记的开放类型记法,或无标记的“DummyReference”(见 GB/T 16262.4—2025 的 8.3),这种情况下它是显式置标记。

注 2: 一旦采用自动置标记,组件的标记完全确定了,而且即使在使用自动置标记转换的另一个“AlternativeTypeLists”内的替换项的定义中引用选择类型时也不能修改。因此,在下面的情况下:

$$T ::= \text{CHOICE}\{a \text{ Ta}, b \text{ Tb}, c \text{ Tc}\}$$

$$E ::= \text{CHOICE}\{f1 \text{ E1}, f2 \text{ T}, f3 \text{ E3}\}$$

应用于 E 组件的自动置标记决不会影响附加在 T 的组件 a、b 和 c 的标记,不管 T 的置标记环境如何。如果 T 在自动置标记环境中定义,而 E 不是在自动置标记环境中,自动置标记仍然适用于 T 的组件 a、b 和 c。

注 3: 分成子类型不影响自动置标记。

注 4: 当自动置标记在某一位置时,在非扩展插入点(见 3.8.35)的任意位置插入新替换项,由于改变标记的副作用而引起与规范旧版本互工作的问题可能导致其他替换项变化。

29.6 在 25.1 中定义了“VersionNumber”,而且,在 25.16 中规定的整个模块中限制一致使用“VersionNumber”应用于本产生式中的“number”。

29.7 加在“ExtensionAdditionAlternativesList”上的每个新“NamedType”的标记应正则地比“ExtensionAdditionAlternativesList”中其他替换项的那些大(见 8.6),而且,应是“ExtensionAdditionAlternativesList”中的最后一个“NamedType”。

29.8 选择类型包含的值并不都具有相同的标记(标记依赖于将值贡献给了选择类型的替换项)。

29.9 当这一类型没有扩展标志且用在本文件要求使用带不同标记的类型的地方时(见 29.3),选择类型的值的所有可能标记应认为在这个要求内。假设“TagDefault”不是 AUTOMATIC TAGS 的下面示例说明了这一要求。

示例:

```
1 A ::= CHOICE{
```

```

        b      B,
        c      NULL}
B ::= CHOICE{
    d      [0]NULL,
    e      [1]NULL}
2 A ::= CHOICE{
    b      B,
    c      C}
B ::= CHOICE{
    d      [0]NULL,
    e      [1]NULL}
C ::= CHOICE{
    f      [2]NULL,
    g      [3]NULL}
3(Incorrect)
A ::= CHOICE{
    b      B,
    c      C}
B ::= CHOICE{
    d      [0]NULL,
    e      [1]NULL}
C ::= CHOICE{
    f      [0]NULL,
    g      [1]NULL}

```

例 1 和例 2 是记法的正确用法。例 3 没有用自动置标记是不正确的,其中类型 d 和类型 f 的标记是相同的,同时类型 e 和类型 g 的标记也是相同的。

29.10 在“AlternativeTypeLists”中所有“NamedType”的“identifier”应与该表中其他“NamedType”的不同。

29.11 定义选择类型值的记法应是“ChoiceValue”,或者当用作“XMLValue”时是“XMLChoiceValue”。这些产生式是:

ChoiceValue ::= identifier " : " Value

XMLChoiceValue ::= "<" & identifier ">" XMLValue "</" & identifier ">"

29.12 “Value”或“XMLValue”应是由“identifier”命名的“AlternativeTypeLists”中类型的值的记法。

30 精选类型的记法

30.1 定义精选类型(见 3.8.66)的记法应是“SelectionType”。

SelectionType ::= identifier "<" Type

这里,“Type”指明选择类型,而“identifier”是出现在该选择类型定义的“Alternative-Types”中的某“NamedType”的标识符。

30.2 当“Type”指明受约束类型时,选择在双亲类型上执行,忽略双亲类型上的任何子类型约束。

30.3 在“SelectionType”用作“NamedType”时,出现“NamedType”的“identifier”,以及“SelectionType”的“identifier”。

30.4 在“SelectionType”用作“Type”时,保留“identifier”,而且指明的类型是被选择替换项的类型。

30.5 精选类型的值的记法应是“SelectionType”引用的类型的值的记法。

31 前缀类型的记法

31.1 概述

31.1.1 前缀类型(见 3.8.78)是一种新类型,它与旧类型同构,但具有不同的或附加的标记,并且可能具有不同或附加的相关编码指令。

31.1.2 前缀类型要么是“TaggedType”,要么是“EncodingPrefixedType”。

31.1.3 作为已标记类型的前缀类型主要用于要求使用带有不同标记类型的本文件(见 25.6~25.7、27.3 和 29.3)。在模块中使用 AUTOMATIC TAGS 的“TagDefault”允许在该模块中不显式出现“TaggedType”的情况下完成。

注:如果协议确定可在任何时间传输来自几种数据类型的值,则需要不同的标记以使接收者能够正确解码该值。

31.1.4 使用“EncodingPrefixedType”分配编码指令仅与相关编码引用标识的编码相关,对类型的抽象值没有影响。

31.1.5 前缀类型的记法应是“PrefixedType”:

```
PrefixedType ::=
    TaggedType
    | EncodingPrefixedType
```

注:如果将置标记描述为编码指令的分配,“PrefixedType”的语法规则会更简单和更清晰。然而,从历史上看,标记是在 ASN.1 规范的最早版本中引入的,并且会影响类型定义的合法性。引入编码前缀类型时,对置标记概念(以及相关的语法描述)进行了最小的更改。置标记在语法上也不同于编码指令的分配:置标记是 EXPLICIT 或 IMPLICIT 的规范出现在标记的结束“]”之后,它不像普通编码指令那样包含在成对的“[”和“]”中。

31.1.6 “PrefixedType”值记法应是“PrefixedValue”,或者当用作“XMLValue”时是“XMLPrefixedValue”。这些产生式是:

```
PrefixedValue ::= Value
XMLPrefixedValue ::= XMLValue
```

其中“Value”或“XMLValue”是“TaggedType”中“Type”的值记法或“PrefixedType”的“Encoding-PrefixedType”的值记法。

注 1:“Tag”和“EncodingPrefix”的任何部分都没有出现在这个记法中。

注 2:编码指令也能指派给编码控制部分中的类型(见第 54 章)。这样的分配对类型的值记法没有影响。

31.2 已标记类型

31.2.1 已标记类型的记法应是“TaggedType”:

```
TaggedType ::=
    Tag Type
    | Tag IMPLICIT Type
    | Tag EXPLICIT Type
Tag ::= "["EncodingReference Class ClassNumber"]"
EncodingReference ::=
    encodingreference ":"
    | empty
```

```

ClassNumber ::=
    number
  | DefinedValue
Class ::=
    UNIVERSAL
  | APPLICATION
  | PRIVATE
  | empty

```

31.2.2 在“Tag”中使用,“encodingreference”应为 TAG。“Tag”中的“EncodingReference”不应为“empty”,除非模块的默认编码引用是 TAG(见 13.5)。

31.2.3 在“DefinedValue”中的“valuereference”应是整数类型,并且分配了非负值。

31.2.4 新类型与旧类型同形,但是带有以“Class”为类别和以“ClassNumber”为序号的标记,除非“Class”是“empty”,这时,标记是上下文规定的类别,而序号是“ClassNumber”。

31.2.5 除本文件定义的类型外,“Class”不应是“UNIVERSAL”。

注 1: 对通用类别标记的使用,ITU-T 和 ISO 会经常协商。

注 2: G.2.12 包含有关使用标记类别格式的指导和提示。

31.2.6 标记的所有应用是隐式置标记或显式置标记。对于那些提供选择的编码规则,隐式置标记指明:在传送期间不需要显式标识“TaggedType”中的“Type”的原始标记。

注: 当是通用类别时,保留旧标记可能有用,因此,不用新类型的 ASN.1 定义的知识无歧义地标识旧类型。然而,通常用 IMPLICIT 实现最小传送八位位组。在 ISO/IEC 8825-1:2021 中给出了用 IMPLICIT 编码的示例。

31.2.7 如果下述任何一项成立,置标记构造规定显式置标记:

- a) 使用“Tag EXPLICIT Type”替换项;
- b) 使用“Tag Type”替换项,而且模块的“TagDefault”值是 EXPLICIT TAGS 或者是空;
- c) 使用“Tag Type”替换项,而且模块的“TagDefault”值是 IMPLICIT TAGS 或者 AUTOMATIC TAGS,但是,“Type”定义的类型是无已标记的选择类型、无已标记的开放类型或者无已标记的“DummyReference”(见 GB/T 16262.4—2025 的 8.3)。

否则,置标记构造规定隐式置标记。

31.2.8 如果“Class”是“empty”,使用“Tag”没有限制,而不是 25.6~25.7、27.3 和 29.3 中不同标记要求暗示的。

31.2.9 如果“Type”定义的类型是无标记选择类型或无标记开放类型或无标记“DummyReference”(见 GB/T 16262.4—2025 的 8.3),则不应使用 IMPLICIT 替换项。

31.3 编码前缀类型

31.3.1 编码前缀类型的记法应是“EncodingPrefixedType”:

```

EncodingPrefixedType ::=
    EncodingPrefix Type
EncodingPrefix ::=
    "["EncodingReference EncodingInstruction"]"

```

“EncodingReference”在 31.2.1 中定义。

31.3.2 “EncodingInstruction”产生式由“EncodingReference”(见附录 E)标识的本文件规定,能够由 ASN.1 词项的任何序列(包括注释、字符串和空白)组成。

注 1: “[”和“]”词项永远不会出现在“EncodingInstruction”中。

注2：本文件的后续版本可能会增加对附录 E 的进一步编码引用。如果“encodingreference”不是附录 E 中规定的内容之一，则推荐 ASN.1 工具（仅）提供警告，然后忽略使用“]”作为终止符的整个“EncodingPrefix”（见上文注 1）。

31.3.3 如果“EncodingReference”为空，则编码前缀的编码引用是模块的默认编码引用。

注：如果模块的默认编码引用是 TAG（见 31.2.2）并且“EncodingReference”是“empty”，那么“PrefixedType”是“TaggedType”，而不是“EncodingPrefixedType”。

31.3.4 对能够组合使用的编码指令（具有相同的编码引用）以及能够应用特定指令或指令组合的类型有一般限制。这些限制在与编码引用相关的本文件中规定（见附录 E），在本文件中没有规定。

32 对象标识符类型的记法

32.1 对象标识符类型（见 3.8.54）应用记法“ObjectIdentifierType”引用：

```
ObjectIdentifierType ::=
    OBJECT IDENTIFIER
```

32.2 该类型的标记是通用类别，编号是 6。

32.3 对象标识符的值记法应是“ObjectIdentifierValue”，或者当用作“XMLValue”时是“XML-ObjectIdentifierValue”。这些产生式是：

```
ObjectIdentifierValue ::=
    "{" ObjIdComponentsList }"
|
    "{" DefinedValueObjIdComponentsList }"
ObjIdComponentsList ::=
    ObjIdComponents
|
    ObjIdComponents ObjIdComponentsList
ObjIdComponents ::=
    NameForm
|
    NumberForm
|
    NameAndNumberForm
|
    DefinedValue
NameForm ::= identifier
NumberForm ::= number | DefinedValue
NameAndNumberForm ::=
    identifier ("NumberForm")
XMLObjectIdentifierValue ::=
    XMLObjIdComponentList
XMLObjIdComponentList ::=
    XMLObjIdComponent
|
    XMLObjIdComponent & "." & XMLObjIdComponentList
XMLObjIdComponent ::=
    NameForm
|
    XMLNumberForm
|
    XMLNameAndNumberForm
XML NumberForm ::= number
```


XML NameAndNumberForm ::=
 identifier & "(" & XMLNumberForm & ")"

32.4 在“NumberForm”的“DefinedValue”中的“valuereference”应是整数类型,并且分配一个非负值。

32.5 在“ObjectIdentifierValue”的“DefinedValue”中的“valuereference”应是对象标识符类型。

32.6 “ObjIdComponents”的“DefinedValue”应是相对对象标识符类型,而且应标识从对象标识符树中的某个开始节点到对象标识符树中的某个后来节点的弧的有序集合。开始节点用较早的“ObjIdComponents”标识,而后面的“ObjIdComponents”(若有)标识取自后面节点的弧。要求开始节点既不能是根、也不能是紧接根下面的节点。

注:相对对象标识符值需要与规定的对象标识符值相关以便无歧义地标识对象。要求对象标识符值(见 32.11)至少有两个组件。这就是为什么要限制开始节点。

32.7 “NameForm”应仅用于那些数字值和标识符在 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 A(也见附录 F)规定的那些对象标识符组件,而且应是 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 A~附录 C 规定的标识符之一。

注:在允许使用“NameForm”的情况下,推荐在某些情况下使用“NameAndNumberForm”,见 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的 A.1.2。

32.8 当 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 规定同义标识符时,同义词可在根据 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 注册同义词时确定的条件下使用。当 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 中规定的标识符和包含“NameForm”的模块内的 ASN.1 值引用都是相同的名称时,则对象标识符值内的名称应视为 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 标识符。

32.9 在“NumberForm”和“XMLNumberForm”中的“number”是分配给对象标识符组件的数值。

32.10 能够在三个顶级弧下的“NameAndNumberForm”和“XMLNameAndNumberForm”中使用的“identifier”具有灵活性。这些标识符不包含在编码中,并且可能会随着时间而改变。这是认可组织的名称可更改。弧标识符通常应在负责弧上方节点的注册权威机构和负责分配后续弧的注册权威机构之间达成一致。

注:Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 中规定了负责三个顶级弧下面的弧的注册权威机构。

32.11 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 中规定了与对象标识符值相关的语义。

注:Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 要求对象标识符值应包含至少两条弧。

32.12 对象标识符组件的重要部分是简化为“NameForm”或“NumberForm”或者“XMLNumberForm”,并为对象标识符组件提供数值。除了 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 A 至附录 C 中(也见附录 F)规定的弧之外,对象标识符组件的数值总是出现在对象标识符值记法实例中。

32.13 在“ObjectIdentifierValue”包含对象标识符值的“DefinedValue”时,它引用的对象标识符组件列表作为前缀加在值中显式出现的组件上。

注:Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 推荐只要分配对象标识符值来标识对象,也分配对象描述符值。

例如,

按 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 规定分配的标识符,值:

{iso standard 8571 application-context(1)}

和

{1 0 8571 1}

将各标识一个对象,ISO 8571 中定义的“application-context”。正如“XMLObjectIdentifierValue”中的

iso.standard.8571.application-context(1)

和

1.0.8571.1

一样。

随着下面的附加定义：

ftam OBJECT IDENTIFIER ::= {iso standard 8571}

下面的值与上述值相等：

{ftam application-context(1)}

33 相对对象标识符类型记法

33.1 相对对象标识符类型(见 3.8.64)应用记法“RelativeOIDType”来引用：

RelativeOIDType ::=

RELATIVE-OID

33.2 该类型的标记是通用类别,编号为 13。

33.3 相对对象标识符的值记法应是“RelativeOIDValue”,或者当用作“XMLValue”时是“XML-RelativeOIDValue”。这些产生式是：

RelativeOIDValue ::=

"{"RelativeOIDComponentsList"}"

RelativeOIDComponentsList ::=

RelativeOIDComponents

| RelativeOIDComponents RelativeOIDComponentsList

RelativeOIDComponents ::=

NumberForm

| NameAndNumberForm

| DefinedValue

XMLRelativeOIDValue ::=

XMLRelativeOIDComponentList

XMLRelativeOIDComponentList ::=

XMLRelativeOIDComponent

| XMLRelativeOIDComponent & "." & XMLRelativeOIDComponentList

XMLRelativeOIDComponent ::=

XMLNumberForm

| XMLNameAndNumberForm

33.4 产生式“NumberForm”“NameAndNumberForm”“XMLNumberForm”“XMLNameAndNumberForm”及它们的语义在 32.3~32.12 中定义。

33.5 “RelativeOIDComponents”的“DefinedValue”应是类型相对对象标识符的,而且应标识从对象标识符树中的某些开始节点到对象标识符树中的某些后续节点的弧的有序集合。开始节点用较早的“RelativeOIDComponents”(若有)标识,而后面的“RelativeOIDComponents”(若有)标识取自后面节点的弧。

33.6 第一个“RelativeOIDComponents”或者“XMLRelativeOIDComponent”标识从对象标识符树中的某些开始节点到对象标识符树中的某些后来节点的一个或多条弧。开始节点能用与类型定义相关的注解定义。如果与类型定义相关的注解内没有开始节点的定义,那么它需要作为通信实例中的对象标识

符值传送(见 G.2.21)。要求开始节点既不能是根、也不能是紧接根下面的节点。

注：相对对象标识符值需要与特定的对象标识符值相关以便无歧义地标识对象。要求对象标识符值(见 32.11)至少有两个组件。这就是为什么要限制开始节点。

示例

用下面的定义：

```
thisUniversity OBJECT IDENTIFIER ::=
    {joint-iso-itu-t example(999) universities(56) thisuni(32)}
firstgroup RELATIVE-OID ::= {science-fac(4) maths-dept(3)}
```

或在 XML 值记法中：

```
thisUniversity ::= <OBJECT_IDENTIFIER>2.999.56.32</OBJECT_IDENTIFIER>
firstgroup ::= <RELATIVE_OID>4.3</RELATIVE_OID>
```

相对对象标识符：

```
relOID RELATIVE-OID ::= {firstgroup room(4) socket(6)}
```

或在 XML 值记法中：

```
relOID ::= <RELATIVE_OID>4.3.4.6</RELATIVE_OID>
```

可用来替代 OBJECT IDENTIFIER 值 {2 999 56 32 4 3 4 6}，只要当前根是 thisUniversity(通过应用知道或应用传送)。

34 OID 国际化资源标识符类型的记法

34.1 OID 国际化资源标识符类型(见 3.8.47)应用记法“IRIType”来引用：

```
IRIType ::= OID-IRI
```

34.2 该类型的标记是通用类别，编号为 35。

34.3 OID 国际化资源标识符的值记法应是“IRIValue”，或者当用作“XMLValue”时，值记法应是“XMLIRIValue”。这些产生式是：

```
IRIValue ::=
    " " "
    FirstArcIdentifier
    SubsequentArcIdentifier
    " " "

FirstArcIdentifier ::=
    "/"ArcIdentifier

SubsequentArcIdentifier ::=
    "/"ArcIdentifier SubsequentArcIdentifier
|
    empty

ArcIdentifier ::=
    integerUnicodeLabel
|
    non-integerUnicodeLabel

XMLIRIValue ::=
    FirstArcIdentifier
    SubsequentArcIdentifier
```

34.4 “FirstArcIdentifier”应标识从 OID 树的根开始的弧(可能是长弧)。

34.5 每个“SubsequentArcIdentifier”应从前面的“ArcIdentifier”中标识一个弧。

示例

按 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 和 ISO/IEC 19785 规定赋值的标识符,对象由以下标识:

```
{iso registration-authority cbeff(19785)organizations(0)jtc1-sc37(257)
patron-formats(1)tlv-encoded(5)}
```

或在 XML 值记法中:

```
<OBJECT_IDENTIFIER>1.1.19785.0.257.1.5</OBJECT_IDENTIFIER>
```

其标识 TLV 编码的 CBEFF 用户格式,也能够有一个 ASN.1 OID-IRI 标识:

```
"/ISO/Registration_Authority/19785.CBEFF/Organizations/JTC1-SC37/Patron-formats/TLV-
encoded"
```

或在 XML 值记法中:

```
<OID-IRI>/ISO/Registration_Authority/19785.CBEFF/Organizations/JTC1-SC37/
Patron-formats/TLV-encoded</OID-IRI>
```

35 相对 OID 国际化资源标识符类型的记法

35.1 相对 OID 国际化资源标识符类型(见 3.8.62)应用记法“RelativeIRIType”来引用:

```
RelativeIRIType ::= RELATIVE-OID-IRI
```

35.2 该类型的标记是通用类别,编号为 36。

35.3 相对 OID 国际化资源标识符的值记法应是“RelativeIRIValue”,或者当用作“XMLValue”时,值记法应是“XMLRelativeIRIValue”。这些产生式是:

```
RelativeIRIValue ::=
    " " "
    FirstRelativeArcIdentifier
    SubsequentArcIdentifier
    " " "
```

```
FirstRelativeArcIdentifier ::=
```

```
    ArcIdentifier
```

```
XMLRelativeIRIValue ::=
```

```
    FirstRelativeArcIdentifier
```

```
    SubsequentArcIdentifier
```

35.4 “FirstRelativeArcIdentifier”应标识从对象标识符树中的某个起始节点到对象标识符树中的某个后续节点的弧。起始节点能够通过与类型定义相关的注解来定义。如果在与类型定义相关的注解中没有定义起始节点,则需要在通信实例中将其作为 OID 国际化资源标识符值传输。

注:相对 OID 国际化资源标识符值一定与特定 OID 国际化引用标识符值相关,以便无歧义地标识资源。

示例

使用以下标识的节点:

```
cbeffPatronFormats OID-IRI ::=
```

```
    "/ISO/Registration_Authority/19785.CBEFF/Patron-formats"
```

相对 OID 国际化资源标识符:

```
tlv-encoded RELATIVE-OID-IRI ::= "TLV-encoded"
```

标识 TLV 编码的用户格式。

36 嵌入式 pdv 类型的记法

36.1 嵌入式 pdv 类型(见 3.8.24)应用记法“EmbeddedPDVType”来引用。

EmbeddedPDVType=EMBEDDED PDV

注：术语“Embedded PDV”是指取自嵌入到报文中的可能不同抽象语法(实质上,在隔开-和标识-协议中定义的报文编码和值)的抽象值。以前,它是指其在 OSI 表示层使用的“嵌入式表示层数据值(Embedded Presentation Data Value)”,但现在已不再使用这个展开式,而且,它要理解为“嵌入式值(embedded value)”。

36.2 该类型的标记是通用类别,编号为 11。

36.3 该类型由表示下面的值组成:

- a) 可以,但不是一定,是 ASN.1 类型的值的单个数据值编码;和
- b) 以下的标识(单独或一起):
 - 1) 抽象语法;
 - 2) 传输语法。

注 1: 数据值可能是 ASN.1 类型的值,或者,例如:可能是静止图像或移动图片的编码。标识由一个或两个对象标识符组成,或者(在 OSI 环境中)引用规定抽象和传输语法的 OSI 表示上下文标识符。

注 2: 抽象语法和/或编码的标识也可能由应用层设计者确定为一个固定值,这时,它在通信实例中没有被编码。

36.4 嵌入式 pdv 类型有一个相关类型,该相关类型用来支持嵌入式 pdv 类型的值和子类型记法。

36.5 值定义和分成子类型的相关类型(假设是自动置标记环境)是(有标准注解):

```
SEQUENCE{
  identification
    syntaxes
      abstract
      transfer
      --抽象和传送语法对象标识符--,
      syntax
      --标识抽象和传送语法的单个对象标识符--,
      presentation-context-id
      --(仅适用于 OSI 环境)
      --协商的 OSI 表示上下文标识抽象和传送语法--,
      context-negotiation
      presentation-context-id
      transfer-syntax
      --(仅适用于 OSI 环境)进程中的上下文协商、
      --表示层上下文标识符只标识抽象语法,
      --因此,应规定传送语法--
      transfer-syntax
      --值的类型(例如,是 ASN.1 类型的值的规范)由应用
      --设计者固定(因此,发送者和接收者都了解),提供这一情况
      --主要是支持 ASN.1 类型的选择-范围-加密(或其他编码转换)--
      fixed
    CHOICE{
      SEQUENCE{
        OBJECT IDENTIFIER,
        OBJECT IDENTIFIER}
      OBJECT IDENTIFIER
      INTEGER
      SEQUENCE{
        INTEGER,
        OBJECT IDENTIFIER}
      NULL
```

--数据值是固定 ASN.1 类型的值(因此,发送者和接收者都了解)--

data-value-descriptor ObjectDescriptor OPTIONAL

--这样提供值的类别的人可读标识--

```
data-value                                OCTET STRING}
```

(WITH COMPONENTS{

...

```
data-value-descriptor  ABSENT))
```

注：嵌入式 pdv 类型不允许包含 data-value-descriptor 值。然而，这里提供的相关类型的定义强调存在于嵌入式 pdv 类型、外部类型与无限制的字符串类型之间的共性。

36.6 当整数值应是 OSI 定义的上下文集中的 OSI 表示上下文标识符时,则 presentation-context-id 替换项只适用于 OSI 环境。这一替换项不应用于 OSI 上下文协商期间。

36.7 context-negotiation 替换项应仅用于 OSI 环境,而且,仅用于 OSI 上下文协商期间。整数值应是提议附加在 OSI 定义的上下文集上的 OSI 表示上下文标识符。对象标识符 transfer-syntax 应标识为那个用来对值编码的 OSI 表示上下文提议的传输语法。

36.8 嵌入式 pdv 类型值的记法应是 36.5 中定义的相关类型的值记法,其中,类型 OCTET STRING 的 data-value 组件的值表示采用 identification 中规定的传送语法的编码。

$$\text{EmbeddedPDVValue} ::= \text{SequenceValue}$$
$$\text{XML EmbeddedPDVValue} ::= \text{XMLSequenceValue}$$

例如,如果强加了单个选项,如使用语法,那么能够通过书写下面的内容完成:

EMBEDDED PDV(WITH COMPONENTS{

...

identification(WITH COMPONENTS{

syntaxes PRESENT}}))

37 外部类型的记法

37.1 外部类型(见 3.8.43)应用记法“ExternalType”来引用:

$$\text{ExternalType} ::= \text{EXTERNAL}$$

37.2 该类型的标记是通用类别,编号为 8。

37.3 类型由表示下列内容的值组成:

- a) 可以,但不是一定,是 ASN.1 类型值的单个数据值编码;和
- b) 下面的标识(单独或一起):
 - 1) 抽象语法;
 - 2) 传输语法;
- c) (可选择)提供数据值类型的人可读描述的对象描述符。除非与使用“ExternalType”记法相关的注解明显地允许,否则不应出现可选择的对象描述符。

注：36.3 的注 1 也适用于外部类型。

37.4 外部类型有相关类型,这个类型用来精确定义外部类型抽象值,也用来支持外部类型的值和子类型记法。

注：编码规则可定义用来衍生编码的不同类型，或可规定不引用任何相关类型的编码，例如，由于历史原因 BER 编码采用不同的序列类型。

37.5 值定义和分成子类型的相关类型(假设是自动置标记环境)是(有标准注解):

```

SEQUENCE{
  identification
    CHOICE{
      syntaxes
        SEQUENCE{
          abstract
            OBJECT IDENTIFIER,
          transfer
            OBJECT IDENTIFIER}
        --抽象和传输语法对象标识符--,
      syntax
        OBJECT IDENTIFIER
        --标识抽象和传送语法的单个对象标识符--,
      presentation-context-id
        INTEGER
        --(仅适用于 OSI 环境)
        --协商的 OSI 表示上下文标识抽象和传送语法--
      context-negotiation
        SEQUENCER{
          presentation-context-id
            INTEGER,
          transfer-syntax
            OBJECT IDENTIFIER}
        --(仅适用于 OSI 环境)进程中的上下文会话、
        --上下文协商进行中,表示上下文标识符只标识抽象语法,
        --因此,应规定传输语法--
      transfei-syntax
        OBJECT IDENTIFIER
        --值的类型(例如,是 ASN.1 类型的值的规范)
        --由应用设计者固定(因此,发送者和接收者都了解),提供这一情况
        --主要是支持 ASN.1 类型的选择范围加密(或其他编码转换)--
      fixed
        NULL
        --数据值是固定的 ASN.1 类型的值(因此,发送者和接收者都了解)--}
  data-value-descriptor
    ObjectDescriptor OPTIONAL
    --提供值的类别的人可读标识--
  data-value
    OCTET STRING}
(WITH COMPONENTS{
  ...,
  identification(WITH COMPONENTS{
    ...,
    syntaxes
      ABSENT,
    transfer-syntax
      ABSENT,
    fixed
      ABSENT}))}

```

注：由于历史原因，外部类型不允许有 identification 的替换项 syntaxes、transfer-syntax 或 fixed。要求这些选项的应用设计者需要使用嵌入式 pdv 类型。这里提供的相关类型的定义强调存在于外部类型、未限制的字符串类型与嵌入式 pdv 类型之间的共性。

37.6 36.6 和 36.7 的内容也适用于外部类型。

37.7 外部类型值的记法应是 37.5 中定义的相关类型的值记法，其中，类型 OCTET STRING 的 data-value 组件的值表示使用 identification 中定义的传输语法的编码。

ExternalValue::=SequenceValue

XMLExternalValue::=XMLSequenceValue

注：由于历史的原因，编码规则能传输编码不是八位精确倍数的 EXTERNAL 中的嵌入式值。这样的值不能使用上述相关类型的值记法中表示。

38 时间类型

38.1 概述

38.1.1 时间类型(见 3.8.83)应用记法“TimeType”来引用：

TimeType::=TIME

38.1.2 此记法定义的类型标记是通用类别，编号为 14。

38.1.3 时间类型的值应用记法“TimeValue”定义，或者当用作“XMLValue”时，用记法“XMLTime-Value”定义。这些记法的语法在 38.3 中定义为“simplestring”的内容，使用 GB/T 7408.1—2023 的 3.4 中定义的记法。

38.2 时间抽象值的时间属性和设置

38.2.1 表 6 在第 1 列中规定了时间抽象值的时间属性的描述和名称。在第 2 列中规定了第 1 列时间属性的可能具有的时间属性设置的名称。第 3 列规定了(通常引用 GB/T 7408.1—2023)时间属性适用的抽象值，并具有相应的时间属性设置。

- 注 1：ASN.1 未规定 GB/T 7408.1—2023 表示法不支持的抽象值。
- 注 2：时间属性和设置的名称出现在属性设置子类型记法的属性声明中(见第 51 章)。
- 注 3：如果 TIME 抽象值具有相同的属性和这些属性的相同设置，ASN.1 会识别它们之间的顺序关系。对于包含时差的抽象值，只有具有相同时差的抽象值之间才能识别顺序关系。

表 6 时间抽象值的属性和设置

时间属性	属性设置名称	具有此属性设置的抽象值
抽象值的基本性质 名称: Basic 注解: 此属性的设置标识抽象值的基本性质。所有时间抽象值都具有此属性。	Date	见 GB/T 7408.1—2023 的 4.1。 仅是日期的所有抽象值
	Time	见 GB/T 7408.1—2023 的 4.2。 仅是当日时间的所有抽象值
	Date-Time	见 GB/T 7408.1—2023 的 4.3。 是日期和当日时间的所有抽象值
	Interval	见 GB/T 7408.1—2023 的 4.4。 所有时间间隔的抽象值
	Rec-Interval	见 GB/T 7408.1—2023 的 4.5。 所有重复间隔的抽象值

表 6 时间抽象值的属性和设置 (续)

时间属性	属性设置名称	具有此属性设置的抽象值
日期的时间刻度和精确性 名称: Date 注解:这仅适用于包含日期标识的抽象值。它标识该日期的时间尺度和精确性。 注:任何标识多个日期(例如,时间间隔)的抽象值都有适用于两个日期的单个 Date 设置。	C(世纪)	见 GB/T 7408.1—2023 的 4.1.2.3 c)。 包含仅代表一个世纪日期的所有抽象值
	Y(仅限年份)	见 GB/T 7408.1—2023 的 4.1.2.3 b)。 包含仅代表年份日期的所有抽象值
	YM(年-月)	见 GB/T 7408.1—2023 的 4.1.2.3 a)。 包含使用年-月时间刻度日期的所有抽象值
	YMD(年-月-日)	见 GB/T 7408.1—2023 的 4.1.2.2。 包含使用年-月-日时间刻度日期的所有抽象值
	YD(年-日)	见 GB/T 7408.1—2023 的 4.1.3.2。 包含使用年-日时间刻度日期的所有抽象值
	YW(年-周)	见 GB/T 7408.1—2023 的 4.1.4.3。 包含使用年-周时间刻度日期的所有抽象值
	YWD(年-周-日)	见 GB/T 7408.1—2023 的 4.1.4.2。 包含使用年-周-日时间刻度日期的所有抽象值
相关年份类型 名称: Year 注解:这仅适用于包含一年或多年或世纪标识的抽象值。其设置标识年份(或世纪)标识是“normal”的年份、公历中的年份(见 J.2.2)、负年份或需要超过四位数字来表示的年份。 注:任何涉及一年以上的抽象值(例如,时间间隔)都有适用于两个年份的单个 Year 设置。	Basic	包含 1582~9999 范围内的年份(或 15~99 范围内的世纪)的所有抽象值
	Proleptic	包含 0~1581 范围内的年份(或 00~14 范围内的世纪)的所有抽象值。 注: 在公历中,零年的含义大致对应于公元前 1 年(见 J.2.2)
	Negative	包含 -9999~-0001 范围内的年份(或 -99~-01 范围内的世纪)的所有抽象值
	L5, L6, L7 等, 至无穷远(大)	分别包含十进制表示法需要 5、6、7 等位数的年份(或十进制表示法需要 3、4、5 等位数的世纪)的所有抽象值,无论是正数还是负数
时间的精确性 名称: Time 注解:这仅适用于包含当日时间标识的抽象值。它标识了当日时间的精确性。 注:任何标识多个当日时间(例如,时间间隔)的抽象值都具有适用于两个当日时间的单个 Time 设置。	H(小时)	见 GB/T 7408.1—2023 的 4.2.2.3 b)。 包含精确到小时的当日时间的所有抽象值
	HM(小时-分钟)	见 GB/T 7408.1—2023 的 4.2.2.3 a)。 包含精确到分钟的当日时间的所有抽象值
	HMS(小时-分钟-秒)	见 GB/T 7408.1—2023 的 4.2.2.2。 包含精确到秒的当日时间的所有抽象值
	HF1、HF2、HF3 等, 直到无穷大(小时-小数-分数)	见 GB/T 7408.1—2023 的 4.2.2.4 c)。 包含小时精确到小数点后 1、2、3 位等的当日时间的所有抽象值

表 6 时间抽象值的属性和设置(续)

时间属性	属性设置名称	具有此属性设置的抽象值
时间的精确性 名称: Time 注解: 这仅适用于包含当日时间标识的抽象值。它标识了当日时间的精确性。 注: 任何标识多个当日时间(例如,时间间隔)的抽象值都具有适用于两个当日时间的单个 Time 设置。	HMF1、HMF2、HMF3 等,到无穷大(小时-分钟-分数)	见 GB/T 7408.1—2023 的 4.2.2.4 b)。包含分钟精确到小数点后 1、2、3 位等的当日时间的所有抽象值
	HMSF1、HMSF2、HMSF3 等,到无穷大(小时-分钟-秒-分数)	见 GB/T 7408.1—2023 的 4.2.2.4 a)。包含秒精确到小数点后 1、2、3 位等的当日时间的所有抽象值
当地或 UTC 时间刻度 名称: Local-or-UTC 注解: 这仅适用于包含时间标识的抽象值。它标识该时间的时间刻度(当地时间、UTC 或当地时间加上与 UTC 的时差)。时差由当地政府决定。ASN.1 支持-15 小时到+15 小时范围内的时差。如果一天中的当地时间早于或等于 UTC,则时差为正(见 GB/T 7408.1—2023 的 4.2.5.1)。也见 J.2.11。 注: 任何标识多个时间(例如,时间间隔)的抽象值都有适用于两个时间的单个 Local-or-UTC 设置。	L(仅限当地时间)	见 38.2.2 和 GB/T 7408.1—2023 的 4.2.2 和 4.2.3。包含仅规定当地时间的当日时间的所有抽象值
	Z(仅限 UTC)	见 GB/T 7408.1—2023 的 4.2.4。包含规定 UTC 而不是当地时间的当日时间的所有抽象值
	LD(当地时间和与 UTC 的时差)	见 GB/T 7408.1—2023 的 4.2.4。包含规定当地时间以及添加到 UTC 以获取当地时间的时间(可能为负数)的当日时间的所有抽象值
时间间隔规范形式 名称: Interval-type 注解: 这仅适用于作为时间间隔或循环时间间隔的抽象值。它标识了时间间隔规范形式(起点和终点、时间长度、起点和时间长度,或带终点的时间长度)。	SE(起点和终点)	见 GB/T 7408.1—2023 的 4.4.1 a)。使用起点和终点规定时间间隔的所有抽象值
	D(仅时间长度)	见 GB/T 7408.1—2023 的 4.4.1 b)和 4.4.3。仅使用时间长度规定时间间隔的所有抽象值
	SD(起点和时间长度)	见 GB/T 7408.1—2023 的 4.4.1 c)。使用起点和时间长度规定时间间隔的所有抽象值
	DE(时间长度和终点)	见 GB/T 7408.1—2023 的 4.4.1 d)。使用时间长度和终点规定时间间隔的所有抽象值

表 6 时间抽象值的属性和设置（续）

时间属性	属性设置名称	具有此属性设置的抽象值
起点和/或终点规范性质 名称:SE-point 注解:这仅适用于使用起点或终点或两者的时间间隔或循环时间间隔。此属性的设置标识了形成此抽象值一部分的起点和/或终点的性质。 注:所有具有起点和终点的时间间隔抽象值都具有此属性以及与日期或当日时间相关的任何相关属性的单个设置。 不具有不同形式的起点和终点的时间间隔抽象值。因此,所有具有时间间隔起点和时间间隔终点的抽象值都具有相同的起点和终点时间组件集(但终点的值记法见表 7)。这与 GB/T 7408.1—2023 有所不同。	Date	见 GB/T 7408.1—2023 的 4.1。 仅使用日期规定起点和/或终点的所有抽象值
	Time	见 GB/T 7408.1—2023 的 4.2。 仅使用当日时间规定起点和/或终点的所有抽象值
	Date-Time	见 GB/T 7408.1—2023 的 4.3。 使用日期和当日时间规定起点和/或终点的所有抽象值
重复规范 名称:Recurrence 注解:这仅适用于作为循环时间间隔的抽象值。它标识了对重复次数的约定限制(或无限制)。	Unlimited (重复次数不限,重复次数用空字符串表达)	见 GB/T 7408.1—2023 的 4.5。 代表时间间隔无限次重复的所有抽象值
	R1、R2、R3 等,直到无穷大(重复位数)	见 GB/T 7408.1—2023 的 4.5。 代表时间间隔重复分别需要 1、2、3 等位数来表示重复次数的所有抽象值
一天中午夜的开始或结束 名称:Midnight 注解:这仅适用于包含表示午夜时间的抽象值。它标识此午夜值是一天的开始(通常表示为 00:00:00)或是一天的结束(通常表示为 24:00:00)。	Start(一天的开始)	见 GB/T 7408.1—2023 的 4.2.3 a)。 包含表示一天开始时的午夜时间的抽象值
	End(一天的结束)	见 GB/T 7408.1—2023 的 4.2.3 b)。 包含表示一天结束时的午夜时间的抽象值
注:ASN.1 不支持使用具有不同时间属性的时间间隔的起点和终点,因为只有单个 SE-point 设置来支配起点和终点的语法。起点和终点需要使用相同的时间格式。这与 GB/T 7408.1—2023 有所不同。		

38.2.2 GB/T 7408.1—2023 为午夜提供了两种基本表示法:“2 400”表示一天结束时的午夜,“0 000”表示一天开始时的午夜(任何秒或秒的小数部分仅包含 0 位)。这些不被视为单个抽象值的不同表示法,而是不同的抽象值。

注 1: 这是因为作为一个独立的时间,它们明显不同,并代表一天的开始和一天的结束。当在一天与另一天的连接处使用时,第 x 天的“2 400”需要被认为小于第 x+1 天的“0 000”,尽管在时间轴上的位置完全相同。

注 2: 它们分别具有时间属性设置“Midnight=End”和“Midnight=Start”。

注 3: 与其他时间一样,在任何特定一天的开始和结束时都有无数不同的抽象值,具体取决于秒的精确性和秒的小数部分。还有基于使用小时或分钟而不是秒的小数的午夜抽象值的无限集。(如果抽象值是午夜值,所有这

些小数部分对于各种不同的精度都将为 0。)

38.2.3 GB/T 7408.1—2023 为时间长度(周,或年、月、日、小时、分钟和秒的某些组合)提供了两种基本表示法,作为时间间隔和循环时间间隔的组成组件。GB/T 7408.1—2023 中表示时间长度的不同字符串在 ASN.1 中被认为表示不同的抽象值,唯一的区别是省略或包含不改变所表示的时间长度(包括时间长度的精确性)的零时间组件。零时间组件的包含或省略完全在规范编码规则以及在 ISO/IEC 8825-2:2021 的所有编码规则中规定。没有与时间长度相关的时间属性(除了“Basic = Interval Interval-type=D”),但是能够对时间长度的时间组件使用限制,要求它们不存在或限制它们的值(见 38.4.4)。

注: GB/T 7408.1—2023 要求事先对组件(尤其是小数部分)的大小达成一致。这通常由不同精度的属性设置处理。但是,在 DURATION 的情况下,为简单起见,没有引入属性设置来确定组件的精确性。相反,等效序列类型上的内部子类型约束能够应用,如 38.4.4 中所述,以记录 DURATION 组件的先前协议。

GB/T 7408.1—2023 要求周组件的使用不应与任何其他日期组件(年、月、日)的使用结合,也不应与小时、分钟或秒时间组件结合。为了与 GB/T 7408.1—2023 保持一致,此限制也适用于 ASN.1。

38.2.4 不同的时间长度抽象值之间没有定义顺序关系,除非它们使用单个时间元素(例如,仅周、月或天)来表示,因为国际上对以秒为单位的一个月或一年的时间长度没有一致的定义。

38.3 具有规定属性设置的时间抽象值的基本值记法和 XML 值记法

38.3.1 具有相同时间属性设置的所有时间抽象值都具有相同的值记法,仅因年、月、周、日、小时、分钟、秒等(在相关的时间刻度上)的值而不同,这些值用于将该抽象值与具有相同属性设置的其他值区分开来。

38.3.2 时间类型的值记法应是“TimeValue”和“XMLTimeValue”:

TimeValue::=tstring

XMLTimeValue::=xmltstring

“tstring”和“xmltstring”的内容在 38.3.4 中使用表 7 的第 3 列中定义的时间组件语法定义。表 7 为不同的组件定义了多种可能的记法(例如,年份组件)。使用精确记法取决于在第 2 列中规定的抽象值的属性设置。第 2 列中未列出的属性对组件使用的记法没有影响。这些时间组件记法通常是通过引用 GB/T 7408.1—2023 表示法(它们与之一致)来定义的,但为了避免值记法的歧义,在时间组件中添加了一个附加的 C 字符来表示一个世纪而不是一整年,如表 7 第 3 列所规定。

38.3.3 表 7 规定(在第 3 列中)时间组件的值记法和 XML 值记法(在第 1 列中列出)。第 1 列标识了一个时间组件。第 2 列根据与抽象值相关的属性设置规定了特定行适用的条件。第 3 列规定用于该时间组件的记法。第 3 列中使用的记法在 GB/T 7408.1—2023 的 3.4 中定义,添加了 C 作为世纪指示符。

注 1: 第 3 列中使用的 GB/T 7408.1—2023 记法能够概括为:Y 是年份数字,M 是月份数字或月份指示符,D 是日期数字,w 是周数字,h 是小时数字,m 是分钟数字,s 是秒数字,n 是 0~9 中的任何一个,± 是正负,下划线表示零次或多次重复(例如“±YYYYY”)。GB/T 7408.1—2023 使用的记法优先于本文件中使用的任何其他记法,以明确与 GB/T 7408.1—2023 的联系。

注 2: J.2 提供了一个关于 GB/T 7408.1—2023 关键概念的辅导,这将有助于理解这个记法。有关结果值记法的示例,也见 G.3。

表 7 具有特定属性和设置的时间抽象值的值记法

时间组件	属性	值记法语法
年份组件	“Year=Basic” 和“Date=C” 或 “Year=Proleptic” 和“Date=C”	GB/T 7408.1—2023 的 4.1.2.3 c) 后跟字符 LATIN CAPITAL LETTER C:[YYC]
年份组件	“Year=Negative” 和“Date=C” 或 “Year=Ln”和“Date=C”	GB/T 7408.1—2023 的 4.1.2.4 d) 后跟字符 LATIN CAPITAL LETTER C:[±YYYY] 对于“Year=Negative”, Y 的重复次数应为 0, 对于“Year=Ln”, 应等于 $n-4$
年份组件	“Year=Basic” 和 Date 不是 C 或 “Year=Proleptic” 和 Date 不是 C	GB/T 7408.1—2023 的 4.1.2.2:[YYYY]
年份组件	“Year=Negative” 和 Date 不是 C 或 “Year=Ln”和 Date 不是 C	GB/T 7408.1—2023 的 4.1.2.4 c):[±YYYY] 对于“Year=Negative”, Y 的重复次数应为 0, 对于“Year=Ln”, 应等于 $n-4$
月份组件	任何	GB/T 7408.1—2023 的 4.1.2.3 a):[-MM]
周组件	任何	GB/T 7408.1—2023 的 4.1.4.3:[-Www]
日组件	“Year=YMD”	GB/T 7408.1—2023 的 4.1.2.2 扩展格式: [-DD]
日组件	“Year=YD”	GB/T 7408.1—2023 的 4.1.3.2 扩展格式: [-DDD]
日组件	“Year=YWD”	GB/T 7408.1—2023 的 4.1.4.2 扩展格式: [-D]
小时组件	“Basic=Time” 或 “Basic=Interval” 和“SE-point=Time” 或 “Basic=Rec-Interval” 和“SE-point=Time”	GB/T 7408.1—2023 的 4.2.2.3 b):[hh] 小时组件值记法 24 应始终用于抽象值“midnight at end of day”, 小时组件值记法 00 应始终用于“midnight at start of day”
小时组件	“Basic=Date-Time” 或 “Basic=Interval” 和“SE-point=Date-Time” 或 “Basic=Rec-Interval” 和“SE-point=Date-Time”	GB/T 7408.1—2023 的 4.3.2 扩展格式:[Thh] 值记法 T24 应始终用于抽象值“midnight at end of day”的小时组件, 值记法 T00 应始终用于抽象值“midnight at start of day”

表 7 具有特定属性和设置的时间抽象值的值记法(续)

时间组件	属性	值记法语法
分钟组件	任何	GB/T 7408.1—2023 的 4.3.2 扩展格式:[:mm]
秒组件	任何	GB/T 7408.1—2023 的 4.3.2 扩展格式:[:ss]
小时、分钟或秒的小数组件	任何	GB/T 7408.1—2023 的 4.2.2.4:[,hh]或[,hh], [,mm]或[,mm],或[,ss]或[,ss] 注:推荐在任何给定的 ASN.1 模块中,逗号或句号始终用于小数点符号。
时间长度中年、月、周或日的小数组件(见 J.2.6, 注)	“Basic=Interval” 和“Interval-type=D” 或 “Basic=Interval” 和“Interval-type=SD” 或 “Basic=Interval” 和“Interval-type=DE”	GB/T 7408.1—2023 的 4.4.3.2:[,nn]或[,nn] 注:推荐在任何给定的 ASN.1 模块中,逗号或句号始终用于小数点符号。
UTC 指示符组件	“Local-or-UTC=Z”	GB/T 7408.1—2023 的 4.2.4:[Z]
时差组件	“Local-or-UTC=LD”	GB/T 7408.1—2023, 4.2.5.2 扩展格式:[±hh]或 [±hh:mm] 如果不是精确的小时倍数,则时差组件应是精确的分钟时差。 注:这意味着一定存在分钟组件,除非当地时间和 UTC 之间的时差是整数个小时。
时间长度组件	“Interval-type=D” 或 “Interval-type=SD” 或 “Interval-type=DE”	GB/T 7408.1—2023 的 4.4.3.2; 见 38.3.6
时间间隔	“Interval-type=SE” 或 “Interval-type=SD” 或 “Interval-type=DE”	GB/T 7408.1—2023 的 4.4 扩展格式: 起点组件(“Interval-type=SE”或“Interval-type=SD”)或时间长度组件(“Interval-type=DE”),后跟 [/],而后跟时间长度组件(“Interval-type=SD”)或 终点组件(“Interval-type=SE”或“Interval-type=DE”)
起点组件	取决于 SE-point 设置	这是由 SE-point 的设置决定的,它应被认为代表这个组件的 Basic 属性的设置。然后应使用 Date、Year、Time 和 Local-or-UTC 属性设置来确定起点组件的格式

表 7 具有特定属性和设置的时间抽象值的值记法(续)

时间组件	属性	值记法语法
终点组件	取决于 SE-point 设置	这是由 SE-point 的设置决定的,它应被认为代表这个组件的 Basic 属性的设置。然后应使用 Date、Year、Time 和 Local-or-UTC 属性设置来确定终点组件的格式。如果终点的 UTC 与当地时间的时差与起点的时差相同,则允许(可选地)省略时差组件。 注:这不像 GB/T 7408.1—2023 那样通用,但为简单起见仅限于这些情况。
循环时间间隔	“Recurrence= Unlimited”	GB/T 7408.1—2023 的 4.5 扩展格式:[R/]后跟时间间隔组件
循环时间间隔	“Recurrence= R1”, “Recurrence= R2”, “Recurrence= R3”等	GB/T 7408.1—2023 的 4.5 扩展格式:[Rnn/]后跟时间间隔组件

38.3.4 “tstring”的值应是时间组件字符编码的连接(根据表 6 由它们的属性设置确定),前面和后面是在 12.17 中规定的双引号字符(“”),“xmltstring”的值应是时间组件字符编码的连接(根据表 6 由它们的属性设置确定),不包括双引号字符。

注 1: 值记法和 XML 值记法是规范的,除了:

- a) 时间长度的不同表示法;
- b) 小数分隔符中逗号或句号的不同用法;
- c) 小时和分钟的不同使用或小时仅用于整数小时的时差组件;
- d) 当终点的时差与起点的时差相同时,在一个时间间隔的终点(有起点和终点)中包含或省略一个时差组件。

注 2: 值记法的示例在 G.3 中提供。

38.3.5 时间组件的记法应按照 GB/T 7408.1—2023 中规定的顺序连接。

注: 这意味着最重要的时间组件在前,区域指示符(时差组件或 Z)最后。

38.3.6 时间长度组件的基本值记法和 XML 值记法在以下小节中规定。

38.3.6.1 值记法应是[P](见 GB/T 7408.1—2023 的 4.4.3),后跟:

- a) 一个年-月-日名称(见 38.3.6.2)可选地后跟时-分-秒名称(见 38.3.6.3);或
- b) 周名称(见 38.3.6.4);或
- c) 时-分-秒名称(见 38.3.6.3)。

38.3.6.2 年-月-日名称应是以下一项或多项(按顺序):

- a) 年份名称(见 38.3.6.5);
- b) 月份名称(见 38.3.6.6);
- c) 日期名称(见 38.3.6.7)。

38.3.6.3 时-分-秒名称应是[T]后跟一个或多个(顺序):

- a) 小时名称(见 38.3.6.8);或
- b) 分钟名称(见 38.3.6.9);或
- c) 秒名称(见 38.3.6.10)。

- 38.3.6.4 周名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[W]。
- 38.3.6.5 年份名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[Y]。
- 38.3.6.6 月份名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[M]。
- 38.3.6.7 日期名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[D]。
- 38.3.6.8 小时名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[H]。
- 38.3.6.9 分钟名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[M]。
- 38.3.6.10 秒名称应由一个或多个数字组成,可选择后跟一个小数部分(见 38.3.6.12),而后跟[S]。
- 38.3.6.11 除非它是单个数字 0,否则名称的整数部分不应包含前导 0,可选择后跟小数部分。如果后面有小数部分,则整数部分应至少有一位数字。
- 38.3.6.12 小数部分应包含一个小数分隔符(应为句号或逗号),后跟一个或多个小数位。
- 38.3.6.13 如果名称包含小数部分,则不应有以下名称。
- 38.3.6.14 表达不同精度的时间长度的值记法表示不同的抽象值。

示例 1:以下值记法都表示不同的抽象值:

- a) P29M(或 P0Y29M) --0 年 29 个月,精度为 1 个月。
- b) P29M0D(或 P0Y29M0D) --0 年 29 个月 0 天,精度为 1 天。
- c) P29MT0S(或 P0Y29M0DT0H0M0S) --0 年 29 个月 0 天 0 小时 0 分 0 秒,精度为 1 秒。
- d) P29MT0.00H(或 P0Y29M0DT0.00H) --0 年 29 个月 0 天 0 小时,精度为百分之一小时。
- e) P29MT0.000S(或 P0Y29M0DT0H0M0.000S) --0 年 29 个月 0 天 0 小时 0 分钟 0 秒,精度为 1 毫秒。

示例 2:以下值记法都表示相同的抽象值(0 年、29 个月、0 天、0 小时、0 分钟),精度为百分之一分钟:

- a) P0Y29M0DT0H0.00M
- b) P0Y29M0DT0.00M
- c) P0Y29MT0H0.00M
- d) P0Y29MT0.00M
- e) P29M0DT0H0.00M
- f) P29M0DT0.00M
- g) P29MT0H0.00M
- h) P29MT0.00M

更多示例见附录 G。

38.4 有用时间类型

定义了以下有用时间类型,并有望涵盖应用设计者的大多数正常要求。

注:这些定义使用第 51 章中规定的属性设置子类型记法。如果需要替代时间刻度,例如,使用年和日日历,能够使用已定义时间类型(见附录 B),或者属性设置子类型记法能够用于定义 TIME 类型的附加子类型(见 G.3 中能够使用的属性和设置示例)。

38.4.1 日期类型应由以下记法引用:

DateType::=DATE

并定义为:

DATE::=[UNIVERSAL 31] IMPLICIT TIME
(SETTINGS "Basic=Date Date=YMD Year=Basic")

38.4.2 当日时间类型应由以下记法引用:

TimeOfDayType::=TIME-OF-DAY

并定义为:

TIME-OF-DAY::=[UNIVERSAL 32] IMPLICIT TIME
(SETTINGS "Basic=Time Time=HMS Local-or-UTC=L")

注：此类型允许在一天的开始时的午夜(00:00:00)以及在一天结束时的午夜(24:00:00)。

38.4.3 日期时间类型应由以下记法引用：

DateTimeType::=DATE-TIME

并定义为：

DATE-TIME::=[UNIVERSAL 33] IMPLICIT TIME

(SETTINGS "Basic=Date-Time Date=YMD Year=Basic Time=HMS Local-or-UTC=L")

注：此类型允许一天开始时的午夜(00:00:00)以及一天结束时的午夜(24:00:00)。

38.4.4 时间长度类型应由以下记法引用：

DurationType::=DURATION

并定义为：

DURATION::=[UNIVERSAL 34] IMPLICIT TIME

(SETTINGS "Basic=Interval Interval-type=D")

TIME 类型的任何子集,其所有抽象值都具有属性设置“Basic=Interval Intervaltype=D”(无论是 UNIVERSAL 34 还是 UNIVERSAL 14),称为时间长度子类型。这种类型能够根据以下各条进行限制。TIME 类型的辅导附录见附录 J。

38.4.4.1 内部子类型约束能够使用等效的序列类型应用于任何时间长度子类型(见 38.4.4.2)。

注：应用于等价序列类型的内部子类型约束可用于禁止或要求时间长度类型中的特定时间组件,或对时间长度类型的某些或所有时间组件的值设置范围约束(也见 51.11.2)。

38.4.4.2 DURATION-EQUIVALENT 等价序列类型是：

```
DURATION-EQUIVALENT::=SEQUENCE{
  years      INTEGER(0..MAX)OPTIONAL,
  months     INTEGER(0..MAX)OPTIONAL,
  weeks      INTEGER(0..MAX)OPTIONAL,
  days       INTEGER(0..MAX)OPTIONAL,
  hours      INTEGER(0..MAX)OPTIONAL,
  minutes    INTEGER(0..MAX)OPTIONAL,
  seconds    INTEGER(0..MAX)OPTIONAL,
  fractional-part SEQUENCE{
    number-of-digits  INTEGER(1..MAX),
    fractional-value  INTEGER(0..MAX)}OPTIONAL}
```

其中等价序列类型的年份组件对应于时间长度类型抽象值的年份时间组件,依此类推。

38.4.4.3 对等价序列类型组件设置的约束也是对时间长度类型相应时间组件的约束。

注 1：时间长度类型的规则要求至少有一个时间组件存在(见 38.2.3),但当周存在时不存在其他时间组件。使用违反这些规则的内部子类型约束将是非法规范。

注 2：小数部分始终适用于抽象值中存在的最不重要的时间组件。

38.4.5 所有有用时间类型的基本值记法和 XML 值记法应是 TIME 类型的值记法(见 38.3.2),仅限于那些出现在有用时间类型中的抽象值的记法。

39 字符串类型

这些类型由取自某些规定的字符汇中的字符串组成。通常用一个或多个表中的字符元来定义一个字符汇及其编码,每个字符元对应于字符汇中的一个字符。通常也给每个字符元赋予图形符号和字符名称,虽然在有些字符汇中,字符元为空或者有名称但没有字型(有名称但没有字型的字符元的示例包

括如 ISO/IEC 646 中 EOF 的控制字符和如 GB/T 13000—2010 中 THIN-SPACE 和 EN-SPACE 的空格字符)。ASN.1 字符串的辅导附录见附录 H。

总之,与字符元相关的信息指明字符汇中不同的抽象字符,即使信息是空的(那个字符元没有被赋予图形符号或名称)。

字符串类型的 ASN.1 基本值记法有三个变量(能组合而成),形式规定如下。

a) 字符串中采用赋值图形符号(可能包含空格字符)的字符表达式;这是“cstring”记法。

注 1: 当字符汇中不只一个字符使用相同的图形符号时,这样的表达式在打印的表达式中可能有歧义。

注 2: 当字符汇中出现不同宽度的空格字符或规范用按比例-空格字体打印时,这样的表达式在打印的表达式中可能有歧义。

b) 通过给出已赋予字符的系列 ASN.1 值引用的字符串值中的字符列表;第 42 章中的模块 ASN1-CHARACTER-MODULE 中为 GB/T 13000—2010 字符汇和 IA5String 字符汇定义了这类值引用的集合。除非用户使用上述 a) 或下述 c) 中描述的值记法指派这类值引用,否则,这一形式不会在其他字符汇中出现。

c) 通过用其字符元在字符汇表中的位置标识每个抽象字符的字符串值中的字符列表。这一形式只在 IA5String、UniversalString、UTF8String 和 BMPString 中出现。

字符串类型的 ASN.1 XML 值记法使用“xmlcstring”记法,它包括对某些特殊字符、和对采用十进制或十六进制(见 12.15)的字符的规范使用转义序列的能力。

40 字符串类型的记法

40.1 引用字符串类型(见 3.8.12)的记法应是:

```
CharacterStringType ::=
    RestrictedCharacterStringType
    | UnrestrictedCharacterStringType
```

“RestrictedCharacterStringType”是受限制的字符串类型的记法并且在第 41 章中定义。

“UnrestrictedCharacterStringType”是无限制的字符串类型的记法并且在 44.1 中定义。

40.2 每个受限制的字符串类型的标记在 41.1 中规定。无限制的字符串类型的标记在 44.2 中规定。

40.3 字符串值的记法应是:

```
CharacterStringValue ::=
    RestrictedCharacterStringValue
    | UnrestrictedCharacterStringValue
XMLCharacterStringValue ::=
    XMLRestrictedCharacterStringValue
    | XMLUnrestrictedCharacterStringValue
```

“RestrictedCharacterStringValue”,和“XMLRestrictedCharacterStringValue”分别在 41.8 和 41.9 中定义。“UnrestrictedCharacterStringValue”和“XMLUnrestrictedCharacterStringValue”是无限制的字符串值的记法并且在 44.7 中定义。

41 受限制字符串类型的定义

本章定义类型的值被限制为取自字符的某些规定集的 0 个、一个或多个字符的序列。引用受限制字符串类型的记法应是“RestrictedCharacterStringType”:

```
RestrictedCharacterStringType ::=
```

- BMPString
- | GeneralString
- | GraphicString
- | IA5String
- | ISO646String
- | NumericString
- | PrintableString
- | TeletexString
- | T61String
- | UniversalString
- | UTF8String
- | VideotexString
- | VisibleString

每个“RestrictedCharacterStringType”替换项通过规定下面的内容定义：

- a) 赋给类型的标记；和
- b) 所被类型引用的名称(例如,数字串)；和
- c) 通过引用列出图形符号的表或引用编码字符集 ISO 国际登记(见所使用的具有转义序列的编码字符集的 ISO 国际登记)中的登记号或引用 GB/T 13000—2010,用于定义类型的字符集中的字符。

表 8 受限制字符串列表

引用类型的名称	通用类别号	定义登记号 ^{a)} 、表号、或 GB/T 16262.1 章	注
UTF8String	12	41.16 条	
NumericString	18	表 9	(注 1)
PrintableString	19	表 10	(注 1)
TeletexString(T61String)	20	6、87、102、103、106、107、126、144、150、153、 156、164、165、168+SPACE+DELETE	(注 2)
VideotexString	21	1、13、72、73、87、89、102、108、126、128、129、 144、150、153、164、165、 168+SPACE+DELETE	(注 3)
IA5String	22	1、6+SPACE+DELETE	
GraphicString	25	所有 G 集合+SPACE	
VisibleString(ISO646String)	26	6+SPACE	
GeneralString	27	所有 G 和所有 C 集合+SPACE+DELETE	
UniversalString	28	见 41.6	
BMPString	30	见 41.15	
注 1：类型风格、大小、颜色、强度或者其他显示特性没有意义。			
注 2：登记号 6 和 156 能代替 102 和 103 使用。			
注 3：对应这些登记号的登录提供 CCITT Rec.T.100:1988 和 Rec.ITU-T T.101 (1994)的功能。			
^{a)} 所使用的具有转义序列的编码字符集的 ISO 国际登记中列出了定义登记号。			

41.1 表 8 列出了所引用的每个受限制字符串类型的名称,赋给类型的通用类别标记的序号,定义登记号或表,或者定义文本章号,以及在必要时,标识与表中登录有关的注解。记法中定义了同义名称时,要用圆括号列出。

41.2 表 9 列出会在 NumericString 类型和 NumericString 字符抽象语法中出现的字符。

表 9 NumericString

名称	图形
Digits	0、1、...9
Space	(space)

41.3 赋予下面的对象标识符、OID 国际化资源标识符和对象描述符值来标识和描述数字串字符抽象语法,指派的对象标识符和 OID 国际化资源标识符值符合附录 D 的要求:

```
{joint-iso-itu-t asn1(1)specification(0)characterStrings(1)numericString(0)}
"/Joint-ISO-ITU-T/ASN.1/Specification/Character_Strings/Numeric_String"
```

和

"NumericString character abstract syntax"

注 1: 这个对象标识符值能用于 CHARACTER STRING 值和需要携带与这些值分开的字符串类型的标识的其他情况中。

注 2: NumericString 字符抽象语法的值可用下面来编码:

- GB/T 13000—2010 给出的编码抽象字符的规则之一。这时,用与 GB/T 13000—2010 附录 N 中的那些规则相关的对象标识符来标识字符传送语法。
- 固有类型 NumericString 的 ASN.1 编码规则。这时,字符传送语法由对象标识符值(joint-iso-itu-t asn1(1)basic-encoding(1))标识。

41.4 表 10 列出会在 PrintableString 类型和 PrintableString 字符抽象语法中出现的字符。

表 10 PrintableString

名称	图形
拉丁大写字母 Latin capital letters	A、B、...Z
拉丁小写字母 Latin small letters	a、b、...z
数字 Digits	0、1、...9
间隔 SPACE	(空)
撇号 APOSTROPHE	'
左圆括号 LEFT PARENTHESIS	(
右圆括号 RIGHT PARENTHESIS	)
正号 PLUS SIGN	+
逗号 COMMA	,
负号 HYPHEN-MINUS	-
句号 FULL STOP	.
斜线 SOLIDUS	/
冒号 COLON	:
等于记号 EQUALS SIGN	=
问号 QUESTION MARK	?

41.5 赋予下面的对象标识符、OID 国际化资源标识符和对象描述符值来标识和描述 PrintableString 字符抽象语法,指派的对象标识符和 OID 国际化资源标识符值符合附录 D 的要求:

```
{joint-iso-itu-t asn1(1)specification(0)characterStrings(1)printableString(1)}
```

"/Joint-ISO-ITU-T/ASN.1/Specification/Character_Strings/Printable_String"

和

"PrintableString character abstract syntax"

注 1: 这个对象标识符值能用于 CHARACTER STRING 值和需要携带与这些值分开的字符串类型的标识的其他情况中。

注 2: PrintableString 字符抽象语法的值可用下面方法编码:

- a) 在 GB/T 13000—2010 中给出的编码抽象字符的规则之一。这时,用与 GB/T 13000—2010 附录 N 中的那些规则相关的对象标识符来标识字符传送语法。
- b) 固有类型 PrintableString 的编码规则。这时,字符传送语法由对象标识符(joint-iso-itu-t asn1(1)basic-encoding(1))标识。

41.6 会在 UniversalString 类型中出现的字符是 GB/T 13000—2010 允许的任何字符。

41.7 使用这种类型要用到 GB/T 13000—2010 规定的一致性要求。

注: 第 42 章为 GB/T 13000—2010 的附录 A 中定义的“子集图形字符集”定义了包含这种类型许多子类型的 ASN.1 模块。

41.8 受限制字符串类型的“RestrictedCharacterStringValue”记法应是“cstring”(见 12.14),“CharacterStringList”“Quadruple”或“Tuple”。“Quadruple”仅能定义长度为 1 的字符串,而且仅用在 UniversalString、UTF8String 或 BMPString 类型的值记法中。“Tuple”仅能定义长度为 1 的字符串,且仅用在 IA5String 类型的值记法中。

```
RestrictedCharacterStringValue ::=
    cstring
  | CharacterStringList
  | Quadruple
  | Tuple
CharacterStringList ::= "{CharSyms}"
CharSyms ::=
    CharsDefn
  | CharSyms, "CharsDefn"
CharsDefn ::=
    cstring
  | Quadruple
  | Tuple
  | DefinedValue
Quadruple ::= "{Group, Plane, Row, Cell}"
Group ::= number
Plane ::= number
Row ::= number
Cell ::= number
Tuple ::= "{TableColumn, TableRow}"
TableColumn ::= number
TableRow ::= number
```

注 1: “cstring”记法仅可无歧义地用在能显示值中出现的字符的图形符号的媒体上。相反,如果媒体没有这种能力,无歧义地规定采用这种图形符号的字符串值的唯一手段是用“CharacterStringList”记法的方法,而且,只有类型是 UniversalString、UTF8String、BMPString 或 IA5String 和使用“CharsDefn”的“DefinedValue”替换项(见 42.1.2)。

注 2：第 42 章定义了指明类型 BMPString(和 UniversalString 及 UTF8String)和 IA5String 的单个字符(大小为 1 的串)的许多“valuereference”。

例如,假设当字符“Σ”不能在现有媒体中表示时,我们希望为 UniversalString 规定“abcΣdef”的值,该值也能表示如下:

```
IMPORTS BasicLatin,greekCapitalLetterSigma FROM ASN1-CHARACTER-MODULE
{joint-iso-itu-t asn1(1)specification(0)modules(0)iso10646(0)};
MyAlphabet ::= UniversalString(FROM(BasicLatin | greekCapitalLetterSigma))
mystring MyAlphabet ::= {"abc",greekCapitalLetterSigma,"def"}
```

注 3：当规定 UniversalString、UTF8String 或 BMPString 类型的值时,除非能解决有类似形状的不同图形字符出现的歧义性,否则不宜采用“cstring”记法。

例如,不宜使用下面的“cstring”记法,因为在 BASIC LATIN、CYRILLIC 和 BASIC GREEK 字母表中出现图形符号'H','O','P'和'E'并且是有歧义的。

```
IMPORTS BasicLatin,Cyrillic,BasicGreek FROM ASN1-CHARACTER-MODULE
{joint-iso-itu-t asn1(1)specification(0)modules(0)iso10646(0)};
MyAlphabet ::= UniversalString(FROM(BasicLatin | Cyrillic | BasicGreek))
mystring MyAlphabet ::= "HOPE"
```

无歧义定义 mystring 的替换项将是:

```
mystring MyAlphabet(BasicLatin) ::= "HOPE"
```

形式上,mystring 是 MyAlphabet 的子集的值引用,但是,它通过附录 C 的值映射规则用于 MyAlphabet 内这个值需要值引用的任何地方。

41.9 “XMLRestrictedCharacterStringValue”记法是:

```
XMLRestrictedCharacterStringValue ::= xmlcstring
```

对于“XMLRestrictedCharacterStringValue”,“XMLTypedValue”(见 16.2)中的“XMLValue”周围不应出现空白,除非在编码中使用该记法并且编码规则明确允许出现空白(见 ISO/IEC 8825-4:2021 的 39.3.2)。

41.10 有些字符不能直接用“xmlcstring”表示。他们应采用 12.15 中规定的转义序列表示。

注:如果受限制字符串值包含 12.15.1 中规定的非 GB/T 13000—2010 字符的字符,他们不能用“xmlcstring”表示,而且这些值不能用 XML 编码规则传送(见 ISO/IEC 8825-4:2021)。

41.11 在“CharsDefn”中的“DefinedValue”应是引用那个类型的值。

41.12 在“Plane”“Row”“Cell”产生式中的“number”应小于 256,而在“Group”产生式中,它应小于 128。

41.13 “Group”规定 UCS 编码范围中的一个组,“Plane”规定组内的平面,“Row”规定平面内的行,“Cell”规定行内的字符元。这个记法标识的抽象字符是“Group”“Plane”“Row”和“Cell”值规定的字符元的抽象字符。在所有情形中,允许的字符集可用分成子类型限制。

注:当使用像 GeneralString、GraphicString 和 UniversalString 等没有应用约束的开放-结尾字符串类型时,应用设计者需要仔细考虑一致性暗示。像 TeletexString 等有界但大的字符串类型,也需要有关一致性的详细文本。

41.14 在“TableColumn”产生式中的“number”应是 0~7 的范围,而“TableRow”产生式中的“number”应是 0~15 的范围。“TableColumn”规定符合 GB/T 2311—2000 图 1 的字符代码表的列,“TableRow”规定其中的行。当代码表包含第 0 列和第 1 列中的登记登录 1,以及第 2 列~第 7 列中的登记登录 6 时,这一记法仅用于 IA5String(见转义序列使用的编码字符集的 ISO 国际登记)。

41.15 BMPString 是有自身唯一标记和只包含 GB/T 13000—2010 的基本多文种平面(那些对应最先 64K-2 字符元的,其编码用来表示基本多文种平面之外的字符的少量字符元)中的字符的 UniversalString 的子类型。它有如下定义的相关类型:

```
UniversalString(Bmp)
```

其中 Bmp 在 ASN.1 模块 ASN1-CHARACTER-MODULE(见第 42 章)中定义为对应 GB/T 13000—2010 的附录 A 中定义的“BMP”集名称的 UniversalString 的子类型。

注 1: 由于 BMPString 是固有类型, 它没有在 ASN1-CHARACTER-MODULE 中定义。

注 2: 定义 BMPString 为固有类型是为了使没有约束的编码规则(如 BER)能用 16 位而不是 32 位编码。

注 3: 在值记法中, 所有的 BMPString 值是有效的 UniversalString 和 UTF8String 值。

41.16 UTF8String 在抽象层是和 UniversalString 同义的, 而且能用于使用 UniversalString 的地方(符合要求不同标记的规则), 但是它有不同标记, 并且是不同的类型。

注: BER 和 PER 使用的 UTF8String 的编码与 UniversalString 的不同, 而且, 对于大部分文本将少有多余的词。

42 命名字符、集合和属性类别集

本章规定包含取自 GB/T 13000—2010 的每个字符的值引用名称定义的 ASN.1 固有模块, 其中, 每个名称引用大小为 1 的 UniversalString 值。这个模块也包含为取自 GB/T 13000—2010 的每个字符集的类型引用名称的定义, 其中每个名称引用 UniversalString 类型的子集。最后, 它包含 Unicode 标准的 4.5 中列出的每个通用字符性质类别中的字符集的“typereference”名称的定义, 其中每个名称都引用 UniversalString 类型的子集。

注: 这些值可用于 UniversalString 类型及其衍生的类型的值记法。在 42.1 规定的模块中已定义的所有值和类型引用都输出, 而且一定通过采用他们的任意模块引入。

42.1 ASN.1 模块“ASN1-CHARACTER-MODULE”的规范:

该模块没有在这里全部打印, 而是规定定义它的方法。

注: 本文件基于 GB/T 13000—2010。它不适用于本文件的后续版本。定义“ASN1-CHARACTER-MODULE”的方法规范只能适用于 GB/T 13000—2010。

42.1.1 模块以下列开始:

```
ASN1-CHARACTER-MODULE{joint-iso-itu-t asn1(1)specification(0)modules(0)
    iso10646(0)}
"/Joint-ISO-ITU-T/ASN.1/Specification/Modules/ISO_10646"
DEFINITIONS ::= BEGIN
--本模块内定义的所有值引用和类型引用隐式输出,
--而且用任何模块引入。
--ISO/IEC 646 控制字符:
nul  IA5String ::= {0,0}
soh  IA5String ::= {0,1}
stx  IA5String ::= {0,2}
etx  IA5String ::= {0,3}
eot  IA5String ::= {0,4}
enq  IA5String ::= {0,5}
ack  IA5String ::= {0,6}
bel  IA5String ::= {0,7}
bs   IA5String ::= {0,8}
ht   IA5String ::= {0,9}
lf   IA5String ::= {0,10}
vt   IA5String ::= {0,11}
ff   IA5String ::= {0,12}
cr   IA5String ::= {0,13}
so   IA5String ::= {0,14}
```

```

si    IA5String::={0,15}
dle   IA5String::={1,0}
dc1   IA5String::={1,1}
dc2   IA5String::={1,2}
dc3   IA5String::={1,3}
dc4   IA5String::={1,4}
nak   IA5String::={1,5}
syn   IA5String::={1,6}
etb   IA5String::={1,7}
can   IA5String::={1,8}
em    IA5String::={1,9}
sub   IA5String::={1,10}
esc   IA5String::={1,11}
is4   IA5String::={1,12}
is3   IA5String::={1,13}
is2   IA5String::={1,14}
is1   IA5String::={1,15}
del   IA5String::={7,15}

```

42.1.2 对于 GB/T 13000—2010 中第 24 章和第 25 章中给出的图形字符(glyphs)的字符名称的每个列表中的每个登录,模块包含形式陈述:

```

<namedcharacter>BMPString::=<tablecell>
--表示字符<iso10646name>,见 GB/T 13000—2010

```

其中:

- a) <iso10646name>是从 GB/T 13000—2010 中列出的一个表衍生出来的字符名称;
- b) <namedcharacter>通过应用 42.2 中规定的程序到<iso10646name>获得的串;
- c) <tablecell>是对应列表登录的 GB/T 13000—2010 中表字符元中的图形字符。

例如:

```

latinCapitalLetterA BMPString::={0,0,0,65}
--表示字符 LATIN CAPITAL LETTER A,见 GB/T 13000—2010
greekCapitalLetterSigma BMPString::={0,0,3,163}
--表示字符 GREEK CAPITAL LETTER SIGMA,见 GB/T 13000—2010

```

42.1.3 对 GB/T 13000—2010 的附录 A 中规定的图形字符的集的名称,形式模块中包含陈述:

```

<namedcollectionstring>::=BMPString
    (FROM(<alternativelist>))
--表示字符集<collectionstring>,
--见 GB/T 13000—2010。

```

其中:

- a) <collectionstring>是在 GB/T 13000—2010 中指派的字符集的名称;
- b) <namedcollectionstring>通过应用 42.3 的程序到<collectionstring>形成的;
- c) <alternativelist>是像为 GB/T 13000—2010 规定的每个字符在 42.2 中产生的一样,通过使用 <namedcharacter>形成的。

产生的类型引用,<namedcollectionstring>,形成了有限的子集。(见附录 H 中的指南)

注:有限子集是规定子集中的字符表。它与选择的子集相反,是 GB/T 13000—2010 的附录 A 中列出的字符集加

BASIC LATIN 集。

例如(部分):

```
space BMPString      ::= {0,0,0,32}
exclamationMark BMPstring ::= {0,0,0,33}
quotationMark BMPSting ::= {0,0,0,34}
...                  --等等
tilde BMPString      ::= {0,0,0,126}
BasicLatin ::= BMPString
    (FROM(space
      | exclamationMark
      | quotationMark
      | ... --等等
      | tilde)
    )
```

--表示字符 BASIC LATIN 集,见 GB/T 13000—2010。

--本示例中的圆括号用来简化并且是“等等”的意思。

--不能在实际的 ASN.1 模块中这样使用。

42.1.4 GB/T 13000—2010 定义实现的三个级。默认情况下,由于这些类型没有对使用组合字符的限制,故在 ASN1-CHARACTER-MODULE 中定义的所有类型,除“Level1”和“Level2”外,都符合 3 级实现。“Level1”指明要求 1 级实现,“Level2”指明要求 2 级实现,而“Level3”指明要求 3 级实现。因此,ASN1-CHARACTER-MODULE 中定义了下列内容:

```
Level1 ::= BMPString(FROM(BMPString(SIZE(1)) EXCEPT CombiningCharacters))
Level2 ::= BMPString(FROM(BMPString(SIZE(1)) EXCEPT CombiningCharactersType-2))
Level3 ::= BMPString
```

注 1: “CombiningCharacters”和“CombiningCharactersType-2”是分别对应于 GB/T 13000—2010 的附录 A 中定义的“COMBINING CHARACTERS”和“COMBINING CHARACTERS B-2”的<namedcollectionstring>。

注 2: “Level1”和“Level2”将用于“IntersectionMark”(见第 50 章)之后或作为“ConstraintSpec”中唯一的约束(示例见 G.2.7.1)。

注 3: 有关本内容的更多信息见 H.2.5。

42.1.5 Unicode 标准的表 4-5 中列出的每个缩写和每个描述,以下形式的模块中包含两个陈述:

```
<categoryabbreviation> ::= UniversalString(FROM(<alternativelist>))
```

--表示具有属性的字符集

--类别<categoryabbreviation>。

```
<categorydescription> UniversalString ::= <categoryabbreviation>
```

其中

- <categoryabbreviation>是 Unicode 标准的表 4-5 中列出的字符性质的一般类别的缩写(例如, Lu 或 Nd 或 Pi);
- <categorydescription>是对相同的通用字符类别的描述,所有单词的首字母大写,去掉逗号和所有空格,去掉括号中的所有描述(例如, LetterUppercase 或 NumericDigit 或 PunctuationInitialQuote);
- 每个<categoryabbreviation>的<alternativelist>是 42.2 为 Unicode 标准的 Unicode 字符数据库(版本 3.2.0)中列出的具有相应<categoryabbreviation>的每个字符生成的<namedcharacter>名称列表。

注：字符的 Unicode 名称与该字符的〈iso10646name〉相同。

42.1.6 Unicode 标准的表 4-5 中列出的每个缩写的首字母，以下形式的模块中包含两个陈述：

〈categoryabbreviationletter〉::=UniversalString(FROM(〈alternativelist〉))

--表示具有任何类别属性的字符集；

--带有首字母〈categoryabbreviationletter〉。

〈maincategorydescription〉UniversalString::=〈categoryabbreviationletter〉

其中：

- a) 〈categoryabbreviationletter〉是 Unicode 标准的表 4-5 中列出的字符性质的一般类别的缩写的首字母(例如, L 或 N 或 P)；
- b) 〈categorydescription〉是相同的通用字符类别(例如, 字母或数字或标点符号)的描述的第一个词；
- c) 每个〈categoryabbreviationletter〉的〈alternativelist〉是 42.2 为 Unicode 标准的 Unicode 字符数据库(版本 3.2.0)中列出的具有相应〈categoryabbreviationletter〉的每个字符生成的〈namedcharacter〉名称列表。

注：字符的 Unicode 名称与该字符的〈iso10646name〉相同。

42.1.7 模块以下面的语句结束：

END

42.1.8 用户定义的, 和 42.1.3 中的示例相当的是：

BasicLatin::=BMPString(FROM(space..tld))

--表示字符 BASIC LATIN 集；

--见 GB/T 13000—2010。

42.2 〈namedcharacter〉是采用〈iso10646name〉(见 42.1.2)和应用下列算法获得的串：

- a) 〈iso10646name〉的每个大写字母转换成对应的小写字母, 除非大写字母是以 SPACE 开头, 此时, 大写字母保持不变；
- b) 每个数字和每个 HYPHEN-MINUS 保持不变；
- c) 删除每个 SPACE。

注：上面的算法, 和 GB/T 13000—2010 附录 K 中的字符命名指南一起将总是为 GB/T 13000—2010 中列出的各个字符名产生无歧义的值记法。

例如：取自 GB/T 13000—2010 的 0 行 60 字符元, 被命名为“LESS-THAN SIGN”并且有图形表达式“<”的字符能用 LESS-THAN SIGN 的“DefinedValue”引用。

42.3 〈namedcollectionstring〉是采用〈collectionstring〉和应用下列算法获得的串：

- a) GB/T 13000—2010 集名的每个大写字母转换成对应的小写字母, 除非大写字母是以 SPACE 开头或它是名字的第一个字母, 此时, 大写字母保持不变；
- b) 每个数字和每个 HYPHEN-MINUS 保持不变；
- c) 删除每个 SPACE。

例如：

- 1) 在 GB/T 13000—2010 的附录 A 中定义为 BASIC LATIN 的集有 ASN.1 类型引用：

BasicLatin

- 2) 由 BASIC LATIN 集中的字符以及 BASIC ARABIC 集构成的字符串类型能被定义为：

My-Character-String::=BMPString(FROM(BasicLatin | BasicArabic))

注：上面的结构是必需的, 因为明显的更简单结构

My-Character-String::=BMPString(FROM(BasicLatin | BasicArabic))

将只允许是整个 BASIC LATIN 或 BASIC ARABIC, 而不是二者混合的串。

43 字符的常规顺序

43.1 为了“ValueRange”分成子类型并为编码规则可使用,为 UniversalString、UTF8String、BMPString、NumericString、PrintableString、VisibleString 和 IA5String 规定了字符的常规顺序。

43.2 仅仅因为本章,字符和代码表中的字符元一一对应,不管那个字符元被赋予字符名称或者字型,也不管它是控制字符还是印刷字符、组合还是非组合字符。

43.3 抽象字符的常规顺序由 GB/T 13000—2010 的 32 位表达式中其值的常规顺序定义,常规顺序中小数字首先出现和大数字最后出现。

43.4 在“PermittedAlphabet”记法(或个别字符)内“ValueRanges”的结束点能用模块 ASN1-CHARACTER-MODULE 中定义的 ASN.1 值引用或(这时,图形符号在规范上下文和用来表示他的媒体中是无歧义的)通过给出“cstring”中的图形符号(42.1 中定义了 ASN1-CHARACTER-MODULE),或通过采用 41.8 的“Quadruple”或“Tuple”记法规定。

43.5 对 NumericString,从左到右逐步增加的常规顺序定义(见 41.2 的表 9)为:

(间隔)0 1 2 3 4 5 6 7 8 9

整个字符集精确地包含 11 个字符。“ValueRange”(或单个字符)的结束点能用“cstring”中的图形符号规定。

注:这个顺序与 GB/T 13000—2010 中 BASIC LATIN 集中对应字符的顺序一致。

43.6 对于 PrintableString,从左到右和从顶到底逐步增加的常规顺序定义(见 41.4 的表 10)为:

(SPACE)(APOSTROPHE)(LEFT PARENTHESIS)(RIGHT PARENTHESIS)(PLUS SIGN)
(COMMA)(HYPHEN-MINUS)(FULL STOP)(SOLIDUS)0123456789(COLON)(EQUAL SIGN)
(QUESTION MARK)ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz

整个字符集精确地包含 74 个字符。“ValueRange”(或单个字符)的结束点能用“cstring”中的图形符号规定。

注:这个顺序与 GB/T 13000—2010 中 BASIC LATIN 集中对应字符的顺序一致。

43.7 对于 VisibleString,字符元的常规顺序根据 ISO/IEC 646 编码(称为 ISO/IEC 646 ENCODING)定义为:

(ISO/IEC 646 ENCODING)—32

注:即,常规顺序与 ISO/IEC 646 代码表的字符元 2/0~7/14 中的字符一样。

整个字符集精确地包含 95 个字符。“ValueRange”(或单个字符)的结束点能用“cstring”中的图形符号规定。

43.8 对于 IA5String,字符元的常规顺序根据 ISO/IEC 646 编码定义为:

(ISO/IEC 646 ENCODING)

整个字符集精确地包含 128 个字符。“ValueRange”(或单个字符)的结束点能用“cstring”中的图形符号或 42.1.1 中定义的 ISO/IEC 646 控制字符值引用规定。

44 无限制字符串类型的定义

本章定义类型值是所有字符抽象语法值的类型。在 OSI 环境中,该抽象语法可能是 OSI 定义的上文集的一部分。否则,可为使用无限制字符串类型的每个实例直接引用。

注 1:抽象字符语法(及一个或多个对应的字符传送语法)可由能分配 ASN.1 OBJECT IDENTIFIERS 的任何组织定义。

注 2:利益相同产生的外形通常将确定对 CHARACTER STRING 规定实例或实例组所支持的字符抽象语法和字

符传送语法。在 OSI 应用中,它通常应包括 OSI 协议实现一致性声明中引用所支持的语法。

44.1 无限制的字符串类型(见 3.8.89)应用记法“UnrestrictedCharacterStringType”引用:

UnrestrictedCharacterStringType ::= CHARACTER STRING

44.2 该类型的标记是通用类别,编号为 29。

44.3 类型由表示下列内容的值组成:

- a) 可能但不必是 ASN.1 字符串类型的值的字符串值;和
- b) 以下的标识(单独或一起):
 - 1) 字符抽象语法;和
 - 2) 字符传送语法。

44.4 无限制字符串类型有相关类型。用该相关类型来支持其值和子类型记法。

44.5 为值定义和分成子类型的相关类型(假设是自动标记环境)是(有标准注解):

```
SEQUENCE{
  identification
    CHOICE{
      syntaxes
        SEQUENCE{
          abstract
            OBJECT IDENTIFIER,
          transfer
            OBJECT IDENTIFIER}
        --抽象和传送语法对象标识符--,
      syntax
        OBJECT IDENTIFIER
        --标识抽象和传送语法的单个对象标识符--,
      presentation-context-id
        INTEGER
        --(仅适用于 OSI 环境)
        --协商的 OSI 表示上下文标识抽象和传送语法--
      context-negotiation
        SEQUENCE{
      presentation-context-id
        INTEGER,
      transfer-syntax
        OBJECT IDENTIFIER}
      --(仅适用于 OSI 环境)
      --进程中的上下文协商、表示上下文标识符只标识抽象语法,
      --因此,应规定传送语法--
      transfer-syntax
        OBJECT IDENTIFIER
      --值的类型(例如,是 ASN.1 类型的值的规范)由应用
      --设计者固定(因此,发送者和接收者都了解),--
      --提供这一情况主要是支持 ASN.1 类型的选择-范围-加密(或其他编码转换)--
      fixed
        NULL
      --数据值是固定 ASN.1 类型的值(因此,发送者和接收者都了解)--}
  data-value-descriptor
    ObjectDescriptor OPTIONAL
    --这样提供值的类别的人可读标识--
  string-value
    OCTET STRING}
(WITH COMPONENTS{
  ...,
  data-value-descriptor ABSENT})
```

注:无限制字符串类型不允许包含与“identification”一起的“data-value-descriptor”值。然而,这里提供的相关类型的定义强调嵌入式 pdv 类型、外部类型与无限制字符串类型之间存在的共性。

44.6 正文的 36.6 和 36.7 也适用于无限制字符串类型。

44.7 值记法应是在 44.5 中定义的相关类型的值记法,其中类型 OCTET STRING 的 string-value 组件的值表示使用“identification”中规定的传送语法的编码。

UnrestrictedCharacterStringValue ::= SequenceValue

XMLUnrestrictedCharacterStringValue ::=

XMLSequenceValue

44.8 在 G.2.8 中给出了无限制字符串类型的示例。

45 第 46 章~第 48 章中定义的类型的记法

45.1 引用第 46 章~第 48 章中定义的类型的记法应是:

UsefulType ::= typereference

其中,“typereference”是第 46 章~第 48 章中用 ASN.1 记法定义的之一。

45.2 每个“UsefulType”的标记在第 46 章~第 48 章中规定。

46 通用时间

注 1: 本文件的早期版本使用了不同的文本(由于 ISO 时间标准的演变),但技术内容与本文件的第一版没有变化。

注 2: 时间类型(见第 38 章)提供了更大的灵活性,宜优先考虑。

46.1 这一类型应用下面名称引用:

GeneralizedTime

46.2 该类型的组成包括日历日期,以及:

- a) 一天中的当地时间,包括一天开始时的午夜,但不包括一天结束时的午夜,精确到:
 - 1) 小时、分钟和秒(或秒和秒的小数部分到任意小数位);或
 - 2) 小时和分钟(或分钟和分钟的小数部分到任意小数位);或
 - 3) 小时(或小时和小时的小数部分到任意小数位);或
- b) 一天的 UTC 时间,包括一天开始时的午夜,但不包括一天结束时的午夜,达到上述 a) 中列出的任何精度;或
- c) 上述 a) 中规定的当地时间,以及当地时间和 UTC 时间之间的时差。

注: 如果一天中的当地时间早于 UTC 时间,则时差组件为正。

46.3 该类型可用 ASN.1 定义如下:

GeneralizedTime ::= [UNIVERSAL 24] IMPLICIT VisibleString

带有“VisibleString”的值限制在下面字符串之一:

- a) 日历日期的规范,后跟一天中的当地时间,包括:
 - 1) 如 GB/T 7408.1—2023 的 4.1.2.2 - 基本格式中规定的,表示日历日期的字符串,后跟:

注 1: 它指定年份的四位数表示形式、月份的两位数表示形式和日期的两位数表示形式,而不使用分隔符。
 - 2) 表示一天中时间的字符串,精度为一小时、一分钟、一秒或几分之一秒(任何精度),使用逗号或句号作为十进制符号(如 GB/T 7408.1—2023 的 4.2.2.2 和 4.2.2.3 基本格式所规定);可选择后跟:
 - 3) 如果省略秒,则为分钟的小数位,如果省略分钟和秒,则为小时的小数位(如 GB/T 7408.1—2023 的 4.2.2.4 中所规定);或

注 2: GB/T 7408.1—2023 规定使用逗号或句号作为十进制符号。不存在其他分隔符。推荐在任何给定的 ASN.1 规范中,始终用逗号或句号作为十进制符号。

- b) 日历日期和 UTC 时间的规范,由上述 a)中的字符后跟大写字母 Z;或
- c) 日历日期、当地时间以及 GB/T 7408.1—2023 中规定的当地时间与 UTC 时间之间精确时差的规范,如果时差是整数个小时,则可选择省略分钟组件。

注 3: ASN.1 规范编码规则的早期工作假设没有实际的精确性概念,因此可能用 3.000 秒组件表示的抽象值与用 3 秒组件表示的抽象值被视为相同,并禁止在规范编码小数部分使用尾随 0,及禁止省略秒或分和秒。它还仅支持使用 UTC 时间,而不支持当地时间或带有时差组件的当地时间。为了向后兼容,在 ASN.1 标准的后续版本中没有修改这一点。TIME 类型(2004 年引入 ASN.1)认可抽象值能够具有相关的精确性,并且(例如)秒以 3.000 和 3 的形式表示会产生不同的抽象值,并且当地时间和 UTC 规范表示不同的抽象值。TIME 的规范编码规则对其抽象值的全部范围进行编码,因此在新规范中使用 TIME 可能优于使用 GeneralizedTime。

在情况 c)中,在情况 a)中形成的字符串部分表示当地时间(t_1),并且(有符号的)时差(t_2)能够确定 UTC。如果 t_2 为正数,则当地时间早于 UTC。因此,我们能将 UTC 确定为:

UTC 是 $t_1 - t_2$

例如:

情况 a):

"19851106210627.3"表示当地时间 1985 年 11 月 6 日下午 9 时 6 分 27.3 秒。

情况 b):

"19851106210627.3Z"是上述的协调世界时。

情况 c):

"19851106210627.3-0500"表示例 a)中的当地时间,协调世界时为 1985 年 11 月 7 日上午 2 时 6 分 27.3 秒。

情况 d):

"198511062106.456"表示当地时间 1985 年 11 月 6 日下午 9 时 6.456 分。

情况 e):

"1985110621.14159"表示当地时间 1985 年 11 月 6 日下午 9 时 0.14159 小时。

46.4 标记应如 46.3 中定义。

46.5 值记法应是 46.3 中定义的“VisibleString”的值记法。

47 世界时间

47.1 该类型应用下列名称引用:

UTCTime

47.2 该类型由表示下列内容的值组成:

- a) 日历日期;和
- b) 精确到一分或一秒的时间;和
- c) (可选择)与协调世界时不同的当地时间。

47.3 该类型用 ASN.1 定义如下:

UTCTime ::= [UNIVERSAL 23] IMPLICIT VisibleString

带有“VisibleString”的值限制为下面平行放置的字符串:

- a) 六位数字 YYMMDD,其中,YY 是公元年号的后两位数,MM 是月份(一月记为 01),DD 是月中的日(01~31);和
- b) 下面二者之一:
 - 1) 四位数字 hhmm,其中 hh 是小时(00~23),mm 是分(00~59);或
 - 2) 六位数字 hhmmss,其中,hh 和 mm 如上面 1)中的时间,ss 是秒(00~59);和

c) 下面二者之一:

1) 字符 Z;或

2) 字符+或-之一,后随 hhmm,其中 hh 是小时,mm 是分。

上面 b)中的替换项允许改变时间规范内的精确度。

在替换项 c)1)中,时间是协调世界时。在替换项 c)2)中,上面 a)和 b)规定的时间(t_1)是当地时间;上面 C)2)规定的时差(t_2)能使协调世界时按下列来确定:

协调世界时是 $t_1 - t_2$

例 1:如果当地时间是 1982 年 1 月 2 日上午 7 时,而协调世界时是 1982 年 1 月 2 日中午 12 时,UTCTime 值是二者之一:

——"8201021200Z";或

——"8201020700-0500"。

例 2:如果当地时间是 2001 年 1 月 2 日上午 7 时,而协调世界时是 2001 年 1 月 2 日中午 12 时,UTCTime 值是二者之一:

——"0101021200Z";或

——"0101020700-0500"。

47.4 标记应如 47.3 中定义。

47.5 值记法应是 47.3 中定义的“VisibleString”的值记法。

48 对象描述符类型

48.1 该类型应用下列名称引用:

ObjectDescriptor

48.2 该类型由用来描述一个对象的人们可读的文本组成。文本不是对象的无歧义标识,但是不同对象的相同文本并不常见。

注:推荐将类型 OBJECT IDENTIFIER 值赋给对象的权威机构也宜将类型 ObjectDescriptor 的值赋给那个对象。

48.3 该类型用 ASN.1 定义如下;

ObjectDescriptor ::= [UNIVERSAL 7] IMPLICIT GraphicString

“GraphicString”含有描述对象的文本。

48.4 标记应如 48.3 中定义。

48.5 值记法应是 48.3 中定义的“Graphicstring”的值记法。

49 受约束类型

49.1 “ConstrainedType”记法允许对(双亲)类型加以约束,约束其值集为双亲类的某些子类型或(在集合或序列类型内)规定组件关系适用于双亲类型的值,以及同一集合或序列值中某些其他组件的值。它也允许有与约束有关的例外标识符。

ConstrainedType ::=

Type Constrain

| TypeWithConstraint

在第一个替换项中,双亲类型是“Type”,而且该约束由 49.6 中定义的“Constraint”规定。第二个替换项在 49.5 中定义。

49.2 当“Constraint”记法紧接单一集合或单一序列类型记法之后时,它适用于(最里面的)单一集合或单一序列记法中的“Type”,而不适合单一集合或单一序列类型。

注：例如，下面的约束“(SIZE(1..64))”适用于 VisibleString，但不适用于 SEQUENCE OF：

NamesOfMemberNations ::= SEQUENCE OF VisibleString(SIZE(1..64))

49.3 当“Constraint”记法紧接精选类型记法之后时，它将适用于选择类型，而不适用于所选替换项的类型。这样的约束将被忽略(见 30.2)。

注：在下面的示例中，约束(WITH COMPONENTS{...,a ABSENT})适用于 CHOICE 类型 T，但不适用于所选的 SEQUENCE 类型，而且不影响 V 的值。

```
T ::= CHOICE{
  a SEQUENCE{
    a INTEGER OPTIONAL,
    b BOOLEAN
  },
  b NULL
}
V ::= a<T(WITH COMPONENTS{...,a ABSENT})
```

49.4 当“Constraint”记法紧接“PrefixedType”记法之后时，全部记法的解释一样，无论“PrefixedType”或“Type”是否认为是双亲类型。

49.5 作为 49.2 中规定的解释的结果，提供特定的记法来允许约束适用于单一集合或单一序列类型。这就是“TypeWithConstraint”：

```
TypeWithConstraint ::=
  SET Constraint OF Type
| SET SizeConstraint OF Type
| SEQUENCE Constraint OF Type
| SEQUENCE SizeConstraint OF Type
| SET Constraint OF NamedType
| SET SizeConstraint OF NamedType
| SEQUENCE Constraint OF NamedType
| SEQUENCE SizeConstraint OF NamedType
```

在第一个和第二个替换项中，双亲类型是“SET OF Type”，而在第三个和第四个中，它是“SEQUENCE OF Type”。在第五个和第六个替换项中，双亲类型是“SET OF NamedType”，而在第七个和第八个中，它是“SEQUENCE OF NamedType”。在第一个、第三个、第五个和第七个替换项中，约束是“Constraint”(见 49.6)，而在第二个、第四个、第六个和第八个中，它是“SizeConstraint”(见 51.5)。

注：尽管“Constraint”替换项包括了对应的“SizeConstraint”替换项，由于历史的原因提供了“SizeConstraint”替换项。

49.6 由记法“Constraint”规定的约束：

```
Constraint ::= ("ConstraintSpec ExceptionalSpec")
ConstraintSpec ::=
  SubtypeConstraint
| GeneralConstraint
```

“ExceptionalSpec”在 53 中定义。除非与“extension marker”(见第 52 章)一起使用，否则，它仅表示“ConstraintSpec”是否包括“DummyReference”(见 GB/T 16262.4—2025 的 8.3)或者是“UserDefinedConstraint”(见 GB/T 16262.3—2025 的第 9 章)的事件。在 GB/T 16262.3—2025 的 8.1 中定义了“GeneralConstraint”。

49.7 记法“SubtypeConstraint”是通用“ElementSetSpecs”记法(见第 50 章)：

```
SubtypeConstraint ::= ElementSetSpecs
```


在这里的上下文中,元素是双亲类型的值(元素集的支配者是双亲类型)。在集中至少应有一个元素。

50 元素集规范

50.1 在某些记法中,能够规定某些标识类型或信息客体类别(支配者)的元素集。这时使用记法“ElementSetSpec”:

```

ElementSetSpecs ::=
    RootElementSetSpec
    | RootElementSetSpec, "... "
    | RootElementSetSpec, "... ", "AdditonalElementSetSpec"
RootElementSetSpec ::= ElementSetSpec
AdditonalElementSetSpec ::= ElementSetSpec
ElementSetSpec ::= Unions
    | ALL Exclusions
Unions ::= Intersections
    | UElms UnionMark Intersections
UElms ::= Unions
Intersections ::= IntersectionElement
    | IElems IntersectionMark IntersectionElements
IElems ::= Intersections
IntersectionElements ::= Elements | Elems Exclusions
Elems ::= Elements
Exclusions ::= EXCEPT Elements
UnionMark ::= "|" | UNION
IntersectionMark ::= "^" | INTERSECTION

```

注1: 脱字符“^”和单词 INTERSECTION 同义。字符“|”和单词 UNION 同义。推荐,作为格式问题,字符或单词能用于整个用户规范。EXCEPT 能以任一种形式使用。

注2: 从最高到最低的优先顺序是: EXCEPT、“^”、“|”。注意,规定了 ALL EXCEPT 以便它不能用没有把“ALL EXCEPT xxx”圆括号括起来的其他约束打断。

注3: 任何有“Elements”的地方,会出现没有圆括号的约束[例如: INTEGER(1..4)]或圆括号括起来的子类型约束[例如: INTEGER((1..4|9))]

注4: 注意,两个“EXCEPT”运算符一定用“|”、“^”、“(或)”分开,因此,(A EXCEPT B EXCEPT C)是不允许的。它一定改成((A EXCEPT B) EXCEPT C)或(A EXCEPT (B EXCEPT C))。

注5: 注意,((A EXCEPT B) EXCEPT C)与(A EXCEPT (B|C))是相同的。

注6: “ElementSetSpecs”引用的元素是“RootElementSetSpec”和“AdditonalElementSetSpec”(出现时)引用的元素的联合。

注7: 当元素是信息客体时(即,支配者是信息客体类别),使用如 GB/T 16262.2—2025 的 12.10 中定义的记法“ObjectSetElements”。

50.2 形成集的元素是:

- 如果选择“ElementSetSpecs”的第一个替换项,“Unions”中规定的那些内容[见 b)],除了“Exclusions”的“Elements”记法中规定的那些内容外,支配者的所有其他元素;
- 如果选择“Unions”的第一个替代项,那么是“Intersections”[见 c)]中规定的那些内容,在

“UElems”或“Intersections”中至少规定一次；

- c) 如果选择“Intersections”的第一个替代项,那么是“IntersectionElements”[见 d)]中规定的那些内容,否则也是由“IntersectionElements”规定的“IElems”规定的那些内容；
- d) 如果选择“IntersectionElements”的第一个替代项,则为“Elements”中规定的那些内容,除了在“Exclusions”中规定外,应在“UElems”中规定。

50.3 当元素是信息客体(即,支配者是信息客体类别)时,不应使用“ElementSetSpec”的第二个替换项。

50.4 如果下面的条件有效,定义值集为可扩展的：

- a) 对于“ElementsSetSpecs”：在较外面的层有扩展标志；
- 注：即使双亲类的所有值包含在新受限类型的根中这也适用。
- b) 对于“Unions”：至少一个“UElems”是可扩展的；
- c) 对于“Intersections”：至少一个“IElems”是可扩展的；
- d) 对于“Exclusions”：EXCEPT 之前的元素集是可扩展的。

否则,值的集不是可扩展的(也见 I.4)。

50.5 如果值的集是可扩展的,如 50.2 中规定的,通过只用集合运算中包含的值集的根值形成集合运算能确定根值。通过采用由扩展附加增加的根值形成值集能确定扩展附加,对集合运算中包含的每个值集,随后排除确定为根值的值。

50.6 “Elements”记法定义为：

```
Elements ::=
    SubtypeElements
    | ObjectSetElements
    | ("ElementSetSpec")
```

本记法规定的元素是：

- a) 如果使用“SubtypeElements”替换项,则在下面第 51 章描述。该记法应仅用于支配者是类型,而且包括的实际类型应进一步约束记法的可能性时,在这一上下文中,支配者称为双亲类型；
- b) 如果采用“ObjectSetElements”记法,则在 GB/T 16262.2—2025 的 12.10 中描述,该记法仅用于当支配者是信息客体类别时；
- c) 如果采用第三种替换项,是“ElementSetSpec”规定的那些内容。

50.7 当支配类型是不可扩展的,形成子类型约束或值集内的集合运算时,集合运算中只采用支配类型的抽象值。这时,集合运算中所用的所有值记法(包括值引用)实例要求引用支配类型的抽象值。要求范围约束的结束点引用支配类型的值,而整个范围规范引用全部(且只有)是支配类型抽象值范围中的那些值。

50.8 当支配类型是可扩展的,形成子类型约束或值集内的集合运算时,集合运算中只采用支配类型的扩展根中的抽象值。这时,集合运算中所用的所有值记法(包括值引用)实例要求引用支配类型的扩展根的抽象值。要求范围约束的结束点引用在支配类型的扩展根中出现的值,而整个范围规范引用全部(且只有)是支配类型扩展根内范围中的那些值。

50.9 当形成含有信息客体集合的集合运算时,所有信息客体都用于集合运算中。如果贡献给集合运算的任何信息客体集合是可扩展的,或如果在“ElementSetSpec”最外层有扩展标志,则集合运算的结果是可扩展的。

50.10 如果子类型约束应用于不可扩展的双亲类型,则其中使用的值记法不应引用不是双亲类型抽象值的值。

50.11 如果子类型约束通过可扩展约束的应用连续地应用到可扩展的双亲类型上,其中用的值记法不

应引用不在双亲类型的扩展根中的值。只要约束应用于没有扩展标志和可能扩展附加的双亲类型,第二个(连续应用的)约束的结果定义为是一样的。

例如:
Foo::=INTEGER(1..6,...,73,..80)
Bar::=Foo(73)--非法的
foo Foo::=73 --合法,因为它是 Foo 的值记法,不是约束的一部分。

由于 73 不在 Foo 的扩展根中,Bar 是非法的。如果 73 已经在 Foo 的扩展根中,示例将是合法的,而且 Bar 将约束 73 的单个值。

注:本条仅适用于“SubtypeConstraint”。如果将“GeneralConstraint”(见 GB/T 16262.3—2025 的 8.1)应用于双亲类型,则该双亲类型的可扩展性不受影响。

51 子类型元素

51.1 概述

对“SubtypeElements”提供了许多不同形式的记法。他们标识如下,而且将他们的语法和语义在下面各条中定义。表 11 和表 12 概括了哪种记法适用于哪种双亲类型。“SubtypeElements”未出现在其中一个表中意味着对应的子类型元素不能应用于该表中列出的任何双亲类型。

SubtypeElements::=
 SingleValue
 | ContainedSubtype
 | ValueRange
 | PermittedAlphabet
 | SizeConstraint
 | TypeConstraint
 | InnerTypeConstraints
 | PatternConstraint
 | PropertySettings
 | DurationRange
 | TimePointRange
 | RecurrenceRange

表 11 “SubtypeElements”对 Time 类型以外的类型的适用性

类型(或通过置标记或分成子类型从该类型派生的)	Single value	Contained subtype	Value range	Size constraint	Permitted alphabet	Type constraint	Inner subtyping	Pattern constraint
Bit string	是	是	否	是	否	否	否	否
Boolean	是	是	否	否	否	否	否	否
Choice	是	是	否	否	否	否	是	否
Embedded-pdv	是	否	否	否	否	否	是	否
Enumerated	是	是	否	否	否	否	否	否
External	是	否	否	否	否	否	是	否
Instance-of	是	是	否	否	否	否	是	否
Integer	是	是	是	否	否	否	否	否

表 11 “SubtypeElements”对 Time 类型以外的类型的适用性 (续)

类型(或通过置标记或分成子类型从该类型派生的)	Single value	Contained subtype	Value range	Size constraint	Permitted alphabet	Type constraint	Inner subtyping	Pattern constraint
Null	是	是	否	否	否	否	否	否
Object class field type	是	是	否	否	否	否	否	否
Object descriptor	是	是	否	是	是	否	否	否
Object identifier	是	是	否	否	否	否	否	否
Octet string	是	是	否	是	否	否	否	否
OID internationalized resource identifier	是	是	否	否	否	否	否	否
Open type	否	否	否	否	否	是	否	否
Real	是	是	是	否	否	否	是	否
Relative object identifier	是 ^{b)}	是 ^{b)}	否	否	否	否	否	否
Relative OID internationalized resource identifier	是 ^{b)}	是 ^{b)}	否	否	否	否	否	否
Restricted character string types	是	是	是 ^{a)}	是	是	否	否	是
Sequence	是	是	否	否	否	否	是	否
Sequence-of	是	是	否	是	否	否	是	否
Set	是	是	否	否	否	否	是	否
Set-of	是	是	否	是	否	否	是	否
GeneralizedTime and UTCTime types	是	是	否	否	否	否	否	否
Unrestricted character string type	是	否	否	是	否	否	是	否
^{a)} 只允许在 BMPString、IA5String、NumericString、PrintableString、VisibleString、UTF8String 和 UniversalString 的“PermittedAlphabet”内。 ^{b)} 约束或值集中所有相对对象标识符和相对 OID 国际化资源标识符类型或值的开始节点应与支配者的开始节点相同。								

表 12 “SubtypeElements”对 Time 类型的适用性

类型(或 通过置标 记或分成 子类型从 该类型派 生的)	Single value	Contained subtype	Property settings	Duration range	Time point range	Recurrence range	Inner subtyping
Time type	是	是	是	是	是	是	(注解)
注：仅当双亲类型的所有抽象值都具有属性设置“Basic=Interval Interval-type=D”时才允许(见 38.4.4)。							

51.2 单值

51.2.1 “SingleValue”记法应是：

SingleValue::=Value

其中,“Value”是双亲类型的值记法。

51.2.2 “SingleValue”规定由“Value”规定的双亲类型的单值。

51.3 包含子类型

51.3.1 “ContainedSubtype”记法应是：

ContainedSubtype::=Includes Type

Includes::=INCLUDES | empty

当“ContainedSubtype”中的“Type”是空类型的记法时,不应使用“Includes”产生式的“empty”替换项。

51.3.2 “ContainedSubtype”规定也在“Type”根中的双亲类型的根中的所有值。要求“Type”从作为双亲类型的相同固有类型派生而来。

51.3.3 包含子类型约束中使用的可扩展“Type”引用的值集不从“Type”中继承扩展标志。忽略不在那一类型扩展根中的“Type”中的任何值,而且,不贡献给受约束类型的值。

注：使用可扩展“Type”本身不会使受约束类型扩展。

51.4 值范围

51.4.1 “ValueRange”记法应是：

ValueRange::=LowerEndpoint".."UpperEndpoint

51.4.2 “ValueRange”规定通过规定范围结束点的值指定的值范围内的值。这一记法仅适用于整数类型、某些受限制字符串类型(只有 IA5String、NumericString、PrintableString、VisibleString、BMPString、UniversalString 和 UTF8String)的“PermittedAlphabet”和实数类型。要求“ValueRange”中规定的所有值是在双亲类型根中。

注：为了分成子类型,NOT-A-NUMBER 超过所有实值,PLUS-INFINITY 超过除 NOT-A-NUMBER 之外的所有实值,负 0 超过所有负实值并且小于正 0,MINUS-INFINITY 小于所有实值。其他将使用正常的数学排序。

51.4.3 范围的每个结束点是封闭(这时规定了那个结束点)或者是开放(这时没有规定结束点)的。在开放时,结束点规范包含一个小于符号("<")：

LowerEndpoint::=LowerEndValue | LowerEndValue"<"

UpperEndpoint::=UpperEndValue | ">UpperEndValue

51.4.4 也可不规定结束点,这时从那个方向延伸到双亲类型允许的范围:

LowerEndValue::=Value | MIN

UpperEndValue::=Value | MAX

注:当 ValueRange 用作“PermittedAlphabet”约束时,“LowerEndpoint”和“UpperEndpoint”的大小应为 1。

51.5 大小约束

51.5.1 “SizeConstraint”的记法应是:

SizeConstraint::=SIZE Constraint

51.5.2 “SizeConstraint”只适用于位串类型、八位位组串类型、字符串类型、单一集合类型或单一序列类型。

51.5.3 “Constraint”为被规定值的长度规定允许的整数值,并采用适用于下列双亲类型的任何约束形式:

INTEGER(0..MAX)

“Constraint”应用“SubtypeConstraint”替代“ConstraintSpec”。

51.5.4 测量单位依赖于双亲类型,如下:

类型	测量单位
位串	位
八位位组串	八位位组
字符串	字符
单一集合	组件值
单一序列	组件值

本条中为确定字符串值大小规定的字符数计数应清楚地与八位位组计数区别开来。字符计数应根据用于类型的,特别是与引用标准、表或能在这种定义中出现的登记中的登记数有关系的字符集的定义来解释。

51.6 类型约束

51.6.1 “TypeConstraint”记法应是:

TypeConstraint::=Type

51.6.2 这一记法仅适用于开放类型记法,而且约束开放类型为“Type”的值。

51.7 允许字母表

51.7.1 “PermittedAlphabet”记法应是:

PermittedAlphabet::=FROM Constraint

51.7.2 “PermittedAlphabet”规定能用双亲串的子字母表构成的全部值。这一记法仅能用于受限制字符串类型。

51.7.3 “Constraint”应使用“ConstraintSpec”的“SubtypeConstraint”替换项。该“SubtypeConstraint”中的每个“SubtypeElements”应是“SingleValue”“ContainedSubtype”“ValueRange”和“SizeConstraint”四个替换项之一。子字母表精确地包括那些出现在“Constraint”允许的双亲串类型的一个或多个值中的字符。

51.7.4 如果“Constraint”是可扩展的,那么允许字母表约束选择的值集是可扩展的。根中的值集是“Constraint”的根允许的那些,而且扩展附加是根与“Constraint”扩展附加一起允许的那些,但不包括已经在根中的那些值。

51.8 内部子类型

51.8.1 “InnerTypeConstraints”记法应是：

```
InnerTypeConstraints ::=
    WITH COMPONENT SingleTypeConstraint
    | WITH COMPONENTS MultiTypeConstraints
```

51.8.2 “InnerTypeConstraints”只规定那些满足约束出现的集合的值和/或双亲类型组件的值。除非满足明显或隐含的所有约束(见 51.8.7),否则,不规定双亲类型的值。这一记法能适用于单一集合、单一序列、集合、序列和选择类型。

注：通过 COMPONENT OF 转换来忽略适用于集合或序列类型的“InnerTypeConstraints”(见 25.5 和 27.2)。

51.8.3 如果“InnerTypeConstraints”包含“GeneralConstraint”(见 49.6),那么它只能(直接或间接)用作产生式“ElementSetSpecs”(见第 50 章)和/或“ObjectSetSpec”的第一个替换项的一部分(见 GB/T 16262.2—2025 的 12.3)和产生式“ElementSetSpec”“Unions”“Intersections”和“IntersectionElements”的第一个替换项的一部分(见第 50 章)。

51.8.4 对于按照单个其他(内部)类型(单一集合和单一序列)定义的类型,提供了采用子类型值规范形式的约束。它的记法是“SingleTypeConstraint”：

```
SingleTypeConstraint ::= Constraint
```

“Constraint”定义单个其他(内部)类型的子类型。如果且只有每个内部值属于“Constraint”用于内部类型获得的子类型,才规定双亲类型的值。

51.8.5 对于按照多个其他(内部)类型(选择、集和序列)定义的类型,提供了这些内部类型的许多约束。它的记法是“MultipleTypeConstraints”：

```
MultipleTypeConstraints ::=
    FullSpecification
    | PartialSpecification
FullSpecification ::= "{" "TypeConstraints" "}"
PartialSpecification ::= "{" "...", "TypeConstraints" "}"
TypeConstraints ::=
    NamedConstraint
    | NamedConstraint, "TypeConstraints"
NamedConstraint ::=
    Identifier ComponentConstraint
```

51.8.6 “TypeConstraints”包含对双亲类型的成分类型的约束列表。对于序列类型,约束一定按顺序出现。约束作用的内部类型用其标识符标识。对于给出的组件,最多应有一个“NamedConstraint”。

51.8.7 “MultipleTypeConstraints”由“FullSpecification”或者“PartialSpecification”组成。当使用“FullSpecification”时,能被约束为缺失(见 51.8.10)且没有明显列出的所有内部类型中有“ABSENCE”的隐式出现约束。采用“PartialSpecification”时,没有隐式约束且能从列表中忽略任何内部类型。

51.8.8 特定的内部类型可按照它的出现(在双亲类型的值中)、它的值或两者约束。记法是“ComponentConstraint”：

```
ComponentConstraint ::= ValueConstraint PresenceConstraint
```

51.8.9 内部类型值的约束用记法“ValueConstraint”表示：

```
ValueConstraint ::= Constraint | empty
```

如果且只有内部值属于作用于内部类型的“Constraint”规定的子类型时,双亲类型的值满足这一约束。

51.8.10 内部类型出现的约束应用记法“PresenceConstraint”表示：

PresenceConstraint ::= PRESENT | ABSENT | OPTIONAL | empty

这些候选项的意义以及允许他们的情形在 51.8.10.1~51.8.10.3 中定义。

51.8.10.1 如果双亲类型是序列或集合，标记为 OPTIONAL 的成分类型可约束为 PRESENT（这时，如果也只有对应的组件值出现才满足约束）或者为 ABSENT（这时，如果也只有对应的组件值缺失才满足约束）或者为“OPTIONAL”（这时，对应的组件值出现上没有加以约束）。

51.8.10.2 如果双亲类型是选择，成分类型能被约束为 ABSENT（此时，如果也只有对应的成分类型没有用在值中才满足约束），或 PRESENT（此时，如果也只有对应的成分类型用在值中才满足约束）；在“MultiTypeConstraints”中最多应有一个 PRESENT 关键字。

注：解释示例见 G.5.6。

51.8.10.3 空“PresenceConstraint”的意义依赖于采用“FullSpecification”还是“PartialSpecification”：

- a) 在“FullSpecification”中，它相当于对标记 OPTIONAL 的集合或序列组件的 PRESENT 约束，且不强加进一步的约束；
- b) 在“PartialSpecification”中，不强加约束。

51.9 模式约束

51.9.1 “PatternConstraint”记法应是：

PatternConstraint ::= PATTERN Value

51.9.2 “Value”应是包含附录 A 中定义的 ASN.1 常规表达式的类型 UniversalString（或引用这种字符串）的“cstring”。“PatternConstraint”选择满足 ASN.1 常规表达式的双亲类型的那些值。整个值应满足整个 ASN.1 常规表达式，即，“PatternConstraint”不选择其开头字符与（整个）ASN.1 常规表达式匹配但包括进一步结尾字符的值。

注：“Value”形式上定义为类型 UniversalString 的值，但是类型 UniversalString 和 UTF8String 的值集是相同的（见 41.16）。因此，全部相当的定义可能已经说明“Value”是类型 UTF8String 的值。

51.10 属性设置

51.10.1 “PropertySettings”记法应是：

PropertySettings ::= SETTINGS simplestring

51.10.2 “simplestring”的内容应是“PropertySettingsList”：

PropertySettingsList ::=

PropertyAndSettingPair

| PropertySettingsList PropertyAndSettingPair

PropertyAndSettingPair ::= PropertyName = "SettingName

PropertyName ::= psname

SettingName ::= psname

51.10.3 “PropertyName”应为表 6 第 1 列中列出的时间属性名称之一，并且应在“PropertySettingsList”中最多出现一次。

51.10.4 “PropertyAndSettingPair”的“SettingName”应是表 6 的第 2 列中包含（在第 1 列中）该“PropertyAndSettingPair”的“PropertyName”的行中列出的属性设置名称之一。

51.10.5 当且仅当抽象值满足所有“PropertyAndSettingPair”的以下条件时，才应将抽象值包含在子类型中。任何一个：

- a) 抽象值没有“PropertyName”的属性设置（对于给定“PropertyName”具有属性设置的抽象值，见表 6 的第 2 列和第 3 列）；或

b) 抽象值具有与“SettingName”相同的属性设置。

注：为了提高人可读性，推荐(但不一定)将 Basic 时间属性的设置始终作为第一个“PropertyAndSettingPair”。

示例：TIME(SETTINGS “Midnight=Start”)将生成 TIME 类型的子集，其中存在所有抽象值(包括仅表示日期的那些)，但具有属性设置“Midnight=End”的那些除外。

51.10.6 TIME 类型的所有抽象值都具有 Basic 时间属性的设置(对于其他时间属性则不然)。为了防止“PropertyAndSettingPair”对抽象值的结果集没有影响的误导性记法，对“PropertyName”设置了一些限制，这些限制能与 Basic 时间属性的特定设置一起使用。这些限制在表 13 中列出。

注：表 13 不是防止使用“PropertyAndSetting”对的详尽规则集，其中一些是多余的(这通常不是非法的)。

表 13 使用具有 Basic 属性设置的属性名称的限制

Basic 属性设置	具有此 Basic 属性设置的禁止属性名称
Date	Time, Local-or-UTC, Midnight, Interval-type, SE-point, Recurrence
Time	Date, Year, Interval-type, SE-point, Recurrence
Date-Time	Interval-type, SE-point, Recurrence
Interval	Recurrence
Rec-Interval	无限制

51.11 时间长度范围

51.11.1 “DurationRange”子类型记法应是：

DurationRange ::= ValueRange

51.11.2 “ValueRange”中的两个“Value”都应标识子类型中存在的时间抽象值(通过值记法或通过值引用)：

TIME(SETTINGS “Basic=Interval Interval-type=D”)

51.11.3 “ValueRange”中的两个“Value”都应使用以下任一方式规定时间长度：

- a) 相同的单个时间组件具有相同的精度(没有小数部分，或小数部分中的位数相同)；或
- b) 除了最低有效时间组件(可能具有不同的值，但应具有相同的精度)之外，具有相同值的多个时间组件。

51.11.4 选定的时间长度抽象值是以下这些：

- a) “ValueRange”中两个“Value”的相同时间组件具有相同的值；和
- b) 在“ValueRange”中两个“Value”的最低有效时间组件的规定范围内；和
- c) 与“ValueRange”中两个“Value”的最低有效时间组件具有相同的精度。

注：这提供了使用内部子类型(见 38.4.4)作为规定时间长度的一个替换项，该时间长度只能使用单个时间组件规定的精度。

示例：TIME(“PT2M0.000S”..“PT2M59.000S”)定义了一个 TIME 子类型，该子类型仅包含表示 2 分钟和 0 到 59 秒时间长度的抽象值，精度为 1 毫秒。

51.12 时间点范围

51.12.1 “TimePointRange”记法应是：

TimePointRange ::= ValueRange

51.12.2 “ValueRange”中的两个“Value”都应标识子类型中存在的时间抽象值(值记法或值引用)：

TIME((SETTINGS"Basic=Date")
 | (SETTINGS"Basic=Time")
 | (SETTINGS"Basic=Date-Time"))

51.12.3 “ValueRange”中的两个“Value”对于除 Midnight 时间属性外的所有时间属性都应具有相同的设置。

51.12.4 如果“ValueRange”中的两个“Value”具有属性设置“Local-or-UTC = LD”，那么这两个“Value”中的时差应相同。

注：这允许使用子类型，例如：

TIME("00:00".. "09:00")或 TIME("21:00".. "24:00")。

51.12.5 此子类型记法从双亲类型中选择那些对所有时间属性 (Midnight 时间属性除外) 设置与“ValueRange”中“Value”设置相同的抽象值 (如果“Value”中有 Local-or-UTC = LD 属性设置，则时差相同)，且其值在规定的“ValueRange”内 (见 51.4)。

注：要求所有相关抽象值对所有时间属性具有相同的设置 (Midnight 时间属性除外)，并且如果它们具有时差，则是相同的时差，确保它们都使用相同的时间刻度，因此它们之间存在顺序关系。

51.13 重复范围

51.13.1 “RecurrenceRange”记法应是：

RecurrenceRange::=ValueRange

51.13.2 “ValueRange”中的两个“Value”都应为整数值。

51.13.3 仅出于排序的目的，属性设置为“Recurrence=Unlimited”的时间值应被视为规定无限次重复 (MAX 的整数值)。

51.13.4 此子类型记法从双亲类型中选择那些也存在于子类型中的抽象值：

TIME(SETTINGS "Basic=Rec-interval")

并且具有在规定的“ValueRange”内的多个重复数字 (见 51.4)。

52 扩展标志

注：像一般的约束记法一样，扩展标志不影响 ASN.1 的某些编码规则，例如，基本编码规则，但是对某些有影响，例如，紧缩编码规则。它对用 ECN 定义的编码的影响由 ECN 规范确定。

52.1 扩展标志，省略号，是希望扩展附加的指示。它不宜说明怎样处理这种附加而是它们在解码期间不应当错误处理。

52.2 扩展标志和例外标识符一起使用 (见第 53 章) 是希望扩展附加的指示，并且如果约束违规时提供标识应采取的行动的方法。该记法宜用在那些使用储存和向前或者任何其他中继形式的情形，以指明 (例如) 任何未识别的扩展附加回到可重新编码和中继的应用。

52.3 包括可扩展子类型约束、值集或信息客体集合的集合运算结果在第 50 章中规定。

52.4 如果用可扩展约束定义的类型在“ContainedSubtype”中引用，则新定义的类型不继承扩展标志或任何他的扩展附加 (见 51.3.3)。能通过在其“ElementSetSpecs”的最外层包含扩展标志使新定义的类型可扩展 (也见 50.4)。例如：

A::=INTEGER(0..10,...,12) --A 是可扩展的。

B::=INTEGER(A) --B 是不可扩展的且约束为 0-10。

C::=INTEGER(A,...) --C 是可扩展的且约束为 0-10。

52.5 如果使用可扩展约束定义的类型进一步使用“ElementSetSpecs”进行约束，则产生的类型既不继承扩展标志也不继承可在前面的约束中出现的任何扩展附加 (见 50.11)。例如：

A ::= INTEGER(0..10,...) --A 是可扩展的。

B ::= A(2..5) --B 是不可扩展的。

C ::= A --C 是可扩展的。

52.6 不应出现约束为缺失的集、序列或选择类型的组件,无论集、序列或选择类型是否是可扩展类型。

注: 内部类型约束不影响可扩展性。

例如:

```
A ::= SEQUENCE{
    a  INTEGER
    b  BOOLEAN OPTIONAL,
    ...
}
```

B ::= A(WITH COMPONENTS{b ABSENT})--B 是可扩展的,但是 b 不应以它的任何值出现。

52.7 当本文件要求不同标记(见 25.6~25.7、27.3 和 29.3)时,进行标记唯一性检查前应概念性应用下面的转换:

52.7.1 下列情况下,在扩展插入点概念性增加新元素或替换项(称作概念增加元素,见 52.7.2):

- 没有扩展标志但在模块开头隐含可扩展性,然后,增加扩展标志而且在扩展标志之后增加新元素作为第一个附加;或者
- 在 CHOICE 或 SEQUENCE 或 SET 中有单个扩展标志,然后,在紧贴结束括号之前的 CHOICE 或 SEQUENCE 或 SET 后面增加新元素;或者
- 在 CHOICE 或 SEQUENCE 或 SET 中有两个扩展标志,那么,新元素加在紧贴第二个扩展标志之前。

52.7.2 这一概念增加元素只是通过规则的应用来检查要求不同标记(见 25.6~25.7、27.3 和 29.3)的合法性。它是在应用自动置标记(如果适用的话)和 COMPONENTS OF 扩展之后概念增加的。

52.7.3 这一概念增加元素定义具有与所有正常 ASN.1 类型的标记不同的标记,但是,与所有这样的概念增加元素的标记匹配,而且如 GB/T 16262.2—2025 的 14.2 注 2 中规定的,与开放类型的不确定标记匹配。

注: 一定有与概念增加元素和开放类型相关的标记唯一性规则以及要求不同标记的规则(见 25.6~25.7、27.3 和 29.3)并足以保证:

- 当 BER 编码被解码时,任何未知的扩展附加能无歧义地归因于单个插入点;和
- 未知的扩展附加绝不可能与 OPTIONAL 元素混淆。

在 PER 中,上面的规则足以保证这些特性,但不是必需的。尽管如此,他们仍然用作 ASN.1 的规则以保证记法与编码规则独立。

52.7.4 如果要求不同类型的规则由于这些概念增加元素而违规,那么规范非法使用了可扩展性记法。

注: 上面规则的目的是精确约束使用插入点(特别是不在各个 SEQUENCE 或 SET 或 CHOICE 尾部的那些)。设计这些约束来保证在 BER、DER 和 CER 中版本 1 系统收到的未知元素可能无歧义地归因于规定的插入点。如果例外处理这种附加元素对不同插入点是不同的,这样做将很重要。

52.8 示例

52.8.1 例 1

```
A ::= SET{
    a  A,
    b  CHOICER
    c  C,
    d  D,
```

```

    ...,
  }
}

```

是合法的,因为,像任何附加的材料一定是“b”的部分一样无歧义。

52.8.2 例 2

```

A ::= SET{
  a  A,
  b  CHOICE{
    c  C,
    d  D,
    ...,
  },
  ...,
  d  D
}

```

是非法的,因为,附加的材料可是“b”的部分,或可在“A”的较外面层,而版本 1 系统不能告诉是哪一个。

52.8.3 例 3

```

A ::= SET{
  a  A,
  b  CHOICE{
    c  C,
    ...
  },
  d  CHOICE{
    e  E,
    ...
  }
}

```

也是非法的,因为附加的材料可是 b 或 d 的部分。

52.8.4 用可扩展选择内的可扩展选择或标志为 OPTIONAL 或 DEFAULT 的序列元素内的可扩展选择能够构成更复杂的示例,但是上面的规则是必需的而且足以保证版本 1 中不出现的元素能由版本 1 系统无歧义地归因于精确的一个插入点。

53 例外标识符

53.1 在复杂的 ASN.1 规范中,在许多地方需要特别指出解码器应处理其中没有完全规定的材料。这些情况特别出现在使用抽象语法参数定义的约束中(见 GB/T 16262.4—2025 第 10 章)。

53.2 在这些情况下,应用设计者需要标识干扰某些依赖实现约束时采取的行动。提供例外的标识符作为引用 ASN.1 规范部分的无歧义手段以便指出采取的行动。标识符由“!”字符构成,紧接着是可选的 ASN.1 类型和那个类型的值。在没有类型时,则假设 INTEGER 是值的类型。

53.3 如果出现“ExceptionSpec”,指明标准正文的文本中有说明怎样处理与“!”字符有关的约束干扰。如果没有,那么,实现者要么需要标识描述他们采取的行动的文本,要么当出现约束干扰时,将采取依赖实现的行动。

53.4 “ExceptionSpec”的记法定义如下：

```
ExceptionSpec ::= "!" ExceptionIdentification | empty
ExceptionIdentification ::=
    SignedNumber
    | DefinedValue
    | Type ":" Value
```

最初两个替换项指明类型整数的例外标识符。第三个替换项指明任意类型(“Type”)的例外标识符(“Value”)。

53.5 用多重约束来约束类型时,不止一个有例外标识符,在最外约束中的例外标识符应认为是那个类型的例外标识符。

53.6 当用于集合运算的类型里出现例外标志时,应忽略例外标识符而且不应由被约束的类型作为集合运算的结果继承。

54 编码控制部分

54.1 “EncodingControlSections”由以下产生式规定：

```
EncodingControlSections ::=
    EncodingControlSection EncodingControlSections
    | empty
EncodingControlSection ::=
    ENCODING-CONTROL
    encodingreference
    EncodingInstructionAssignmentList
```

54.2 ASN.1 模块中的每个“EncodingControlSection”应具有不同的“encodingreference”,并将该编码引用的编码指令指派给模块中的一种或多种类型。

54.3 “encodingreference”不应是 TAG。

54.4 “EncodingInstructionAssignmentList”产生式和相关语义在“encodingreference”标识的本文件中规定(见附录 E),并可由 ASN.1 词项(包括注解、字符串和空白)的任何序列组成,但“EncodingInstructionAssignmentList”中不会出现的词项 END 和 ENCODING-CONTROL 除外。

注 1: 本文件的未来版本可在附件 E 中添加更多编码引用。如果“EncodingControlSection”中的“encodingreference”不是附件 E 中规定的那些,则推荐 ASN.1 工具(仅)提供警告,然后忽略“EncodingControlSection”直到下一次出现 END 或 ENCODING-CONTROL,以先到者为准。

注 2: “EncodingControlSection”中的“encodingreference”不能省略。模块的默认编码引用对“EncodingControlSection”没有影响。

54.5 对于使用类型前缀和“EncodingControlSection”将编码指令(有相同的编码引用)赋值给类型,存在相互作用和限制。仅使用“EncodingControlSection”始终是可能的(作为一种风格),但通常有些编码指令(尤其是适用于模块中所有类型的指令)只能在“EncodingControlSection”中被指派。对特定指令或指令组合可应用的类型也有限制。这些相互作用和限制在与编码引用相关的本文件(见附录 E)中有规定,而该本文件中没有规定。

附 录 A

(规范性)

ASN.1 正则表达式

A.1 定义

A.1.1 ASN.1 正则表达式是一种模式,用于描述一组字符串,这些字符串的格式符合此模式。正则表达式本身是字符串;它与算法表达式构造类似,通过使用各种运算符将较小的表达式组合。最小的表达式(通常)由一个或两个字符组成,表示一组字符的占位符。

这里提供的正则表达式与 Perl 等脚本语言的那些表达式以及 XML 模式的那些表达式非常相似,那里能够找到用法的某些其他示例。

A.1.2 大部分字符,包括全部字母和数字,是与它们自身匹配的正则表达式。

例如:

正则表达式“fred”只与字符串“fred”相匹配。

A.1.3 可拼接两个正则表达式;产生的正则表达式与通过拼接两个分别与子表达式相匹配的子串形成的任意字符串相匹配。

A.2 元字符

A.2.1 元字符序列(或元字符)是一个或多个在正则表达式上下文中有特定意义的相邻字符的集合。下面的列表包含所有的元字符序列。它们的意义在下面各条中解释。

[]		与用“-”指明范围的集合中的任意字符匹配。
		开括号之后的“-”补充紧跟其后的集合。
{g,p,r,c}		标识 GB/T 13000—2010 字符的四元组(见 41.8)
\N{名称}		如 A.2.4 中定义,与已命名字符(或已命名字符集的任意字符)匹配
.		与任意字符匹配(除非它是 12.1.6 中定义的换行字符之一)
\d		与任何数字匹配(相当于“[0-9]”)
\w		与任何字母数字字符匹配(相当于“[a-zA-Z0-9]”)
\t		与 HORIZONTAL TABULATION(9)字符匹配(见 12.1.6)
\n		与 12.1.6 中定义的任何换行字符匹配
\r		与 CARRIAGE RETURN(13)字符匹配(见 12.1.6)
\s		与任意一个空白字符匹配(见 12.1.6)
\b		与单词边界匹配
\	(前缀)	引用下一个元字符并使它从字面上解释
\\		与 REVERSE SOLIDUS(92)字符匹配“\”匹配
""		与 QUOTATION MARK(34)字符(“”)匹配
	(中缀)	两个表达式之间的替换项
()		封闭表达式的分组
*	(后缀)	与前面表达式匹配 0、1 或多次
+	(后缀)	与前面表达式匹配 1 次或多次
?	(后缀)	与前面表达式匹配 1 次或一次也没有
#n	(后缀)	与前面表达式准确地匹配 n 次(n 是单个数字)
#(n)	(后缀)	与前面表达式准确地匹配 n 次

#(n,)	(后缀)	与前面表达式匹配至少 n 次
#(n,m)	(后缀)	与前面表达式匹配至少 n 次但不超过 m 次
#(,m)	(后缀)	与前面表达式匹配不超过 m 次

注 1: 字符 CIRCUMFLEX ACCENT(94)“~”和 HYPHEN-MINUS(45)“-”是 A.2.2 中定义的字符串中某些位置上的附加元字符。

注 2: 本附录中字符名称后的圆括号内的值是 GB/T 13000—2010 中字符的十进制值。

该记法不提供元字符“^”和“\$”来分别与串的开始和结尾匹配。因此,字符串应整体上与正则表达式相匹配,除后者在它的开头、结尾或者两端包括“.*”之外。

注 3: 除非空白出现在紧接换行之前或之后,否则,下面元字符序列不能包含空白(见 12.1.6):

```
{g,p,r,c}
\N{名称}
#n
#(n)
#(n,)
#(n,m)
#(,m)
```

如果正则表达式包含换行、紧接换行之前或之后出现的任何间隔字符没有意义且与什么也不匹配(见 12.14.1)。

A.2.2 用“[”和“]”封闭的字符列表与该列表中的任意单个字符相匹配,如果列表中的第一个字符是脱字符号“~”,那么它与不在列表中的任意字符匹配。通过给出第一个和最后一字符、用连字符隔开(根据 43.3 中定义的顺序关系)可规定字符的范围。除“]”和“\”外,所有元字符序列失去它们在列表内的特殊意义。为了包括文字 CIRCUMFLEX ACCENT(94)“~”,将它放在除第一个位置外的任何位置或在它前放置反斜线。为了包括文字 HYPHEN-MINUS(45)“-”,将它放在列表的最前或最后,或在它前放置反斜杠。为了包括文字 CLOSING SQUARE BRACKET(93)“]”,它放在最前面。如果列表中的第一个字符是脱字符号“~”,那么当它们紧接在那个脱字符号之后时,字符“-”和“]”也与它们自身相匹配。A.2.3、A.2.4、A.2.6 和 A.2.7 中定义的元字符序列能用于它们保持其意义的方括号之间。

例如:

正则表达式“[0123456789]”,或相当的“[0-9]”与任何单个数字相匹配。

正则表达式“[~0]”与除 0 之外的任何单个字符相匹配。

正则表达式“[~d.-]”与任何单个数字、脱字符号、连字符或句号相匹配。

A.2.3 为了避免有相同字形字符的两个 GB/T 13000—2010 字符之间的任何歧义,提供了两个记法。形式“{组、面、行、字符元}”的记法按照 41.8 中定义的“四元组”产生式引用(单个)字符。

A.2.4 如果“valuereference”是引用当前模块中定义或引入的大小为 1(见第 41 章)的受限制字符串值,则形式“\N{valuereference}”的记法与引用的字符相匹配。如果“typereference”是引用当前模块中定义的“RestrictedCharaterStringType”的子类型或第 41 章中定义的“RestrictedCharaterStringType”之一,则形式“\N{typereference}”的记法与引用的字符集的任何字符相匹配。正则表达式“\N{Letter-Uppercase}”和“\N{Lu}”与 Unicode 标准定义的通用类别“Letter,uppercase”(缩写为“Lu”)的任意(单个)字符相匹配。

注: 在特定情况下,“valuereference”或“typereference”可能是模块 ASN1-CHARACTER-MODULE(见 42.1)中定义和引入到当前模块(见 41.8)的引用之一。

例如:

正则表达式“\N{greekCapitalLetterSigma}”与 GREEK CAPITAL LETTER SIGMA 相匹配。

正则表达式“\N{BasicLatin}”与 BASIC LATIN 字符集的任意(单个)字符相匹配。

“[\N{BasicLatin}\N{Cyrillic}\N{BasicGreek}]+”,或相当的“(\N{BasicLatin} | \N{Cyrillic} | \N{BasicGreek})+”是与任何取自规定的三个字符集字符的(非空)数字构成的字符串相匹配的正则

表达式。

A.2.5 句号“.”除非是 12.1.6 中定义的换行字符之一,否则,它与任意单个字符相匹配。

A.2.6 符号“\d”与“[0-9]”同义,即:它与任意单个数字相匹配。符号“\t”与字符 HORIZONTAL TABULATION(9)相匹配。符号“\w”与“[a-zA-Z0-9]”同义,即:它与任意单个(大写或小写)字符或任意单个数字相匹配。

例如:

正则表达式“\w+(\s\w+)*\.”匹配一个句子,该句子包含至少一个(字母数字)词。该句子中的词用 12.1.6 中定义的一个空白字符分隔。结束句号前没有空白字符。

A.2.7 符号“\r”与 CARRIAGE RETURN(13)字符相匹配。符号“\n”与 12.1.6 中定义的任意一个换行字符相匹配。符号“\s”与 12.1.6 中定义的任意一个空白字符相匹配。符号“\b”与词开始或结束的空串相匹配。

例如:

正则表达式“.* \bfred\b.”与包含词“fred”(该词不仅仅是一串四个字符;它是划界了的)的任意串相匹配。因此,它与“fred”或“I am fred the first”等类似的串相匹配,但不与“My name is freddy”或“I am afred I don't know how to spell'afraid'!”等类似的串相匹配。

A.2.8 通常用作元字符的字符能通过用“\”作其前缀来字面说明。如果正则表达式包含 QUOTATION MARK(34),该字符应用一对 QUOTATION MARK 字符表示。

例如:

正则表达式“\.”与(单个)串“.”相匹配,但不与任何单个字符的任何串相匹配;

正则表达式“\"”与包含单个 QUOTATION MARK 的串相匹配;

正则表达式“\)”与串“)”相匹配;

正则表达式“\a”与字符“a”相匹配。

注:第四个示例表明,允许反斜线在非元字符的字符前,但是不赞成这种用法(因为在本文档将来的版本中会允许其他元字符)。

A.2.9 通过中缀操作符“|”可合并两个或多个正则表达式。产生的正则表达式与任一子表达式相匹配的任意字符串相匹配。

A.2.10 不以重复操作符结尾的正则表达式可后接重复操作符。如果操作符是“?”,则前面的项是可选的而且最多匹配一次。如果操作符是“*”,则前面的项将匹配 0 或多次。如果操作符是“+”,则前面的项将匹配一次或多次。如果操作符是“#(n)”的形式,则前面的项准确地匹配 n 次;在这一特定情况下,如果 n 由一位数字组成,则能够省略圆括号。如果它是“#(n,)”的形式,则项匹配 n 或更多次。如果它是“#(,m)”的形式,则项是可选的而且匹配最多 m 次。如果它是“#(n,m)”的形式,则项匹配至少 n 次,但不超过 m 次。

注:将元字符“*”“+”“?”或“#”用作正则表达式的第一个字符或紧跟在重复操作符之后是非法的,将元字符“#”或“|”用作正则表达式的最后一个字符也是非法的。

例如:

“555-1212”这样的电话号码是通过正则表达式“\d#3-\d#4”来相匹配的,或者相当于“\d#(3)-\d#(4)”。

“\$12345.90”这样的美元价格是通过正则表达式“\$\d#(1,)(\.\d#(1,2))?”来相匹配的。注意,当“#”符号后接范围时要求有圆括号。

“123-45-5678”这样的美国社会保险号码是通过正则表达式“\d#3-? \d#2-? \d#4”来相匹配的。

A.2.11 重复(见 A.2.10)优先于拼接(见 A.1.3),而拼接优先于交替(见 A.2.9)。全部子表达式可包围在圆括号中以打破这些优先规则。

A.2.12 当正则表达式包含圆括号中的子表达式时,每个(非引用的)开圆括号从正则表达式的左边到

右边连续地赋予不同的(严格地讲正)整数。然后每个子表达式能够被引用到有像使用相关整数的“\1”“\2”等记法的注解中去。不允许空的子表达式“()”。

例如:

"(\d#2)(\d#2)(\d#4))" --\1 是日期,其中\2 是月、\3 是星期几和\4 是年。

注: 对形式引用正则表达式的子表达式在许多情况下有要求。一个这样的实例是需要写出文本将 ASN.1 模块中的正则表达式归档。这是能用来提供这种引用的记法。该记法不在本文件中的其他地方使用。

附 录 B

(规范性)

定义的时间类型

B.1 概述

B.1.1 本附录包含一个规定已定义时间类型的 ASN.1 模块。这些类型能够导入到 ASN.1 规范中并在该规范中使用,或者能够用作定义附加时间类型的模型。它们不能在没有导入的情况下使用。

B.1.2 在某些情况下,已定义时间类型仅在此模块中规定的日期或当日时间子集(或两者)之一进行子类型化时才有用。在这种情况下,在类型的定义中明确说明。

示例:使用

```
APPLICATION-DATE-TIME ::= DATE-TIME(YEAR-MONTH-DAY-SUBSET)(SECONDS-SUBSET)
```

定义日期时间,即年、月、日、时、分、秒。要使用它,应导入类型和两个子类型。

B.2 ASN.1 定义的时间类型模块

```
DefinedTimeTypes {joint-iso-itu-t asn1(1) specification(0) modules(0) defined-types-  
module(3)}
```

```
DEFINITIONS AUTOMATIC TAGS ::= BEGIN
```

```
EXPORTS ALL;
```

--日期类型

```
CENTURY ::= TIME((SETTINGS "Basic=Date Date=C Year=Basic") |  
                  (SETTINGS "Basic=Date Date=C Year=Proleptic"))
```

```
ANY-CENTURY ::= TIME((SETTINGS "Basic=Date Date=C Year=Negative") |  
                     (SETTINGS "Basic=Date Date=C Year=L5"))
```

--如果是正数,这只允许 3 位数的世纪。

--能够将具有更多位数的类型定义为附加时间类型。

--注意,如果年的规范需要 5 位数字,则 L5 用于世纪。见表 6。

```
YEAR ::= TIME((SETTINGS "Basic=Date Date=Y Year=Basic") |  
              (SETTINGS "Basic=Date Date=Y Year=Proleptic"))
```

```
ANY-YEAR ::= TIME((SETTINGS "Basic=Date Date=Y Year=Negative") |  
                  (SETTINGS "Basic=Date Date=Y Year=L5"))
```

--如果是正数,这只允许 5 位数的年份。

--能够将具有更多位数的类型定义为附加时间类型。

```
YEAR-MONTH ::= TIME((SETTINGS "Basic=Date Date=YM Year=Basic") |  
                    (SETTINGS "Basic=Date Date=YM Year=Proleptic"))
```

```
ANY-YEAR-MONTH ::= TIME((SETTINGS "Basic=Date Date=YM Year=Negative") |  
                        (SETTINGS "Basic=Date Date=YM Year=L5"))
```

--如果是正数,这只允许 5 位数的年份。

--能够将具有更多位数的类型定义为附加时间类型。

```
YEAR-MONTH-DAY ::= TIME((SETTINGS "Basic=Date Date=YMD Year=Basic") |  
                        (SETTINGS "Basic=Date Date=YMD Year=Proleptic"))
```

```
ANY-YEAR-MONTH-DAY ::= TIME((SETTINGS "Basic=Date Date=YMD Year=
```

Negative") |

(SETTINGS "Basic=Date Date=YMD Year=L5"))

--如果是正数,这只允许 5 位数的年份。

--能够将具有更多位数的类型定义为附加时间类型。

YEAR-WEEK::=TIME((SETTINGS "Basic=Date Date=YW Year=Basic") |

(SETTINGS "Basic=Date Date=YW Year=Proleptic"))

ANY-YEAR-WEEK::=TIME((SETTINGS "Basic=Date Date=YW Year=Negative") |

(SETTINGS "Basic=Date Date=YW Year=L5"))

--如果是正数,这只允许 5 位数的年份。

--能够将具有更多位数的类型定义为附加时间类型。

YEAR-WEEK-DAY::=TIME((SETTINGS "Basic=Date Date=YWD Year=Basic") |

(SETTINGS "Basic=Date Date=YWD Year=Proleptic"))

ANY-YEAR-WEEK-DAY::=TIME((SETTINGS "Basic=Date Date=YWD Year=Negative") |

(SETTINGS "Basic=Date Date=YWD Year=L5"))

--如果是正数,这只允许 5 位数的年份。

--能够将具有更多位数的类型定义为附加时间类型。

--与当日时间相关的类型

HOURS::=TIME(SETTINGS "Basic=Time Time=H Local-or-UTC=L")

HOURS-UTC::=TIME(SETTINGS "Basic=Time Time=H Local-or-UTC=Z")

HOURS-AND-DIFF::=TIME(SETTINGS "Basic=Time Time=H Local-or-UTC=LD")

MINUTES::=TIME(SETTINGS "Basic=Time Time=HM Local-or-UTC=L")

MINUTES-UTC::=TIME(SETTINGS "Basic=Time Time=HM Local-or-UTC=Z")

MINUTES-AND-DIFF::=TIME(SETTINGS "Basic=Time Time=HM Local-or-UTC=LD")

SECONDS::=TIME(SETTINGS "Basic=Time Time=HMS Local-or-UTC=L")

SECONDS-UTC::=TIME(SETTINGS "Basic=Time Time=HMS Local-or-UTC=Z")

SECONDS-AND-DIFF::=TIME(SETTINGS "Basic=Time Time=HMS Local-or-UTC=LD")

HOURS-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HF3 Local-or-UTC=L")

--3 位小数

HOURS-UTC-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HF3 Local-or-UTC=Z")

--3 位小数

HOURS-AND-DIFF-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HF3 Local-or-UTC=LD")

--3 位小数

MINUTES-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMF3 Local-or-UTC=L")

--3 位小数

MINUTES-UTC-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMF3 Local-or-UTC=Z")

--3 位小数

MINUTES-AND-DIFF-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMF3 Local-or-UTC=LD")

--3 位小数

SECONDS-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMSF3 Local-or-UTC=L")

--3 位小数

SECONDS-UTC-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMSF3 Local-or-UTC=Z")

--3 位小数

SECONDS-AND-DIFF-AND-FRACTION::=TIME(SETTINGS "Basic=Time Time=HMSF3 Local-or-UTC=LD")

--3 位小数

--时间间隔类型(不包括 DURATION,因为这是一种有用的类型)。

START-END-DATE-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SE SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

START-END-TIME-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SE SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

START-END-DATE-TIME-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SE SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

START-DATE-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SD SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

START-TIME-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SD SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

START-DATE-TIME-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=SD SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

DURATION-END-DATE-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=DE SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

DURATION-END-TIME-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=DE SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

DURATION-END-DATE-TIME-INTERVAL::=TIME(SETTINGS "Basic=Interval Interval-type=DE

SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

--循环时间间隔类型。

REC-START-END-DATE-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval Interval-type=SE

SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

REC-START-END-TIME-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval Interval-type=SE

SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

REC-START-END-DATE-TIME-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval Interval-type=SE

SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

REC-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval Interval-type=D")

REC-START-DATE-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval

Interval-type=SD

SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

REC-START-TIME-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval

Interval-type=SD

SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

REC-START-DATE-TIME-DURATION-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval Interval-type=SD

SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

REC-DURATION-END-DATE-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval

Interval-type=DE

SE-point=Date")

--这仅在使用 DATE 子集进行子类型化时才有用(见下文)。

REC-DURATION-END-TIME-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval

Interval-type=DE

SE-point=Time")

--这仅在使用 TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

REC-DURATION-END-DATE-TIME-INTERVAL::=TIME(SETTINGS "Basic=Rec-Interval

Interval-type=DE
SE-point=Date-Time")

--这仅在使用 DATE 子集和

--TIME-OF-DAY 子集进行子类型化时才有用(见下文)。

--日期子集

CENTURY-SUBSET::=TIME((SETTINGS "Date=C Year=Basic") |
(SETTINGS "Date=C Year=Proleptic"))

ANY-CENTURY-SUBSET::=TIME((SETTINGS "Date=C Year=Negative") |
(SETTINGS "Date=C Year=L5"))

YEAR-SUBSET::=TIME((SETTINGS "Date=Y Year=Basic") |
(SETTINGS "Date=Y Year=Proleptic"))

ANY-YEAR-SUBSET::=TIME((SETTINGS "Date=Y Year=Negative") |
(SETTINGS "Date=Y Year=L5"))

YEAR-MONTH-SUBSET::=TIME((SETTINGS "Date=YM Year=Basic") |
(SETTINGS "Date=YM Year=Proleptic"))

ANY-YEAR-MONTH-SUBSET::=TIME((SETTINGS "Date=YM Year=Negative") |
(SETTINGS "Date=YM Year=L5"))

YEAR-MONTH-DAY-SUBSET::=TIME((SETTINGS "Date=YMD Year=Basic") |
(SETTINGS "Date=YMD Year=Proleptic"))

ANY-YEAR-MONTH-DAY-SUBSET::=TIME((SETTINGS "Date=YMD Year=Negative") |
(SETTINGS "Date=YMD Year=L5"))

YEAR-WEEK-SUBSET::=TIME((SETTINGS "Date=YW Year=Basic") |
(SETTINGS "Date=YW Year=Proleptic"))

ANY-YEAR-WEEK-SUBSET::=TIME((SETTINGS "Date=YW Year=Negative") |
(SETTINGS "Date=YW Year=L5"))

YEAR-WEEK-DAY-SUBSET::=TIME((SETTINGS "Date=YWD Year=Basic") |
(SETTINGS "Date=YWD Year=Proleptic"))

ANY-YEAR-WEEK-DAY-SUBSET::=TIME((SETTINGS "Date=YWD Year=Negative") |
(SETTINGS "Date=YWD Year=L5"))

--时间子集

HOURS-SUBSET::=TIME(SETTINGS "Time=H Local-or-UTC=L")

HOURS-UTC-SUBSET::=TIME(SETTINGS "Time=H Local-or-UTC=Z")

HOURS-AND-DIFF-SUBSET::=TIME(SETTINGS "Time=H Local-or-UTC=LD")

MINUTES-SUBSET::=TIME(SETTINGS "Time=HM Local-or-UTC=L")

MINUTES-UTC-SUBSET::=TIME(SETTINGS "Time=HM Local-or-UTC=Z")

MINUTES-AND-DIFF-SUBSET::=TIME(SETTINGS "Time=HM Local-or-UTC=LD")

SECONDS-SUBSET::=TIME(SETTINGS "Time=HMS Local-or-UTC=L")

SECONDS-UTC-SUBSET::=TIME(SETTINGS "Time=HMS Local-or-UTC=Z")

SECONDS-AND-DIFF-SUBSET::=TIME(SETTINGS "Time=HMS Local-or-UTC=LD")

HOURS-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HF3 Local-or-UTC=L")

HOURS-UTC-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HF3 Local-or-UTC=Z")

HOURS-AND-DIFF-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HF3 Local-or-

UTC=LD")

MINUTES-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMF3 Local-or-UTC=L")

MINUTES-UTC-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMF3 Local-or-UTC=Z")

MINUTES-AND-DIFF-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMF3 Local-or-UTC=LD")

SECONDS-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMSF3 Local-or-UTC=L")

SECONDS-UTC-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMSF3 Local-or-UTC=Z")

SECONDS-AND-DIFF-AND-FRACTION-SUBSET::=TIME(SETTINGS "Time=HMSF3 Local-or-UTC=LD")

END

附 录 C

(规范性)

类型和价值兼容的规则

本附录希望主要为工具建造者使用以保证他们能相同地解释语言。本附录的目的是清楚地规定什么是合法的 ASN.1 和什么是不合法的 ASN.1, 而且能够规定任意值引用名标识符的精确值, 以及任意类型或值集引用名标识符的精确值集。不准备用它来提供任何上面描述以外目的的 ASN.1 记法的有效转换的定义。

C.1 需要值映射概念(辅导介绍)

C.1.1 考虑下面 ASN.1 定义:

```
A ::= INTEGER
B ::= [1]INTEGER
C ::= [2]INTEGER(0..6,...)
D ::= [2]INTEGER(0..6,...,7)
E ::= INTEGER(7..20)
F ::= INTEGER{red(0),white(1),blue(2),green(3),purple(4)}
a A ::= 3
b B ::= 4
c C ::= 5
d D ::= 6
e E ::= 7
f F ::= green
```

C.1.2 很清楚, 值引用 a、b、c、d、e 和 f 能用在分别由 A、B、C、D、E 和 F 支配的值记法中。例如,

```
W ::= SEQUENCE{w1 A DEFAULT a}
```

和

```
x A ::= a
```

和

```
Y ::= A(L.a)
```

都是 C.1.1 中给出定义的有效值。然而, 如果上面的 A 用 B、或 C、或 D、或 E、或 F 替代, 产生的语句将合法吗? 同样, 如果上面的值引用 a 在每一个这种情况下用 b、或 c、或 d、或 e、或 f 替代, 产生的语句合法吗?

C.1.3 更复杂的问题将考虑在每种情况下用赋值右边的显式文本来代替类型引用。考虑下边的示例:

```
f INTEGER{red(0),white(1),blue(2),green(3),purple(4)} ::= green
W ::= SEQUENCE{
    w1 INTEGER{red(0),white(1),blue(2),green(3),purple(4)} DEFAULT f
    x INTEGER{red(0),white(1),blue(2),green(3),purple(4)} ::= f
    Y ::= INTEGER{red(0),white(1),blue(2),green(3),purple(4)}(1..f)
```

上面的是合法的 ASN.1 吗?

C.1.4 上面的一些例子, 即使是合法的(大多数都是合法的——见后面的文本), 用户也不写类似的文本, 因为它们是模糊的, 甚至是令人困惑的。但是, 在支配者中应用标记或子类型时, 经常使用对某些类型的值(不一定只是 INTEGER 类型)的值引用作为该类型的默认值。引入值映射概念是为了提供一

种清晰而精确的方法来确定哪些结构(如上述结构)是合法的。

C.1.5 再考虑:

$C ::= [2] \text{INTEGER}(0..6, \dots)$

$E ::= \text{INTEGER}(7..20)$

$F ::= \text{INTEGER}\{\text{red}(0), \text{white}(1), \text{blue}(2), \text{green}(3), \text{purple}(4)\}$

在每种情况下产生新的类型。对于 F, 我们能在它的值与通用类型 INTEGER 的值之间清楚地标识 1-1 对应。在 C 和 E 的情况下, 我们能在它们的值与通用类型 INTEGER 的值的子集之间清楚地标识 1-1 对应。我们把这种关系称作两个类型中的值之间的值映射。而且, 因为 F、C 和 E 中的值都与 INTEGER 值的 (1-1) 映射, 我们能用这些映射提供 F、C 和 E 它们本身值之间的映射。对 F 和 C, 在图 C.1 中示出。

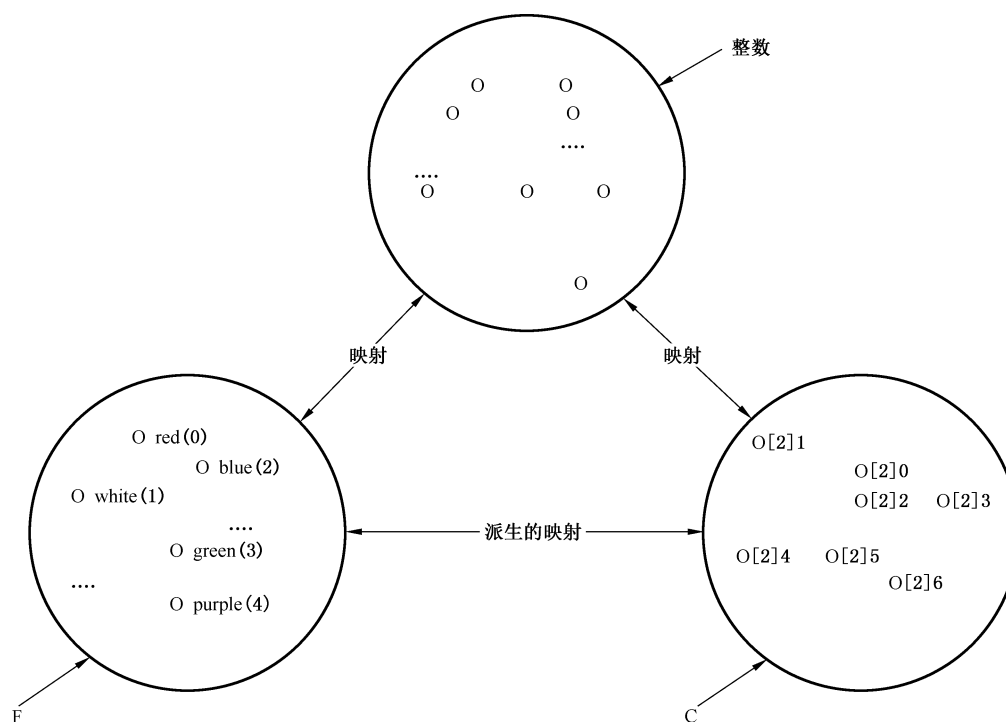


图 C.1 派生映射

C.1.6 现在当我们有下列值引用:

$c \ C ::= 5$

对于在某些上下文中要求用 C 中的值来标识 F 中值, 那么, 只要 C 中的那个值和 F 中的(单个)值存在值映射, 我们能(且确实)定义 c 是 F 中值的合法引用。这在图 C.2 中示出, 其中值引用 c 用来标识 F 中的值, 而且能用来代替直接引用 f1, 否则我们需要定义:

$f1 \ F ::= 5$

C.1.7 需注意, 在某些情况下, 在一个类型中应有值(例如, C.1.1 的 A 中的 7~20)与另一个类型中的值(例如, C.1.1 的 E 中的 7~20)有值映射, 但是其他值(A 的 21 以上)没有这种映射。A 中这类值的引用不会提供 E 中值的有效引用(在本例中, 整个 E 与 A 的子集有值映射。在一般情况下, 在有值映射的两种类型中可能都有值的子集, 其他值在没有映射的两种类型中)。

C.1.8 在 ASN.1 主体中标注, 用正常的英语文本来规定上面和类似情况中的合法性。C.6 章给出合法性的精确要求并且宜在对复杂结构有疑虑时引用。

注: “Type”结构的两个事件之间定义存在值映射的事实允许采用(使用一个“Type”结构建立的)值引用来标识另一

个充分相似的“Type”结构中的值。它允许用两个文本隔开、没有与虚拟和实际参数兼容性规则相冲突的“Type”结构来将虚拟和实际参数归类。也允许用一个“Type”结构来规定信息客体类别的范围和用充分相似的、不同“Type”结构规定信息客体中的对应值(这些示例不是无遗漏的)。然而,推荐利用这一自由的优点只是为了简单的情形,如,SEQUENCE OF INTEGER,或 CHOICE{int INTEGER,id OBJECT IDENTIFIER},而不能用于更复杂的“Type”结构。

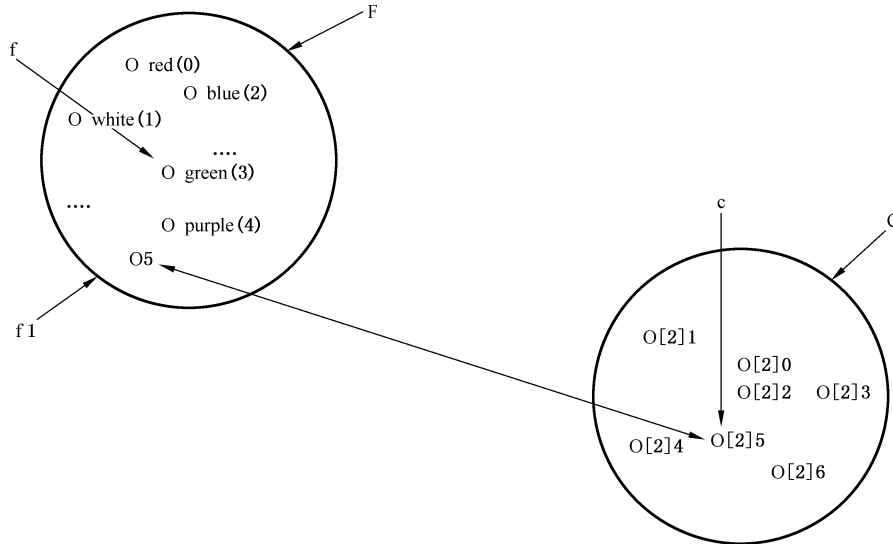


图 C.2 值引用的映射

C.2 值映射

C.2.1 基础的模型是类型,像非重叠的容器,包含值,有定义不同新类型(见图 C.1 和图 C.2)的每个 ASN.1“Type”结构事件。本附录规定当这些类型间存在值映射,以及在需要引用某些其他类型中的值时,可使用引用一个类型中的值。

例如,考虑:

X ::= INTEGER

Y ::= INTEGER

X 和 Y 是两个不同类型的类型引用名(指针),但是,这些类型间存在值映射,因此,当由 Y 支配时(例如,紧跟 DEFAULT 后面),可采用 X 值的任意值引用。

C.2.2 在所有可能的 ASN.1 值集中,值映射把一对值关联起来。值映射的整个集合是数学关系。这个关系具有下面的特性:它是反身的(每个 ASN.1 值与本身有关),对称的(如果从值 x1 到值 x2 以定义存在值映射,那么从 x2 到 x1 自动存在值映射),而且,它是可传递的(如果从值 x1 到值 x2 有值映射,而且从值 x2 到值 x3 有值映射,那么,从 x1 到 x3 自动存在值映射)。

C.2.3 另外,给出任意两个类型 X1 和 X2,视为值集,从 X1 中的值到 X2 中的值映射集是一一对应关系,即,对于 X1 中所有的值 x1、X2 中的 x2,如果从 x1 到 x2 有值映射,那么:

- 从 x1 到 X2 中另一个不同于 x2 的值没有值映射;和
- 从 X1 中的任意值(非 x1)到 x2 没有值映射。

C.2.4 在值 x1 和值 x2 之间存在值映射时,如果某些支配类型这样要求的话,能自动地采用任一个的值引用。

注: 某些“Type”结构中的值之间定义了存在值映射的事实仅仅为了提供使用 ASN.1 记法的灵活性。这些映射的存在不携带暗示两种类型携带相同的应用语义,但是,推荐只有对应的类型确实携带相同的应用语义,才使用无值映射就是非法的 ASN.1 结构。注意,值映射将经常存在于同一 ASN.1 结构中的两个类型之间任意的大规

范中,但是,携带完全不同应用语义,而且这些值映射的存在从不用于确定整个规范的合法性。

C.3 相同类型定义

C.3.1 用相同类型定义的概念来使值映射能在相同或足够相似到将正常希望它们的使用可交换的“Type”的两个实例间定义。为了给出“足够相似”的精确意义,本条规定一系列应用于每个“Type”实例的转换为那些“Type”实例产生正常形式。如果(且只有)两个“Type”实例,当它们的正常形式是相同词项的相同有序列表时,则定义两个“Type”实例是相同类型定义(见第 12 章)。

C.3.2 在 ASN.1 规范中的每个“Type”事件是第 12 章中定义的词项的有序列表。通过以那样的顺序应用 C.3.2.1~C.3.2.6 中定义的转换可获得正常形式。

C.3.2.1 删去所有的注解(见 12.6)。

C.3.2.2 下面的转换不递归,因此,只需要以任意顺序应用一次。

- a) 对于由“ValueSetTypeAssignment”定义的类型,其定义由“TypeAssignment”用如 16.6 中规定的“ValueSet”内容的、相同的“Type”和子类型约束代替。
- b) 对于每个整数类型:“NamedNumberList”(见 19.1),如果有的话,重新排序以便“identifier”按字母表顺序排列(“a”开头,“z”最后)。
- c) 对于每个枚举类型:如 20.3 中的规定,数字增加到是“identifier”(没有数字)的任意“EnumerationItem”(见 20.1);然后,“RootEnumeration”重新排序以便“identifiers”按字母表顺序排列(“a”开头,“z”最后)。
- d) 对于每个 bitstring 类型:“NamedBitList”(见 22.1),如果有的话,重新排序以便“identifiers”按字母表顺序排列(“a”开头,“z”最后)。
- e) 对于每个对象标识符值:每个“ObjIdComponent”转换成它对应的、符合第 32 章语义的“NumberForm”(见 32.13 中的示例)。
- f) 对于每个相对对象标识符值(见 33.3):每个“RelativeOIDComponent”转换成它对应的、符合第 33 章语义的“NumberForm”。
- g) 对于序列类型(见第 25 章)和集合类型(见第 27 章):形式“ExtensionAndException”“ExtensionAdditions”的任何扩展被切割并粘贴到“ComponentTypeLists”的尾部;删去“OptionalExtensionMarker”,如果出现的话。

如果“TagDefault”是 IMPLICIT TAGS,关键字 IMPLICIT 被加到“Tag”的所有实例上(见 31.2),除非有下面情况之一:

- 它已经出现;或
- 出现保留字 EXPLICIT;或
- 被标记的类型是 CHOICE 类型;或
- 它是开放类型。

如果“TagDefault”是 AUTOMATIC TAGS,根据 25.3 来决定是否应用自动置标记(自动置标记将在后面进行)。

注:在 25.4 和 27.2 中规定,作为替换“Component of Type”的结果插入的“ComponentType”中出现的“Tag”本身不阻止自动置标记转换。

如果“ExtensionDefault”是 EXTENSIBILITY IMPLIED,若没有出现省略号(“...”)的话,则在“ComponentTypeLists”后加上。

- h) 对于选择类型(见第 29 章):“RootAlternativeTypeList”重新排序以便“NameType”的标识符按字母表顺序排列(“a”开头,“z”最后)。“OptionalExtensionMarker”,如果出现的话,被删去。如果“TagDefault”是 IMPLICIT TAGS,关键字 IMPLICIT 被加到“Tags”的所有实例上(见 31.2),除非有下面情况之一:

- 它已经出现;或
- 出现保留字 EXPLICIT;或
- 被标记的类型是 CHOICE 类型;或
- 它是开放类型。

如果“TagDefault”是 AUTOMATIC TAGS,根据 29.5 来决定是否应用自动置标记。如果“ExtensionDefault”是 EXTENSIBILITY IMPLIED,若没有出现省略号(“...”)的话,则在“AlternativeTypeLists”后加上。

C.3.2.3 以规定的顺序递归地应用下面的转换直至达到固定点。

- a) 对每个对象标识符值(见 32.3):如果值定义以“DefinedValue”开头,则“DefinedValue”用它的定义代替。
- b) 对每个相对对象标识符值(见 33.3):如果值定义包含“DefinedValue”,则“DefinedValue”用它们的定义代替。
- c) 对于序列类型和集合类型:按照第 25 章和第 27 章转换所有的“COMPONENT OF Type”实例(见第 25 章)。
- d) 对于序列、集合和选择类型:如果它较早就决定自动加标记[见 C.3.2.2 g) 和 h)],根据第 25 章、第 27 章和第 29 章应用自动置标记。
- e) 对于精选类型:结构按照第 30 章用选择的替换项代替。
- f) 按照下面的规则,所有类型引用由它们的定义代替:
 - 如果代替的类型是引用正被转换的类型,类型引用用只与除某项本身外无其他项相匹配的特定项代替;
 - 如果代替的类型是单一序列类型或单一集合类型,约束紧接在被替代类型的后面,如果有的话,移到关键字 OF 的前面;
 - 如果被代替的类型是参数化类型或参数化值集合(见 GB/T 16262.4—2025 的 8.2),则每个“DummyReference”用对应的“ActualParameter”替代。
- g) 所有的值引用由它们的定义替代:如果被替代的值是参数化值(见 GB/T 16262.4—2025 的 8.2),则每个“DummyReference”用对应的“ActualParameter”替代。替代任意值引用之前,应应用本附录的规程以保证值引用通过值映射或直接地标识其支配类型中的值。

C.3.2.4 对于集合类型:“RootComponentTypeList”重新排序以便“ComponentType”按字母表顺序排列(“a”开头,“z”最后)。

C.3.2.5 应将下面的转换应用到值定义上:

- a) 如果用标识符定义整数值,则用相关的数字替代那个标识符;
- b) 如果用标识符定义位串值,则用删去所有结尾 0 位的对应“bstring”替代;
- c) 删去“cstring”中每个紧连换行(包括换行)前后的所有空白;
- d) 删去“bstring”和“hstring”中的所有空白;
- e) 规格化用底数 2 定义的每个实数值以使尾数是奇数,规格化用底数 10 定义的每个实数值以使尾数最后的数字不是 0;
- f) 每个 GeneralizedTime、UTCTime、TIME、TIME-OF-DAY、DATE、DATE-TIME 和 DURATION 值用符合 DER 和 CER 中编码时采用的规则串替代(见 ISO/IEC 8825-1:2021 的 11.7、11.8 和 11.9);
- g) 应用 c) 之后,每个 UTF8String、NumericString、PrintableString、IA5String、VisibleString (ISO/IEC 646 String)、BMPString 和 UniversalString 值用以“四元组”记法书写的 Univer-

salString 类型的相当值替代(见 41.8)。

C.3.2.6 任何“realnumber”事件应转换成以 10 为“base”的相关“SequenceValue”。任何与“SequenceValue”相关的“RealValue”事件应转换成相同“base”的相关“SequenceValue”，以使尾数的最后数字不是 0。

C.3.3 如果两个“Type”实例，当转换成它们的正常形式时，是词项的相同列表(见第 12 章)，那么，定义两个“Type”实例为相同类型定义，但下列除外：如果“objectclassreference”(见 GB/T 16262.2—2025 的 7.1)，“objectreference”见 GB/T 16262.2—2025 的 7.2)或“objectsetreference”(见 GB/T 16262.2—2025 的 7.3)出现在“Type”的规格化形式内，那么，这两个类型没有定义为相同类型定义，而且它们之间将不存在值映射(见 C.4)。

注：插入该例外是避免需要为关于信息客体类别、信息客体和信息客体集合记法的语法元素提供正常形式的转换规则。类似地，这时还不包括规格化所有值记法和集合运算记法的规范。如果要证明是该规范的需求，可在本文件的将来版本中提供。引入相同类型定义和值映射的概念以保证能够通过采用引用名或通过复制文本来使用简单的 ASN.1 结构。不必为更复杂的包括信息客体类别等的“Type”实例提供这一功能。

C.4 值映射规范

C.4.1 如果“Type”的两个事件是 C.3 规则下的相同类型定义，那么，一个类型的每个值与另一个类型对应值之间存在值映射。

C.4.2 对于通过加标记(见 31.2)，从任意类型 X2 产生的类型 X1，X1 的所有成员与 X2 的对应成员之间定义为存在值映射。

注：当上面 C.4.2 中 X1 与 X2 的值之间，以及 C.4.3 中 X3 与 X4 的值之间定义存在值映射时，如果这种类型嵌入到其他方面相同但不同类型定义(如，SEQUENCE 或 CHOICE 类型定义)中，产生的类型定义(SEQUENCE 或 CHOICE 类型)将不是相同类型定义，而且它们之间没有值映射。

C.4.3 对于通过元素集结构或通过分成子类型、并通过从任意支配类型 X4 选择值产生的类型 X3，新类型成员和由元素集或分成子类型结构选择的支配类型的那些成员之间被定义为存在值映射。有无扩展标志出现不影响这一规则。

C.4.4 在 C.5 中规定了某些字符串类型之间的附加值映射。

C.4.5 定义为已命名值的整数类型的任意类型的所有值与定义为无已命名值、或有不同已命名值、或已命名值不同名称、或两者都有的任意整数类型之间被定义为存在值映射。

注：值映射的存在不影响使用已命名值名称的任何范围规则要求。它们只能用于由在其中定义了它们的类型、或那个类型的类型引用名支配的范围中。

C.4.6 定义为已命名位的位串类型的任意类型的所有值与定义无已命名位、或有不同已命名位、或已命名位不同名称、或两者都有的任意位串类型之间被定义为存在值映射。

注：值映射的存在不影响使用已命名值名称的任何范围规则要求。它只能用于由在其中定义了它们的类型、或那个类型的类型引用名支配的范围中。

C.5 为字符串类型定义的附加值映射

C.5.1 有两组受约束的字符串类型，A 组(见 C.5.2)和 B 组(见 C.5.3)。A 组中所有类型之间被定义为存在值映射，而且当由其他类型之一支配时，能够使用这些类型值的值引用。对于 B 组中的类型，这些不同类型之间从不会存在值映射，A 组中的任意类型和 B 组中的任意类型之间也不会存在值映射。

C.5.2 A 组组成组件：

UTF8String
NumericString

PrintableString

IA5String

VisibleString(ISO/IEC 646 String)

UniversalString

BMPString

C.5.3 B 组组成组件:

TeletexString(T61String)

VideotexString

GraphicString

GeneralString

C.5.4 将每种类型的字符串值与 UniversalString 映射来规定 A 组中的值映射,然后采用值映射的传递性特性。为了将来自 A 组类型之一的值映射到 UniversalString,串用相同长度、每个字符按如下规定映射的 UniversalString 替代。

C.5.5 形式上,UTF8String 中的抽象值集是存在于 UniversalString 中但带不同标记的抽象值的相同集(见 41.16),而且 UTF8String 中的每个抽象值被定义为与 UniversalString 中对应的抽象值映射。

C.5.6 用于形成类型 NumericString 和 PrintableString 的字符的图形字符(打印的字符形状)与分配给 GB/T 13000—2010 的最初 128 个字符的图形字符的子集有可识别和无歧义的映射。使用图形字符的这种映射来定义这些类型的映射。

C.5.7 通过将每个字符映射成具有如 IA5String 和 VisibleString 的 BER 编码值(8 位)的 UniversalString 的 BER 编码中的相同(32 位)值的 UniversalString 字符来把 IA5String 和 VisibleString 映射成 UniversalString。

C.5.8 BMPString 形式上是 UniversalString 的子集,而且对应的抽象值有值映射。

C.6 特定类型和值兼容性要求

本章使用值映射概念来为某些 ASN.1 结构的合法性提供精确的文本。

C.6.1 具有支配类型 Y 的任何“Value”事件 x-记法,标识在支配类型 Y 中的值 y-val,该类型有一个到由 x-记法规定的值 x-val 的值映射。要求有这样的值存在。

例如,考虑下面最后一行中的 x 事件:

X ::= [0]INTEGER(0..30)

x X ::= 29

Y ::= [1]INTEGER(25..35)

Z1 ::= Y(x|30)

这些 ASN.1 结构是合法的,在最后的赋值中 x-记法 x 引用 X 中的 x-val 29,并且通过值映射标识 Y 中的 y-val 29。x-记法 30 引用 Y 中的 y-val 30,同时 Z1 是值 29 和 30 的集合。另一方面,赋值:

Z2 ::= Y(x|20)

是非法的,因为没有 x-记法 20 能引用的 y-val。

C.6.2 任何“Type”事件,具有支配类型 V 的 t-记法,标识与“Type”类型 t-记法根中任意值有值映射的支配类型 V 根中值的完整集合。要求该集合至少含有一个值。

例如,考虑下面最后一行中的 W 事件:

V ::= [0]INTEGER(0..30)

W ::= [1]INTEGER(25..35)

$Y ::= [2] \text{INTEGER}(31..35)$

$Z1 ::= V(W|24)$

W 贡献值 25-30 给集合运算导致 Z1 具有值 24-30。另一方面,赋值:

$Z2 ::= V(Y|24)$

是非法的,因为 Y 中没有与 V 中值映射的值。

C.6.3 作为实际参数提供的任何值的类型要求从那个值到支配虚拟参数的类型中的值之一有值映射,而且,它是那个被标识的支配类型的值。

C.6.4 如果“Type”作为是值集虚拟参数的虚拟参数的实际参数提供,那么那个“Type”的所有值要求与值集虚拟参数的支配者中的值具有值映射。实际参数选择与“Type”有映射的支配者中值的全部集。

C.6.5 在规定是值或值集参数的虚拟参数的类型 A 中,除非对 A 的所有值及对赋值右边使用 A 的每个实例,A 的那个值能合法地应用于虚拟参数的地方,否则,它是非法的规范。

C.7 示例

C.7.1 本章提供说明 C.3 和 C.4 的示例。

C.7.2 示例 1

$X ::= \text{SEQUENCE}$	$X1 ::= \text{SEQUENCE}$
{name VisibleString,	{name VisibleString,
age INTEGER	--注解--
	age INTEGER
$X2 ::= [8] \text{SEQUENCE}$	$X3 ::= \text{SEQUENCE}$
{name VisibleString,	{name VisibleString,
age INTEGER	age AgeType}
	AgeType ::= INTEGER

X、X1、X2 和 X3 都是相同类型定义。不能看到空白和注解的差别,X3 中使用 AgeType 类型引用也不影响类型定义。然而,注意,如果序列元素的任何标识符改变,则类型将不再是相同定义,而且它们之间将没有值映射。

C.7.3 示例 2

$B ::= \text{SET}$	$B1 ::= \text{SET}$
{name VisibleString,	{age INTEGER
age INTEGER}	name VisibleString}

它们是相同类型定义并且限定于不是在模块头中有 AUTOMATIC TAGS 的模块中提供,否则它们不是相同类型定义,而且它们之间将没有值映射。能用 CHOICE 和 ENUMERATED 写出类似的示例(用“EnumerationItem”的“identifier”形式)。

C.7.4 示例 3

$C ::= \text{SET}$	$C1 ::= \text{SET}$
{name[0]VisibleString,	{name VisibleString,
age INTEGER}	age INTEGER(1..64)}

它们不是相同类型定义,它们中的任一个与 B 或 B1 中的任一个也不是相同类型定义,而且 C 和 C1 的任何值之间没有值映射,它们中的任一个和 B 或 B1 中的任一个之间也没有值映射。

C.7.5 示例 4

$x \text{ INTEGER}\{y(2)\} ::= 3$

z INTEGER::=X

是合法的,并且通过 C.4.5 中定义的值映射把值 3 赋给 z 。

C.7.6 示例 5

b1 BIT STRING::='101'B

b2 BIT STRING::={version1(0),version2(1),version3(2)}::=b1

它们是合法的,并且把值{version1,version3}赋给 b2。

C.7.7 示例 6

根据 C.1.1 的定义,形式:

X DEFAULT y

的 SEQUENCE 元素是合法的,其中 X 是 A、B、C、D、E 或 F 中的任一个,或者是这些名称类型赋值右边的任意文本,而 y 是 a、b、c、d、e 或 f 中的任一个,下面例外:E DEFAULT y 对于所有的 a、b、c、d、f 是非法的,而且,C DEFAULT e 是非法的,因为在这些情况下,默认值引用到被默认的类型没有出现值映射。

附录 D

(规范性)

指派的对象标识符和 OID 国际化资源标识符值

本附录记录 ASN.1 系列标准中所赋的对象标识符、OID 国际化资源标识符和对象描述符值,并提供 ASN.1 模块以便在引用这些对象标识符值时使用。

D.1 本文件中指派的值

本文件中赋予下面的值:

41.3 条:

Object Identifier Value:

```
{joint-iso-itu-t ans1(1)specification(0)characterStrings(1)
  numericString(0)}
```

OID internationalized Resource Identifier Value:

```
"/Joint-ISO-ITU-T/ASN.1/Specification/Character_Strings/Numeric_String"
```

Object Descriptor Value: "NumericString ASN.1 Type"

41.5 条:

Object Identifier Value:

```
{joint-iso-itu-t ans1(1)specification(0)characterStrings(1)
  printableString(1)}
```

OID Internationalized Resource Identifier Value:

```
"/Joint-ISO-ITU-T/ASN.1/Specification/Character_Strings/Printable_String"
```

Object Descriptor Value: "PrintableString ASN.1 Type"

42.1 条:

Object Identifier Value:

```
{joint-iso-itu-t ans1(1)specification(0)modules(0)iso10646(0)}
```

OID Internationalized Resource Identifier Value:

```
"/Joint-ISO-ITU-T/ASN.1/Specification/Modules/ISO_10646"
```

Object Descriptor Value: "ASN.1 Character Module"

D.2 条:

Object Identifier Value:

```
{joint-iso-itu-t ans1(1)specification(0)modules(0)
  object-identifiers(1)}
```

OID Internationalized Resource Identifier Value:

```
"/Joint-ISO-ITU-T/ASN.1/Specification/Modules/Object_Identifiers"
```

Object Descriptor Value: "ASN.1 Object Identifier Module"

D.2 ASN.1 中的对象标识符及其编码规则标准

本章规定对 ASN.1 标准 (GB/T 16262.1—2025 ~ GB/T 16262.4—2025, ISO/IEC 8825-1:2021, ISO/IEC 8825-2:2021, Rec.ITU-T X.692 (2021) | ISO/IEC 8825-3:2021, ISO/IEC 8825-4:2021) 中

定义的对象标识符值包含的值引用名定义的 ASN.1 模块。

注：这些值可用于 OBJECT IDENTIFIER 类型和其衍生的类型的值记法中。本章规定的模块中定义的所有值引用均会被导出，并且任何希望使用这些值引用的模块都要导入它们

```
ASN1-Object-Identifier-Module{joint-iso-itu-t asn1(1)specification(0)
modules(0)object-identifiers(1)}
"/Joint-ISO-ITU-T/ASN.1/Specification/Modules/Object_Identifiers"
DEFINITIONS::=BEGIN
--NumericString ASN.1 类型(见 41.3)--
numericString OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)specification(0)characterStrings(1)
numericString(0)}
--PrintableString ASN.1 类型(见 41.5)--
printableString OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)specification(0)characterStrings(1)
printableString(1)}
--ASN.1 Character Module(见 42.1)--
asn1CharacterModule OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)specification(0)modules(0)iso10646(0)}
--ASN.1 Object Identifier Module(本模块)--
asn1ObjectIdentifierModule OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)specification(0)modules(0)
object-identifiers(1)}
--单个 ASN.1 类型的 BER 编码--
ber OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)basic-encoding(1)}
--单个 ASN.1 类型的 CER 编码--
cer OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)ber-derived(2)canonical-encoding(0)}
--单个 ASN.1 类型的 DER 编码--
der OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)ber-derived(2)distinguished-encoding(1)}
--单个 ASN.1 类型的 PER 编码(基本对齐)--
perBasicAligned OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)packed-encoding(3)basic(0)aligned(0)}
--单个 ASN.1 类型的 PER 编码(基本非对齐)--
perBasicUnaligned OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)packed-encoding(3)basic(0)unaligned(1)}
--单个 ASN.1 类型的 PER 编码(正则对齐)--
perCanonicalAligned OBJECT IDENTIFIER::=
{joint-iso-itu-t asn1(1)packed-encoding(3)canonical(1)aligned(0)}
--单个 ASN.1 类型的 PER 编码(正则非对齐)--
```

```

perCanonicalUnaligned OBJECT IDENTIFIER ::=
    {joint-iso-itu-t asn1(1)packed-encoding(3)canonical(1)unaligned(1)}
--单个 ASN.1 类型的 XER 编码(基本)--
xerBasic OBJECT IDENTIFIER ::=
    {joint-iso-itu-t asn1(1)xml-encoding(5)basic(0)}
--单个 ASN.1 类型的 XER 编码(正则)--
xerCanonical OBJECT IDENTIFIER ::=
    {joint-iso-itu-t asn1(1)xml-encoding(5)canonical(1)}
--单个 ASN.1 类型的 EXER 编码(扩展)--
xerExtended OBJECT IDENTIFIER ::=
    {joint-iso-itu-t asn1(1)xml-encoding(5)extended(2)}
END--ASN1-Object-Identifier-Module--

```

附 录 E
(规范性)
编码引用

E.1 本附录规定了当前定义的编码引用和规定了具有该编码引用(TAG 编码引用除外,它没有相关的编码指令)的编码指令的语法形式(和语义)的本文件。

注：如果此处未指定的编码引用出现在 ASN.1 规范中,则忽略相关的编码指令,并(仅)发出警告诊断。

E.2 当前定义了表 E.1 第 1 列中的编码引用。相关编码指令(如适用)的语法和语义在表 E.1 第 2 列中引用的本文件中定义。

表 E.1 定义与给定编码引用相关的语义的标准

编码引用	引用标准
TAG	本文件
XER	ISO/IEC 8825-4:2021
PER	Rec.ITU-T X.695 (2021) ISO/IEC 8825-6:2021

附录 F

(资料性)

国际对象标识符树中弧的分配和使用

F.1 概述

F.1.1 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 A 中规定了国际对象标识符树。它定义了注册权威机构的层次结构,每个机构指派:

- a) 每个从属弧的主要整数标识符(无歧义且唯一),标识其负责的节点下方的节点;
- b) (可选)每个从属弧的辅标识符,能够提高从属弧标识的人可读性,但不一定是无歧义的;
- c) 一个整数值 Unicode 标号(无歧义),它是弧的主要整数值的字符编码;
- d) (可选)提供从属弧的替代标识的进一步的 Unicode 标号(无歧义)。

F.1.2 Unicode 标号是(具有较小限制的)任何 Unicode 字符序列。

F.1.3 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 A 中(及其引用的本文件和程序)定义了国际对象标识符树。

注:有关 OID 分配的非正式信息库可在 <http://www.oid-info.com> 获得。

F.2 对象标识符(OBJECT IDENTIFIER)类型对国际对象标识符树的使用

F.2.1 这种类型(及其值记法和编码)提供了一种仅使用每个弧的主要整数值来标识国际对象标识符树节点的方法(在值记法、XML 值记法和 XML 编码中可选地包含辅标识符)。

F.2.2 它已被长期使用,并提供了一种在二进制编码计算机通信中标识国际对象标识符树节点的紧凑方法。

F.3 OID 国际化资源标识符(OID-IRI)类型对国际对象标识符树的使用

F.3.1 这种类型(及其值记法和编码)提供了一种仅使用每个弧的 Unicode 标号来标识国际对象标识符树节点的方法。

F.3.2 相同的语法也构成了向 IANA 注册的“oid”IRI 和 URI 方案的主体[见 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 F]。

附 录 G
(资料性)
举例和提示

本附录包含描述(假定)数据结构中采用 ASN.1 的示例。它也包含有使用 ASN.1 各种特性的提示或指南。除非另有说明,否则假设是 AUTOMATIC TAGS 环境。

G.1 人事记录的示例

用一个简单、假设的人事记录来说明 ASN.1 的用法。

G.1.1 人事记录的非形式描述

人事记录的结构和某个特定个人的值如下所示:

Name:	John P Smith
Title:	Director
Employee Number:	51
Date of Hire:	1971 年 9 月 17 日
Name of Spouse:	Mary T Smith
Number of Children:	2
Child Information	
Name:	Ralph T Smith
Date of Birth	1957 年 11 月 11 日
Child Information	
Name:	Susan B Jones
Date of Birth	1959 年 7 月 17 日

G.1.2 记录结构的 ASN.1 描述

下面使用数据类型标准记法形式地描述每个人事记录的结构。

```
PersonnelRecord ::= [APPLICATION 0]SET
{
    name           Name,
    title          VisibleString,
    number         EmployeeNumber
    dateOfHire     Date,
    nameOfSpouse   Name,
    children       SEQUENCE OF ChildInformation DEFAULT {}
}
ChildInformation ::= SET
{
    name           Name,
    dateOfBirth    Date
}
Name ::= [APPLICATION 1]SEQUENCE
{
    givenName      VisibleString
    initial        VisibleString
}
```

```

        familyName      VisibleString
    }
    EmployeeNumber ::= [APPLICATION 2] INTEGER
    Date ::= [APPLICATION 3] VisibleString --YYYY MMDD

```

本例说明了 ASN.1 语法分析的一个方面。语法结构 DEFAULT 只能适用于 SEQUENCE 或 SET 组件,它不能适用于 SEQUENCE OF 的元素。因此,在 PersonnelRecord 中的 DEFAULT{} 适用于 children,而不适用于 ChildInformation。

G.1.3 记录值的 ASN.1 描述

下面使用数据值的标准记法形式地描述 John Smith 的个人记录值。

```

{ name          {givenName"John",initial"P",familyName"Smith"},
  title         "Director",
  number        51,
  dateOfHire    "19710917"
  nameOfSpouse  {givenName"Mary",initial"T",familyName"Smith"},
  children      { {name {givenName"Ralph",initial"T",familyName"Smith"},
                    dateOfBirth"19571111"},
                  {name {givenName"Susan",initial"B",familyName"Jones"},
                    dateOfBirth"19590717"}
                }
}

```

或者以 XML 值记法:

```

person ::=
  <PersonnelRecord>
    <name>
      <givenName>John</givenName>
      <initial>P</initial>
      <familyName>Smith</familyName>
    </name>
    <title>Director</title>
    <number>51</number>
    <dateOfHire>19710917</dateOfHire>
    <nameOfSpouse>
      <givenName>Mary</givenName>
      <initial>T</initial>
      <familyName>Smith</familyName>
    </nameOfSpouse>
    <children>
      <ChildInformation>
        <name>
          <givenName>Ralph</givenName>
          <initial>T</initial>

```

```

        <familyName>Smith</familyName>
    </name>
    <dateOfBirth>19571111</dateOfBirth>
</ChildInformation>
<ChildInformation>
    <name>
        <givenName>Susan</givenName>
        <initial>B</initial>
        <familyName>Jones</familyName>
    </name>
    <dateOfBirth>19590717</dateOfBirth>
</ChildInformation>
</children>
</PersonnelRecord>

```

G.2 使用记法指南

本文件定义的数据类型及形式记法是灵活的,可广泛地用于各种协议的描述。然而,这种灵活性有时会带来混淆,尤其是在第一次用这个记法时。本附录通过给出使用记法的指南和示例,意于将混淆减到最少。对于每个固有数据类型,提供了一个或多个使用指南。这里不考虑字符串类型(如:Visible-String)和第 46~48 章定义的类型。

G.2.1 布尔

G.2.1.1 用布尔类型为逻辑(即:双态)变量值建模,例如,问题的答案为“是或否”。

例:

```
Employed::=BOOLEAN
```

G.2.1.2 当赋给布尔类型引用名称时,选择描述 true 状态的那个。

例:

```
Married::=BOOLEAN
```

而不是

```
MaritalStatus::=BOOLEAN
```

G.2.2 整数

G.2.2.1 用整数类型为坐标或整数变量的值(为满足通用性而不限界)建模。

例:

```
CheckingAccountBalance::=INTEGER    --精确到分;负为超支。
```

```
balance CheckingAccountBalance::=0
```

或者用 XML 值记法:

```
balance::=<CheckingAccountBalance>0</CheckingAccountBalance>
```

G.2.2.2 定义整数类型有名数字的最小和最大允许值。

例:

```
DayOfTheMonth::=INTEGER{first(1),last(31)}
```

```
today DayOfTheMonth::=first
```

```
unknown DayOfTheMonth::=0
```


或者用 XML 值记法:

```
today::=<DayOfTheMonth><first/></DayOfTheMonth>
unknown::=<DayOfTheMonth>0</DayOfTheMonth>
```

注意,选择有名数字 first 和 last 是因为他们对于读者语法的重要性,并且不排除有其他可能小于 1、大于 31 或在 1 与 31 之间的 DayOfTheMonth 的可能性。

要限制 DayOfTheMonth 的值刚好是 first 和 last,它需要写作:

```
DayOfTheMonth::=INTEGER{first(1),last(31)}(first|last)
```

以及要限制 DayOfTheMonth 的值在 1 与 31(含)之间的所有值,它需要写作:

```
DayOfTheMonth::=INTEGER{first(1),last(31)}(first..last)
dayOfTheMonth DayOfTheMonth::=4
```

或者用 XML 值记法:

```
dayOfTheMonth::=<DayOfTheMonth>4</DayOfTheMonth>
```

G.2.3 枚举

G.2.3.1 用枚举类型为有不少于三个状态的变量值建模。如果它们仅有的约束是不同的,则从 0 开始赋值。

例:

```
DayOfTheWeek::=ENUMERATED{sunday(0),monday(1),tuesday(2),
                             wednesday(3),thursday(4),friday(5),saturday(6),}
firstDay DayOfTheWeek::=Sunday
```

或者用 XML 值记法:

```
firstDay::=<DayOfTheWeek><sunday/></DayOfTheWeek>
```

注意,由于他们对读者语义的重要性,在选择枚举 sunday、monday 等等时,DayOfTheWeek 限制为假设这些值之一且没有其他值。另外,只有名称 sunday、monday 等等能赋给值;不允许有相当的整数值。

G.2.3.2 用可扩展的枚举类型为变量值建模,变量值在目前只有两个状态、但是在协议的今后版本中可能有附加状态。

例:

```
MaritalStatus::=ENUMERATED{single,married} --MaritalStatus 第一版
```

在下列预料中

```
MaritalStatus::=ENUMERATED{single,married,...,widowed}--MaritalStatus 第二版
```

今后还有:

```
MaritalStatus::=ENUMERATED{single,married,...,widowed,divorced}
--MaritalStatus 第三版
```

G.2.4 实数

G.2.4.1 用实数类型为大约数目建模。

例:

```
AngleInRadians::=REAL
pi REAL::={mantissa 3141592653589793238462643383279,base10,exponent-30}
```

或者用实数的替换值记法:

```
pi REAL::=3.14159265358979323846264338327
```

或者用 XML 值记法:

```

pi::=
  <REAL>
    3.14159265358979323846264338327
  </REAL>

```

G.2.4.2 应用设计者可能希望保证与实数值全部互工作,尽管在浮点硬件中不同,而且在实现决定中为应用使用(例如)单个或双倍长度浮点,它能够通过下面实现:

```

App-X-Real::=REAL(WITH COMPONENTS{
    mantissa(−16777215..16777215),
    base(2),
    exponent(−125..128)})
/*

```

发送者不应传输这些范围之外的值,而且合格的接受者应能够接受和处理这些范围内的值。

```

*/
girth App-X-Real::={mantissa 16,base 2,exponent 1}

```

或者用 XML 值记法:

```

girth::=
  <App-X-Real>
    32
  </App-X-Real>

```

G.2.5 位串

G.2.5.1 用位串类型为没有规定格式和长度,或者在其他地方规定,而且其位的长度不必是 8 的整数倍的二进制数据建模。

例:

```

G3FacsimilePage::=BIT STRING
--符合 ITU-T T.4 的位序列。
image G3FacsimilePage::='100110100100001110110'B
trailer BIT STRING::='0123456789ABCDEE'H
body1 G3FacsimilePage::='1101'B
body2 G3FacsimilePage::='1101000'B

```

或者用 XML 值记法:

```

image::=<G3Facsimile>100110100100001110110</G3Facsimile>
trailer::=
  <BIT_STRING>
    0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011
    1100 1101 1110 1111
  </BIT_STRING>
body1::=<G3Facsimile>1101</G3Facsimile>
body2::=<G3Facsimile>1101000</G3Facsimile>

```

注意,因为结尾 0 位很重要(由于 G3FacsimilePage 的定义中没有“NamedBitList”),因此,“body1”和“body2”是不同的抽象值。

G.2.5.2 用有大小约束的位串类型为固定的依大小排列的位字段的值建模。

例:

```
BitField::=BIT STRING(SIZE(12))
map1 BitField::='100110100100'B
map2 BitField::='9A4'H
map3 BitField::='100110100'B --非法的——与大小约束冲突。
```

或者用 XML 值记法：

```
map1::=<BitField>100110100100</BitField>
```

注意,“map1”和“map2”是相同的抽象值,因为“map2”的四个结尾位没有意义。

G.2.5.3 用位串类型为 bit map 值建模,bit map 是指明是否为每个对应的对象有序集合保持特定条件的逻辑变量的有序集合。

```
DaysOfTheWeek::=BIT STRING{
    sunday(0),monday(1),tuesday(2),wednesday(3),
    thursday(4),friday(5),saturday(6)}(SIZE(0..7))
sunnyDaysLastWeek1 DaysOfTheWeek::={sunday,monday,wednesday}
sunnyDaysLastWeek2 DaysOfTheWeek::='1101'B
sunnyDaysLastWeek3 DaysOfTheWeek::='1101000'B
sunnyDaysLastWeek4 DaysOfTheWeek::='11010000'B --非法的
```

或者用 XML 值记法：

```
sunnyDaysLastWeek1::=
    <DaysOfTheWeek>
        <sunday/><monday/><wednesday/>
    </DaysOfTheWeek>
sunnyDaysLastWeek2::=<DaysOfTheWeek>1101</DaysOfTheWeek>
sunnyDaysLastWeek3::=<DaysOfTheWeek>1101000</DaysOfTheWeek>
```

注意,如果位串值的长度小于 7 位,那么丢失的位指明那些天为阴天。也就是上面的最初三个值有相同的抽象值。

G.2.5.4 用位串类型为 *bit map* 值建模,是指明是否为每个对应的对象有序集合保持特定条件的逻辑变量的固定大小的有序集合。

```
DaysOfTheWeek::=BIT STRING{
    sunday(0),monday(1),tuesday(2),wednesday(3),
    thursday(4),friday(5),saturday(6)}(SIZE(7))
sunnyDaysLastWeek1 DaysOfTheWeek::={sunday,monday,wednesday}
sunnyDaysLastWeek2 DaysOfTheWeek::='1101'B --非法的——与大小约束冲突
sunnyDaysLastWeek3 DaysOfTheWeek::='1101000'B
sunnyDaysLastWeek4 DaysOfTheWeek::='11010000' --非法的——与大小约束冲突
```

注意,第一个和第三个值有相同的抽象值。

G.2.5.5 用带已命名位的位串类型为相关逻辑变量集合的值建模。

例：

```
PersonalStatus::=BITS TRING
    {married(0),employed(1),veteran(2),collegeGraduate(3)}
billClinton PersonalStatus::={married,employed,collegeGraduate}
hillaryClinton PersonalStatus::='110100'B
```

或者用 XML 值记法：

```
billClinton::=
```

```

    <PersonalStatus>
      <married/>
      <employed/>
      <collegeGraduate/>
    </PersonalStatus>

```

```
hillaryClinton::=<PersonalStatus>110100</PersonalStatus>
```

注意,“billClinton”和“hillaryClinton”有相同的抽象值。

G.2.6 八位位组串

G.2.6.1 用八位位组串类型为没有规定格式和长度,或者在其他地方规定,而且其位的长度是 8 的整数倍的二进制数据建模。

例:

```

G4FacsimileImage::=OCTET STRING
--符合 ITU-T T.5 和 ITU-T T.6 的八位位组序列。
image G4FacsimilePage::='3FE2EBAD471005' H

```

或者用 XML 值记法:

```
image::=<G4FacsimileImage>3FE2EBAD471005</G4FacsimileImage>
```

G.2.6.2 在有合适的受限制字符串类型时,优先用它而不用八位位组串类型。

例:

```

Surname::=PrintableStrins
president Surname::="Clinton"

```

或者用 XML 值记法:

```
president::=<Surname>Clinton</Surname>
```

G.2.7 UniversalString、BMPStnn 和 UTF8String

用 BMPString 类型或 UTF8String 类型来为只由取自 GB/T 13000—2010 基本多文种平面(BMP)的字符组成的任何信息串建模,而 UniversalString 或 UTF8String 则由不属于 BMP 的 GB/T 13000—2010 字符构成的任何串建模。

G.2.7.1 用 Level1 或 Level2 指明实现级数对使用组合字符的限制。

例:

```

RussianName::=Cyrillic(Level1)
--RussianName 用无组合字符。
SaudiName::=BasicArabic(SIZE(1..100)·Level2)
--SaudiName 用组合字符的子集。

```

Σ的表达式:

```
greekCapitalLetterSigma BMPString::={0,0,3,163}
```

或用 XML 值记法:

```
greekCapitalLetterSigma::=<BMPString>&#x03a3;</BMPString>
```

串“f→∞”的表达式:

```

rightwardsArrow UTF8String::={0,0,33,146}
infinity UTF8String::={0,0,34,30}
property UTF8String::={"f",rightwardsArrow,"",infinity}

```

或用 XML 值记法:

property::=<UTF8String>f→∞</UTF8String>

G.2.7.2 通过使用“UnionMark”(见第 50 章),集合可扩大成选择的子集(即:包括 BASIC LATIN 集合中的所有字符)。

例:

KatakanaAndBasicLatin:=UniversalString(FROM(Katakana|BasicLatin))

G.2.8 CHARACTER STRING

用无限制字符串类型来为不能用受限制字符串类型之一建模的任何信息串建模。确保规定了字符汇和八位位组代码。

例:

```
PackedBCDString::=CHARACTER STRING(WITH COMPONENTS{
    identification(WITH COMPONENTS{
        fixed PRESENT}))
/* 抽象和传送语法应是下面定义的 packedBCDString-AbstractSyntaxId
   和 packedBCDString-TransferSyntaxId。
*/
})
/* 字母为数字 0 至 9 的字符抽象语法(字符集)的对象标识符值。
*/
packedBCDString-AbstractSyntaxId OBJECT IDENTIFIER::=
    {joint-iso-itu-t example(999)packedBCD(2)charSet(0)}
/* 每八位位组紧缩两位数字,每位数字编码为 0000 至 1001,11112用作填充的
   字符传送语法的对象标识符值。
*/
packedBCDString-TransferSyntaxId OBJECT IDENTIFIER::=
    {joint-iso-itu-t example(999)packedBCD(2)
    characterTransferSyntax(1)}
/* PackedBCDString 的编码将只包含字符的已定义编码,有任意必需长的字段,
   以及在 BER 时,携带标记的字段。当“fixed”已规定时,不携带对象标识符值。
*/
```

或者用 XML 值记法:

```
packedBCDString-AbstractSyntaxId::=
    <OBJECT_IDENTIFIER>
    joint-iso-itu-t.example(999).packedBCD(2).charSet(0)
    </OBJECT_IDENTIFIER>
packedBCDString-TransferSyntaxId::=
    <OBJECT_IDENTIFIER>
    joint-iso-itu-t.example(999).packedBCD(2).
    characterTransferSyntax(1)
    </OBJECT_IDENTIFIER>
```

或者

```
packedBCDString-AbstractSyntaxId::=
    <OBJECT_IDENTIFIER>2.999.2.0</OBJECT_IDENTIFIER>
```

```
packedBCDString-TransferSyntaxId ::=
  <OBJECT_IDENTIFIER>2.999.2.1</OBJECT_IDENTIFIER>
```

注：编码规则不必将类型 CHARACTER STRING 的值编码成总是包含对象标识符值的形式，尽管它们保证编码中保留抽象值。

G.2.9 空

用空类型来指明序列组件的有效缺失。

例：

```
PatientIdentifier ::= SEQUENCE {
    name          VisibleString,
    roomNumber    CHOICE {
        room      INTEGER,
        outPatient NULL  --如果一个门诊病人--
    }
}

lastPatient PatientIdentifier ::= {
    name "Jane Doe",
    roomNumber outPatient: NULL
}
```

或者用 XML 值记法：

```
lastPatient ::=
  <PatientIdentifier>
    <name>Jane Doe</name>
    <roomNumber><outPatient/></roomNumber>
  </PatientIdentifier>
```

G.2.10 序列和单一序列

G.2.10.1 用单一序列类型来为类型相同、数字大或不可预知、顺序有意义的变量集合建模。

例：

```
NamesOfMemberNations ::= SEQUENCE OF VisibleString
--按字母表顺序

firstTwo NamesOfMemberNations ::= {"Australia", "Austria"}
```

或用可选标识符：

```
NamesOfMemberNations2 ::= SEQUENCE OF memberNation VisibleString
--按字母表顺序

firstTwo2 NamesOfMemberNations2 ::=
  {memberNation "Australia", memberNation "Austria"}
```

采用 XML 值记法，上面的两个值如下：

```
firstTwo ::=
  <NamesOfMemberNations>
    <VisibleString>Australia</VisibleString>
    <VisibleString>Austria</VisibleString>
  </NamesOfMemberNations>
```

```

firstTwo2 ::=
  <NamesOfMemberNations2>
    <memberNation>Australia</memberNation>
    <memberNation>Austria</memberNation>
  </NamesOfMemberNations2>

```

G.2.10.2 用序列类型来为类型相同、数字已知和适中、及顺序有意义的变量集合建模，只要集合的构成不可能从协议的一个版本改变到下一个。

例：

```

NamesOfOfficers ::= SEQUENCE{
  president      VisibleString,
  vicePresident   VisibleString,
  secretary       VisibleString}
acmeCorp NamesOfOfficers ::= {
  president      "Jane Doe",
  vicePresident   "John Doe",
  secretary       "Joe Doe"}

```

或者用 XML 值记法：

```

acmeCorp ::=
  <NamesOfOfficers>
    <president>Jane Doe</president>
    <vicePresident>John Doe</vicePresident>
    <secretary>Joe Doe</secretary>
  </NamesOfOfficers>

```

G.2.10.3 用不可扩展的序列类型来为类型不同、数字已知和适中、顺序有意义的变量集合建模，只要集合的构成不可能从协议的一个版本改变到下一个。

例：

```

Credentials ::= SEQUENCE{
  username      VisibleString,
  password      VisibleString,
  accountNumber INTEGER}

```

G.2.10.4 用可扩展序列类型来为顺序有意义、数字当前已知和适中但预计会增加的变量集合建模。

例：

```

Record ::= SEQUENCE{ --包含“Record”的协议的第一版
  username      VisibleString,
  password      VisibleString,
  accountNumber INTEGER,
  ...,
  ...
}

```

在预料中：

```

Record ::= SEQUENCE{ --包含“Record”的协议的第二版
  username      VisibleString,
  password      VisibleString,

```

```

    accountNumber  INTEGER
    ...,
    [[2:--协议第二版中增加的扩展附加
        lastLoggedIn      GeneralizedTime OPTIONAL,
        minutesLastLoggedIn  INTEGER
    ]],
    ...
}

```

且后来还有(没有附加到记录的协议的第三版):

```

Record ::= SEQUENCE{
    --包含“Record”的协议的第三版
    username      VisibleString,
    password      VisibleString,
    accountNumber  INTEGER,
    ...,
    [[2:--协议第二版中增加的扩展附加
        lastLoggedIn      GeneralizedTime OPTIONAL,
        minutesLastLoggedIn  INTEGER
    ]],
    [[4:--协议第三版中增加的扩展附加
        certificate  Certificate,
        thumb        ThumbPrint OPTIONAL
    ]],
    ...
}

```

G.2.11 集合和单一集合

G.2.11.1 用集合类型来为数字已知和适中且顺序无意义的变量集合建模。如果自动置标记无效,用如下所示的特殊上下文置标记来标识每个变量(有自动置标记时,不必加标记)。

例:

```

UserName ::= SET{
    personalName      [0] VisibleString,
    organizationName  [1] VisibleString,
    countryName       [2] VisibleString}
user UserName ::= {
    countryName      "Nigeria",
    personalName     "Jonas Maruba",
    organizationName "Meteorology,Ltd."}

```

或者用 XML 值记法:

```

user ::=
    <UserName>
        <countryName>Nigeria</countryName>
        <personalName>Jonas Maruba</personalName>
        <organizationName>Meteorology,Ltd.</organizationName>

```


⟨/UserName⟩

G.2.11.2 用有 OPTIONAL 的集合类型来为数字已知和合理的小且顺序无意义的变量的另一个集的（正确或不正确）子集的变量集合建模。如果自动置标记无效，用如下所示的特殊上下文置标记来标识每个变量。（有自动置标记时，不必加标记。）

例：

```
UserName ::= SET{
    personalName          [0]VisibleString,
    organizationName      [1]VisibleString OPTIONAL
    --默认为当地组织的那个--
    countryName           [2]VisibleString OPTIONAL
    --默认为当地国家的那个--
}
```

G.2.11.3 用可扩展集合类型来为构成可能从一个协议版本改变到下一个的变量集合建模。下面假设模块定义中规定了 AUTOMATIC TAGS。

例：

```
UserName ::= SET{
    personalName          VisibleString, --“UserName”的第一版
    organizationName      VisibleString OPTIONAL,
    countryName           VisibleString OPTIONAL,
    ...,
    ...
}

user userName ::= {personalName"Jonas Maruba"}
```

或者用 XML 值记法：

```
user ::=
    ⟨UserName⟩
    ⟨personalName⟩Jonas Maruba⟨/personalName⟩
    ⟨/UserName⟩
```

在预料中：

```
UserName ::= SET{ --“UserName”第二版
    personalName          VisibleString,
    organizationName      VisibleString OPTIONAL,
    countryName           VisibleString OPTIONAL,
    ...,
    [[2: --协议第二版中增加的扩展附加
        internetEmailAddress VisibleString,
        faxNumber           VisibleString OPTIONAL
    ]],
    ...
}

user UserName ::= {
    personalName          "Jonas Maruba",
    internetEmailAddress  "jonas@meteor.ngo.com"
}
```

或者用 XML 值记法：

```
user::=
  <UserName>
    <personalName>Jonas Maruba</personalName>
    <internetEmailAddress>jonas@meteor.ngo.com</internetEmailAddress>
  </UserName>
```

且后来还有(没有附加到用户名的协议的第三和四版)：

```
UserNamel::=SET{ --包含“UserName”的协议第五版
  personalName      VisibleString,
  organizationName   VisibleString OPTIONAL,
  countryName        VisibleString OPTIONAL,
  ...,
  [[2: --协议第二版中增加的扩展附加
    internetEmailAddress VisibleString,
    faxNumber             VisibleString OPTIONAL
  ]],
  [[5: --第五版中增加的扩展附加
    phoneNumber          VisibleString OPTIONAL
  ]],
  ...
}
user UserNamel::={
  personalName      "Jonas Maruba",
  internetEmailAddress "jonas@meteor.ngo.com"
}
```

或者用 XML 值记法：

```
user::=
  <UserName>
    <personalName>Jonas Maruba</personalName>
    <internetEmailAddress>jonas@meteor.ngo.com</internetEmailAddress>
  </UserName>
```

G.2.11.4 用单一集合类型来为类型相同和顺序无意义的变量集合建模。

例：

```
Keywords::=SET OF VisibleString --按任意顺序
someASN1Keywords Keywords::={ "INTEGER", "BOOLEAN", "REAL" }
```

或者采用可选标识符：

```
Keywords::=SET OF keyword VisibleString --按任意顺序
someASN1Keywords2 Keywords2::={ keyword"INTEGER", keyword"BOOLEAN",
  keyword"REAL" }
```

采用 XML 值记法,上面的两个值如下：

```
someASN1Keywords::=
  <Keywords>
    <VisibleString>INTEGER</VisibleString>
```

```

    <VisibleString>BOOLEAN</VisibleString>
    <VisibleString>REAL</VisibleString>
  </Keywords>
someASN1Keywords2 ::=
  <Keywords2>
    <keyword>INTEGER</keyword>
    <keyword>BOOLEAN</keyword>
    <keyword>REAL</keyword>
  </Keywords2>

```

G.2.12 已标记

介绍 AUTOMATIC TAGS 结构之前,ASN.1 规范常常包含标记。下面各条描述典型应用置标记的方法。随着 AUTOMATIC TAGS 的介绍,新 ASN.1 规范不必采用标记记法,尽管那些修改旧记法可能使其本身与标记有关。鼓励 ASN.1 记法的新用户使用 AUTOMATIC TAGS,因为它使记法更可读。

G.2.12.1 通用类别标记仅用于本文件内。提供记法[UNIVERSAL 30](示例)只是为了“UsefulTypes”(见 45.1)的定义精确。它不宜用在其他地方。

G.2.12.2 使用标记经常碰到的形式是在整个规范中分配应用层类别精确地标记一次,用它标识在规范内发现宽度、分散、用途的类型。也常常使用应用层类别标记(只有一次)来标记应用的最外 CHOICE 的类型,如果通过应用类别标识个人信息。下面是前面情形的举例用法:

例:

```

FileName ::= [APPLICATION 8]SEQUENCE{
    directoryName          VisibleString,
    directoryRelativeFileName VisibleString}

```

上面的示例假设默认编码引用为“empty”或 TAG。否则,上面的示例将被改写为:

```

FileName ::= [TAG: APPLICATION 8]SEQUENCE{
    directoryName          VisibleString,
    directoryRelativeFileName VisibleString}

```

在随后的示例中将需要进行类似的更改。

G.2.12.3 特定上下文置标记常常以代数方式应用于 SET、SEQUENCE 或者 CHOICE 的所有组件中。然而,要注意,AUTOMATIC TAGS 设施为你轻松地完成这些。

例:

```

CustomerRecord ::= SET{
    name          [0]VisibleString,
    mailingAddress [1]VisibleString,
    accountNumber [2]INTEGER,
    balanceDue     [3]INTEGER--以分计--}
CustomerAttribute ::= CHOICE{
    name          [0]VisibleString,
    mailingAddress [1]VisibleString,
    accountNumber [2]INTEGER,
    balanceDue     [3]INTEGER--以分计--}

```

G.2.12.4 专用类别置标记通常不宜用在国际化规范(尽管没有禁止这样做)。企业生产的应用通

常使用应用层和特定上下文标记类别。然而,特定企业的规范试图扩大国际标准化规范可能是常见的情形,并且在这种情况下,采用专用类别标记可能会在部分地保护特定企业规范变化成国际标准化规范中获得一些利益。

例:

```
AcmeBadgeNumber ::= [PRIVATE 2] INTEGER
```

```
badgeNumber AcmeBadgeNumber ::= 2345
```

或者用 XML 值记法:

```
badgeNumber ::= <AcmeBadgeNumber>2345</AcmeBadgeNumber>
```

G.2.12.5 文本使用带各个标记的 IMPLICIT 通常只能在较早的规范中找得到。使用隐式置标记时,BER 产生比使用显式置标记时更不简洁的表达式。PER 在两种情况下产生同样简洁的编码。用 BER 和显式置标记,已编码的数据中的重要类型(INTEGER、REAL、BOOLEAN,等等)具有更好的可视性。在这样做合法的示例中,这些指南采用隐式置标记。根据编码规则,这样可能产生在某些应用程序中非常渴望的简洁表达式。在其他应用中,例如,简洁可能没有进行强类型检查的能力重要。在后面的情况下,能够采用显式置标记。

例:

```
CustomerRecord ::= SET {
    name                [0]IMPLICIT VisibleString,
    mailingAddress       [1]IMPLICIT VisibleString,
    accountNumber        [2]IMPLICIT INTEGER,
    balanceDue           [3]IMPLICIT INTEGER--以分计--}
CustomerAttribute ::= CHOICE {
    name                [0]IMPLICIT VisibleString,
    mailingAddress       [1]IMPLICIT VisibleString,
    accountNumber        [2]IMPLICIT INTEGER,
    balanceDue           [3]IMPLICIT INTEGER--以分计--}
}
```

G.2.12.6 使用引用本文件的新 ASN.1 规范中的标记的指南十分简单:不要用标记。将 AUTOMATIC TAGS 放在模块头中,然后,不必管标记。如果你需要在后面的版本中增加新的组件给 SET、SEQUENCE 或者 CHOICE,将它们加在末尾。

G.2.13 选择

G.2.13.1 用 CHOICE 来为从总数已知且适中的变量集中选择的变量建模。

例:

```
FileIdentifier ::= CHOICE {
    relativeName VisibleString,
    --文件名称(例如,“MarchProgressReport”)
    absoluteName VisibleString,
    --文件名称并包含目录(例如,<Williams>MarchProgressReport”)
    serialNumber INTEGER
    --指派的系统的文件的标识符--}
file FileIdentifier ::= serialNumber:106448503
```

或者用 XML 值记法:

```
fileIdentifier ::=
```

```

<FileIdentifier>
  <serialNumber>106448503</serialNumber>
</FileIdentifier>

```

G.2.13.2 用可扩展 CHOICE 来为构成可能从协议的一个版本改变为另一个的变量集中选择的变量建模。

例：

```

FileIdentifier ::= CHOICE{ --FileIdentifier 的第一版
    relativeName      VisibleString,
    absoluteName      VisibleString,
    ..., ...
}

```

```
fileId1 FileIdentifier ::= relativeName: "MarchProgressReport.doc"
```

或者用 XML 值记法：

```

fileId1 ::=
  <FileIdentifier>
    <relativeName>MarchProgressReport.doc</relativeName>
  </FileIdentifier>

```

在预料中：

```

FileIdentifier ::= CHOICE{ --FileIdentifier 的第二版
    relativeName      VisibleString,
    absoluteName      VisibleString,
    ...,
    serialNumber      INTEGER, --第二版中增加的扩展附加
    ...
}

```

```
fileId1 FileIdentifier ::= relativeName: "MarchProgressReport.doc"
```

```
fileId2 FileIdentifier ::= serialNumber: 214
```

或者用 XML 值记法：

```

fileId1 ::=
  <FileIdentifier>
    <relativeName>MarchProgressReport.doc</relativeName>
  </FileIdentifier>

fileId2 ::=
  <FileIdentifier>
    <serialNumber>214</serialNumber>
  </FileIdentifier>

```

且后来还有：

```

FileIdentifier ::= CHOICE{ --FileIdentifier 的第三版
    relativeName      VisibleString,
    absoluteName      VisibleString,
    ...,
    serialNumber      INTEGER, --第二版中增加的扩展附加
    [[
    --第三版中增加的扩展附加

```

```

        vendorSpecific VendorExt,
        unidentified NULL
    ]],
    ...
}
fileId1 FileIdentifier ::= relativeName: "MarchProgressReport.doc"
fileId2 FileIdentifier ::= serialNumber: 214
fileId3 FileIdentifier ::= unidentified: NULL

```

或者用 XML 值记法:

```

fileId1 ::=
    <FileIdentifier>
        <relativeName>MarchProgressReport.doc</relativeName>
    </FileIdentifier>
fileId2 ::=
    <FileIdentifier>
        <serialNumber>214</serialNumber>
    </FileIdentifier>
fileId3 ::=
    <FileIdentifier>
        <unidentified/>
    </FileIdentifier>

```

G.2.13.3 预计在将来允许可能不止一种类型时,使用仅有一个类型的可扩展 CHOICE。

例:

```

Greeting ::= CHOICE{
    --"Greeting"的第一版
    postcard VisibleString,
    ...,
    ...
}

```

在预料中:

```

Greeting ::= CHOICE{
    --"Greeting"的第二版
    postCard VisibleString,
    ...,
    [[2: --第二版中增加的扩展附加
        audio Audio,
        video Video
    ]],
    ...
}

```

G.2.13.4 当选择值嵌套在另一选择值内时,要求用多个冒号。

例:

```

Greeting ::= [APPLICATION 12]CHOICE{
    postCard VisibleString,
    recording Voice}

```

```
Voice ::= CHOICE{
    english    OCTET STRING,
    swahili    OCTET STRING}
myGreeting Greeting ::= recording:english:'O19838547EO'H
```

或者用 XML 值记法:

```
myGreeting ::=
    <Greeting>
        <recording><english>019838547E0</english></recording>
    </Greeting>
```

G.2.14 精选类型

G.2.14.1 用精选类型为其类型是前面定义的 CHOICE 某些特定替换项的变量建模。

G.2.14.2 考虑定义:

```
FileAttribute ::= CHOICE{
    date-last-used  INTEGER,
    file-name       VisibleString}
```

然后,可能是下面的定义:

```
AttributeList ::= SEQUENCE{
    first-attribute  date-last-used<FileAttribute,
    second-attribute file-name<FileAttribute>}
```

可能的值记法:

```
listOfAttributes AttributeList ::=
    first-attribute    27,
    second-attribute   "PROGRAM"}
```

或者用 XML 值记法:

```
listOfAttributes ::=
    <AttributeList>
        <first-attribute>27</first-attribute>
        <second-attribute>PROGRAM</second-attribute>
    </AttributeList>
```

G.2.15 对象类别字段类型

G.2.15.1 用对象类别字段类型来标识用信息客体类别(见 GB/T 16262.2—2025)定义的类型。例如,信息客体类别 ATTRIBUTE 的范围可用于定义类型、Attribute。

例:

```
ATTRIBUTE ::= CLASS{
    &.Attribute Type,
    &.attributeId    OBJECT IDENTIFIER UNIQUE
}
Attribute ::= SEQUENCE{
    attributeID      ATTRIBUTE.&.attributeId, --这样通常是约束的
    attributeValue   ATTRIBUTE.&.AttributeType --这样通常是约束的
}
```

ATTRIBUTE.&.attributeId 和 ATTRIBUTE.&.AttributeType 都是对象类别字段类型,因为它们引用信息客体类别(ATTRIBUTE)定义的类型。因为类型 ATTRIBUTE.&.attributeId 在 ATTRIBUTE 中显式地定义为 OBJECT IDENTIFIER,因此它是固定的。然而,类型 ATTRIBUTE.&.AttributeType 能携带用 ASN.1 定义的任何类型的值,因为其类型在信息客体类别 ATTRIBUTE 的定义中没有固定。具有能携带任何类型值的这一特性的记法称为“开放类型记法”,即:ATTRIBUTE.&.AttributeType 是开放类型。

G.2.16 嵌入式 pdv

G.2.16.1 用嵌入式 pdv 类型来为其类型没有规定、或在无限制用来规定类型的记法的地方规定的变量建模。

例:

```
FileContents ::= EMBEDDED PDV
DocumentList ::= SEQUENCE OF document EMBEDDED PDV
```

G.2.17 外部

外部类型与嵌入式 pdv 类型相似,但是标识选择更少。新规范通常更愿意使用嵌入式 pdv,因为其更大的灵活性和用某些编码规则对其值进行编码更有效的事实。

G.2.18 单一实例

G.2.18.1 用单一实例规定包含对象标识符字段和其类型是对象标识符确定的开放类型值的类型。如果用从 TYPE-IDENTIFIER(见 GB/T 16262.2—2025 的附录 A 和附录 C)衍生来的类别的信息客体规定对象标识符值和类型之间的联系,只采用单一实例类型。

例:

```
ACCESS-CONTROL-CLASS ::= TYPE-IDENTIFIER
Get-Invoke ::= SEQUENCE{
    objectClass      ObjectClass,
    objectInstance   ObjectInstance,
    accessControl     INSTANCE OF ACCESS-CONTROL-CLASS, --这样通常是受
                                                             约束的
    attributeID       ATTRIBUTE.&.attributeId}
```

那么,调用相当于:

```
Get-Invoke ::= SEQUENCE{
    objectClass      ObjectClass,
    objectInstance   ObjectInstance,
    accessControl     [UNIVERSAL 8]IMPLICIT SEQUENCE{
        type-id      ACCESS-CONTROL-CLASS.&.id, --这样通常是受约束的。
        value        [0]ACCESS-CONTROL-CLASS.&.Type --这样通常是受约束的。
    },
    attributeID       ATTRIBUTE.&.attributeId
}
```

单一实例类型的真实效果要到用信息客体集合约束它才能看到,但是这样的示例超出了本文件的范围。信息客体集合的定义见 GB/T 16262.3—2025,而怎样用信息客体集合约束单一实例类型见 GB/T 16262.3—2025 的附录 A。

G.2.19 对象标识符

当二进制编码中需要 OID 树节点的紧凑数字标识时,需使用 OBJECT IDENTIFIER。

G.2.20 OID 国际化资源标识符

当需要使用包含所有大多数 Unicode 字符的名称,并且能够接受字符编码时,需使用 OID-IRI。OID-IRI 值也能够用作使用“oid”IRI/URI 方案的 IRI 或 URI[见 Rec.ITU-T X.660 (2011) | ISO/IEC 9834-1 的附录 F]。

G.2.21 相对对象标识符

G.2.21.1 用相对对象标识符类型来在已知对象标识符值的早期部分的上下文中以更简洁的形式传送对象标识符值。会出现三种情况:

- a) 对象标识符值的早期部分对给定规范(这是一个特定工业标准,而且所有的 OIDs 是相对于分配给标准化机构的 OID 的)是固定的。这时,使用:

RELATIVE-OID --相对对象标识符值是相对于{iso identified-organization set(22)}

- b) 对象标识符值的早期部分常常是在规范的时间内已知的值,但是,通常可能是更通用的值。这时,使用:

CHOICE

{a RELATIVE-OID --值是相对{1 3 22}--,

b OBJECT IDENTIFIER --任意对象标识符值--}

- c) 对象标识符值的早期部分直到通信时才知道,但是,常常将是许多需要发送的值共同的,而且在规范时间总是已知的值。这时,使用:

SEQUENCE

{oid-root OBJECT IDENTIFIER DEFAULT{1 3 22},

reloids SEQUENCE OF RELATIVE-OID --相对于 oid-root--}

G.3 值记法和属性设置(TIME 类型和有用时间类型)

本节提供时间类型的值记法示例。相同的值记法用于有用时间类型,但仅限于表示这些类型中存在的抽象值。每个示例都以正常的人类记法给出时间抽象值,然后对该值进行赋值,如果有一个有用时间类型包含它,则使用有用时间类型,否则使用 TIME 类型。以下注解给出了定义包含所有相似抽象值的 TIME 类型的子类型所需的设置。

G.3.1 日期

示例:

日历日期-1985 年 4 月 12 日:

date1 DATE::="1985-04-12" --Basic=Date Date=YMD Year=Basic

序号日期-1985 年 4 月 12 日:

date2 TIME::="1985-102" --Basic=Date Date=YD Year=Basic

星期日期-1985 年 4 月 12 日星期五:

date3 TIME::="1985-W15-5" --Basic=Date Date=YWD Year=Basic

日历周-1985 年的第 15 周:

date4 TIME::="1985-W15" --Basic=Date Date=YW Year=Basic

日历月-1985 年 4 月:

date5 TIME::="1985-04" --Basic=Date Date=YM Year=Basic

日历年-1985 年:

date6 TIME::="1985" --Basic=Date Date=Y Year=Basic

日历日期-11985 年 4 月 12 日:

date7 TIME::="+11985-04-12" --Basic=Date Date=YMD Year=L5

0000 年前第二年的 4 月 12 日:

date8 TIME::="-0002-04-12" --Basic=Date Date=YMD Year=Negative

20 世纪:

date9 TIME::="19C" --Basic=Date Date=C Year=Basic

G.3.2 当日时间

示例:

15 时 27 分 46 秒:

time1 TIME-OF-DAY::="15:27:46"

--Basic=Time Time=HMS Local-or-UTC=L

到最近的分钟:

time2 TIME::="15:28"

--Basic=Time Time=HM Local-or-UTC=L

使用逗号的小数部分的当地时间-15 时 27 分 35 秒半:

time3 TIME::="15:27:35,5"

--Basic=Time Time=HMSF1 Local-or-UTC=L

UTC-23 时 20 分 30 秒:

time4 TIME::="23:20:30Z"

--Basic=Time Time=HMS Local-or-UTC=Z

到最近的小时:

time5 TIME::="23Z"

--Basic=Time Time=H Local-or-UTC=Z

当地时间和与 UTC 的时差-日内瓦当地 15 时 27 分 46 秒(比 UTC 提前一小时):

time6 TIME::="15:27:46+01:00"

--Basic=Time Time=HMS Local-or-UTC=LD

相同抽象值的替代值记法:

time7 TIME::="15:27:46+01"

--Basic=Time Time=HMS Local-or-UTC=LD

纽约当地 15 时 27 分 46 秒(比 UTC 晚 5 小时):

time8 TIME::="15:27:46-05:00"

--Basic=Time Time=HMS Local-or-UTC=LD

G.3.3 日期和当日时间

示例:

日历日期和当地时间的组合:

date-time1 DATE-TIME::="1985-04-12T10:15:30"

--Basic=Date-Time Date=YMD Year=Basic Time=HMS

--Local-or-UTC=L

日历日期和带有时差的当地时间的组合;当地时间是 1985 年 4 月 1 日 01:30;该位置的 UTC 时间是 1985 年 3 月 31 日 23:30:

date-time2 TIME::="1985-04-01T01:30:00+02.00"

--Basic=Date-Time Date=YMD Time=HMS Local-or-UTC=LD

顺序日期和 UTC 的组合:

```
date-time3 TIME::="1985-10T23:50:30Z"
--Basic=Date-Time Date=YD Year=Basic Time=HMS Local-or-UTC=Z
星期日期和当地时间的组合:
date-time4 TIME::="1985-W14-5T23:50:30"
--Basic=Date-Time Date=YWD Year=Basic Time=HMS
--Local-or-UTC=L
```

G.3.4 时间间隔

示例:

时间间隔从 1985 年 4 月 12 日 23 时 20 分 50 秒开始,到 1985 年 6 月 25 日 10 时 30 分结束:

```
interval1 TIME::="1985-04-12T23:20:50/1985-06-25T10:30:00"
```

```
--Basic=Interval Interval-type=SE SE-point=Date-Time
--Date=YMD Year=Basic Time=HMS Local-or-UTC=L
```

时间间隔从 1985 年 4 月 12 日的当地时间 12 时 30 分(UTC 时间 10 时 30 分)开始,到 4 月 12 日 13 时 30 分结束,时差相同(这不作要求):

```
interval2 TIME::="1985-04-12T12:30:00+02:00/1985-04-12T13:30:00+02:00"
```

```
--Basic=Interval Interval-type=SE SE-point=Date-Time
--Date=YMD Year=Basic Time=HMS Local-or-UTC=L
```

相同抽象值的替代值记法,省略了秒时差:

```
interval3 TIME::="1985-04-12T12:30:00+02:00/1985-04-12T13:30:00"
```

```
--Basic=Interval Interval-type=SE SE-point=Date-Time
--Date=YMD Year=Basic Time=HMS Local-or-UTC=L
```

时间间隔从 1985 年 4 月 12 日开始到 1985 年 6 月 25 日结束:

```
interval4 TIME::="1985-04-12/1985-06-25"
```

```
--Basic=Interval Interval-type=SE SE-point=Date
--Date=YMD Year=Basic
```

2 年 10 个月 15 天 10 时 20 分 30 秒的时间间隔:

```
duration1 DURATION::="P2Y10M15DT10H20M30S"
```

```
--Basic=Interval Interval-type=D
```

1 年 6 个月的时间间隔:

```
duration2 DURATION::="P1Y6M"
```

```
--Basic=Interval Interval-type=D
```

72 小时的时间间隔:

```
duration3 DURATION::="PT72H"
```

```
--Basic=Interval Interval-type=D
```

时间间隔为 1 年 2 个月 15 天 12 时,从 1985 年 4 月 12 日 23 时 20 分开始:

```
interval5 TIME::="1985-04-12T23:20:00/P1Y2M15DT12H"
```

```
--Basic=Interval Interval-type=SD SE-point=Date-Time
--Date=YMD Year=Basic Time=HMS Local-or-UTC=L
```

时间间隔为 1 年 2 个月 15 天 12 时,于 1985 年 4 月 12 日 23 时 20 分结束:

```
interval6 TIME::="P1Y2M15DT12H/1985-04-12T23:20:00"
```

```
--Basic=Interval Interval-type=DE SE-point=Date-Time
--Date=YMD Year=Basic Time=HMS Local-or-UTC=L
```

G.3.5 循环时间间隔

示例:

以 2 年 10 个月 15 天 10 时 20 分 30 秒的时间间隔重复十五次:

```

rec-int1 TIME::="R15/P2Y10M15DT10H20M30S"
--Basic=Rec-Interval Recurrence=R2 Interval-type=D
时间间隔为 2 年 15 天 10 时 20 分 30 秒的无限次重复:
rec-int2 TIME::="R/P2Y15DT10H20M30S"
--Basic=Rec-Interval Recurrence=Unlimited Interval-type=D
时间间隔分别为 1 年和 6 个月的两次重复:
rec-int3 TIME::="R2/P1Y6M"
--Basic=Rec-Interval Recurrence=R1 Interval-type=D
时间间隔为 1 年 2 个月 15 天 12 时的无限次重复,其中最后一次重复在 1985 年 4 月 12 日 23 时 20 分 50 秒时结束:
rec-int4 TIME::="R/P1Y2M15DT12H/1985-04-12T23:20:50"
--Basic=Rec-Interval Recurrence=Unlimited Interval-type=DE
--SE-point=Date-Time Date=YMD Year=Basic Time=HMS
--Local-or-UTC=L

```

G.4 标识抽象语法

G.4.1 常常用语义与单个 ASN.1 类型(在选择类型时典型)的每个值相关来定义协议。(该 ASN.1 类型有时非形式地引用为“应用的顶层类型”。)抽象值的集合形式上称为应用的抽象语法。抽象语法能通过给它一个 ASN.1 类型对象标识符的抽象语法名来标识。

G.4.2 能用 GB/T 16262.2—2025 中定义的固有信息客体类别 ABSTRACT-SYNTAX 完成抽象语法的对象标识符赋值。这也用来清楚地标识应用层的顶层类型。

G.4.3 下面是可能在应用规范中出现的文本示例:

例:

```

Application-ASN1 DEFINITIONS::=
    BEGIN
    EXPORTS Application-PDU;
    Application-PDU::=CHOICE{
        connect-pdu.....,
        data-pdu CHOICE{
            .....,
            .....,
        },
        .....,
    }
    .....
END
Abstract-Syntax-Module DEFINITIONS::=
    BEGIN
    IMPORTS Application-PDU FROM Application-ASN1;
--本应用定义下面的抽象语法:
    abstract-Syntax ABSTRACT-SYNTAX::=
        {Application-PDU IDENTIFIED BY
            application-abstract-syntax-object-id}
    application-abstract-syntax-object-id OBJECT IDENTIFIER::=
        {joint-iso-itu-t asn1(1)examples(123)application-abstract-syntax(3)}

```

--对应的对象描述符是:

application-abstract-syntax-descriptor ObjectDescriptor ::=

"Example Application Abstract Syntax"

--ASN.1 对象标识符和对象描述符值:

--编码规则对象标识符

--编码规则对象描述符

--指派给 ISO/IEC 8825-1:2021 或 ISO/IEC 8825-2:2021 的编码规则能用作和

--这一传送语法一起的传送语法标识符。

END

G.4.4 为了保证能互工作,标准可能额外地标识一个强制传送语法[典型是,ISO/IEC 8825-1:2021 或 ISO/IEC 8825-2:2021 或 Rec.ITU-T X.692 (2021) | ISO/IEC 8825-3:2021 的编码规则中定义的那些之一]。

G.5 子类型

G.5.1 用子类型限制特殊情况下允许的现存类型的值。

例:

AtomicNumber ::= INTEGER(1..104)

TouchToneString ::= IA5String

(FROM("0123456789"|"*"|"#"))(SIZE(1..63))

ParameterList ::= SET SIZE(1..63) OF Parameter

SmallPrime ::= INTEGER(2|3|5|7|11|13|17|19|23|29)

G.5.2 用可扩展子类型约束来为允许的值集小且很好定义了,但预计会增加的 INTEGER 类型建模。

例:

SmallPrime ::= INTEGER(2|3,...)--SmallPrime 的第一版

在预料中:

SmallPrime ::= INTEGER(2|3,...,5|7|11)

--SmallPrime 的第二版的

且以后还有:

SmallPrime ::= INTEGER(2|3,...,5|7|11|13|17|19)

--SmallPrime 的第三版

注: 对于某些类型,某些编码规则(如 PER)为子类型约束扩展根值(即:"..."之前出现的值)提供极好的优化编码,而为子类型约束扩展附加值(即:"..."之后出现的值)提供较少的优化编码,而在某些其他编码规则(如 BER)中子类型约束不影响编码。

G.5.3 当两个或多个相关的类型有重要共性时,考虑显性定义其共同的父类为一个类型而对单独的类型使用子类型。这样使其关系和共性清楚,并鼓励(尽管不强迫)这样随类型发展而继续进行。因此,便于采用处理这些类型的值的共同实现方法。

例:

Envelope ::= SET{

typeA TypeA,

typeB TypeB OPTIONAL,

typeC TypeC OPTIONAL}

--共同的父类

ABEnvelope ::= Envelope(WITH COMPONENTS

```

{...,
  typeB PRESENT, typeC ABSENT})
--typeB 一定要总是出现, 而 typeC 不是
ACEnvelope ::= Envelope(WITH COMPONENTS
  {...,
  typeB ABSENT, typeC PRESENT})
--typeC 一定要总是出现, 而 typeB 不是

```

后者的定义能用下面的表达式替换:

```

ABEnvelope ::= Envelope(WITH COMPONENTS{typeA, typeB})
ACEnvelope ::= Envelope(WITH COMPONENTS{typeA, typeC})

```

根据双亲类型中组件数量, 以及那些可选择的数量、个体类型之间差异的程度和可能发展的策略等因素考虑替换项之间的选择。

G.5.4 用子类型来部分地定义值, 例如, 在测试仅关心 PDU 的某些组件时, 一致性测试中用来测试的协议数据单元。

例:

```

Given:
PDU ::= SET
{alpha    INTEGER,
 beta     IA5String OPTIONAL,
 gamma    SEQUENCE OF Parameter,
 delta    BOOLEAN}

```

那么在要求布尔为假和整数为负的综合测试中, 写作:

```

TestPDU ::= PDU(WITH COMPONENTS
{...,
 delta(FALSE),
 alpha(MIN..<0)})

```

而进一步, 如果 IA5string、beta 出现, 而且长度为 5 或 12 个字符, 则写成:

```

FurtherTestPDU ::= TestPDU(WITH COMPONENTS{..., beta(SIZE(5|12))PRESENT})

```

G.5.5 如果一般目的数据类型已经被定义为 SEQUENCE OF, 用子类型来定义一般类型的受限制子类型。

例:

```

Text-block ::= SEQUENCE OF VisibleString
Address ::= Text-block(SIZE(1..6))(WITH COMPONENT(SIZE(1..32)))

```

G.5.6 如果一般目的数据类型已经被定义为 CHOICE, 用子类型来定义一般类型的受限制子类型。

例:

```

Z ::= CHOICE{
  a    A,
  b    B,
  c    C,
  d    D,
  e    E
}
V ::= Z(WITH COMPONENTS{..., a ABSENT, b ABSENT})

```

--a 和 b 一定缺失，
--c、d 或 e 可在值中出现。

W::=Z(WITH COMPONENTS{...,a PRESENT}) --只有 a 能出现(见 51.8.10.2)。

X::=Z(WITH COMPONENTS{a PRESENT}) --只有 a 能出现(见 51.8.10.2)。

Y::=Z(WITH COMPONENTS{a ABSENT,b,c})

--a、d 和 e 一定缺失，
--b 或 c 可在值中出现。

注：W 和 X 语义相同。

G.5.7 用包含的子类型来从现存的子类型中形成新子类型。

例：

```
Months::=ENUMERATED{
    january          (1),
    february         (2),
    march            (3),
    april            (4),
    may              (5),
    june             (6),
    july             (7),
    august           (8),
    september        (9),
    october          (10),
    november         (11),
    december         (12)}

First-quarter::=Months(january|february|march)
Second-quarter::=Months(april|may|june)
Third-quarter::=Months(july|august|september)
Fourth-quarter::=Months(october|november|december)
First-half::=Months(First-quarter|Second-quarter)
Second-half::=Months(Third-quarter|Fourth-quarter)
```

G.5.8 38.4 中提供了时间类型的子类型示例,G.3 中的注解中给出了一些有用的设置。后面还有其他示例,并附有注解。注意,子类型的所有示例也能够应用于有用时间类型,但只会选择那些类型中已经存在的抽象值。此记法的主要用途是提供有用时间类型的变体。

示例：

```
My-Date::=TIME
(SETTINGS"Basic=Date Year=Basic Date=YD")
--使用年和日的日期类型

My-Date1::=TIME
(SETTINGS"Basic=Date Year=Basic Date=YD")
("2000-001"..<"2011-001")
--使用年和日的日期类型,仅限于
--从公元 2000 年 1 月 1 日到公元 2010 年 12 月 31 日(含)。

My-Date2::=TIME
("2000-001"..<"2011-001")
--与 My-Date1 相同的日期类型,但可能对于人类用户不太清晰。
```

--它依赖于从值记法(见附录 K)中推导出来的属性设置。

My-Illegal-Date::=TIME

("1500-01"..<"2011-01")

--下限是预测日期,而上限是

--基本日期,因此它们不具有相同的属性

--且这是非法的。

My-time-of-day-1::=TIME

(SETTINGS"Basic=Time Time=HMS Local-or-UTC=L

Midnight=Start")

--这与 TIME-OF-DAY 相同,但在一天结束时的午夜

--不包括在内,唯一的午夜

--用值记法“00:00:00”表示。

My-time-of-day-2::=TIME

(SETTINGS"Basic=Time Time=HMS Local-or-UTC=L

Midnight=End")

--这与 TIME-OF-DAY 相同,但在一天开始时的午夜

--不包括在内,唯一的午夜

--用值记法“24:00:00”表示。

My-time-of-day-3::=TIME

(SETTINGS"Basic=Time Time=HMS Local-or-UTC=Z")

--这与 TIME-OF-DAY 相同,但时间是 UTC,而不是

--当地时间。

附 录 H

(资料性)

ASN.1 字符串的辅导附录

H.1 ASN.1 中的字符串支持

H.1.1 在 ASN.1 中有四个字符串支持组。这四个组是：

- a) 基于与转义序列一起使用的编码字符集的 ISO 国际登记(即：基于 ISO/IEC 646 的结构)和相关编码字符集的国际登记,以及类型 VisibleString、IA5String、TeletexString、VideotexString、GraphicString 和 GeneralString 提供的字符串类型；
- b) 基于 GB/T 13000—2010 的,用有 GB/T 13000—2010 中定义的子集的 UniversalString、UTF8String 或 BMPString 类型的分成子集或用已命名字符提供的字符串类型；
- c) 提供本文件中规定的字符的简单小集的,以及预定特殊用途的字符串类型,这些是 NumericString 和 PrintableString 类型；
- d) 使用类型 CHARACTER STRING,与所用的字符集协商(或宣称使用的集)；它允许实现采用赋予了 OBJECT IDENTIFIERs(包括与转义序列一起使用的编码字符集的 ISO 国际登记)、ISO/IEC 7350、GB/T 13000—2010 的字符集和编码的任意集合,以及字符和编码的专用集合(文件可能强制要求或限制所用的字符集-字符抽象语法)。

H.2 UniversalString, UTF8String 和 BMPString 类型

H.2.1 UniversalString 和 UTF8String 类型携带取自 GB/T 13000—2010 的任何字符。GB/T 13000—2010 中的字符集对于所要求的意义相一致的场合通常太大,而且正常情况下,宜分成 GB/T 13000—2010 的附录 A 中字符的标准集合组合的子集。

H.2.2 BMPString 类型携带取自 GB/T 13000—2010 基本多文种平面的任何字符。正常情况下,基本多文种平面分成 GB/T 13000—2010 的附录 A 中字符的标准集合组合的子集。

H.2.3 对于 GB/T 13000—2010 的附录 A 中定义的集合,存在固有 ASN.1 模块“ASN1-CHARACTER-MODULE”(见第 42 章)中定义的类型引用。“子类型约束”机制允许定义现存子类型组合的 UniversalString 的新子类型。

H.2.4 在 ASN1-CHARACTER-MODULE 中定义的类型引用的示例与它们对应 GB/T 13000—2010 的汇集名是：

BasicLatin	BASIC LATIN
Latin-1 Supplement	LATIN-1 SUPPLEMNT
LatinExtended-a	LATIN EXTENDED-A
LatinExtended-b	LATIN EXTENDED-B
IpaExtensions	IPA EXTENSIONS
SpacingModifierLetters	SPACING MODIFIER LETTERS
CombiningDiacriticalMarks	COMBINING DIACRITICAL MARKS

H.2.5 GB/T 13000—2010 规定三个“实现级”,并要求 GB/T 13000—2010 的所有用法规定实现级。

实现级与为字符汇中组合用字符给出支持的程度有关,因此,用 ASN.1 术语,定义 UniversalString 和 BMPString 受限字符串类型的子集。

在 1 级实现中,不允许组合字符,而且通常 ASN.1 字符串中的抽象字符与串的物理表现中的打印字符一一对应。

在 2 级实现中,能够使用某些组合用字符(列于 GB/T 13000—2010 的附录 B 中),但是也有禁止其使用的其他情况。

在 3 级实现中,不限制使用组合用字符。

H.2.6 用如下的子类型记法能限制 BMPString 和 UniversalString 排除所有的控制功能:

```
VanillaBMPString ::= BMPString (FROM (BMPString (SIZE (1)) EXCEPT ({0,0,0,0}..{0,0,0,31} | {0,0,0,128}..{0,0,0,159})))
```

或相当的:

```
C0 ::= BMPString (FROM ({0,0,0,0}..{0,0,0,31})) --C0 控制功能
```

```
C1 ::= BMPString (FROM ({0,0,0,128}..{0,0,0,159})) --C1 控制功能
```

```
VanillaBMPString ::= BMPString (FROM (BMPString (SIZE (1)) EXCEPT (C0 | C1)))
```

H.3 关于 GB/T 13000—2010 一致性要求

使用 ASN.1 类型定义中 UniversalString、BMPString 或 UTF8String (或其子类型) 要求指出 GB/T 13000—2010 的一致性要求。

这些一致性要求需要使用这种 ASN.1 类型的标准 (例如 X) 实现者为其标准 X 的实现提供 GB/T 13000—2010 合适的子集和实现级 (支持组合字符) 的声明。

使用规范中 UniversalString、BMPString 或 UTF8String 的 ASN.1 子类型要求实现支持那个 ASN.1 子类型中包括的全部 GB/T 13000—2010 字符,也就是,实现的合适子集中 (至少) 出现那些字符。也要求对所有那种 ASN.1 子类型支持陈述的级。

注: ASN.1 规范 (缺少抽象语法参数和例外规范) 确定被传输的 (最大) 字符集和收到后进行处理的 (最小) 字符集。

被采用的 GB/T 13000—2010 的集要求不传输超过这个集的字符,而且收方支持这个集内的所有字符。因此,采用的集要求确切地是 ASN.1 规范允许的所有字符的集。下面讨论抽象语法有参数的情形。

H.4 对 ASN.1 用户的 GB/T 13000—2010 一致性建议

ASN.1 用户宜清楚,如果满足他们的标准的实现,将形成采用的实现子集 (和要求的实现水平) 的 GB/T 13000—2010 字符集。

通过定义包含标准需要的所有字符的 UniversalString、UTF8String 或 BMPString 的 ASN.1 子类型能够方便地完成这一任务,如果合适的话,并通过限制它到“Level1”或“Level2”。这个类型的方便名字可能是“ISO-10646-String”。

例:

```
ISO-10646-String ::= BMPString
```

```
(FROM (Level2 INTERSECTION (BasicLatin UNION HebrewExtended UNION Hiragana)))
```

——这是定义为实现该标准采用的子集中字符最小集的类型。

——要求实现级至少是 2 级。

在 OSI 环境中,OSI 协议实现一致性声明那时将包含一个简单的声明:采用的 GB/T 13000—2010 子集是“ISO-10646-String”定义的有限子集 (和级),而且,“ISO-10646-String” (可能分成了子类型) 将在包含 GB/T 13000—2010 串的整体标准中使用。

EXAMPLE COMFORMANCE STATEMENT

采用的 GB/T 13000—2010 子集是由模块〈模块名称〉中定义的 ASN.1 类型 ISO-10646-String 中的所有字符组成的有限子集,具有 2 级实现。

EXAMPLE USE IN PROTOCOL

```
Message ::= SEQUENCE {
```

```
    first-field ISO-10646-String, --采用子集中的所有字符会出现
```

```

second-field ISO-10646-String
    (FROM(latinSmallLetterA..latinSmallLetterZ))--只有小写拉丁字母
third-field ISO-10646-String,(FROM(digitZero..digitNine))--只有数字
}

```

H.5 用子集作为抽象语法的参数

GB/T 13000—2010 要求显式地定义实现采用的子集和级数。当 ASN.1 用户不希望约束被定义标准的某些部分中的 GB/T 13000—2010 字符字段时,这能够通过定义 ISO-10646-String(例如)为 UniversalString、BMPString 或 UTF8String 的子类型,它们带由(或包括)留下作为抽象语法参数的 ImplementorsSubset 构成的子类型约束。

告诫 ASN.1 用户,在这样的情况下,合格的发送者可传送不能被接收者处理的字符给合格的接收者,因为它们处于接收者(实现依赖)采用的子集或级数之外,同时宜在这样的情况下,ISO-10646-String 定义中包括例外处理规范。

例:

```

ISO-10646-String{UniversalString:ImplementorsSubset,ImplementationLevel}::=
    UniversalString(FROM((ImplementorsSubset UNION BasicLatin)
        INTERSECTION ImplementationLevel)! characterSetProblem)
——GB/T 13000—2010 采用的子集应包括“BasicLatin”,但可能也包括抽象语法参数。
——“ImplementorsSubset”中规定的任何附加字符。抽象语法参数“ImplementationLevel”。
——定义实现级数。合格的接收者需准备接收它采用的子集和实现级数之外的字符。
——这时,调用关于“characterSetProblem”的第<这里增加你的章号>章规定的例外处理。
——注意,如果通信实例中采用的实际字符限制为“BasicLatin”,
——合格的接收者从不可能调用它。

My-Level2-String::=ISO-10646-String{{HebrewExtended UNION Hiragana},Level2}

```

H.6 CHARACTER STRING 类型

H.6.1 CHARACTER STRING 类型给出字符集的选择和编码方法中的完整灵活性。

注:当单个连接提供端到端数据传送(无中继)时,而且使用 OSI 协议,那么,所用的字符集协商及其编码能作为字符抽象语法的 OSI 表达式上下文的定义的一部分完成。否则,由一对对象标识符值声明抽象和传送字符语法(字符表和编码)。

H.6.2 在形式术语中,字符抽象语法是对可能值(它们是所有字符串,而且确实是取自字符的某些集合形成的所有字符串)有某些限制的普通抽象语法。因此,对字符抽象和传送语法的对象标识符值的分配以普通的方式进行。

H.6.3 CHARACTER STRING 编码要声明所用字符汇中的抽象和传送语法(即,字符集和编码)。在 OSI 环境中,这两种语法都可能协商。

H.6.4 字符抽象语法(和对应的字符传送语法)已经在许多我国标准、ITU-T 建议和国际标准中定义,而且任何能分派对象标识符的组织能定义附加字符抽象语法(和/或字符传送语法)。

H.6.5 在 GB/T 13000—2010 中,具有为整个字符集合、为每个子集(BASIC LATIN,BASIC SYMBOLS,等等)定义的字符集合,以及为字符集合定义的每个可能组合所定义的字符抽象语法(和分派的对象标识符)。还定义了两个字符传送语法来标识 GB/T 13000—2010 中的各种选择(特别是 16 位和 32 位)。

附录 I

(资料性)

类型扩展 ASN.1 模块的辅助附录

I.1 概述

I.1.1 ASN.1 类型会随着时间的过去通过称为扩展附加的系列扩展手段从 extension root 类型转变而来。

I.1.2 能进行特定实现的 ASN.1 类型可能是扩展根类型,或可能是扩展根类型加一个或多个扩展附加。每个这种包含扩展附加的 ASN.1 类型也包含所有前面定义的扩展附加。

I.1.3 将本系列中的 ASN.1 类型定义说成是相关扩展(“相关扩展”的更精确定义见 3.8.38),而且要求编码规则以下面的方式对相关扩展类型编码:如果两个系统使用相关扩展的两个不同类型,在两个系统之间的转换将成功地传送两个系统共同的相关扩展类型那部分的信息内容。也要求不属于两个系统共同的那部分能解除限制并在后面的传输中重新传输(也许对第三方),倘若使用相同的传送语法。

注:发送者可能用扩展附加系列中较早或者较晚的类型。

I.1.4 逐步增加根类型获得的系列类型称为扩展系列。为了这些编码规则有合适的准备来传输相关扩展类型(可能要求行里有多位),这种类型(包括扩展根类型)需要语法上加标志。标志是一个省略号(...),并称作扩展标志。

例:

Extension root type	1 st extension	2 nd extension	3 rd extension
A ::= SEQUENCE{ a INTEGER, ... }	A ::= SEQUENCE{ a INTEGER, ... b BOOLEAN, c INTEGER }	A ::= SEQUENCE{ a INTEGER, ... b BOOLEAN, c INTEGER, d SEQUENCE{ e INTEGER, f IA5String } }	A ::= SEQUENCE{ a INTEGER, ... b BOOLEAN, c INTEGER, d SEQUENCE{ e INTEGER, ... g BOOLEAN OPTIONAL, h BMPString, ... f IA5String } }

I.1.5 将序列、集和选择类型中的所有扩展附加插入在扩展标志对之间。允许有单个扩展标志,只要(在扩展根类型中)它在类型中作为最后一项出现,这时,假设一个匹配的扩展标志正好在类型的结束括号之前存在;在这些情况下,将所有扩展附加插入在类型的末尾。

I.1.6 有扩展标志的类型能嵌套在没有的类型内,或能嵌套在扩展根中的类型内,或者能嵌套在扩展附加类型内。在这些情况下,应独立地对待扩展系列,而且有扩展标志的被嵌套类型没有与它嵌套其中的类型相冲突。在任何规定的结构中只能出现一个扩展插入点(如果使用单个扩展标志,则在类型末尾,或者如果使用一对扩展标志,则刚好在第二个扩展标志之前)。

I.1.7 单个扩展附加组(一个或多个嵌套在“[[”“]]”内的类型)或单个加在扩展插入点上的类型的术语

中定义了扩展系列中的新扩展附加。在下面的示例里,第一种扩展定义了扩展附加组,其中 b 和 c 一定在类型“A”的值中同时出现或同时缺失。第二种扩展定义了单个成分类型 d,它可在类型“A”的值中缺失。第三种扩展定义了扩展附加组,其中只要值中出现新增加的扩展附加组,h 一定在类型“A”的值中出现。

例:

Extension root type	1 st extension	2 nd extension	3 rd extension
A ::= SEQUENCE{ a INTEGER, ... }	A ::= SEQUENCE{ a INTEGER, ... , [[b BOOLEAN, c INTEGER]] }	A ::= SEQUENCE{ a INTEGER, ... , [[b BOOLEAN, c INTEGER,]] , d SEQUENCE{ e INTEGER, ... , ... , f IA5String } }	A ::= SEQUENCE{ a INTEGER, ... , [[b BOOLEAN, c INTEGER,]] , d SEQUENCE{ e INTEGER, ... , [[g BOOLEAN OPTIONAL, h BMPString,]] , ... , f IA5String } }

I.1.8 也可能为版本括号增加新的版本号,但是只有它出现在模块内的所有括号中,而且只有模块中的所有扩展在版本括号内时才如此。宜使用版本号。省略号码和版本括号的能力是历史的原因(本文件的较早版本中不允许有版本括号和版本号)。也见 I.3。

I.1.9 随着时间的过去,对扩展附加将增加正常实用时,重要的 ASN.1 模块和规范不包括时间。如果一个能通过扩展附加从另一个“生成”,则这两个类型是相关扩展。也就是,一个包含另一个的所有组件。可能有在相反方向(尽管不太可能)“生成”的类型。随着时间的过去,它甚至可能是一个以许多逐步删去的扩展附加开头的类型。ASN.1 及其编码规则关注的一切是一对扩展规范是否是相关扩展。如果它们是,那么所有 ASN.1 编码规则保证它们的用户之间能互工作。

I.1.10 我们从一个类型开始,然后,如果我们后来要扩展它,决定是否要与较早的版本实现互工作。如果是的话,我们要包含现在的扩展标志。然后,我们能增加后面的扩展附加给带通过较早的系统定义好处理扩展值的类型。然而,注意增加一个扩展标志给以前没有(或删除扩展标志)的类型可能阻碍互工作。

注:当使用 ECN 时,在第二版中可能在第一版中没有扩展标志的地方增加扩展,而且,在第一版与第二版之间仍然保持互工作。

I.1.11 表 I.1 表明能形成 ASN.1 扩展系列的扩展根类型的 ASN.1 类型,以及那一类型(当然,连续或作为扩展组一起能构成多扩展附加)允许的单个扩展附加的性质。

表 I.1 扩展附加

扩展根类型	扩展附加性质
ENUMERATED	在“AdditionalEnumeration”后面附加单个进一步枚举,用比已经增加的任何枚举的都大的枚举值
SEQUENCE 和 SET	在“ExtensionAdditionList”后面附加单个类型或扩展附加组。是扩展附加(不包含在扩展附加组中)的“ComponentType”不要求被标志 OPTIONAL 或 DEFAULT,尽管这将是经常发生的情况
CHOICE	在“ExtensionAdditionAlternativesList”后面附加单个“NamedType”
约束记法	将单个“AdditionalElementSetSpec”附加到“ElementSetSpecs”记法中

I.2 版本号的意义

- I.2.1 在 BER 和 PER 编码中不使用版本号。由 ECN 规范确定它们在 ECN 编码中的用途(如果有的话)。
- I.2.2 当它们与解码完整 PDU 的方式而不是与单个的类型相关时版本号最有用。类型用作几个协议的组件并且因此贡献给不同的完整 PDU 时,那个类型的附加通常要求增加它贡献的所有 PDU 的版本号。
- I.2.3 当用来提供被开发系统间的互工作时,版本号宜以下面的方式用在扩展附加组:被开发的系统有带出版版本号(不管它们在协议内哪里出现)的所有扩展附加组的语法和语义知识,以及带较早版本号的所有扩展附加组的知识。ECN 规定者通常将假设已经按照这一原则分配了版本号(给适用于 ECN 的类型的部分)。

I.3 编码规则要求

- I.3.1 抽象语法能定义为可扩展类型的单个 ASN.1 类型的值。然后,它含有所有能用加上或删除扩展附加获得的值。这样的抽象语法称作相关扩展抽象语法。
- I.3.2 相关扩展抽象语法的很好形成的编码规则集满足 I.3.3~I.3.5 中描述的附加要求。
注:所有 ASN.1 编码规则满足这些要求。
- I.3.3 将抽象值转换成编码来传输和将收到的编码转换成抽象值的程序定义应认识到,发送者和接收者使用不等同、但均为相关扩展的抽象语法的可能性。
- I.3.4 在这种情况下,编码规则应保证发送者有比接收者在扩展系列中较早的类型规范时,发送者的值应以接收者能确定扩展附加不出现的方式传送。
- I.3.5 编码规则应保证发送者有比接收者在扩展系列中更晚的类型规范时,应可能将那个类型的值传送到接收者。

I.4 (可能可扩展的)约束的组合

I.4.1 模块

- I.4.1.1 应用约束的基本 ASN.1 模块是简单的:类型是抽象值集,而且加在其上的约束选择这些抽象值的子集。如果未约束的类型是不可扩展的,那么产生的类型如果且只有所应用的约束被定义为可扩展时才定义为可扩展的。
- I.4.1.2 即使在这种简单的情况下,要澄清一个特点:类型可能在形式上可扩展,即使从来不可能有任何扩展附加。考虑:

A::=INTEGER(MIN..MAX,...,1..10)

像本附录中给出的许多示例一样,这是没人曾经写过的东西,但是哪个工具零售商都应写代码,因为 ASN.1 标准给出的是简单和通用的,而因此,这个示例是合法的 ASN.1。在这个示例中,A 在形式上是可扩展整数,并具有根中整数值的整个范围。

I.4.1.3 复杂性的三个主要来源:

- 约束应用于已经有可扩展约束作用于它的类型(约束的系列应用见——I.4.2);
- 用 UNION 和 INTERSECTION 和 EXCEPT 组合可扩展约束(集合运算——见 I.4.3);
- 当类型引用去引用可扩展类型时(也许有实际扩展附加——见 I.4.4),使用约束的集合运算中的类型引用(包含的子类型)。

I.4.2 约束的系列应用

I.4.2.1 当类型受到约束(在类型引用的赋值中)和类型引用随后与应用于它的进一步约束一起使用时,出现约束的系列应用。

I.4.2.2 当类型以系列的方式有多种约束直接应用于它时,它也能出现,但是不常见。本附录中的很多示例采用后面的形式(为了简单说明),但是类型引用链接两个(或多个)约束的情况是实际规范中通常出现系列应用的形式。

I.4.2.3 在约束的系列应用中有以下两个重点:

- 如果受约束类型是可扩展的(和也许已扩展的),如果随后系列应用进一步约束,则丢弃“可扩展”标志和所有扩展附加。受约束类型(和任何扩展附加)的可扩展性仅仅依赖于所用的最后约束,能只引用要进一步受约束(双亲类型)的类型的根中的值。在根中或结果类型的扩展附加中包含的值只能是双亲类型根中的值。
- 约束的系列应用(对复杂的情况)与集合运算交叉不一样,即使在不包括可扩展性时也如此。首先,解释 MIN 和 MAX 的环境,其次是能在第二个约束中引用的抽象值在系列应用中与规定两个约束为取自共同父类的交叉值时的情况完全不同。

注:在整数类型的约束中使用 20..28 这样的范围是合法的,如果(也只有)20 和 28 都在双亲类型(根)中,但是该范围规范引用的值仅仅是父类(根)中的那些。因此,如果父类已经约束排除值 24 和 25,则范围 20..28 只引用 20~23 和 26~28。

下面有一些示例:

A1::=INTEGER(1..32,...,33..128)

- A1 是可扩展的,并且包含值 1~218,1~32 在根中,而且 33~128 作为扩展附加。

B1::=A1(1..128)

- 或相当的

B1::=INTEGER(1..32,...,33..128)(1..128)

- 这些是非法的,因为 128 不在双亲类型中,当进一步约束它时失去它的扩展附加

B2::=A1(1..16)

- 这是合法的。B2 不是可扩展的,而且包含 1~16。

A2::=INTEGER(1..32)(MIN..63)

- MIN 是 1,而 63 是非法的。

A3::=INTEGER((1..32)INTERSECTION(MIN..63))

- 这是合法的。MIN 是负无穷,而 A3 包含 1~32。

I.4.3 使用集合运算

I.4.3.1 结果在很大程度上凭直觉,并遵循交叉、联合和集差(EXCEPT)的通常数学规则。特别是,交

叉和联合都是可交换的,即:

(⟨some set 1 of values⟩INTERSECTION⟨some set 2 of values⟩)

和

(⟨some set 2 of values⟩INTERSECTION⟨some set 1 of values⟩)

对于 UNION 类似的一样。

I.4.3.2 可交换性是真实的,不管什么值集是可扩展的,也不管出现什么扩展附加。

I.4.3.3 如果交叉不能使扩展附加值存在将出现误解。这与 INTEGER(MIN..MAX,...)情况类似。

I.4.3.4 例如:

$A := \text{INTEGER}((1..256, \dots, B)(1..256))$

—A 总是(只)包含值 1..256,不管 B 包含什么值。

I.4.3.5 记住这点也很重要:虽然双亲类型在进一步约束时失去其可扩展性和扩展附加,且包含的子类型失去其可扩展性和扩展附加,但是集合运算中直接规定的值集既不失去其可扩展性也不失去其扩展附加。

I.4.3.6 由集合运算产生的值集可扩展性规则已在 50.4 和 50.5 中清楚地陈述了,而且不依赖于集合运算是否可能产生实际的扩展附加。

I.4.3.7 这里,为了完整而总结了规则,其中用 E 指明有“可扩展”标志集的值集,用 N 指明在形式上无可扩展的值集。用 R 指明每个集的根中的值,用 X 指明扩展附加(若有的话),而且为每种情况给出产生的内容。

注 1: 为了本附录的目的,并为了简化说明,如果值集不是可扩展的,那么我们将它的各个值都描述为根值。

注 2: 如果系列应用约束中使用的值的任何产生集的根是空,这是非法的规范。

注 3: 为避免在下面冗长,在“扩展附加”更正确的地方使用“扩展”。

I.4.3.8 这些规则是:

$N1 \text{ INTERSECTION } N2 = \rangle N$

Root:R1 INTERSECTION R2

$N1 \text{ INTERSECTION } E2 = \rangle E$

Root:R1 INTERSECTION R2,Extensions:R1 INTERSECTION X2

$E1 \text{ INTERSECTION } E2 = \rangle E$

Root:R1 INTERSECTION R2,Extensions:((R1 UNION X1)

INTERSECTION

(R2 UNION X2)

EXCEPT

(R1 INTERSECTION R2)

$N1 \text{ UNION } N2 = \rangle N$

Root:R1 UNION R2

$N1 \text{ UNION } E2 = \rangle E$

Root:R1 UNION R2,Extensions:X2

$E1 \text{ UNION } E2 = \rangle E$

Root:R1 UNION R2,Extensions:(R1 UNION X1 UNION R2 UNION X2)

EXCEPT

(R1 UNION R2)

$N1 \text{ EXCEPT } N2 = \rangle N$

Root:R1 EXCEPT R2

$N1 \text{ EXCEPT } E2 = \rangle N$


```

      Root;R1 EXCEPT R2
E1 EXCEPT N2=>E
      Root;R1 EXCEPT R2,Extensions:(X1 EXCEPT R2)
                                EXCEPT
                                (R1 EXCEPT R2)
E1 EXCEPT E2=>E
      Root;R1 EXCEPT R2,Extensions:(X1 EXCEPT(R2 UNION X2))
                                EXCEPT
                                (R1 EXCEPT R2)
N1...N2=>E
      Root;R1,Extensions;R2 EXCEPT R1
E1...N2=>E
      Root;R1,Extensions;X1 UNION R2
                                EXCEPT
                                R1
N1...E2=>E
      Root;R1,Extensions;R2 UNION X2
                                EXCEPT
                                R1
E1...E2=>E
      Root;R1,Extensions;X1 UNION R2 UNION E2
                                EXCEPT
                                R1

```

注：如果值的可扩展集的集合运算结果没有实际的扩展附加，或者甚至决不可能有实际扩展附加（不管什么扩展附加加到可扩展输入上），对于上面的结果 E，其结果在形式上仍然被定义为可扩展的。

I.4.4 使用包含子类型记法

包含的子类型可能是可扩展的，也可能不是可扩展的，但是当它用在集合运算中时，它常常当作不可扩展的来处理，而且所有它的扩展附加被丢弃。

附 录 J

(资料性)

TIME 类型的辅导附录

J.1 时间和日期的 ASN.1 类型的集合

J.1.1 从历史上看,ASN.1 将自己的时间类型 UTCTime 定义为“UsefulType”,因为它是基于规定 VisibleString 类型的内容。后来,GeneralizedTime 被添加,允许四位数的年份,并通过引用一组标准定义,这些标准是 GB/T 7408.1—2023 的第一个(1988 年)版本的前身,以规定 VisibleString 的内容。(通过规定 GraphicString 的内容定义的另一个“UsefulType”是 ObjectDescriptor。)传统上,所谓的“UsefulType”使用混合的大写和小写字母作为其类型引用名称,而另一个仅使用大写字母的固有 ASN.1 类型。然而,有用的类型有它们自己的 UNIVERSAL 类标记,并且能够在编码规则规范中独立引用。

J.1.2 虽然这些类型(UTCTime、GeneralizedTime 和 ObjectDescriptor)无疑是有用的,但通过术语“UsefulType”将它们与其他类型分开(仅仅是因为它们是根据其他-字符串-类型定义的),以及使用混合大写和小写在它们的类型引用名称中越来越被认为是历史上的偶然。

J.1.3 随着时间类型的引入以支持 GB/T 7408.1—2023 的 2004 版,认识到需要一个主要时间类型(TIME),但是一些常用的时间类型(DATE、TIME-OF-DAY、DATE-TIME 和 DURATION),定义为基本 TIME 类型的子集(使用 ASN.1 子类型记法),也是必需的。决定将这些称为“Useful time types”,并将它们的名称全部大写,以尽量减少向后兼容性问题,因为它们是新的保留字。它们都有不同的 UNIVERSAL 类标记,与 TIME 类型的标记不同(以启用优化的 BER 编码),并且都列在产生式“BuiltinType”下(见 17.2)。

J.2 GB/T 7408.1—2023 关键概念

J.2.1 GB/T 7408.1—2023 为识别瞬时及其字符表示法提供了明确的引用。它构成了 ASN.1 TIME 类型规范的基础,既涉及时间相关的概念,也涉及 ASN.1 值记法和基本编码规则(BER)中使用的实际表示法。

J.2.2 GB/T 7408.1—2023 完全基于 1582 年引入的公历,以及所谓的预测公历,即从 1582 年开始按时间顺序向后扩展公历,使用平年(非闰年)和闰年定义的常规规则。通常没有简单的方法确定从使用预测公历规定的日期开始使用儒略历来确定公元后(AD)或公元前(BC)日期,但特别是公元后 1 年大致(但不完全)与预测公历 1 年一致,并且公元前 1 年(前一年)大致(但不完全)与预测公历 0 年一致。

J.2.3 GB/T 7408.1—2023 中的关键定义和概念包括时间轴的多个时间刻度的概念。每个时间刻度由时间轴上的有序标记集组成。每个标记代表一个时间(瞬时)。

J.2.4 GB/T 7408.1—2023 定义了三个主要的时间刻度。

J.2.4.1 第一个称为日历日期时间刻度。这具有对应于日历年、日历月和日历月内一天的序号的标记(天数从 01 到 28、29、30 或 31,具体取决于月份)。

J.2.4.2 第二个称为顺序日期时间刻度。这具有对应于日历年的标记,以及日历年内一天的顺序(1 月 1 日的天数为 001 到 365 或 366,具体取决于年份)。

J.2.4.3 第三个称为星期日期时间刻度。这具有对应于日历年的标记,该日历年内一周的序号,以及该周内一天的序号(第 1 天是星期一)。周编号为 01 到 52 或 53(取决于年份),第 01 周被定义为包含 1 月 4 日的那一周,而上一年的一周被定义为前一周(这就是为什么有些年份包含 53 周)。

J.2.5 在每个时间刻度上的一天标记之间是小时、分钟和秒标记。然而,时间轴是瞬时的连续体,所有三个时间刻度也包含了在时间轴上到处密集的标志。

注:表达这一点的另一种方式是说,在任何两个标记之间存在无限多的其他标记,每个标记标识一秒的小数部分到任意大的精度。

J.2.6 日历日期时间刻度的变体是不表示秒的时间刻度,但在每分钟之间有无限多的其他标记,表示该分钟的小数部分到任意大的精度。小时的小数部分也是如此。

注:GB/T 7408.1—2023 中没有使用一天的小数部分或任何更大的时间单位来规定时间的概念,尽管能够使用一年、一个月、一周或一天的小数部分来指定时间长度。

J.2.7 因为有理数 $1/60$ 没有确定的小数表示,所以在任何有限表示法中,使用秒的时间刻度上的某些时间不能表示为使用分钟的小数时间刻度上的时间。

J.2.8 同样,在不知道哪些年份是闰年的情况下,不可能标识一个时间刻度上的某一天的标记对应于不同时间刻度上的某一天的标记。使用基于起点和终点、或起点和时间长度(例如,以秒为单位)或时间长度和终点的刻度来识别时间间隔的闰秒也会出现类似的问题。

J.2.9 GB/T 7408.1—2023 也认能够不同的精度标识标记的概念。因此,在任何给定的时间刻度上,能够同时有不同的标记,一个标记规定为(例如)3.100 秒,另一个标记规定为 3.1 秒。

注:在有关时间类型(UTCTime 和 GeneralizedTime)的早期 ASN.1 工作中,没有解决以不同精度表示同一时间的不同抽象值的问题。在 TIME 类型的情况下,时间轴上具有不同精度但放置在同一时刻的标记被牢固地标识为不同的抽象值。因此,可能由 3.100 表示的抽象值与可能由 3.1 表示的抽象值不同,并且可能带有不同的应用语义。

J.2.10 对所使用的精度控制,以及对 GB/T 7408.1—2023 的某些其他方面的控制,在 GB/T 7408.1—2023 中被规定为“by mutual agreement”。一般而言,如果 GB/T 7408.1—2023 标识了需要相互同意的地方,则 ASN.1 中为应用设计者提供了在 ASN.1 类型定义中规定要假定的相互同意的记法。这是通过使用与每个时间抽象值相关联的时间属性来选择 TIME 类型中多个无穷大抽象值的子集来实现的。

J.2.11 GB/T 7408.1—2023 认可时差的概念。这是特定世界时区的当地时间与 UTC 之间的时差。没有就不同世界时区的时差达成一致或记录的国际权威机构。这是当地政府的事情,尽管 HM Nautical Almanac Office(UK)试图保持世界各地当前指派的时差的权威记录。截至 2005 年,各地方政府已定义了 $-12 \sim +14$ 范围内的时差。考虑到将来可能发生的变化,ASN.1(仅)支持 $-15 \sim +16$ 范围内的时差。

J.3 TIME 类型的抽象值

J.3.1 具有每个精度的每个时间刻度上的每个标记都被标识为 TIME 类型的不同抽象值,因此在所有 ASN.1 编码规则中具有不同的 ASN.1 值记法和不同的编码。

J.3.2 GB/T 7408.1—2023 主要关注时间的标识,但区分仅标识日期、仅标识时间以及日期和时间。这些不同的标识也会在 TIME 类型中产生不同的抽象值。

J.3.3 在一天的时间标识内,能够使用一天中的当地时间或 UTC 或两者。同样,这些不同的标识会产生不同且不相关的抽象值,因为正在使用不同的时间刻度(并且通常会携带不同的应用语义)。

J.3.4 GB/T 7408.1—2023 的另一个特性是使用起点和终点(能够使用各种时间刻度中的任何一个来标识)、时间长度、或具有起点或终点的时间长度来标识时间间隔。同样,这些提供了四个主要的抽象值集,它们与表示时间的抽象值不同,但具有这些主要时间间隔集的许多子集,取决于时间间隔的起点和终点规范中使用的抽象值,或用于规定时间长度的时间组件。

注:在 ASN.1 中,不能对时间间隔的起点和终点使用不同的时间尺度。这是为了规范的简单性,并且预计不会成为

应用设计者的问题。

J.3.5 最后,GB/T 7408.1—2023 具有规定循环时间间隔的概念。循环时间间隔映射为与表示时间间隔和时间抽象值不同的抽象值。

J.4 时间抽象值的时间属性

J.4.1 标识具有公共时间属性的时间抽象值集是可能的。某些时间属性(例如它是时间、时间间隔或循环时间间隔)适用于所有时间抽象值。其他时间属性,例如时间间隔是表示为起点和时间长度,还是表示为时间长度和终点(例如),仅适用于作为时间间隔的时间抽象值。类似地,指示时间抽象值是否表示当地时间、UTC 或两者的时间属性,仅适用于具有至少一个与当日时间相关的组件的时间抽象值。

J.4.2 38.2 和表 6 规定了与时间抽象值相关的完整时间属性集,以及每个时间属性的可能设置。时间属性的设置能够在子类型记法中用于选择 TIME 类型的子集,对于给定的时间属性,所有这些子集的抽象值都具有相同的设置。

注:使用术语“setting of a time property”而不是“value of a time property”来避免与术语“abstract value”中的“value”的使用混淆。

J.4.3 如上所述,时间抽象值上某些时间属性的存在取决于其他时间属性的设置。

J.4.4 在子类型记法中,TIME 类型的抽象值是使用时间属性和设置对的列表来规定的。能够被规定的时间属性和设置的组合具有的限制(见 51.10.6),但属性和设置对的顺序无关紧要(见 J.4.5)。如果且只有它具有适用于它的所有列出的属性的规定设置,则 TIME 类型的抽象值包含在子类型中。像之前一样,如果指派给类型引用的抽象值的结果集为空(尽管在集合运算中不禁止空集),则它是非法的 ASN.1 规范。

J.4.5 为了使读者更清楚和避免错误,宜但不只允许属性和设置对的规范顺序从主要属性(例如“Basic=Date-Time”)到更详细的属性(例如“Time=HMS”)。这通常意味着按表 6 的顺序规定时间属性和设置对。本文件的所有示例都使用了该约定(见 38.4、G.3 和 G.5.8)。

J.4.6 对于时间间隔规范和使用值范围分成子类型来说,在 TIME 类型的抽象值上具有顺序关系是很重要的。通常,如果且只有它们都具有相同的时间属性设置,则表示时间的抽象值之间存在顺序关系(基于它们在时间轴上的位置)。类似地,两个时间抽象值之间的秒数通常只有在它们具有相同的属性设置时才能确定。这意味着在某些子类型记法和时间间隔规范中使用的时间需要具有相同的属性设置。仅当它们具有相同的精度并且仅在单个时间组件上有所不同时,才为时间长度定义顺序关系(见 51.11)。

J.5 值记法

J.5.1 抽象值的值记法(和编码)取决于其相关的时间属性及其设置。值记法在 38.3 中规定。

J.5.2 GB/T 7408.1—2023 通常规定两种单独的(基于字符的)表示法,称为基本格式和扩展格式,用于在时间刻度上标识标记。

J.5.3 通常,基本格式是简单的数字字符串,仅在需要在 GB/T 7408.1—2023 所说明的抽象值的常用子集内提供无歧义表示法的情况下才使用非数字分隔符(例如小数分隔符)。ASN.1 值记法不使用此格式。

注:例如,在基本格式中,字符串 2020 既代表 2020,也代表时间下午 8:20。

J.5.4 扩展格式包含附加的非数字分隔符,旨在使表示法对用户更具可读性,并且通常(但有一个例外-见下面的注 1)对 GB/T 7408.1—2023 说明的所有抽象值是无歧义的。GB/T 7408.1—2023 推荐将扩展格式用于纯文本。

注 1: 一个例外是用四位数字表示一个世纪的日期表示法,这可能与表示一年的四位数字混淆。在 ASN.1 值记法中,为所有世纪表示(包括两位数表示)添加一个 C 世纪记法,以解决歧义。

注 2: 例如,在扩展格式中,2020 年将由 2020 表示,但时间下午 8:20 将由 20:20 表示。

J.5.5 扩展格式的使用(为世纪添加大写 C)使得所表示的抽象值的属性设置从值记法确定,并且该记法无歧义地标识一个抽象值,只知道它的类型是 TIME 类型。

J.5.6 基本格式需要知道所表示的抽象值的一些属性设置,以解决表示法中的歧义,并且在 ASN.1 中未使用。

J.5.7 基本 ASN.1 值记法和 XML 值记法(在 38.3 中规定)以及 ISO/IEC 8825-4:2021 中规定的 XML 编码规则使用 GB/T 7408.1—2023 扩展格式。ISO/IEC 8825-1:2021 中指定的基本编码规则使用 GB/T 7408.1—2023 扩展格式(但删除了一些指示符和分隔符,例如时间长度的 P,当日时间的冒号和日期的连字符)。不同的 ASN.1 标记被指派给有用的类型,以使 BER 能够标识需要的属性设置,以解决有用时间类型的编码中可能存在的歧义。ISO/IEC 8825-2:2021 中规定的紧缩编码规则使用与 GB/T 7408.1—2023 不相关(并且超出其范围)的二进制编码。PER 编码提供了非常紧凑的日期、时间和时间长度表示[通常一个日期为 17 位,一个时间为 15 位,一个日期时间为 32 位(4 个八位字节),时间长度通常小于 16 位]。

J.6 使用 ASN.1 子类型记法

J.6.1 这种类型(见第 51 章和表 12)允许使用六种形式的子类型记法(加上受限情况下的内部子类型—见 J.6.8)。

注: 在 38.4、G.3 和 G.5.8 中能够找到 TIME 类型和有用时间类型的子类型记法示例。

J.6.2 属性设置子类型记法允许为一个或多个列出的时间属性选择具有给定设置的所有抽象值。这是为应用生成附加自定义时间类型的常用方法,并且在 J.7 中进行了更全面的讨论。

J.6.3 允许使用单值子类型,但通常不会有用。

J.6.4 允许包含子类型,并且有望在应用设计者的自定义时间类型规范中普遍使用。

J.6.5 能够使用时间长度范围子类型(包含有序的时间长度对)。它们仅从双亲类型中选择那些作为时间长度规定的时间间隔抽象值,并将时间长度限制在规定范围内(见 51.11)。

J.6.6 能够使用时间点范围子类型(包含有序的时间对)。它们仅从双亲类型中选择那些与时间点范围两端具有相同属性设置的时间抽象值(要求具有相同的属性设置),并将时间限制在规定范围内。

注: 这种子类型约束限制了时间值的范围,并且与直接使用记法来标识作为时间间隔的单个时间抽象值是完全分开的。

J.6.7 能够使用重复范围子类型(包含有序的整数对)。它们仅从双亲类型中选择那些循环时间间隔的抽象值,并限制可用于规定重复次数的位数。

J.6.8 如果时间类型已经被限制为时间长度(通常使用 DURATION 有用时间类型),则能够使用内部子类型。这使得能够对时间长度规范的形式进行限制。

J.6.9 不允许其他形式的子类型约束。

J.7 属性设置子类型记法

J.7.1 对于给定时间属性具有相同设置的时间抽象值形成时间类型的自然子集。属性设置子类型记法通过列出一个或多个时间属性的设置来选择抽象值。如果且只有抽象值具有适用于它的所有列出的属性的规定设置时,抽象值才会包含在生成的子类型中。

示例: 以下记法可用于定义时间子类型,该子类型仅包含是日期的所有抽象值,使用四位数的年份、年内的周和日

期规定：

My-time ::= TIME(SETTINGS"Basic=Date Date=YWD Year=Basic")

在 38.4、G.3 和 G.5.8 中给出了一组更广泛的子类型记法的示例。

J.7.2 ASN.1 集合运算(或简单地使用多个约束)能够以正常方式用于定义时间子类型的组合(使用 INTERSECTION、UNION 和 EXCEPT),以生成适用于特定应用的类型。

J.7.3 表 6 中规定了时间属性、它们的名称、可能的设置以及具有此设置的抽象值。

J.7.4 使用属性设置子类型记法(见 38.4)规定少量有用时间子类型,并为其赋予人性化名称。预计这些有用的子类型((DATE、TIME-OF-DAY、DATE-TIME 和 DURATION)对于许多应用来说将是足够的。在附录 B 的 ASN.1 定义的时间类型模块中规定了一组更广泛的具有一般效用的已定义时间类型。这些类型能够直接导入和使用,也能够用于定义特定应用的时间类型。它们支持 GB/T 7408.1—2023 的全部功能。此外,在必要时,设计者能够使用属性设置子类型记法将附加类型定义为 TIME 类型或有用或已定义时间类型的子类型。这些类型能够使用 ASN.1 集合运算进一步组合。

注：有用时间类型被赋予了不同于 TIME 类型的 ASN.1 的 UNIVERSAL 类标记,以允许在 BER 中进行有效编码。

它们需被视为独立类型而不是子类型,但如果双亲类型是 TIME,它们也能够在已包含子类型约束中使用。

附 录 K

(资料性)

TIME 类型的值记法分析

K.1 概述

K.1.1 本文件的正文规定具有给定时间属性的抽象值的值记法。

K.1.2 该值记法的每个实例都无歧义地标识了时间类型的单个抽象值及其属性。

K.1.3 本附录描述了一种可能的算法,用于确定由值记法实例表示的抽象值的时间属性设置。有许多替代的(可能更好的)算法,提供这个附录只是为了证明这些算法的存在。

注:如果将此算法应用于随机字符串,它将标识出该字符串只能表示具有给定时间属性设置集的抽象值。在接受记法并标识抽象值之前,有必要检查字符串的语法是否符合具有这些属性设置的抽象值所需的语法。

K.1.4 如果两个抽象值具有相同的属性设置,则它们的值记法仅在记法中出现的数值上不同,但以下例外:

- a) 时间长度抽象值有几种不同的表示法,具体取决于使用的时间单位,以及是否使用小数;
- b) 逗号或句号都能够用作小数分隔符;
- c) 整数个小时的时差组件可仅用小时表示,也可用小时和分钟表示;
- d) 如果时差与起点上的时差相同,则表示 UTC 的时间间隔的终点可省略时差组件;
- e) 仅通过时差组件的加或减而不同的抽象值仍然具有相同的时间属性。

K.1.5 如果两个字符串仅在字符串中出现的数字的实际值不同,则它们具有相同的属性设置,但使用 0000 到 1581 范围内的年份日期意味着日期具有属性设置“Year=Proleptic”而不是“Year=Basic”(类似于世纪记法)。

K.2 分析完整的字符串

K.2.1 如果字符串以 LATIN CAPITAL LETTER R 开头,那么它具有属性设置“Basic=RecInterval”。“R”后面会跟着一些重复的字符串(空字符串表示无界),然后接斜线(“/”)。如果“R”之后和“/”之前的字符串部分为空,则它具有属性设置“Recurrence=Unlimited”。否则,如果“R”之后和“/”之前的字符串部分中的位数是 1、2、3 等,则它具有属性设置“Recurrence=R1”,“Recurrence=R2”,“Recurrence=R3”等。字符串的其余部分能够被分析为包含时间间隔的字符串(见 K.3)以确定附加的属性设置。

K.2.2 否则,如果字符串包含一个斜线(“/”),那么它具有属性设置“Basic=Interval”,并且能够被分析为一个包含时间间隔的字符串(见 K.3)以确定附加的属性设置。

K.2.3 否则,如果字符串以 LATIN CAPITAL LETTER P 开头,那么它具有设置“Basic=Interval”和“Interval-type=D”,完成属性分析。

K.2.4 否则,如果字符串包含 LATIN CAPITAL LETTER T,那么它具有属性设置“Basic=Date-Time”,并且“T”之前的字符串部分能够被分析为包含日期的字符串(见 K.4)，“T”后面的部分能够分析为包含时间的字符串(见 K.7)以确定进一步的属性设置。

K.2.5 否则,如果字符串以 LATIN CAPITAL LETTER C 结尾,那么它具有属性设置“Basic=Date”和“Date=C”,并且能够被分析为包含世纪的字符串(见 K.6)以确定进一步的属性设置。

K.2.6 否则,如果字符串包含冒号(“:”),或少于四个字符,或多于四个字符,第三个字符是加号(“+”)或负号(“-”),那么它具有属性设置“Basic=Time”,能够分析为包含时间的字符串(见 K.7)以确定进一步的属性设置。

K.2.7 否则,它具有属性设置“Basic=Date”,能够分析为包含日期的字符串(见 K.4)以确定进一步的属性设置。

K.3 分析包含时间间隔的字符串

K.3.1 如果字符串以 LATIN CAPITAL LETTER P 开头并且不包含斜线(“/”),则它具有属性设置“Interval-type=D”,完成属性分析。

K.3.2 如果字符串包含斜线,则斜线之前的字符串部分或斜线之后的字符串部分将以 LATIN CAPITAL LETTER P 开头,或者两者都不会以“P”开头(但不能同时使用两者)。如果:

- a) 斜线之前的部分以“P”开头,则它具有属性设置“Interval-type=DE”,并且可通过分析 K.3 后续子条款中规定的斜线之后的部分来确定进一步的属性;
- b) 斜线之后的部分以“P”开头,那么它具有属性设置“Interval-type=SD”,并且可通过分析 K.3 后续子条款中规定的斜线之前的部分来确定进一步的属性;
- c) 如果两个部分都不以“P”开头,那么它具有属性设置“Interval-type=SE”,并且可通过分析 K.3 后续子条款中规定的斜线之前的部分来确定进一步的属性。

注: ASN.1(但 GB/T 7408.1—2023 中没有)要求终点具有与起点相同的时间属性设置。这意味着在确定属性设置时不需要分析字符串的终点部分。然而,需要注意的是,如果与起点的时差相同,则允许省略时差组件的终点表示法。在确定终点的抽象值时需要考虑这一点。

K.3.3 如果该部分包含 LATIN CAPITAL LETTER T,则它具有属性设置“SE-point = Date-Time”,并且“T”之前的部分能够被分析为包含日期的字符串(见 K.4)，“T”后面的部分能够分析为包含时间的字符串(见 K.7)以确定进一步的属性设置。

K.3.4 如果该部分以 LATIN CAPITAL LETTER C 结尾,则它具有属性设置“SE-point = Date Date = C”,并且能够将其分析为包含世纪的字符(见 K.6)以确定进一步的属性设置。

K.3.5 否则,如果该部分包含一个冒号(“:”),则它具有属性设置“SE-point = Time”,并且能够将其分析为包含时间的字符串(见 K.7)以确定进一步的属性设置。

K.3.6 否则,它具有属性设置“SE-point = Date”,并且能够将其分析为包含日期的字符串(见 K.4)以确定进一步的属性设置。

K.4 分析包含日期的字符串

K.4.1 如果字符串以 LATIN CAPITAL LETTER C 结尾,则它具有属性设置“Date=C”,并且字符串的其余部分能够被分析为包含世纪的字符串(见 K.6)。

K.4.2 如果字符串以负号(“—”)开头,则在 K.4 的其余部分的分析中宜忽略它。

注: 在这种情况下,连字符表示负号,而不是分隔符。

K.4.3 否则,字符串将包含 0 个、一个或两个负号(“—”),并且在最后两种情况下可能包含或不包含 LATIN CAPITAL LETTER W。

K.4.4 如果字符串不包含 LATIN CAPITAL LETTER W,那么如果:

- a) 该字符串不包含负号字符,则它具有属性设置“Date=Y”,能够分析为包含年份的字符串(见 K.5)以进一步确定属性设置;
- b) 该字符串包含一个负号字符,则它具有属性设置“Date=YM”;月份的两位数字将跟在负号之后,负号之前的部分能够分析为包含年份的字符串(见 K.5)以确定进一步的属性设置;
- c) 该字符串包含两个负号字符,则它具有属性设置“Date=YMD”;第一个负号后面是月份的两位数,第二个负号后面是天的两位数,并且第一个负号之前的字符串部分能够被分析为包含年份的字符串(见 K.5)以确定进一步的属性设置。

K.4.5 如果字符串包含 LATIN CAPITAL LETTER W,那么如果:

- a) 该字符串包含一个负号字符,则它具有属性设置“Date=YW”;周的两位数字将在负号之后,负号之前的部分能够分析为包含年份的字符串(见 K.5)以确定进一步的属性设置。
- b) 如果字符串包含两个负号字符,则它具有属性设置“Date=YWD”;第一个负号之后是周的两位数字,第二个负号之后是天的一位数字,第一个负号之前的字符串部分能够分析为包含年份的字符串(见 K.5)以确定进一步的属性设置。

K.5 分析包含年份的字符串

K.5.1 如果该字符串以负号(“—”)开头,并且是五个字符,则它具有属性设置“Year=Negative”,完成属性分析。

K.5.2 否则,如果字符串超过四个字符并且不以负号(“—”)开头,则它具有属性设置“Year=L5”“Year=L6”“Year=L7”等,分别对应字符个数等于 5、6、7 等。

K.5.3 否则,如果字符串多于四个字符并以负号(“—”)开头,则它具有属性设置“Year=L5”“Year=L6”“Year=L7”等,分别对应字符个数等于 6、7、8 等。

K.5.4 否则,如果四位字符串的值小于 1 582,则它具有属性设置“Year=Proleptic”,完成属性分析。

K.5.5 否则,它具有属性设置“Year=Basic”,完成属性分析。

K.6 分析包含世纪的字符串

K.6.1 如果字符串以负号(“—”)开头,并且是三个字符,则它具有属性设置“Year=Negative”,完成属性分析。

K.6.2 否则,如果字符串多于两个字符且不以负号(“—”)开头,则它具有属性设置“Year=L5”“Year=L6”“Year=L7”等,分别对应字符个数等于 3、4、5 等。

K.6.3 否则,如果字符串多于两个字符并以负号(“—”)开头,则它具有属性设置“Year=L5”“Year=L6”“Year=L7”,等,分别对应字符个数等于 4、5、6 等。

K.6.4 否则,如果两位数字字符串的值小于 15,则它具有属性设置“Year=Proleptic”,完成属性分析。

K.6.5 否则,它具有属性设置“Year=Basic”,完成属性分析。

K.7 分析包含时间的字符串

K.7.1 如果字符串以 LATIN CAPITAL LETTER Z 结尾,则它具有属性设置“Local-or-UTC=Z”,并且“Z”之前的字符串部分能够分析为包含简单时间的字符串(见 K.8)以确定进一步的属性设置。

K.7.2 否则,如果字符串包含加号(“+”)或减号(“—”),则它具有属性设置“Local-or-UTC=LD”,以及加号或减号之前的字符串部分能够分析为一个简单时间(见 K.8)以确定进一步的属性设置。

注:在确定属性设置时不需要分析加号或减号(时差)之后的字符串部分。

K.7.3 否则,它具有属性设置“Local-or-UTC=L”并且能够分析为一个简单时间(见 K.8)以确定进一步的属性设置。

K.8 分析包含简单时间的字符串

K.8.1 该字符串将包含 0 个、一个或两个冒号(:),并且可能包含一个小数点符号,即句点(“.”)或逗号(“,”)。

K.8.2 如果字符串不包含小数点符号,那么如果:

- a) 该字符串不包含冒号,则它具有属性设置“Time=H”,完成属性分析;
- b) 该字符串包含一个冒号,则它具有属性设置“Time=HM”,完成属性分析;
- c) 该字符串包含两个冒号,则它具有属性设置“Time=HMS”,完成属性分析。

K.8.3 如果字符串包含小数点符号,那么如果:

- a) 该字符串不包含冒号,如果小数点后的位数分别为 1、2、3 等,则它具有属性设置“Time=HF1”“Time=HF2”“Time=HF3”等,完成属性分析;
- b) 该字符串包含一个冒号,如果小数点后的位数为 1、2、3 等,则它具有属性设置“Time=HMF1”“Time=HMF2”“Time=HMF3”等,完成属性分析;
- c) 该字符串包含两个冒号,如果小数点后的位数为 1、2、3 等,则它具有属性设置“Time=HMSF1”“Time=HMSF2”“Time=HMSF3”等,完成属性分析。

附 录 L
(资料性)
ASN.1 记法总结

在第 12 章中定义的下列词项：

typereference
 identifier
 valuereference
 modulereference
 comment
 empty
 number
 realnumber
 bstring
 xmlbstring
 hstring
 xmlhstring
 cstring
 xmlestring
 simplestring
 tstring
 xmltstring
 psname
 " :: = "
 " .. "
 " ... "
 "[["
 "]] "
 encodingreference
 integerUnicodeLabel
 non-integerUnicodeLabel
 "< /"
 "> /"
 "true"
 extended-true
 "false"
 extended-false
 "NaN"
 "INF"
 xmlasn1typename
 "{ "
 " } "

"<"
">"
","
."
"/"
"("
")"
"["
"]"
"-(HYPHEN-MINUS)
":"
"=
""(QUOTATION MARK)
"'(APOSTROPHE)
"(SPACE)
";"
"@"
"|"
"!"
"^"

ABSENT
ABSTRACT-SYNTAX
ALL
APPLICATION
AUTOMATIC
BEGIN
BIT
BMPString
BOOLEAN
BY
CHARACTER
CHOICE
CLASS
COMPONENT
COMPONENTS
CONSTRAINED
CONTAINING
DATE
DATE-TIME
DEFAULT
DEFINITIONS
DURATION
EMBEDDED

ENCODED
ENCODING-CONTROL
END
ENUMERATED
EXCEPT
EXPLICIT
EXPORTS
EXTENSIBILITY
EXTERNAL
FALSE
FROM
GeneralizedTime
GeneralString
GraphicString
IA5String
IDENTIFIER
IMPLICIT
IMPLIED
IMPORTS
INCLUDES
INSTANCE
INSTRUCTIONS
INTEGER
INTERSECTION
ISO646String
MAX
MIN
MINUS-INFINITY
NOT-A-NUMBER
NULL
NumericString
OBJECT
ObjectDescriptor
OCTET
OF
OID-IRI
OPTIONAL
PATTERN
PDV
PLUS-INFINITY
PRESENT
PrintableString
PRIVATE

REAL
RELATIVE-OID
RELATIVE-OID-IRI
SEQUENCE
SET
SETTINGS
SIZE
STRING
SYNTAX
T61String
TAGS
TeletexString
TIME
TIME-OF-DAY
TRUE
TYPE-IDENTIFIER
UNION
UNIQUE
UNIVERSAL
UniversalString
UTCTime
UTF8String
VideotexString
VisibleString
WITH

在本文件中使用的下列产生式,并且用上面的词法项作为终止符号:

```
ModuleDefinition ::=
    ModuleIdentifier
    DEFINITIONS
    EncodingReferenceDefault
    TagDefault
    ExtensionDefault
    " : : = "
    BEGIN
    ModuleBody
    EncodingControlSections
    END
ModuleIdentifier ::=
    modulereference
    DefinitiveIdentification
DefinitiveIdentification ::=
    DefinitiveOID |
    DefinitiveOIDandIRI |
```

```

    empty
DefinitiveOID ::=
    "{ "DefinitiveObjIdComponentList" }"
DefinitiveOIDandIRI ::=
    DefinitiveOID
    IRIValue
DefinitiveObjIdComponentList ::=
    DefinitiveObjIdComponent |
    DefinitiveObjIdComponent DefinitiveObjIdComponentList
DefinitiveObjIdComponent ::=
    NameForm |
    DefinitiveNumberForm |
    DefinitiveNameAndNumberForm
DefinitiveNumberForm ::= number
DefinitiveNameAndNumberForm ::= identifier("DefinitiveNumberForm")
EncodingReferenceDefault ::=
    encodingreference INSTRUCTIONS |
    empty
TagDefault ::=
    EXPLICIT TAGS |
    IMPLICIT TAGS |
    AUTOMATIC TAGS |
    empty
ExtensionDefault ::=
    EXTENSIBILITY IMPLIED | empty
ModuleBody ::=
    Exports Imports AssignmentList |
    empty
Exports ::=
    EXPORTS SymbolsExported";" |
    EXPORTS ALL";" |
    empty
SymbolsExported ::=
    SymbolList |
    empty
Imports ::=
    IMPORTS SymbolsImported";" |
    empty
SymbolsImported ::=
    SymbolsFromModuleList |
    empty
SymbolsFromModuleList ::=
    SymbolsFromModule |

```

SymbolsFromModuleList SymbolsFromModule
 SymbolsFromModule::=SymbolList FROM GlobalModuleReference SelectionOption
 SelectionOption::=
 empty |
 WITH"SUCCESSORS" |
 WITH"DESCENDANTS"
 GlobalModuleReference::=modulereference AssignedIdentifier
 AssignedIdentifier::=
 ObjectIdentifierValue |
 DefinedValue |
 empty
 SymbolList::=Symbol | SymbolList","Symbol
 Symbol::=Reference | ParameterizedReference
 Reference::=
 typereference |
 valuereference |
 objectclassreference |
 objectreference |
 objectsetreference
 AssignmentList::=Assignment | AssignmentList Assignment
 Assignment::=
 TypeAssignment |
 ValueAssignment |
 XMLValueAssignment |
 ValueSetTypeAssignment |
 ObjectClassAssignment |
 ObjectAssignment |
 ObjectSetAssignment |
 ParameterizedAssignment
 DefinedType::=
 ExternalTypeReference |
 typereference |
 ParameterizedType |
 ParameterizedValueSetType
 DefinedValue::=
 ExternalValueReference |
 valuereference |
 ParameterizedValue
 NonParameterizedTypeName::=
 ExternalTypeReference |
 typereference |
 xmlasnltypename
 ExternalTypeReference::=


```

    modulereference
    "."
    typereference
ExternalValueReference::=
    modulereference
    "."
    valuereference
AbsoluteReference::=
    "@ModuleIdentifier
    "."
    ItemSpec
ItemSpec::=
    typereference |
    ItemId"."ComponentId
ItemId::=ItemSpec
ComponentId::=
    identifier | number | "*"
TypeAssignment::=
    Typereference
    "::="
    Type
ValueAssignment::=
    valuereference
    Type
    "::="
    Value
XMLValueAssignment::=
    valuereference
    "::="
    XMLTypedValue
XMLTypedValue::=
    "<"&.NonParameterizedTypeName">"
    XMLValue
    "</"&.NonParameterizedTypeName">" |
    "<"&.NonParameterizedTypeName"/>"
ValueSetTypeAssignment::=
    Typereference
    Type
    "::="
    ValueSet
ValueSet::="{"ElementSetSpecs"}"
Type::=BuiltinType | ReferencedType | ConstrainedType
BuiltinType::=

```

BitStringType	
BooleanType	
CharacterStringType	
ChoiceType	
DateType	
DateTimeType	
DurationType	
EmbeddedPDVType	
EnumeratedType	
ExternalType	
InstanceOfType	
IntegerType	
IRIType	
NullType	
ObjectClassFieldType	
ObjectIdentifierType	
OctetStringType	
RealType	
RelativeIRIType	
RelativeOIDType	
SequenceType	
SequenceOfType	
SetType	
SetOfType	
PrefixedType	
TimeType	
TimeOfDayType	
ReferencedType ::=	
DefinedType	
UsefulType	
SelectionType	
TypeFromObject	
ValueSetFromObjects	
NamedType ::= identifier Type	
Value ::= BuiltinValue ReferencedValue ObjectClassFieldValue	
XMLValue ::= XMLBuiltinValue XMLObjectClassFieldValue	
BuiltinValue ::=	
BitStringValue	
BooleanValue	
CharacterStringValue	
ChoiceValue	
EmbeddedPDVValue	
EnumeratedValue	

ExternalValue	
InstanceOfValue	
IntegerValue	
IRIValue	
NullValue	
ObjectIdentifierValue	
OctetStringValue	
RealValue	
RelativeIRIValue	
RelativeOIDValue	
SequenceValue	
SequenceOfValue	
SetValue	
SetOfValue	
PrefixedValue	
TimeValue	
XMLBuiltinValue ::=	
XMLBitStringValue	
XMLBooleanValue	
XMLCharacterStringValue	
XMLChoiceValue	
XMLEmbeddedPDVValue	
XMLEnumeratedValue	
XMLExternalValue	
XMLInstanceOfValue	
XMLIntegerValue	
XMLIRIValue	
XMLNullValue	
XMLObjectIdentifierValue	
XMLOctetStringValue	
XMLRealValue	
XMLRelativeIRIValue	
XMLRelativeOIDValue	
XMLSequenceValue	
XMLSequenceOfValue	
XMLSetValue	
XMLSetOfValue	
XMLPrefixedValue	
XMLTimeValue	
ReferencedValue ::=	
DefinedValue	
ValueFromObject	
NamedValue ::= identifier Value	

```

XMLNamedValue ::=
    "< "&identifier">"XMLValne"</"&identifier">"
BooleanType ::= BOOLEAN
BooleanValue ::= TRUE | FALSE
XMLBooleanValue ::=
    EmptyElementBoolean
    TextBoolean |
EmptyElementBoolean ::=
    "< "&"true""/>" |
    "< "&"false""/>"
TextBoolean ::=
    extended-true |
    extended-false
IntegerType ::=
    INTEGER |
    INTEGER{"NamedNumberList"}
NamedNumberList ::=
    NamedNumber |
    NamedNumberList,"NamedNumber
NamedNumber ::=
    identifier("SignedNumber") |
    identifier("DefinedValue")
SignedNumber ::= number | "-"number
IntegerValue ::= SignedNumber | identifier
XMLIntegerValue ::=
    XMLSignedNumber |
    EmptyElementInteger |
    TextInteger
XMLSignedNumber ::=
    number |
    "-"&number
EmptyElementInteger ::=
    "< "&identifier"/>"
TextInteger ::=
    identifier
EnumeratedType ::=
    ENUMERATED{"Enumerations"}
Enumerations ::=
    RootEnumeration |
    RootEnumeration,"..."ExceptionSpec |
    RootEnumeration,"..."ExceptionSpec","AdditionalEnumeration
RootEnumeration ::= Enumeration
AdditionalEnumeration ::= Enumeration

```

```

Enumeration::=EnumerationItem | EnumerationItem", "Enumeration
EnumerationItem::=identifier | NamedNumber
EnumeratedValue::=identifier
XMLEnumeratedValue::=
    EmptyElementEnumerated |
    TextEnumerated
EmptyElementEnumerated::="<"&identifier"/>"
TextEnumerated::=identifier
RealType::=REAL
RealValue::=
    NumericRealValue | SpecialRealValue
NumericRealValue::=
    realnumber |
    "-"realnumber |
    SequenceValue
SpecialRealValue::=
    PLUS-INFINITY |
    MINUS-INFINITY |
    NOT-A-NUMBER
XMLRealValue::=
    XMLNumericRealValue | XMLSpecialRealValue
XMLNumericRealValue::=
    realnumber |
    "-"&realnumber
XMLSpecialRealValue::=
    EmptyElementReal |
    TextReal
EmptyElementReal::=
    "<"&PLUS-INFINITY"/>" |
    "<"&MINUS-INFINITY"/>" |
    "<"&NOT-A-NUMBER"/>"
TextReal::=
    "INF" |
    "-"&"INF" |
    "NaN"
BitStringType::=BIT STRING | BIT STRING{"NamedBitList"}
NamedBitList::=NamedBit | NamedBitList", "NamedBit
NamedBit::=identifier("number") |
    identifier("DefinedValue")
BitStringValue::=bstring | hstring | "{"IdentifierList"}" | "{}" |
    CONTAINING Value
IdentifierList::=identifier | IdentifierList", "identifier
XMLBitStringValue::=

```

```

XMLTypedValue |
xmlbstring |
XMLIdentifierList |
empty
XMLIdentifierList ::=
    EmptyElementList |
    TextList
EmptyElementList ::=
    "<\"&identifier\"/>" |
    EmptyElementList "<\"&identifier\"/>"
TextList ::=
    identifier |
    TextList identifier
OctetStringType ::= OCTET STRING
OctetStringValue ::= bstring | hstring | CONTAINING Value
XMLOctetStringValue ::=
    XMLTypedValue |
    xmlhstring
NullType ::= NULL
NullValue ::= NULL
XMLNullValue ::= empty
SequenceType ::=
    SEQUENCE{"{"}" |
    SEQUENCE{"ExtensionAndException OptionalExtensionMarker"} |
    SEQUENCE{"ComponentTypeLists"}
ExtensionAndException ::= "... | "...ExceptionSpec
OptionalExtensionMarker ::= ",..." | empty
ComponentTypeLists ::=
    RootComponentTypeList |
    RootComponentTypeList, "ExtensionAndException ExtensionAdditions
        OptionalExtensionMarker |
    RootComponentTypeList, "ExtensionAndException ExtensionAdditions
        ExtensionEndMarker", "RootComponentTypeList |
    ExtensionAndException ExtensionAdditions ExtensionEndMarker", "
        RootComponentTypeList |
    ExtensionAndException ExtensionAdditions OptionalExtensionMarker
RootComponentTypeList ::= ComponentTypeList
ExtensionEndMarker ::= ",..."
ExtensionAdditions ::= ",ExtensionAdditionList | empty
ExtensionAdditionList ::=
    ExtensionAddition |
    ExtensionAdditionList, "ExtensionAddition
ExtensionAddition ::= ComponentType | ExtensionAdditionGroup

```

ExtensionAdditionGroup::="["VersionNumber ComponentTypeList"]"

VersionNumber::=empty | number":

ComponentTypeList::=

ComponentType |

ComponentTypeList", "ComponentType

ComponentType::=

NamedType |

NamedType OPTIONAL |

NamedType DEFAULT Value |

COMPONENTS OF Type

SequenceValue::="{ "ComponentValueList" }" | "{ "" }"

ComponentValueList::=

NamedValue |

ComponentValueList", "NamedValue

XMLSequenceValue::=

XMLComponentValueList |

empty

XMLComponentValueList::=

XMLNamedValue |

XMLComponentValueList XMLNamedValue

SequenceOfType::=SEQUENCE OF Type | SEQUENCE OF NamedType

SequenceOfValue::="{ "ValueList" }" | "{ "NamedValueList" }" | "{ "" }"

ValueList::=Value | ValueList", "Value

NamedValueList::=

NamedValue |

NamedValueList", "NamedValue

XMLSequenceOfValue::=

XMLValueList |

XMLDelimitedItemList |

empty

XMLValueList::=

XMLValueOrEmpty |

XMLValueOrEmpty XMLValueList

XMLValueOrEmpty::=

XMLValue |

"<"&.NonParameterizedTypeName"/>"

XMLDelimitedItemList::=

XMLDelimitedItem |

XMLDelimitedItem XMLDelimitedItemList

XMLDelimitedItem::=

"<"&.NonParameterizedTypeName">"XMLValue

"</"&.NonParameterizedTypeName">" |

"<"&.identifier">"XMLValue"</"&.identifier">"

```

SetType ::=
    SET{" "}" |
    SET{"ExtensionAndException OptionalExtensionMarker"} |
    SET{"ComponentTypeLists"}
SetValue ::= {"ComponentValueList"} | {""}
XMLSetValue ::= XMLComponentValueList | empty
SetOfType ::= SET OF Type | SEQUENCE OF NamedType
SetOfValue ::= {"ValueList"} | {"NamedValueList"} | {""}
XMLSetOfValue ::=
    XMLValueList |
    XMLDelimitedItemList |
    empty
ChoiceType ::= CHOICE{"AlternativeTypeLists"}
AlternativeTypeLists ::=
    RootAlternativeTypeList |
    RootAlternativeTypeList", "
        ExtensionAndException ExtensionAdditionAlternatives
        OptionalExtensionMarker
RootAlternativeTypeList ::= AlternativeTypeList
ExtensionAdditionAlternatives ::= ", "ExtensionAdditionAlternativesList | empty
ExtensionAdditionAlternativesList ::=
    ExtensionAdditionAlternative |
    ExtensionAdditionAlternativesList", "ExtensionAdditionAlternative
ExtensionAdditionAlternative ::= ExtensionAdditionAlternativesGroup | NamedType
ExtensionAdditionAlternativesGroup ::=
    "[["VersionNumber AlternativeTypeList"]]"
AlternativeTypeList ::=
    NamedType |
    AlternativeTypeList", "NamedType
ChoiceValue ::= identifier": "Value
XMLChoiceValue ::= "<"&identifier">"XMLValue"</"&identifier">"
SelectionType ::= identifier"<"Type
PrefixedType ::=
    TaggedType |
    EncodingPrefixedType
PrefixedValue ::= Value
XMLPrefixedValue ::= XMLValue
TaggedType ::=
    Tag Type |
    Tag IMPLICIT Type |
    Tag EXPLICIT Type
Tag ::= "[["EncodingReference Class ClassNumber"]]"
EncodingReference ::=

```



```

        encodingreference": " |
        empty
ClassNumber::=number | DefinedValue
Class::=
    UNIVERSAL |
    APPLICATION |
    PRIVATE |
    empty
EncodingPrefixedType::=
    EncodingPrefix Type
EncodingPrefix::=
    "["EncodingReference EncodingInstruction"]"
ObjectIdentifierType::=OBJECT IDENTIFIER
ObjectIdentifierValue::=
    "{"ObjIdComponentsList}" |
    "{"DefinedValue ObjIdComponentsList}"
ObjIdComponentsList::=
    ObjIdComponents |
    ObjIdComponents ObjIdComponentsList
ObjIdComponents::=
    NameForm |
    NumberForm |
    NameAndNumberForm |
    DefinedValue
NameForm::=identifier
NumberForm::=number | DefinedValue
NameAndNumberForm::=identifier("NumberForm")
XMLObjectIdentifierValue::=
    XMLObjIdComponentList
XMLObjIdComponentList::=
    XMLObjIdComponent |
    XMLObjIdComponent&."&XMLObjIdComponentList
XMLObjIdComponent::=
    NameForm |
    XMLNumberForm |
    XMLNameAndNumberForm
XMLNumberForm::=number
XMLNameAndNumberForm::=
    identifier&("&XMLNumberForm&.")
RelativeOIDType::=
    RELATIVE-OID
RelativeOIDValue::=
    "{"RelativeOIDComponentsList}"

```

RelativeOIDComponentsList ::=
 RelativeOIDComponents |
 RelativeOIDComponents RelativeOIDComponentsList
 RelativeOIDComponents ::=
 NumberForm |
 NameAndNumberForm |
 DefinedValue
 XMLRelativeOIDValue ::=
 XMLRelativeOIDComponentList
 XMLRelativeOIDComponentList ::=
 XMLRelativeOIDComponent |
 XMLRelativeOIDComponent & "." & XMLRelativeOIDComponentList
 XMLRelativeOIDComponent ::=
 XMLNumberForm |
 XMLNameAndNumberForm
 IRIType ::= OID-IRI
 IRIValue ::=
 " " "
 FirstArcIdentifier
 SubsequentArcIdentifier
 " " "
 FirstArcIdentifier ::=
 "/"ArcIdentifier
 SubsequentArcIdentifier ::=
 "/"ArcIdentifier SubsequentArcIdentifier |
 empty
 ArcIdentifier ::=
 integerUnicodeLabel |
 non-integerUnicodeLabel
 XMLIRIValue ::=
 FirstArcIdentifier
 SubsequentArcIdentifier
 RelativeIRIType ::= RELATIVE-OID-IRI
 RelativeIRIValue ::=
 " " "
 FirstRelativeArcIdentifier
 SubsequentArcIdentifier
 " " "
 FirstRelativeArcIdentifier ::=
 ArcIdentifier
 XMLRelativeIRIValue ::=
 FirstRelativeArcIdentifier
 SubsequentArcIdentifier

EmbeddedPDVType ::= EMBEDDED PDV
 EmbeddedPDVValue ::= SequenceValue
 XMLEmbeddedPDVValue ::= XMLSequenceValue
 ExternalType ::= EXTERNAL
 ExternalValue ::= SequenceValue
 XMLExternalValue ::= XMLSequenceValue
 TimeType ::= TIME
 TimeValue ::= tstring
 XMLTimeValue ::= xmltstring
 DateType ::= DATE
 TimeOfDayType ::= TIME-OF-DAY
 DateTimeType ::= DATE-TIME
 DurationType ::= DURATION
 CharacterStringType ::=
 RestrictedCharacterStringType |
 UnrestrictedCharacterStringType
 CharacterStringValue ::=
 RestrictedCharacterStringValue |
 UnrestrictedCharacterStringValue
 XMLCharacterStringValue ::=
 XMLRestrictedCharacterStringValue |
 XMLUnrestrictedCharacterStringValue
 RestrictedCharacterStringType ::=
 BMPString |
 GeneralString |
 GraphicString |
 IA5String |
 ISO646String |
 NumericString |
 PrintableString |
 TeletexString |
 T61String |
 UniversalString |
 UTF8String |
 VideotexString |
 VisibleString
 RestrictedCharacterStringValue ::= cstring | CharacterStringList | Quadruple | Tuple
 CharacterStringList ::= "{ "CharSyms" }"
 CharSyms ::= CharsDefn | CharSyms, "CharsDefn"
 CharsDefn ::= cstring | Quadruple | Tuple | DefinedValue
 Quadruple ::= "{ "Group", "Plane", "Row", "Cell" }"
 Group ::= number
 Plane ::= number

Row::=number
 Cell::=number
 Tuple::="{ "TableColumn", "TableRow" }"
 TableColumn::=number
 TableRow::=number
 XMLRestrictedCharacterStringValue::=xmlcstring
 UnrestrictedCharacterStringType::=CHARACTER STRING
 UnrestrictedCharacterStringValue::=SequenceValue
 XMLUnrestrictedCharacterStringValue::=XMLSequenceValue
 UsefulType::=typereference

在 41.1 中定义的下列字符串类型:

UTF8String	GraphicString
NumericString	VisibleString
PrintableString	ISO646String
TeletexString	GeneralString
T61String	UniversalString
VideotexString	BMPString
IA5String	

在第 46 章~第 48 章中定义的下列实用类型:

GeneralizedTime
 UTCTime
 ObjectDescriptor

在第 49 章~第 51 章中使用的下列产生式:

ConstrainedType::=
 Type Constraint |
 TypeWithConstraint

TypeWithConstraint::=
 SET Constraint OF Type |
 SET SizeConstraint OF Type |
 SEQUENCE Constraint OF Type |
 SEQUENCE SizeConstraint OF Type |
 SET Constraint OF NamedType |
 SET SizeConstraint OF NamedType |
 SEQUENCE Constraint OF NamedType |
 SEQUENCE SizeConstraint OF NamedType

Constraint::="{ "ConstraintSpec ExceptionSpec" }

ConstraintSpec::=
 SubtypeConstraint |
 GeneralConstraint

SubtypeConstraint::=ElementSetSpecs

ElementSetSpecs::=
 RootElementSetSpec |
 RootElementSetSpec, "... " |

```

RootElementSetSpec", "...", "AdditionalElementSetSpec
RootElementSetSpec::= ElementSetSpec
AdditionalElementSetSpec::= ElementSetSpec
ElementSetSpec::= Unions | ALL Exclusions
Unions::=
    Intersections |
    UElms UnionMark Intersections
UElms::= Unions
Intersections::=
    IntersectionElements |
    IElems IntersectionMark IntersectionElements
IElems::= Intersections
IntersectionElements::= Elements | Elms Exclusions
Elms::= Elements
Exclusions::= EXCEPT Elements
UnionMark::= " | " | UNION
IntersectionMark::= "^" | INTERSECTION
Elements::=
    SubtypeElements |
    ObjectSetElements |
    ("ElementSetSpec")
SubtypeElements::=
    SingleValue |
    ContainedSubtype |
    ValueRange |
    PermittedAlphabet |
    SizeConstraint |
    TypeConstraint |
    InnerTypeConstraints |
    PatternConstraint |
    PropertySettings |
    DurationRange |
    TimePointRange |
    RecurrenceRange |
SingleValue::= Value
ContainedSubtype::= Includes Type
Includes::= INCLUDES | empty
ValueRange::= LowerEndpoint.."UpperEndpoint
LowerEndpoint::= LowerEndValue | LowerEndValue"<"
UpperEndpoint::= UpperEndValue | "<"UpperEndValue
LowerEndValue::= Value | MIN
UpperEndValue::= Value | MAX
SizeConstraint::= SIZE Constraint

```

```

TypeConstraint::=Type
PermittedAlphabet::=FROM Constraint
InnerTypeConstraints::=
    WITH COMPONENT SingleTypeConstraint |
    WITH COMPONENTS MultipleTypeConstraints
SingleTypeConstraint::=Constraint
MultipleTypeConstraints::=FullSpecification | PartialSpecification
FullSpecification::="{ "TypeConstraints" }"
PartialSpecification::="{ "...", "TypeConstraints" }"
TypeConstraints::=
    NamedConstraint |
    NamedConstraint", "TypeConstraints
NamedConstraint::=
    identifier ComponentConstraint
ComponentConstraint::=ValueConstraint PresenceConstraint
ValueConstraint::=Constraint | empty
PresenceConstraint::=PRESENT | ABSENT | OPTIONAL | empty
PatternConstraint::=PATTERN Value
PropertySettings::=SETTINGS simplestring
PropertySettingsList::=
    PropertyAndSettingPair |
    PropertySettingsList PropertyAndSettingPair
PropertyAndSettingPair::=PropertyName="SettingName
PropertyName::=psname
SettingName::=psname
DurationRange::=ValueRange
TimePointRange::=ValueRange
RecurrenceRange::=ValueRange
ExceptionSpec::="!"ExceptionIdentification | empty
ExceptionIdentification::=
    SignedNumber |
    DefinedValue |
    Type": "Value

```

参 考 文 献

- [1] ISO/IEC 7350:1991 (Information technology—Registration of repertoires of graphic characters from ISO/IEC 10367)
 - [2] ISO/IEC 8824 (所有部分) Information technology—Abstract Syntax Notation One (ASN.1)
 - [3] ISO/IEC 10646:2023 Information technology—Universal coded character set(UCS)
 - [4] Rec.ITU-R TF.460-5 Standard-frequency and timesignal emissions
 - [5] Rec.ITU-T T.101(1994) International interworking for Videotex services
 - [6] Rec.ITU-T X.692 (2021) | ISO/IEC 8825-3: 2021 Information technology—ASN.1 encoding rules: Specification of Encoding Control Notation(ECN)
 - [7] Rec.ITU-T X.695 (2021) | ISO/IEC 8825-6: 2021 Information technology—ASN.1 encoding rules: Registration and application of PER encoding instructions
 - [8] CCITT Rec.T.100:1988 International information exchange for interactive Videotex
-

中 华 人 民 共 和 国
国 家 标 准
信息技术 抽象语法记法一(ASN.1)
第 1 部分:基本记法规范

GB/T 16262.1—2025/ISO/IEC 8824-1:2021

*

中国标准出版社出版发行
北京市朝阳区和平里西街甲 2 号(100029)

网址:www.spc.net.cn

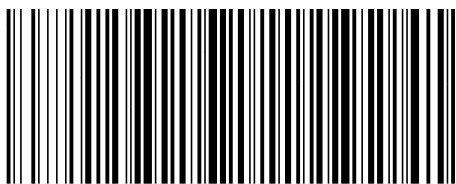
服务热线:400-168-0010

2025 年 3 月第 1 版

*

书号:155066 • 1-78879

版权专有 侵权必究



GB/T 16262.1-2025